The LLM Handbook

How large language models work, from tokenization to production.
Built from first principles: Why, How, What.

- **Derived, not catalogued.** Every component appears because the previous step left a problem unsolved.
- **From tokens to production.** Tokenization, attention and training; fine-tuning, quantization and Mixture of Experts; reasoning models, serving and agents.
- **Real numbers, small enough to follow by hand.** One full training step on a tiny GPT, with every value visible and gradient-checked.
- **Plain Markdown on GitHub.** Load the chapters into ChatGPT or Claude and use them as a tutor.

Created by Anand
LinkedIn: linkedin.com/in/anand-balagopal
GitHub: github.com/anandcoding-cloud/llm-handbook-how-large-language-models-work
Drafted with AI assistance (ChatGPT, Claude) and reviewed by the author.
Text licensed under CC BY 4.0. Code under the MIT licence.

Contents

START

Introduction: how this handbook teaches 3

BUILD A GPT

Chapter 1 — The Big Picture: How a Large Language Model Predicts the Next Token 7
Chapter 2 — Tokenization: Turning Text into Tokens with BPE 13
Chapter 3 — Embeddings: Turning Token IDs into Meaning and Position 19
Chapter 4 — Attention: Queries, Keys, Values and the Causal Mask 26
Chapter 5 — Multi-Head Attention: Many Attention Patterns at Once 40
Chapter 6 — The Feed-Forward Network: Expansion, Activation and GELU 49
Chapter 7 — The Transformer Block: Residual Connections and LayerNorm 63
Chapter 8 — Stacking Blocks into GPT: From Text to a Next-Word Probability 74

TRAIN IT

Chapter 9 — Training I: Learning from Text and Measuring Error with Cross-Entropy Loss 84
Chapter 10 — Training II: Gradients and Backpropagation 89
Chapter 11 — Training III: Gradient Descent, AdamW and the Training Loop 96

RUN IT, ADAPT IT, SHRINK IT

Chapter 12 — Inference: The KV Cache, Sampling, Temperature and Top-p 103
Chapter 13 — Fine-tuning: SFT, LoRA, RLHF and DPO, from Base Model to Assistant 115
Chapter 14 — Quantization: Storing the Same Knowledge in Fewer Bits 145

HOW MODERN MODELS ARE BUILT

Chapter 15 — The Transformer Family: Encoder, Decoder, Encoder–Decoder 161
Chapter 16 — Modern Building Blocks: RoPE, RMSNorm and SwiGLU 170
Chapter 17 — Attention Variants: MQA, GQA, MLA, Sliding Window and FlashAttention 176
Chapter 18 — Mixture of Experts: A Huge Model That Only Uses Part of Itself 185
Chapter 19 — Multimodal Models: Letting a Language Model See and Hear 199

HOW MODELS REASON, AND REAL MODELS

Chapter 20 — Reasoning Models: Test-Time Compute, Thinking Budgets and Reasoning Effort 209
Chapter 21 — Inside Modern Models: How Qwen and DeepSeek Combine These Ideas 233

INTERFACING THE MODEL WITH ITS USERS

Chapter 22 — Serving a Model: GGUF, llama.cpp and vLLM 240
Chapter 23 — Speed and Scale: Speculative Decoding, Parallelism and Hardware 256

IN PRODUCTION

Chapter 24 — Production AI: Tool Calling, MCP, RAG, Agents and Reliability 267

FAST DECISIONS WITHOUT TEXT

Chapter 25 — System 1 Decision Models: Fast, Typed, Calibrated Decisions Without Generating Text 279
Chapter 26 — Inside a Decision Model: BERT, Laya and Vega 294
Chapter 27 — Training and Trusting a Decision Model: Data, Calibration and Evaluation 313

APPENDIX AND SOURCES

Appendix A — TinyGPT by Hand: One Complete Training Step with Real Numbers 330
References and Sources 343

Introduction

What this handbook is

A guide to how large language models work: how they are built, trained, adapted, compressed, served and used. It is written for readers who want to understand *why* things are the way they are, without needing a research background. If you can follow a simple diagram and a little arithmetic, you can follow this.

It is **not** a tour of the Transformer architecture, listing each part with its formula. It is closer to a guided reconstruction. You meet each part of an LLM at the moment you need it, because the previous step left a problem unsolved.

How it teaches: Why, then How, then What

Most material shows the finished architecture: here are the parts, here is what each one does. This handbook goes the other way. It **derives** the model, one problem at a time. For every new piece, it asks the same four questions before moving on:

1. **What problem do we have?**
2. **Why is what we have so far not enough?**
3. **How does the new piece solve it?**
4. **What does it produce?** (concretely, with numbers)

The *why* comes first, and it gets the most space. The *how* and the *what* then feel like consequences, not rules to memorize.

Here is an example from Chapter 4 (attention):

Question	Answer
What problem?	The meaning of a word depends on the words around it ("it" in "The cat drank the milk because *it* was thirsty").
Why isn't the previous step enough?	An embedding is the same vector for a word in every sentence. It knows nothing about its neighbours.
How is it solved?	Each word asks a question (a query), compares it with what every other word offers (keys), and blends their information (values).
What does it produce?	A new, context-aware vector for each word. The chapter computes it with real numbers.

The whole first half of the handbook is one chain of such problems, each created by the solution before it:

```
Computers need numbers, not text           →  Tokenization              (Chapter 2)
Token IDs are meaningless addresses        →  Embeddings                (Chapter 3)
Embeddings know meaning, not context       →  Attention                 (Chapter 4)
One attention pattern is not enough        →  Multi-head attention      (Chapter 5)
Gathering information is not thinking      →  The feed-forward network  (Chapter 6)
Layers must not lose the input or blow up  →  Residuals and LayerNorm   (Chapter 7)
One block is not deep enough               →  Stacking blocks, the LM head (Chapter 8)
The random weights know nothing yet        →  Loss, gradients, AdamW    (Chapters 9 to 11)
```

From Chapter 12 on, the same method is applied to real engineering problems. Generating text recomputes everything, so we get the KV cache (12). A base model only continues text, so we fine-tune it (13). The weights are too big, so we quantize them (14). Attention costs too much on long text (17). Every token pays for the whole model, so we split the FFN into experts (18). One pass is not enough for hard problems, so models learn to reason (20). One GPU serves one user, so serving systems batch (22). Each technique is the answer to one specific, expensive problem.

A habit used throughout: fixed, learned or computed?

Whenever a new component appears, the handbook says what kind of thing it is. Almost every confusing moment in learning LLMs comes from not knowing which of these three it is:

Kind	Meaning	Examples
Fixed by the architect	Chosen before training, never trained	Embedding size, number of layers, number of heads, the GELU formula
Learned during training	Starts random, shaped by backpropagation	The embedding table, `W_Q`, `W_K`, `W_V`, `W_O`, the FFN weights
Computed on every forward pass	Built fresh from the input	Token IDs, Q, K, V, attention scores, the next-word probabilities

When you wonder "where did this number come from?", the answer is always one of these three. Chapter 3 introduces the idea, and the chapters that follow use it.

What you will find in every chapter

- **The problem first.** Each chapter opens with what was left unsolved, and why that matters.
- **An analogy, then the math.** Intuition comes before any formula, and the formula is introduced only when the intuition is solid.
- **Small worked examples.** Real numbers small enough to follow by hand. Appendix A runs one complete training step on a tiny GPT with every number visible.
- **The engineering view.** Tensor shapes, memory, cost, and how the idea shows up in real systems such as llama.cpp and vLLM.
- **A running diagram.** Chapters 2 to 8 each add one piece to a single picture ("The architecture so far"), so that by Chapter 8 you hold the whole model.

- **Common misconceptions.** These come from real questions asked while learning, not from a list of generic myths.
- **A quick reference and a pointer to the next chapter.**

How this differs from a typical textbook or course

- **Derived, not catalogued.** A textbook lists components. Here each one earns its place by solving a problem the previous one created.
- **Real questions are answered.** Questions that learners actually get stuck on have their own sections: *Why can't we just compare the raw embeddings?* (Chapter 4), *Why not just one giant block?* (Chapter 8), *Why do we cache only Keys and Values, and can't the cache be compressed?* (Chapter 12). Most books skip questions like these.
- **The order follows understanding, not the usual table of contents.** For example, the feed-forward network comes before residual connections and LayerNorm, because the need for those only becomes visible once the block has two halves. Modern attention variants come after inference and fine-tuning, because their purpose (a smaller KV cache) only makes sense once you have seen the cache. Serving comes near the end, because it applies to any model.
- **Nothing is hidden.** Every number in a worked example comes from the step before it, and Appendix A shows the whole chain, including the backward pass.
- **Theory and systems in one thread.** The same document that explains attention also explains why a 70B model needs several GPUs, and what a `Q4_K_M` file is.

How to read it

- **In order the first time.** Each chapter builds on the one before it.
- **As a reference afterwards.** Jump straight to the chapter you need. Each one opens with what it covers and why it matters.
- **Numbers are examples unless stated otherwise.** Many worked examples use small, made-up numbers to show a principle, and they say so. Figures taken from papers, model cards or documentation are listed in References.
- **The first eleven chapters follow the Why → How → What pattern most closely.** The later chapters apply it to larger, more engineering-heavy topics, so they lean more on explanation and comparison tables.

Learning with an AI assistant

These files are plain Markdown, so you can give them to an AI assistant (ChatGPT, Claude or similar) and use it as a tutor. The whole handbook is about 140,000 tokens, which is more than many assistants can hold at once, so load the Introduction plus the chapter you are on. Use the Markdown files rather than the PDF for this: they are smaller and cleaner for an AI to read. llms.txt is a one-page topic index, and AGENTS.md tells an assistant how to use the material.

A prompt to start with (attach this file and the chapter):

```
Use the attached handbook as your main source. Teach me this chapter in its
own style: Why, then How, then What. Go one idea at a time, use its examples,
and ask me a short question after each section to check I understood.
For anything newer than October 2026, or about a specific model, price or
version, search the web and tell me which parts come from the handbook and
which from the web.
```

A note on how current it is

The first fourteen chapters describe ideas that change slowly. Chapters 16 to 23 and 25 to 27 describe a fast-moving area: model names, sizes, defaults and prices in particular may be out of date by the time you read them, so treat specific figures as a snapshot and the ideas as the lasting part. The References list the sources to check.

Chapter 1 — The Big Picture: How a Large Language Model Predicts the Next Token

Introduction

Large Language Models can feel like a black box — text goes in, coherent (sometimes brilliant) text comes out, and the mechanism in between is easy to wave away as "AI magic." It isn't magic. It's a long chain of genuinely simple ideas — counting, looking things up in tables, comparing vectors, weighted averages — stacked hundreds of layers deep and trained on an enormous amount of text.

This handbook exists to take that chain apart, one link at a time, and put it back together in your head so that "how does ChatGPT actually work?" stops being a mystery and starts being something you can explain, sketch, and reason about.

The approach throughout is **intuition first, then math, then engineering**:

1. **Intuition** — a real-world analogy for the concept, before any notation.
2. **Concept** — what the piece actually does, and why it exists at all.
3. **Mathematics** — the formulas, introduced only once the intuition is solid enough that the notation feels like a natural way to write down something you already understand.
4. **Engineering perspective** — how the idea actually shows up in real model code, tensor shapes, and production systems.

Underneath that sequence is one habit, and it is what makes this handbook different from a list of components: **every new piece enters the story because the previous one left a problem unsolved.** For each one we ask what problem we have, why what we have so far isn't enough, how the new piece solves it, and what it produces. In short: *why, then how, then what.* (The Introduction explains this approach in more detail.)

Every chapter builds directly on the ones before it, so it's worth reading in order the first time through. After that, treat it as a reference and jump straight to whichever chapter you need.

By the end, you should be able to explain, from memory, how a sentence you type turns into a reply — and, further in, how that same model gets trained, shrunk down to run on a laptop, and served to thousands of users at once.

Let's start with the single idea everything else in this handbook is built on top of.

Where this handbook is going

Before we zoom into any single piece, it helps to see the whole shape of the journey:

```
Chapter 1  →  The Big Picture            (you are here)

Build a GPT
Chapter 2  →  Tokenization
Chapter 3  →  Embeddings
Chapter 4  →  Attention
Chapter 5  →  Multi-Head Attention
Chapter 6  →  The Feed-Forward Network
Chapter 7  →  The Transformer Block
Chapter 8  →  Stacking Blocks into GPT

Train it
Chapter 9  →  Training I: Learning from Text and Measuring Error
Chapter 10 →  Training II: Gradients and Backpropagation
Chapter 11 →  Training III: Gradient Descent, AdamW and the Training Loop

Run it, adapt it, shrink it
Chapter 12 →  Inference (KV Cache, Sampling)
Chapter 13 →  Fine-tuning (SFT, LoRA, RLHF, DPO)
Chapter 14 →  Quantization

How modern models are built
Chapter 15 →  The Transformer Family (BERT, GPT, T5)
Chapter 16 →  Modern Building Blocks (RoPE, RMSNorm, SwiGLU)
Chapter 17 →  Attention Variants (MQA, GQA, MLA, sliding window, sparse, hybrid)
Chapter 18 →  Mixture of Experts (MoE)
Chapter 19 →  Multimodal Models

How models reason, and real models
Chapter 20 →  Reasoning Models
Chapter 21 →  Inside Modern Models (Qwen and DeepSeek)

Interfacing the model with its users
Chapter 22 →  Serving a Model (GGUF, llama.cpp, vLLM)
Chapter 23 →  Speed and Scale (Speculative Decoding, Parallelism, Hardware)

In production
Chapter 24 →  Production AI (agents, tool routing, memory, RAG)

Fast decisions without text
Chapter 25 →  System 1 Decision Models (typed, calibrated choices in one pass)
Chapter 26 →  Inside a Decision Model (BERT, Laya, Vega)
Chapter 27 →  Training and Trusting a Decision Model (data, calibration, evaluation)

Appendix
Appendix A →  TinyGPT by Hand (one full forward and backward pass with real numbers)
References →  Papers, documentation and the study material behind the handbook
```

Every chapter builds on the one before it. Nothing here is magic — it's a long chain of fairly simple ideas, stacked on top of each other until the result *looks* like magic.

The one-sentence answer

> **A Large Language Model is, underneath everything, a very good "guess the next word" machine.**

That's it. That's the whole trick. Everything else in this handbook — attention, transformers, training, fine-tuning, quantization — exists purely to make that one guess as good as possible, as efficiently as possible.

An analogy you already know

You've used this technology before, just a much weaker version of it: **predictive text on your phone keyboard.**

You type:

```
"I'll see you at the"
```

and your phone suggests:

```
"store"   "park"   "gym"
```

Your phone isn't *thinking*. It's just asking:

> "Given everything typed so far, what word is statistically most likely to come next?"

An LLM does exactly this — except instead of looking at the last 2-3 words like your keyboard, it looks at **thousands of words of context**, and instead of a crude frequency table, it uses a very deep, very trained neural network to make the guess. Scale that trick up enough, and the guesses start looking like reasoning, writing, and conversation.

The complete pipeline (bird's-eye view)

Here is the entire journey a piece of text takes through an LLM, from the moment you hit "send" to the moment a reply starts appearing:

```
Your text
   ↓
Tokenizer            (chop text into small pieces called tokens)
   ↓
Token IDs             (turn each piece into a number)
   ↓
Embedding Lookup       (turn each number into a vector of meaning)
   ↓
Positional Information (tell the model the order of the words)
   ↓
Transformer Block × N   (the "thinking" layers — this is most of the model)
   ↓
Contextual Embedding    (a final, context-aware vector for each token)
   ↓
LM Head                 (turn that vector into a score for every possible next word)
   ↓
Logits                  (raw scores)
   ↓
Softmax                 (turn scores into probabilities)
   ↓
Sampler                 (pick one word, according to those probabilities)
   ↓
Next token
```

Then — and this is the part people often miss —

```
Next token
   ↓
Add it to the text
   ↓
Run the ENTIRE pipeline again
```

An LLM doesn't write a sentence. It writes one token, then re-reads everything (including its own new token) and writes the next one. Over and over. A whole paragraph is really just this loop running hundreds of times in a row, fast enough that it feels instant.

Every chapter after this one is really just a deep dive into **one box** in that diagram above. Tokenization is Chapter 2. Embeddings is Chapter 3. Attention (the biggest, most important box, hidden inside "Transformer Block") gets Chapters 4 and 5 all to itself. And so on.

Walking through it once, with a real sentence

Suppose the model has seen:

```
"The cat sat on the ___"
```

Roughly, here's what happens:

1. **Tokenizer** breaks this into pieces: `["The", "cat", "sat", "on", "the"]`
2. Each piece becomes a **token ID** — just an integer, like an index in a giant dictionary.

3. Each ID is looked up in an **embedding table** and turned into a vector — a long list of numbers that captures *meaning*.
4. The model adds **positional information** so it knows "cat" came second, not fifth.
5. These vectors pass through many **Transformer Blocks**, where each word gets to "look at" every other word and update its understanding based on context (this is **attention**, and it's the real star of the show).
6. After the last block, the model has one final vector representing "everything relevant about what comes next, given this whole sentence so far."
7. The **LM Head** turns that vector into a score for *every word in the vocabulary* — tens of thousands of candidates.
8. **Softmax** turns those scores into probabilities:

```
"mat"      →  61%
"floor"    →  14%
"chair"    →   9%
"roof"     →   2%
...
```

9. A **sampler** picks one — usually the highest-probability option, sometimes a slightly less likely one on purpose (more on why in Chapter 12).
10. The chosen word — `"mat"` — is appended, and the *entire process runs again* to predict the word after that.

That's the whole engine. Every "intelligent-sounding" thing an LLM does is this loop, running over and over, on top of a model that has seen an enormous amount of text during training.

Why does something this simple look intelligent?

Two reasons, and both matter:

1. Scale. A modern LLM isn't guessing based on a few rules — it has been trained on a huge fraction of the internet, and it has billions of internal parameters to encode subtle patterns of grammar, facts, reasoning style, and even code. "Guess the next word" done at this scale ends up requiring the model to implicitly learn grammar, facts, logic, and style — because all of those *help* it guess better.

2. Context. The model doesn't just look at the last word — it looks at everything in the conversation so far (up to its context limit), and it can relate *any* word to *any* other word through attention (Chapter 4). That's what lets it stay on topic, remember what you said three paragraphs ago, and follow instructions.

Common misconceptions, cleared up early

Misconception	Reality
"The LLM looks up answers in a database."	No database lookup happens at inference time. Everything is generated fresh, one token at a time, from learned parameters.
"It plans the whole sentence before writing it."	Mostly no — it predicts one token at a time. (Reasoning models, Chapter 20, add a mechanism that gets closer to planning, but the base mechanism is still one-token-at-a-time.)
"Bigger context window means it remembers everything perfectly."	It only "sees" what's inside the context window for that pass — nothing more, nothing magically remembered outside it.
"It understands language the way humans do."	It has learned extremely rich *statistical* and *structural* patterns of language — which produces understanding-*like* behavior, but the underlying mechanism is prediction, not comprehension in the human sense.

Quick reference

```
LLM = a next-token predictor, run in a loop.

Pipeline:
Text → Tokens → Token IDs → Embeddings → +Position →
Transformer Blocks → Contextual Embedding → LM Head →
Logits → Softmax → Sample → Next Token → repeat

Two things make it feel intelligent: SCALE + CONTEXT.
```

Keep this loop in mind

keep this loop in the back of your mind every time you design something that sits around an LLM call — a tool router, a memory system, a planner. All of it is ultimately feeding tokens into, and reading tokens out of, exactly this loop. Nothing your framework does changes the loop itself; it changes what gets put into the context window before the loop runs, and what gets done with the tokens that come out. That framing will make a lot of later engineering chapters (especially Chapter 24) click faster.

What's next

Step one of the pipeline above was "Tokenizer" — text gets chopped into small pieces before anything else happens. Chapter 2 goes deep on **why** we do this, how a tokenizer actually decides where to cut, and why token IDs are just arbitrary index numbers with no built-in meaning of their own.

Chapter 2 — Tokenization: Turning Text into Tokens with BPE

Introduction

In Chapter 1 we said the very first step of the pipeline is:

```
Your text  →  Tokenizer  →  Token IDs
```

That looks like a small, almost boring step compared to attention or training. It isn't. Tokenization decides *what a model is even capable of seeing*. Get an intuition for it now, and a dozen confusing things later in this handbook — why the model sometimes misspells rare words, why some languages are more expensive to run than others, why the vocabulary size shows up in formulas — will make immediate sense.

The core question

A neural network only understands numbers. It has no idea what the letters `"c"`, `"a"`, `"t"` are. So before any "understanding" can happen, we need a strict, mechanical answer to one question:

> **How do we turn text into numbers, and back again?**

That's the tokenizer's entire job. Nothing more.

Step 1: Chop the text into pieces

Suppose the sentence is:

```
"The cat slept"
```

A tokenizer first breaks this into smaller pieces called **tokens**. For a simple example, let's pretend each whole word is one token:

```
"The cat slept"
        ↓
   ["The", "cat", "slept"]
```

(We'll come back to why real tokenizers don't actually split on whole words — see "The reality: subword tokenization" below. For now, whole words make the idea easiest to see.)

Step 2: Turn each piece into a number

Every tokenizer comes with a fixed **vocabulary** — a big lookup table that was decided once, before training ever started, and never changes afterward. Suppose ours looks like this:

Word	Token ID
The	10
cat	25
slept	41

So our sentence becomes:

```
"The cat slept"
        ↓
    [10, 25, 41]
```

That's it. That's tokenization. A sentence in, a list of integers out.

The most important thing to understand: the ID means nothing

This is the part almost every beginner tutorial glosses over, and it's worth sitting with.

> **The number 25 does not mean "cat-like" in any way.** It's not a measurement of anything. It's just an arbitrary index — a row number in a table.

Think of it exactly like an **employee ID**:

```
Employee ID 25 → "works in Cat Department"?
```

No. Employee 25 is just whoever got assigned that number when they joined. The ID carries no information about the person — it's purely an address you use to go look up their actual record somewhere else.

```
"cat" → 25 → (just an address, not a meaning)
```

The *meaning* of "cat" doesn't live in the number 25 at all. It lives one step later, in the **embedding table** — which is exactly what Chapter 3 is about. For now, the only thing to lock in is:

```
Tokenization = text → arbitrary ID numbers.
Nothing about "meaning" has happened yet.
```

Why bother chopping text up at all?

You might reasonably ask: why not just feed the model raw letters, or even raw computer bytes?

You *can* — some models do exactly that (character-level or byte-level models). But it comes with a tradeoff, and it's worth seeing why most large models avoid it:

```
Character-level:
"tokenization" → t-o-k-e-n-i-z-a-t-i-o-n   (12 tokens for one word!)

Word-piece level:
"tokenization" → token + ization             (2 tokens for the same word)
```

Every extra token means:

- more steps the model has to process,
- more opportunities to lose track of long-range structure,
- more compute and more memory (remember, cost scales with the number of tokens).

So there's a real engineering incentive to make each token carry as much "chunk of meaning" as possible, while keeping the total vocabulary (the size of the lookup table) manageable. That balance is exactly what real-world tokenizers are designed to hit.

The reality: subword tokenization

Real tokenizers (the ones GPT, Claude, and similar models actually use) don't split on whole words like our toy example. They split on **subword pieces**, using an algorithm typically called **Byte-Pair Encoding (BPE)** or a close relative of it.

Here's the intuition, without the algorithm details:

```
Common words → stay as ONE token
"the", "is", "cat", "dog"

Rare or long words → get split into pieces that ARE common
"unhappiness"  →  "un" + "happi" + "ness"
"tokenization" →  "token" + "ization"
```

The tokenizer was built by scanning a huge amount of text beforehand and asking: "*What chunks of characters show up together so often that they deserve their own single token?*" Frequent chunks (whole common words, common prefixes like `"un"`, common suffixes like `"ing"`) get merged into single tokens. Rare or unfamiliar words get broken down into smaller, more common pieces — down to individual letters if truly necessary. This is also why a tokenizer never truly gets "stuck" on a word it has never seen: worst case, it falls back to spelling it out piece by piece.

```
Word never seen before?
        ↓
Break into smaller known pieces
        ↓
Worst case: fall back to individual characters
        ↓
There is always SOME valid tokenization
```

This is also a nice explanation for a real, observable quirk: LLMs are sometimes noticeably worse at character-level tasks (like counting letters in a word, or reversing a string) — because the model often never sees individual letters as separate tokens. It sees `"strawberry"` as one or two chunks, not as ten individual letters, so counting the letter `"r"` in it is genuinely harder for the model than it sounds.

Special tokens

Alongside ordinary word-piece tokens, the vocabulary also reserves a handful of **special tokens** that don't correspond to any word at all — they're control signals:

```
<bos>   → "beginning of sequence"
<eos>   → "end of sequence, stop generating"
<pad>   → "empty filler, ignore this position"
```

For example, generation keeps producing tokens in a loop (Chapter 1) until the model itself produces `<eos>`, or a maximum length is hit. We'll see more special tokens later (Chapter 13) when we look at how chat models separate "system," "user," and "assistant" turns — that separation is also implemented with special tokens, not magic.

And decoding — the reverse direction

The tokenizer's job isn't only text → IDs. It also has to go the other way, turning the model's output back into readable text:

```
Model predicts token ID 41
        ↓
Tokenizer looks up ID 41 in the vocabulary
        ↓
"slept"
        ↓
Appears on your screen
```

Same table, same lookup, just used in reverse. Nothing new to learn here — it's worth naming explicitly only so the word "tokenizer" doesn't feel like it does two unrelated jobs. It does one job (a fixed mapping between text pieces and numbers) used in both directions.

The complete picture so far

```
"The cat slept"
        │
        ▼
    Tokenizer  (chop into pieces using the vocabulary)
        │
        ▼
    [10, 25, 41]   ← just addresses, no meaning yet
        │
        ▼
   (Chapter 3 picks up here: Embeddings)
```

Common misconceptions, cleared up early

Misconception	Reality
"Token IDs encode meaning — bigger numbers mean something."	IDs are arbitrary row numbers in a lookup table. `25` and `26` could be totally unrelated words.
"Tokens are always whole words."	Real tokenizers mostly use subword pieces. Common words are one token; rare or long words get split into smaller, more frequent chunks.
"The tokenizer changes during training."	No — the vocabulary and the tokenizer's splitting rules are fixed *before* training starts, and never change afterward. Only the embedding table (Chapter 3) changes during training.
"A word the tokenizer has never seen breaks it."	It can't — worst case, an unfamiliar word gets spelled out as individual characters, which are always in the vocabulary.

Quick reference

```
Tokenization = a fixed, two-way mapping between text pieces and integer IDs.

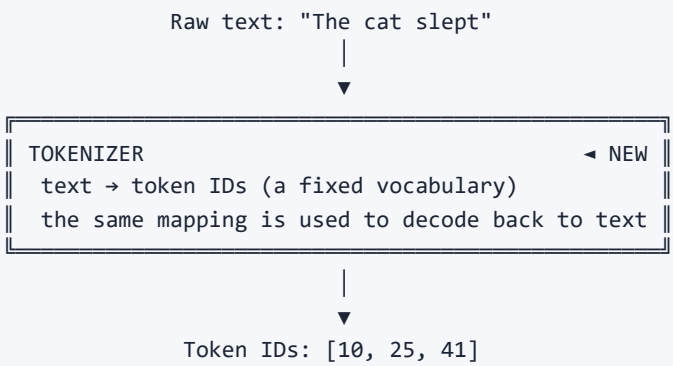
Text → Tokenizer → Token IDs   (encoding)
Token IDs → Tokenizer → Text   (decoding)

Real tokenizers split into SUBWORD pieces (via BPE or similar),
not whole words — common chunks get one token, rare words get split.

Token IDs carry NO meaning by themselves.
They are addresses. Meaning is added in the next step: Embeddings.
```

The architecture so far

From this chapter on, each chapter adds one more piece to a single diagram of the whole model. By Chapter 7 you will have a complete Transformer block, and Chapter 8 stacks it into a full GPT. Boxes drawn with double lines are the new addition in the chapter you are reading. Boxes with single lines were added in earlier chapters, shown in compact form.



Learned so far: nothing yet. The vocabulary is fixed before training starts.

What's next

We now have a list of meaningless integers: `[10, 25, 41]`. Chapter 3 answers the question this naturally raises — how does an arbitrary ID like `25` turn into something that actually captures what "cat" *means*, and how does the model ever learn that in the first place?

Chapter 3 — Embeddings: Turning Token IDs into Meaning and Position

Introduction

Chapter 2 left us with a small cliffhanger. Our sentence:

```
"The cat slept"
        ↓
    [10, 25, 41]
```

turned into a list of integers — and we were very insistent that those integers **mean nothing**. `25` is not "more cat-like" than `24`. It's just a row number in a table.

So where does actual *meaning* come from? That's this entire chapter.

The big idea: use the ID as an address, not a value

Here's the concept that unlocks everything: the model never does math directly on the token ID. Instead, it uses the ID purely as an **address** to look something else up.

```
Token ID = 25
        ↓
   "Go to row 25"
        ↓
  Read whatever is stored there
```

Think of a paper dictionary:

```
Word
  ↓
"CAT"
  ↓
Page 145
```

You don't *calculate* page 145 from the letters C-A-T. You just look it up. Embedding lookup is exactly this — no multiplication, no neural network involved in this particular step, just a table lookup.

Or think of an employee ID at a company: the ID `25` doesn't tell you anything about the person. It's an address you use to pull up their actual record — name, role, department — from a separate database. The **token embedding table** is that database, and the "record" it stores for each word is a list of numbers called a **vector**.

```
Embedding Table

Token ID     Embedding Vector
5      →     [0.2, 0.8]
10     →     [0.7, 0.1]
11     →     [0.3, 0.6]
12     →     [0.9, 0.5]
```

So:

```
Token ID 10
     ↓
"Go to row 10"
     ↓
Read the vector
     ↓
[0.7, 0.1]
```

This is exactly the same idea as a dictionary in most programming languages:

```
embedding_table = {
    5:  [0.2, 0.8],
    10: [0.7, 0.1],
    11: [0.3, 0.6],
    12: [0.9, 0.5],
}

token = 10
embedding = embedding_table[token]
```

A real model doesn't literally use a Python dictionary — it stores this table as one big matrix, with one row per vocabulary entry — but the idea is identical: **ID in, vector out, via lookup, not calculation.**

Wait — where did those numbers come from?

This is the part that feels like magic the first time you see it, and it isn't.

When training a model **starts**, this table is filled with completely random numbers:

```
Token ID     Vector
5   →   [0.12, -0.43]
10  →   [-0.87, 0.91]
11  →   [0.44, -0.18]
12  →   [-0.22, 0.73]
```

Meaningless. Random noise.

Then training happens (we'll cover *how* this actually works, in detail, in Chapters 9–11 — for now, just the shape of the idea): the model makes a prediction, the prediction is wrong by some amount, and a process called **backpropagation** nudges every number that contributed to the mistake, ever so slightly, in the

direction that would have made the prediction better. That includes the embedding vectors.

```
[-0.87, 0.91]
      ↓  (tiny nudge)
[-0.84, 0.95]
      ↓  (tiny nudge, next example)
[-0.80, 0.99]
      ↓
      ⋮
   (millions of nudges later)
      ↓
[1.92, -0.63, ..., 0.17]
```

One crucial detail: **the token ID itself never changes.** `"cat"` is always `10`. What changes, slowly, over millions of training examples, is *only* the content of row 10 in the table. By the end of training, that row has settled into a vector that genuinely captures something useful about how the word "cat" behaves in language.

```
Sentence: "The cat slept"
        │
        ▼
   Tokenizer
        │
        ▼
  [5, 10, 12]
        │
        ▼
Embedding Lookup
        │
   ┌────┼────┐
   ▼    ▼    ▼
 Row5  Row10 Row12
  │      │     │
[0.2,  [0.7, [0.9,
 0.8]   0.1]  0.5]
```

Nothing mysterious happened. The token ID was simply used as an address into a table that *itself* got trained.

What does a "dimension" actually mean?

In our tiny examples the vectors only have 2 numbers, e.g. `[0.7, 0.1]`. A real model uses vectors with hundreds or thousands of numbers — this size is called the **embedding dimension**, and it's a number the model's *architect* chooses before training even begins (not something the model learns on its own).

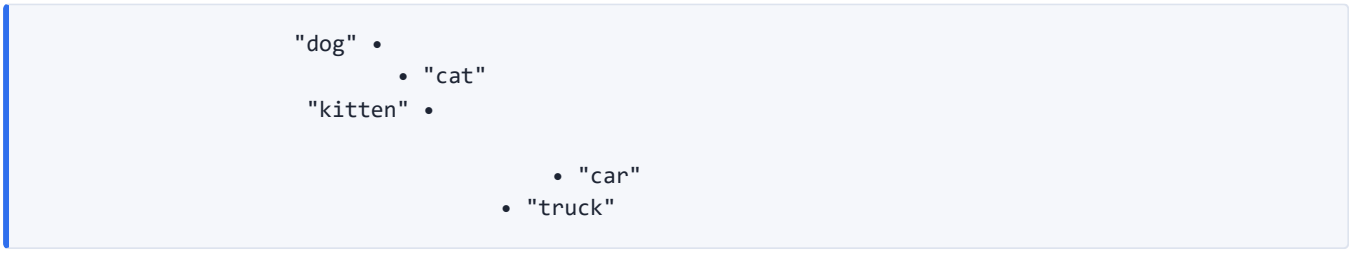
```
Architect decides (fixed before training):
  Vocabulary size    — how many possible tokens exist
  Embedding dimension — how many numbers per vector (e.g. 768, 4096, ...)

Training learns (changes during training):
  The actual numbers inside the embedding table
```

It's tempting to hope that each individual number in the vector has a clean label — like "number 3 measures fuzziness" or "number 7 measures animal-ness." It doesn't work that cleanly in practice. No single dimension has a tidy, human-readable meaning on its own. What *does* end up meaningful is the vector **as a whole**, and especially its position relative to other vectors — which brings us to the next point.

Why does this actually capture "meaning"?

Here's the payoff. Because training pushes these vectors around based on how words are *used*, words that get used in similar ways end up with similar vectors — which means they land **close together** in this abstract vector space.



`"cat"` and `"dog"` end up near each other, because they tend to appear in similar contexts ("the ___ ran", "feed the ___", "my ___ is sleeping"). `"car"` and `"truck"` cluster somewhere else entirely. Nobody programmed this clustering by hand — it's a side effect of training on the simple goal of predicting the next word well. Similar usage patterns naturally pull similar words toward similar vectors.

This is also why the classic embedding party trick works: if you take the vector for `"king"`, subtract the vector for `"man"`, and add the vector for `"woman"`, you land very close to the vector for `"queen"` — the vector space ends up organized enough that relationships like "male → female" or "capital city → country" show up as consistent directions you can walk in.

```
king − man + woman  ≈  queen
```

You won't need to compute this yourself anywhere in this handbook — it's included here purely because it's the clearest possible proof that these vectors capture real structure, not just arbitrary numbers.

One more thing embeddings forget: order

Here's a problem hiding in plain sight. Suppose we rearrange a sentence:

```
"The cat sat"       →   "sat cat The"
```

The embedding for `"cat"` is looked up from the exact same row of the exact same table, no matter where `"cat"` appears in the sentence. **The embedding for a word never changes based on its position.** That means, so far, our model would see these two sentences as containing exactly the same information:

```
"Dog bites man."
"Man bites dog."
```

Same words, same embeddings, wildly different meaning. Word order clearly matters, but nothing we've built so far encodes it. That's a real gap — not a detail we can skip.

The fix: a second table, just for positions

The solution is refreshingly simple: alongside the token embedding table, the model learns a completely separate **position embedding table** — one vector per position number (position 0, position 1, position 2, ...), learned the exact same way (random at first, nudged during training).

```
Position   Position Embedding
   0    →  [0.1, 0.0, 0.2, 0.1]
   1    →  [0.0, 0.3, 0.1, 0.0]
   2    →  [0.2, 0.1, 0.0, 0.4]
```

Then, for every token, we simply **add** its word embedding and its position embedding together:

```
"The" word embedding        [0.2, 0.6, 0.4, 0.1]
"The" is at position 0  +   [0.1, 0.0, 0.2, 0.1]
                        ____________________________

Combined vector              [0.3, 0.6, 0.6, 0.2]
```

This combined vector — meaning **plus** position — is what actually enters the Transformer blocks (Chapter 4 onward), never the raw word embedding alone. Now the same word in two different positions produces two different final vectors, and order is no longer invisible to the model.

> **Common question:** doesn't attention (next chapter) already figure out relationships between words, including order? Attention can learn relationships like *"this word refers to that word,"* but without positional information it has no built-in sense of *before* or *after* at all — it would treat a sentence more like an unordered bag of words than a sequence. Position embeddings are what give it that missing sense of order to work with in the first place.

The full picture, updated

```
Sentence
   │
   ▼
Tokenizer  →  Token IDs
   │
   ▼
Token Embedding Lookup  +  Position Embedding Lookup
   │
   ▼
Combined Vectors  (meaning + position)
   │
   ▼
   (Chapter 4 picks up here: Attention)
```

And our running list of what's fixed-by-design versus learned-through-training now looks like this:

```
Fixed by the architect (chosen once, never trained)
──────────────────────────────────────────────────
Vocabulary size
Embedding dimension

Learned during training
──────────────────────────────────────────────────
Token Embedding Table     — meaning of each word
Position Embedding Table  — meaning of each position
```

Keep this "architect-defined vs. learned vs. computed-on-the-fly" split in the back of your mind — it's a useful lens for almost every new component in the rest of this handbook, including attention, the feed-forward network, and even much more advanced things like Mixture of Experts.

Common misconceptions, cleared up early

Misconception	Reality
"Embeddings are hand-designed to represent grammar or meaning."	They start as random numbers and are shaped entirely by training — nobody designs their content directly.
"One number in the vector represents one specific concept."	No single dimension has a clean, human-readable meaning. Meaning lives in the whole vector, and in its relationship to other vectors.
"Embeddings already encode word order."	They don't. Order is added separately, via a second, position-based table, and the two are combined by simple addition.
"A bigger embedding dimension always means a smarter model."	It gives the model more room to represent nuance, but it's an architectural choice with real memory and compute cost — not a free win.

Quick reference

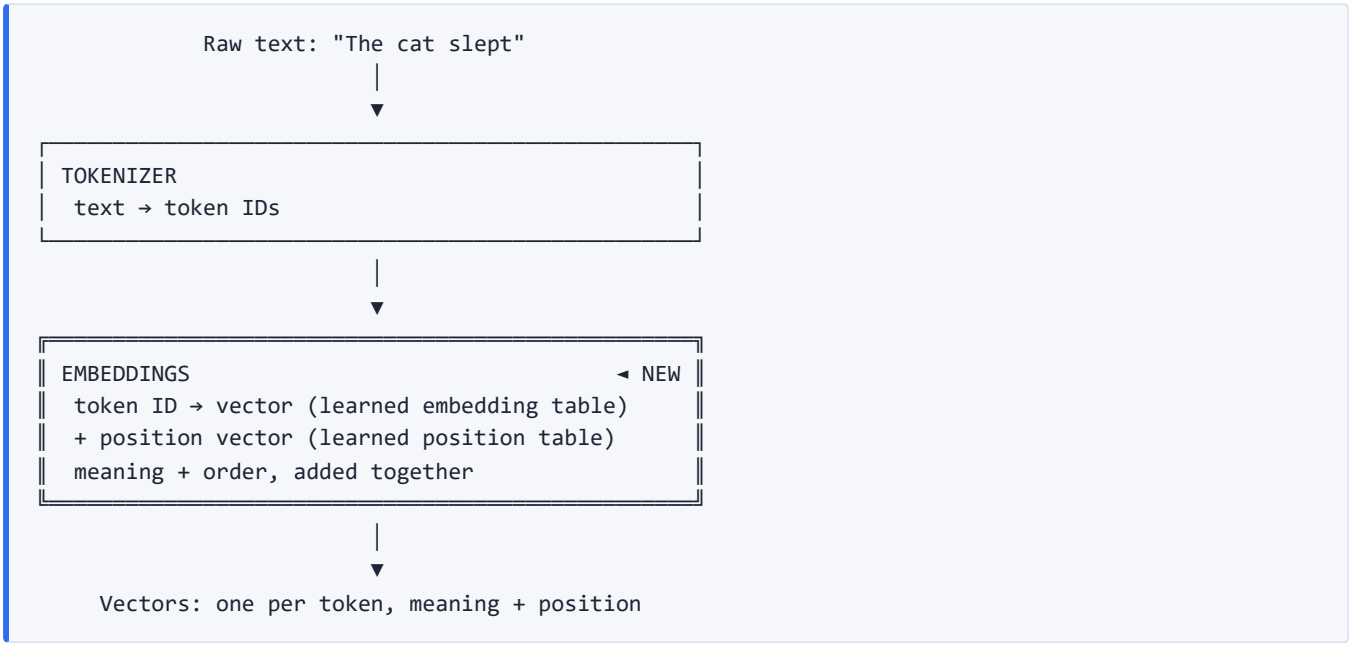
```
Token ID  →  Embedding Table lookup  →  Word meaning vector
Position  →  Position Table lookup   →  Position vector

Final input to the Transformer = Word vector + Position vector

Embedding tables start random, are shaped entirely by training.
Similar-meaning words end up with similar (nearby) vectors.
Embedding dimension is chosen by the architect; the vectors inside
the table are what training actually learns.
```

The architecture so far

Here is the diagram so far. The double-lined box is this chapter's addition.



Learned so far: the token embedding table, and the position embedding table.

What's next

We now have, for every token, a single vector that encodes both *what it means* and *where it sits* in the sentence. Chapter 4 finally asks the question this whole handbook has been building toward: how does a word figure out which *other* words in the sentence it should pay attention to?

Chapter 4 — Attention: Queries, Keys, Values and the Causal Mask

Introduction

If this chapter feels slower and more deliberate than the last three, that's on purpose. Attention is the single most important idea in this handbook, and it's also the idea most tutorials rush through or explain backwards — introducing the letters Q, K, and V before ever explaining why they need to exist. We're not going to do that. We're going to build up to them one problem at a time, and then spend real time on the part most explanations skate past entirely: the learned weight matrices that make the whole thing trainable in the first place. Those matrices are, quite literally, where the model's intelligence about *relationships between words* actually lives — so they deserve more than a passing mention.

Part 1 — Why do we need attention at all?

Step 1: What your brain already does

Before any math, notice you already perform attention constantly, without thinking about it. Read this sentence:

> **The cat drank the milk because it was thirsty.**

When you hit the word **"it,"** your brain automatically asks: *who is "it"?* It quietly scans backward over the earlier words:

```
The       → No
cat       → Maybe
drank     → No
the       → No
milk      → Maybe
because   → No
```

and settles on:

```
cat   → very likely
milk  → not very likely
```

You just performed attention. No math, no vectors, no Q/K/V — just deciding *which previous words deserve my attention right now.* That's the entire idea. Everything below is teaching a computer, which only understands numbers, how to do the same thing.

Step 2: Words start out isolated

After tokenization and embedding (Chapters 2 and 3), suppose our sentence is:

```
"The cat drank milk"
        ↓
"The"   → [0.2, 0.8]
"cat"   → [0.7, 0.1]
"drank" → [0.4, 0.5]
"milk"  → [0.8, 0.2]
```

Here's the problem: each of these vectors only knows about *itself*.

```
The    → knows only "The"
cat    → knows only "cat"
drank  → knows only "drank"
milk   → knows only "milk"
```

The vector for **"cat"** has no idea **"milk"** even exists in the same sentence. They're isolated islands. But the meaning of `"drank"` clearly depends on knowing that a cat is doing the drinking, and milk is what's being drunk. Without some way for words to exchange information, the model can never combine them.

```
Before attention:            After attention:

cat    (isolated)            cat  ──────┐
drank  (isolated)      →      drank ←─────┼── exchange information
milk   (isolated)            milk ──────┘
```

Attention is simply a mechanism that lets words communicate with each other. That's the whole motivation, before a single formula appears.

Step 3: Should every word listen equally?

Now a second, sharper problem appears. Take a longer sentence:

```
"The cat drank milk because it was thirsty."
```

When updating the word `"it"`, should it listen equally to *every* earlier word — `"The"`, `"cat"`, `"drank"`, `"milk"`, `"because"`? Obviously not. Some words are far more relevant to resolving `"it"` than others. So the real problem attention has to solve isn't just "*let words talk to each other,*" it's:

```
Listen MORE to important words,
and listen LESS to irrelevant ones.
```

And how do we decide what counts as "important"? This is where we finally need a precise, computable definition of *relevance* — and the simplest tool for measuring how related two vectors are is a **dot product**, a similarity score between two vectors that's larger when they point in similar directions.

```
Attention = finding which word-vectors are most relevant
            to the current word, where relevance is
            measured by similarity.
```

That's the concept. Suppose the model produces these raw relevance scores for `"it"`:

Previous word	Score
The	0.1
cat	9.8
drank	1.2
milk	0.6

These aren't probabilities yet — just relative signals of relevance. Notice something important, though: **we don't replace `"it"` with `"cat"`.** Instead, we build a brand-new representation of `"it"` that *incorporates* information from the words it found relevant:

```
Old representation of "it"
        ↓
     Attention
        ↓
New representation = "it" + information gathered from "cat", etc.
```

> **This is probably the single most important sentence in this chapter: attention enriches a word's representation. It doesn't replace it.**

Part 2 — Why can't we just compare the raw embeddings?

Here's a completely reasonable next question: if attention is just "compare vectors for similarity," why not compare the embeddings we already have, directly?

The problem: if I tell you *"compare every word with every other word,"* you immediately have to ask — **compare what, exactly?** The whole embedding? Part of it? Something else? And once you decide a word is relevant, what should get copied over — the whole embedding again, unchanged?

This is precisely the problem that **Q, K, and V** exist to solve: they give the model a way to define *what* to compare, separately from *what information to pass along* once a match is found.

Part 3 — Introducing Q, K, and V

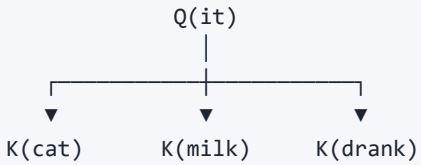
Here's the intuition, using a library. Imagine searching for a book. You don't compare the entire contents of every book against every other book. Each book instead has three separate things:

```
Book
 ↓
Title            (how you search for it)
 ↓
Shelf number     (where it's matched/found)
 ↓
Contents         (the actual information you wanted)
```

You search using the **title**, not the full contents. Once you find the right book, *then* you read the contents. Attention does exactly this. Every word creates three different versions of its own embedding:

```
Embedding
   ↓
Q = "What am I looking for?"       (Query)
K = "What can I be matched on?"    (Key)
V = "What information do I carry?" (Value)
```

Think of a teacher: sometimes they're asking a question, sometimes they're the one being asked, sometimes they're handing over the actual material. Same person, three different roles depending on the moment. Q, K, and V are the same idea, applied to a word — and when processing `"it"`, the model creates `Q(it)` and compares it against the **Keys** of every earlier word:



Whichever Key scores highest against `Q(it)` "wins" the most attention — and then the model reads that word's **Value**, not its Key. **You match using K. You copy from V.**

Part 4 — Where do Q, K, and V actually come from?

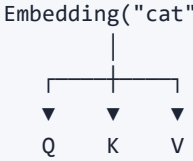
We now know *what job* Q, K, and V do. The natural next question is: how does a word actually get one of each? Are they just three more random numbers assigned per token, separately trained, sitting alongside the embedding?

They can't just be random — they have to come from the embedding

Imagine, for a moment, that Q, K, and V were completely independent of the embedding — say, three more lookup tables, keyed by token ID, filled with their own numbers that have nothing to do with the embedding table from Chapter 3.

That can't be right, and it's worth being precise about *why*. The entire point of attention is to compare words based on **meaning** — the exact meaning that Chapter 3 spent a whole chapter carefully building into the embedding through training. If Q, K, and V were disconnected from that embedding, attention would end up comparing something arbitrary, with no guaranteed relationship to what the word actually means. Whatever `"cat"` ends up looking like as a Query, a Key, and a Value, all three **must be built out of `"cat"`'s embedding** — otherwise there would be nothing tying the comparison back to meaning at all.

So: Q, K, and V are not separately stored, and they are not independently random. They are **three different representations, derived from the one embedding we already have** — three different lenses applied to the same vector.



But "derived from the embedding" raises a harder question

Fine — Q, K, and V come *from* the embedding. But derived **how**, exactly? What decides how to turn "the meaning of cat" into "what cat is looking for" (Q), versus "how cat can be found" (K), versus "what cat has to offer" (V)?

This is the real crux of the idea, and it's worth sitting with, because there's no hand-writable answer to it. Nobody sat down and wrote a formula that says *"to compute a Query, take the embedding and do X to it."* We don't actually know, in advance, what mathematical operation correctly extracts "what this word is looking for" out of a meaning vector — it isn't obvious, and it isn't fixed, because the right answer depends on the language, the sentence, and the specific relationship being detected.

This is exactly the moment where **training** enters the picture, and it's really the whole idea behind attention being a *learned* mechanism instead of a hand-engineered rule:

> Instead of hand-designing the rule that turns an embedding into a Query, we let the model **learn** that rule — by making the rule itself a grid of adjustable numbers (a matrix), and letting training discover which numbers make the whole system predict better.

That adjustable rule is `W_Q`. The same logic applies separately to `W_K` (the rule for "how should I be found") and `W_V` (the rule for "what do I hand over"). Three different jobs, so three different learned rules — three different matrices.

An important correction: it's the *rule*, not the vectors, that starts random and gets trained

It's tempting — and very natural — to picture Q, K, and V themselves as "random values that get trained, then fine-tuned." That framing is close, but mixes up two different things, so it's worth separating them precisely:

```
W_Q, W_K, W_V   →  the RULES.
                   These start random. Training reshapes THEM.

Q, K, V         →  the OUTPUTS of applying those rules to an embedding.
                   Freshly computed, every single time, for every
                   single token. Never stored. Never trained directly.
```

Once training is finished, computing `Q` for a brand-new sentence the model has never seen isn't a fuzzy, learned guess — it's a fixed, fast, completely deterministic multiplication: `embedding × (the now-trained) W_Q`. Nothing random happens at that point. What *was* random, and what training actually reshaped, is the content of `W_Q` itself — the numbers inside the rule, not the numbers that come out of applying it.

The mechanics: how W_Q actually turns an embedding into Q

Let's make "apply a learned rule" completely concrete — this is the moment the embedding's information genuinely gets **projected into three useful parts**. Suppose a word's embedding is a 2-number vector:

```
x = [2, 1]
```

And suppose `W_Q` is a small matrix — a grid of learned numbers, one column per output number we want:

```
        col→Q₀   col→Q₁
W_Q  =  [ 1        4 ]
        [ 3        0 ]
```

To compute `Q`, we take the dot product of `x` with *each column* of `W_Q`:

```
Q₀ = (2 × 1) + (1 × 3) = 2 + 3 = 5
Q₁ = (2 × 4) + (1 × 0) = 8 + 0 = 8

Q = [5, 8]
```

Now apply two *different* rules — `W_K` and `W_V` — to that exact same embedding:

```
W_K = [ 0   1 ]      W_V = [ 3   2 ]
      [ 2   1 ]            [ 1   2 ]

K₀ = (2×0)+(1×2) = 2      V₀ = (2×3)+(1×1) = 7
K₁ = (2×1)+(1×1) = 3      V₁ = (2×2)+(1×2) = 6

K = [2, 3]                V = [7, 6]
```

Look at what just happened: **one embedding, `[2, 1]`, produced three genuinely different vectors** — `Q = [5, 8]`, `K = [2, 3]`, `V = [7, 6]` — purely because three different rules were applied to it. Nothing about the word's underlying meaning changed; that one vector simply got projected through three different lenses, splitting and re-emphasizing its information three different ways. In a real model, embeddings have hundreds or thousands of numbers, and `W_Q`, `W_K`, `W_V` are correspondingly large grids — but this exact same operation, dot-product-with-each-column, is all that's happening, at any scale.

(Quick note if matrix multiplication is rusty: multiplying a vector by a matrix just means "compute one dot product per output column, using the same input vector every time." That's the entire operation, repeated as many times as there are output numbers.)

Why three separate rules, instead of one, or none?

Imagine someone asks you to build a recommendation engine from scratch, starting only from a customer ID.

First question: "*What should a customer ID become?*" You'd probably invent something like a **profile vector** — a list of numbers summarizing that customer. That's exactly an embedding.

Next question: "*Which customers are similar to each other?*" Comparing raw profile vectors directly is crude — you'd probably invent a small transformation first, one that emphasizes the specific traits relevant to *similarity*, and apply it to every customer before comparing them. That transformation is exactly what `W_Q` and `W_K` are — two separate rules, because "what I'm looking for" and "how I present myself to be found" are genuinely different jobs, even though both are ultimately in service of matching.

Final question: "*Once you know two customers are similar, what information should actually flow between them?*" That's a different question again — so you'd invent yet another transformation, specialized for *that* job. That's `W_V`.

The Transformer wasn't designed all at once as a monolithic idea — it's the accumulation of exactly these kinds of small, individually-motivated design decisions, each solving one specific sub-problem, each therefore earning its own learned rule.

There's nothing inherently "question-like" about Q

One more correction worth making explicit: it's tempting to think of Q, K, V as manually assigned roles — as if the architecture *tells* a vector "you are now a question." That's not quite right. There is nothing intrinsically question-shaped about the numbers sitting inside a Q vector. The model *learns to use it that way*, purely because doing so improves its predictions during training. The labels Q, K, V describe the *role a vector plays in the computation*, not some inherent property baked into the numbers — that role emerges entirely from how `W_Q` gets shaped by training, not from anything the architecture assigns up front.

So how does training actually shape W_Q into something useful?

This is the last piece of the puzzle: what does it actually mean for the *rule itself* to "figure out relevance," rather than some vector just happening to land on the right answer?

Every matrix starts filled with small random numbers — `W_Q` included. It has no idea, yet, what makes a good Query. After the model sees one training example and makes a prediction, a process called backpropagation (the full mechanics are in Chapters 10 and 11) nudges every number inside every matrix that contributed to the mistake, very slightly, in whichever direction would have made the prediction a little better:

```
W_Q[0][0]:  0.83  →  0.84  →  0.85  →  0.86  → ...
            (after training example 1, 2, 3, 4...)
```

A single nudge might change a value by something like `0.0000003` — utterly negligible on its own. But after billions of tokens and trillions of such nudges, those microscopic adjustments accumulate into a matrix that reliably does its job: `W_Q` and `W_K` end up shaped so that, whenever they're applied, *genuinely related* words score highly against each other — and `W_V` ends up shaped so that the information it hands over is actually useful once found.

So, to directly answer the question this whole section opened with — *how does the model figure out relevance and relationships between vectors?* — the honest answer is: it isn't computed by any single clever formula. It's **discovered**, gradually, by reshaping `W_Q`, `W_K`, and `W_V` across an enormous number of examples, until the same simple operation — "multiply the embedding by this matrix" — reliably produces vectors that behave the way attention needs them to. In a very real sense, *this is what training a Transformer means*: not memorizing facts, but shaping these matrices until relevance and relationship between words falls naturally out of ordinary multiplication.

A more useful mental picture than "the model is learning facts" is: **the model is learning a geometric space.** Recall from Chapter 3 that similar words end up as nearby vectors after training. The exact same thing happens to Q-space and K-space — training organizes them so that words which *should* attend to each other end up geometrically close once projected through `W_Q` and `W_K`, even if they weren't especially close as raw embeddings.

Which parts of attention are actually learnable?

It's worth being explicit about this, because it's easy to lose track once formulas start appearing. Here is the attention computation, with every step labeled as either a learned parameter or pure, fixed mathematics:

```
Embedding                       (learned — Chapter 3)
     │
     ▼
Matrix Multiply (W_Q)           ← LEARNED
     │
     ▼
Matrix Multiply (W_K)           ← LEARNED
     │
     ▼
Matrix Multiply (W_V)           ← LEARNED
     │
     ▼
Dot Product (Q · K)             — pure math, nothing to learn
     │
     ▼
Scale (÷ √dₖ)                   — pure math, nothing to learn
     │
     ▼
Causal Mask                     — pure math, nothing to learn
     │
     ▼
Softmax                         — pure math, nothing to learn
     │
     ▼
Weighted Sum (with V)           — pure math, nothing to learn
```

Notice how few boxes are actually learnable. `W_Q`, `W_K`, and `W_V` are the *entire* set of trainable knobs inside one attention head — every other step is deterministic arithmetic that has no numbers of its own to adjust. This is exactly why these three matrices carry so much weight (pun intended): when people say "attention is learned," this — three matrices, and nothing else in the mechanism — is literally the whole truth of what that means.

Part 5 — A full worked example, start to finish

Let's now see one complete attention computation, using real (if small) numbers throughout. We're processing the sentence:

```
"The animal didn't cross the road because it was tired."
                                          ↑
                                    current word
```

We're standing at the word `"it"`, with everything before it already processed. Suppose that, after multiplying each word's embedding through `W_Q`, `W_K`, and `W_V` (exactly the operation we just walked through above), we get:

Word	Q	K	V
animal	[3,1]	[2,1]	[9,2]
road	[2,3]	[0,2]	[4,8]
because	[1,2]	[2,2]	[2,5]
it	**[2,1]**	[1,1]	[3,3]

We only need `Q(it) = [2,1]` — that's the question being asked right now. We compare it against every earlier Key using a dot product:

```
Q(it)·K(animal)   = 2×2 + 1×1 = 5
Q(it)·K(road)     = 2×0 + 1×2 = 2
Q(it)·K(because)  = 2×2 + 1×2 = 6
```

Word	Score
animal	5
road	2
because	6

Wait — "because" scored highest. Does the model think "it" means "because"?

No — and this is an important nuance. An attention score is never saying *"this is the correct referent."* It's saying *"this word has useful information for updating the current word."* "Because" might score highly because it signals a causal relationship that's useful context, even though "animal" is the actual thing being referred to. Attention is richer, and messier, than simple coreference resolution — multiple words can legitimately be useful for different reasons at once.

Turning scores into weights: Softmax

Raw scores like `5`, `2`, `6` aren't directly usable — we want them to behave like percentages that add up to 100%. **Softmax** does exactly this conversion, turning any list of numbers into positive weights that sum to 1, while keeping larger scores proportionally larger. Suppose it gives us:

Word	Weight
animal	0.26
road	0.01
because	0.73

In plain terms: "*When updating the word 'it', listen 73% to 'because', 26% to 'animal', and basically ignore 'road'.*"

Building the new representation: a weighted blend of Values

Now — and only now — do the **Value** vectors get used. We blend them together using the softmax weights as a recipe:

```
New representation of "it"
   =  0.26 × V(animal)  +  0.01 × V(road)  +  0.73 × V(because)
```

Computing each position:

```
First number:  0.26×9 + 0.01×4 + 0.73×2 = 2.34 + 0.04 + 1.46 = 3.84
Second number: 0.26×2 + 0.01×8 + 0.73×5 = 0.52 + 0.08 + 3.65 = 4.25
```

```
New representation of "it" = [3.84, 4.25]
```

We started with the word `"it"` on its own, meaning nothing beyond itself. We end with a new vector that has genuinely absorbed information from `"animal"`, `"road"`, and `"because"`, weighted by how relevant each one was judged to be. **This is the output of one attention head** — the core operation of the entire Transformer, running once.

(A quick honest caveat: these numbers were invented purely to show the mechanics clearly. In a real trained model, `W_Q`, `W_K`, and `W_V` would very likely make "animal" score higher than "because" for this sentence — but the arithmetic works identically at any scale, and with real, trained weights instead of made-up ones.)

Part 6 — Keeping the scores well-behaved

There's a subtle numerical problem hiding in the process above. A dot product sums one multiplication per dimension — and real models don't use 2-dimensional vectors, they use hundreds. Summing more numbers tends to produce bigger totals:

```
dk = 2    →  dot products around single digits     (fine)
dk = 128  →  dot products can reach 100, 150, 180+   (a problem)
```

Why is a big number a problem? Because Softmax uses the exponential function, `e^x`, which explodes extremely fast:

Input	e^x
1	2.7
5	148
10	22,026
20	~485 million

If Softmax receives scores like `[180, 175, 172]`, the gap between them gets wildly exaggerated, and the result collapses into something like `[0.9999999, 0.0000001, 0.0]` — the model ends up ignoring every word except one, almost at random, simply because of arithmetic scale rather than genuine relevance. That's not what we want, especially early in training.

The fix: divide every score by $\sqrt{d_k}$ (the square root of the vector's dimension) before applying softmax. This keeps the scores in a sane range no matter how large the vectors get.

Putting it all together, this whole chapter compresses into one formula:

$$\text{Attention}(Q, K, V) = \text{softmax}\left((Q \cdot K^T) / \sqrt{d_k} \right) \cdot V$$

Every symbol in that formula now has a story behind it: $Q \cdot K^T$ is "compare my question against every key," $/ \sqrt{d_k}$ is "keep the scores numerically sane," $\text{softmax}(...)$ is "turn scores into a percentage recipe," and $\cdot V$ is "blend the actual information using that recipe."

Part 7 — The one rule GPT adds: no peeking at the future

There's one more constraint, specific to how models like GPT generate text one word at a time: **a word is only allowed to attend to itself and the words before it — never words that come later.**

Think of a school exam: if the answer key were accidentally printed on the question paper, every student would score perfectly without learning anything. Letting a model "attend to" a future word it's supposed to be predicting is exactly that mistake.

So GPT builds a **causal mask** — a simple allow/deny grid:

```
          The    cat    sat
The        ✓      ✗      ✗
cat        ✓      ✓      ✗
sat        ✓      ✓      ✓
```

Notice the triangle shape — this lower-triangular pattern shows up in nearly every GPT-style implementation, and it's one of the easiest ways to recognize decoder-only code at a glance.

How it's actually implemented: before softmax runs, every forbidden (future) position in the score grid gets replaced with negative infinity:

```
          The    cat    sat
The        3     -∞     -∞
cat        1      4     -∞
sat        6      3      2
```

Why negative infinity specifically? Because softmax involves `e^x`, and `e^(-∞) = 0`. So after softmax:

```
          The    cat    sat
The        1      0      0
cat      0.05   0.95     0
sat      0.94   0.05   0.02
```

Forbidden positions receive *exactly* zero attention — they don't fade out, they vanish completely. Nothing about the softmax formula itself changed; we just quietly rigged the input so the impossible positions could never win any share of attention.

The complete picture

```
Embedding (meaning + position)
        │
        ▼
   x · W_Q  →  Q        ┐
   x · W_K  →  K        ├── the ONLY learnable steps in this whole diagram
   x · W_V  →  V        ┘
        │
        ▼
   Q · Kᵀ  for every pair of positions
        │
        ▼
   Apply causal mask   (future positions → -∞)
        │
        ▼
   Divide by √dₖ
        │
        ▼
   Softmax              (scores → percentages)
        │
        ▼
   Weighted sum of V     (blend information)
        │
        ▼
   New, context-aware representation of the word
```

Common misconceptions, cleared up early

Misconception	Reality
"A high attention score means 'this is the correct referent.'"	It means "this word carries useful information for updating the current word" — which is broader and messier than pronoun resolution.
"Q, K, and V are three unrelated things, manually assigned by the architecture."	They're three different learned *projections* of the exact same embedding — the "roles" are only meaningful because training makes them useful, not because they're inherently different in kind.
"We compare Q against V."	We compare Q against K to decide *how much* to listen, then read from V to decide *what* we actually receive.
"The model just creates as many random representations as possible and magically tunes them."	It creates a *specific, fixed set* of representations, each assigned a distinct job by the architecture (embedding = meaning, Q = what I seek, K = how I'm found, V = what I offer) — training fills in the numbers, it doesn't invent the jobs.
"Q, K, and V start random and get trained, then fine-tuned."	Close, but it's `W_Q`, `W_K`, and `W_V` — the *rules* — that start random and get trained. Q, K, and V themselves are freshly *computed* from the embedding every time, using whatever the current (trained) rule is. They are never stored, and never trained directly.
"W_Q, W_K, and W_V are some special, unique kind of object."	They're plain matrices — the same *kind* of learnable object as the embedding table (Chapter 3) or the feed-forward weights (Chapter 6), just used for a different purpose.
"Scaling by $\sqrt{d_k}$ changes what the model can learn."	It doesn't change *what* the model can represent — it only keeps the numbers well-behaved so training doesn't collapse into near-random guessing early on.
"Masking is a special separate mechanism."	It's just a small edit to the scores (forbidden positions → $-\infty$) before the exact same softmax step runs.

Quick reference

```
Attention(Q, K, V) = softmax( (Q·Kᵀ) / √dₖ  [+ causal mask] ) · V

Q = x · W_Q   "what am I looking for?"
K = x · W_K   "what can I be matched on?"
V = x · W_V   "what information do I carry?"

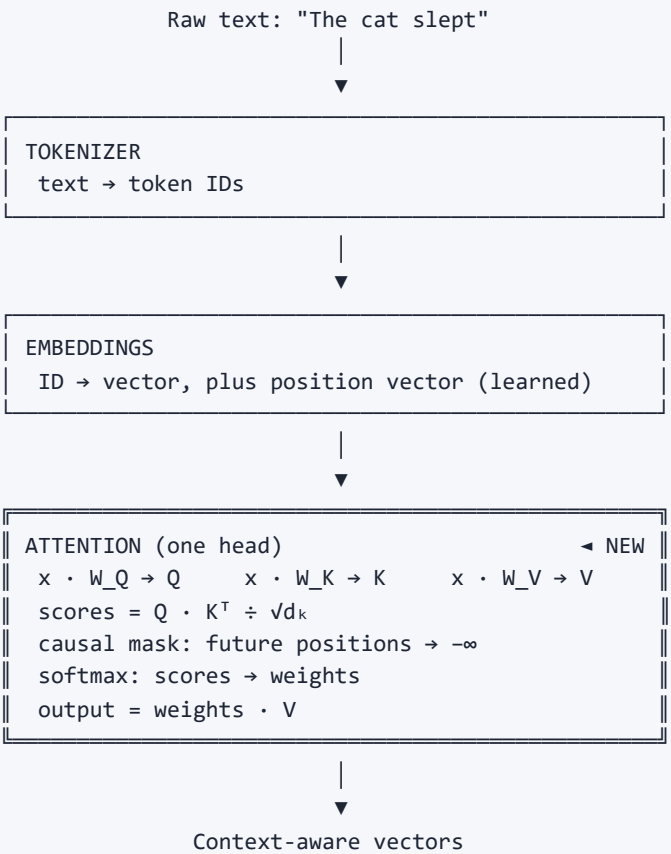
W_Q, W_K, W_V  =  the RULES.        Start random. TRAINED.
Q, K, V        =  the OUTPUTS.      Computed fresh every time. NOT trained directly.

W_Q, W_K, W_V are LEARNED MATRICES — the only trainable parts
of the whole attention computation. Everything else (dot product,
scaling, softmax, weighted sum) is fixed math with nothing to learn.

Match using K.  Copy from V.
Causal mask: future positions get score = -∞, so softmax gives them 0.
Divide by √dₖ before softmax, so scores don't blow up as dimension grows.
```

The architecture so far

The diagram so far. The double-lined box is this chapter's addition.



Learned so far: embedding table, position table, and W_Q, W_K, W_V.

What's next

We just walked through **one** attention computation — one "search" happening once, using one set of `W_Q`, `W_K`, `W_V` matrices. Real models run many of these searches in parallel, each with its *own* separate set of Q/K/V matrices, free to specialize in a different kind of relationship (grammar, meaning, reference...). Chapter 5 covers exactly that: **Multi-Head Attention** — and because we've now built single-head attention this thoroughly, multi-head will turn out to be a surprisingly small step.

Chapter 5 — Multi-Head Attention: Many Attention Patterns at Once

Introduction

Chapter 4 built one complete attention computation, end to end: one set of `W_Q`, `W_K`, `W_V`, one pass of compare-scale-mask-softmax-blend, one new representation per word. That's genuinely the hard part. This chapter takes that fully-understood mechanism and simply **runs it several times in parallel**, each copy free to notice a different kind of relationship. If Chapter 4 clicked, this chapter will feel like a natural, almost easy extension — so we'll move a little faster, but without skipping the "why."

Part 1 — Why isn't one attention head enough?

Go back to our running sentence:

```
"The cat drank the milk because it was thirsty."
```

A single attention head has to answer one comparison question at a time — whatever question its `W_Q` and `W_K` happen to be shaped to ask. But a sentence like this actually needs several *different* questions answered simultaneously to be fully understood:

```
"Who is the subject?"        →  cat
"What action happened?"      →  drank
"Why did it happen?"         →  because, thirsty
```

One head, with one fixed pair of "what to search for" and "what to be found by" rules, can't easily chase all of these at once — it's built to specialize in one kind of relationship, not several unrelated ones. So the natural fix is almost embarrassingly simple: **run several attention heads side by side**, each with its own separate rules, and let each one focus on whatever relationship it finds most useful.

```
Sentence
   │
   ▼
Head 1 (say, grammar-ish relationships)
Head 2 (say, meaning-ish relationships)
Head 3 (say, pronoun-ish relationships)
Head 4 (say, cause-and-effect-ish relationships)
   │
   ▼
Combine everything
```

That's Multi-Head Attention: not one smarter attention mechanism, but several *ordinary* attention mechanisms — each exactly like the one you just learned in Chapter 4 — running in parallel and then merged.

(A quick honest flag before we go further: the "Head 1 does grammar, Head 2 does pronouns" framing above is a teaching simplification, not a guarantee. We'll come back to exactly how literally to take it once we've seen a real numerical example.)

Part 2 — How do the heads actually end up different?

This is where everything from Chapter 4 pays off directly. Recall the central idea: `W_Q`, `W_K`, and `W_V` aren't values — they're *learned rules* that get applied to an embedding. Multi-head attention simply gives **each head its own separate rules to work with**:

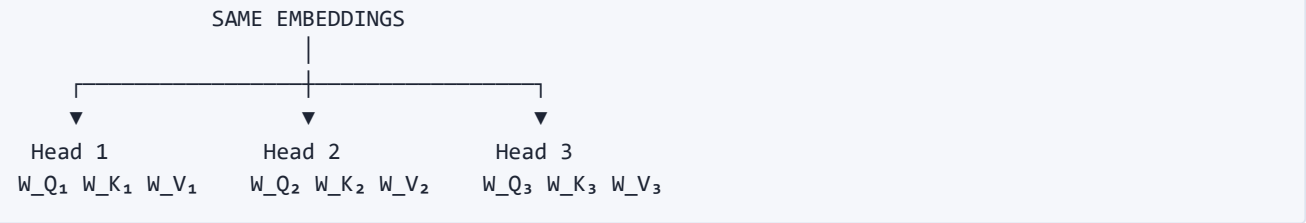
```
A single attention head uses one set of rules:

    W_Q, W_K, W_V

Multi-head attention instead gives every head its own
independent set, each starting from its own separate
random values — not copies of one another:

    Head 1:  W_Q₁, W_K₁, W_V₁
    Head 2:  W_Q₂, W_K₂, W_V₂
    Head 3:  W_Q₃, W_K₃, W_V₃
```

Every head starts from **exactly the same embeddings** — nothing about the input changes per head. What differs is purely which matrices get applied to those embeddings:



All of these matrices are randomly initialized independently, and all of them get reshaped by training, exactly as described in Chapter 4. Because each head's matrices start at different random values and get nudged by slightly different gradient signals over the course of training, they naturally *end up* specializing in different things — not because anyone assigned them a job, but because having heads specialize in different, complementary relationships is a better strategy for reducing prediction error than having every head converge on the same thing. Nobody tells one head "you're the grammar head." That division of labor **emerges** purely from training.

Part 3 — How many heads, and who decides?

This raises an obvious engineering question: how many heads should a model have?

Short answer: the number of heads is chosen by the model's designer, before training even begins. It is not learned. Think of it like staffing an investigation team: you, the architect, decide whether to hire 2 investigators, 8, or 32 — the investigators themselves don't get a vote. In a Transformer, the engineer

decides things like:

```
Embedding size = 768,   Attention heads = 12
Embedding size = 4096,  Attention heads = 32
```

What **is** learned is the content of every head's own `W_Q`, `W_K`, `W_V` — never how many heads exist in the first place.

Why not just use one huge head instead?

You could — a single head is free to use the *entire* embedding dimension. But there's a real tradeoff, well captured by a medical analogy: one generalist doctor looking at everything, versus a heart doctor, a brain doctor, an eye doctor, and a skin doctor, each looking closely at one thing and then combining opinions. Multiple focused specialists often catch more than one generalist spreading their attention thin across everything at once.

Concretely, the embedding dimension gets **split** evenly across the heads:

```
Embedding size 768, split across 12 heads
        ↓
    64  64  64  64  64  64  64  64  64  64  64  64
   (each head works on its own 64-dimensional "view")
```

Why not use hundreds of heads, then?

Every additional head costs real compute: more parameters, more GPU memory, slower inference, higher training cost — and past a certain point, extra heads stop adding meaningfully new information. That's diminishing returns, and it's why real models settle on a specific number rather than maximizing it. There's no single formula for the "correct" number — researchers arrive at it experimentally — but real models cluster around a similar pattern:

Model	Embedding size	Heads	Dimension per head
GPT-2 Small	768	12	64
GPT-2 Medium	1024	16	64
Llama 2 7B	4096	32	128
GPT-3 175B	12288	96	128

Notice the per-head dimension tends to land somewhere around 64–128 across very different model sizes — not a strict law, but a common, empirically-successful design pattern.

An honest correction to the "grammar head / pronoun head" story

It's a genuinely useful *first* intuition to say "one head learns grammar, another learns pronouns, another learns cause and effect" — but don't take it too literally. In real trained models:

- Some heads do specialize cleanly, in ways researchers can identify and name.
- Some heads stay fairly generic, contributing a little to many things.
- Some heads turn out almost redundant.

- Some heads are so unimportant that researchers have removed them from trained models with barely any drop in quality.

A more accurate mental model:

```
Multi-Head Attention
        ↓
Many independent attention mechanisms
        ↓
Each learns whatever turns out to be
most useful for reducing prediction error —
no more, no less.
```

Part 4 — A worked example: three real heads, one sentence

Let's watch this play out concretely. We're back at our sentence, updating the word `"it"`:

```
"The animal didn't cross the road because it was tired."
```

Suppose, after training, three heads produce these attention weight distributions over the earlier words:

Head 1

Word	Weight
animal	80%
road	5%
because	15%

Interpretation: this head has become good at figuring out **who** "it" refers to — mostly `"animal"`. Nobody programmed this; it emerged because attending this way reliably reduced prediction error during training.

Head 2

Word	Weight
animal	10%
road	5%
because	85%

Interpretation: this head discovered something different — it's picked up on `"because"` as a signal that this sentence expresses causality. Head 2 isn't wrong or worse than Head 1; it's solving a different problem entirely.

Head 3

Word	Weight
animal	20%
road	70%
because	10%

Interpretation: this one is less obviously interpretable. Maybe it's picked up on something about physical locations or objects. Or — and this genuinely happens in real models — maybe it's just a fairly useless head that training didn't manage to specialize well. Researchers really have found attention heads like this, contributing very little.

Each head runs the *entire* attention pipeline from Chapter 4 independently — its own Q, K, V, its own scores, its own softmax, its own weighted sum of V — and produces its own output vector. Suppose:

```
Head 1 output → [7, 2]
Head 2 output → [1, 8]
Head 3 output → [5, 6]
```

Three heads, three completely independent opinions about how to update the word `"it"`.

Part 5 — Combining the heads: concatenation

What do we do with three separate output vectors? The answer surprises a lot of people the first time they see it: **we don't average them, and we don't add them — we glue them end-to-end.** This operation is called **concatenation**:

```
Head 1  [7, 2]
Head 2  [1, 8]    →  Concatenate  →  [7, 2, 1, 8, 5, 6]
Head 3  [5, 6]
```

Why concatenate instead of average?

Imagine three specialists each write you a report: a doctor says *"heart is healthy,"* a mechanic says *"engine needs oil,"* a lawyer says *"paperwork expires next month."* Would you average those reports into one blended opinion? Of course not — each one contains genuinely unique, non-overlapping information. Averaging would destroy exactly the specialization we worked so hard to get. Concatenation keeps every head's finding fully intact, side by side, for later layers to make sense of.

The dimension math works out on purpose

Concatenating grows the vector's size: 3 heads of 2 numbers each becomes 6 numbers. In a real model, this scales up the exact same way — say, 32 heads each producing 128 numbers:

```
128 (per head) × 32 (heads) = 4096
```

And 4096 happens to be exactly the model's original embedding size. **That is not a coincidence.** Model designers deliberately choose the number of heads and the per-head dimension so that:

```
Embedding size = Number of heads × Dimension per head
```

This isn't an accident of nature — it's a design constraint, chosen precisely so the concatenated output lines back up with the size the rest of the model expects.

Part 6 — The last learned matrix: W_O

There's one problem left. After concatenation we have `[7, 2, 1, 8, 5, 6]` — but what does that number actually *mean*? It's not one coherent representation; it's three completely independent opinions, stapled together with no interaction between them.

Picture three engineers independently reviewing your code: one scores performance `9/10`, one scores security `8/10`, one scores readability `6/10`. Someone hands you `[9, 8, 6]`. What now? Eventually, *someone* has to combine these separate opinions into one coherent judgment. That's exactly the job of the final learned matrix in attention, called `W_O` — the **output projection**.

```
Concatenated heads
        │
        ▼
    Multiply by W_O
        │
        ▼
Final Multi-Head Attention Output
```

Think of `W_O` as an **opinion merger**. Before it runs, each head's findings sit in complete isolation — without `W_O`, the heads would never actually interact; they'd remain forever-separate streams of information bolted together. `W_O` is what lets the model mix information *across* heads: "when Head 17 says X and Head 4 says Y, that combination actually means Z."

A tiny worked example

Suppose our concatenated vector is `X = [7, 2, 1, 8, 5, 6]`, and (for illustration) `W_O` is a very simple matrix:

```
        col→0  col→1
W_O  =  [ 1     0 ]
        [ 0     1 ]
        [ 1     0 ]
        [ 0     1 ]
        [ 1     0 ]
        [ 0     1 ]
```

Multiplying `X` by `W_O` (dot product of `X` with each column):

```
Output₀ = 7×1 + 2×0 + 1×1 + 8×0 + 5×1 + 6×0 = 7+1+5 = 13
Output₁ = 7×0 + 2×1 + 1×0 + 8×1 + 5×0 + 6×1 = 2+8+6 = 16

Final output = [13, 16]
```

In this toy example, `W_O` just happened to add certain positions together — but only because we deliberately picked an extremely simple matrix to make the arithmetic obvious. A real `W_O` is a dense grid of thousands of trained numbers, mixing every head's output into every output position in far more intricate ways — but the operation is identical: dot-product-with-each-column, exactly like `W_Q`, `W_K`, and `W_V`.

Notice also the shape: the input to `W_O` has 6 numbers (3 heads × 2 each), and the output has 2 — back to the original embedding size. `W_O` isn't just mixing information, it's also **compressing the concatenated output back down** to the size the rest of the Transformer block expects to work with.

The complete picture

```
Embeddings (same input to every head)
        │
   ┌────┬────┬──────────┐
   ▼    ▼    ▼          ▼
 Head1 Head2 Head3 ... HeadN
   │    │    │          │     (each: its own W_Q, W_K, W_V,
   │    │    │          │      its own full attention pipeline
   ▼    ▼    ▼          ▼      from Chapter 4)
[out1][out2][out3]...[outN]
        │
        ▼
   Concatenate
        │
        ▼
   Multiply by W_O
        │
        ▼
Final Multi-Head Attention Output
```

Take a moment to notice what this entire chapter — and Chapter 4 before it — has actually accomplished:

```
Sentence → Tokenization → Embeddings → Q,K,V → Attention →
Softmax → Weighted V → Multiple Heads → Concatenate → W_O
```

We're still only doing **one thing**: making each word's representation richer by letting it gather relevant information from every other word. We haven't predicted anything, reasoned about anything, or generated any text — we've simply improved *what each token knows*. That distinction matters, because it sets up exactly where this handbook goes next.

Common misconceptions, cleared up early

Misconception	Reality
"Each head is literally assigned a job like 'grammar' or 'pronouns.'"	That's a useful first intuition, not a guarantee. Real heads range from clearly specialized, to generic, to nearly redundant — some get pruned from trained models with little effect.
"More heads always means a better model."	Every extra head costs compute and memory, with diminishing returns past a certain point — it's a deliberate design tradeoff, not "more is always better."
"Multi-head attention is a fundamentally different, more complex mechanism than single-head attention."	It's the exact same mechanism from Chapter 4, run several times in parallel with separate learned matrices, then merged. Nothing new is invented.
"The head outputs get averaged together."	They're concatenated (joined end-to-end), specifically to preserve each head's unique findings rather than blending them into mush.
"W_O is just for resizing the vector back down."	Resizing is a side effect. Its real job is mixing information *across* heads — without it, heads would never interact with each other at all.
"Attention alone lets the model 'think' about the sentence."	Attention (single or multi-head) only lets words *gather* information from each other. Genuine per-token processing of that gathered information is a separate step — coming up next.

Quick reference

```
Multi-Head Attention:

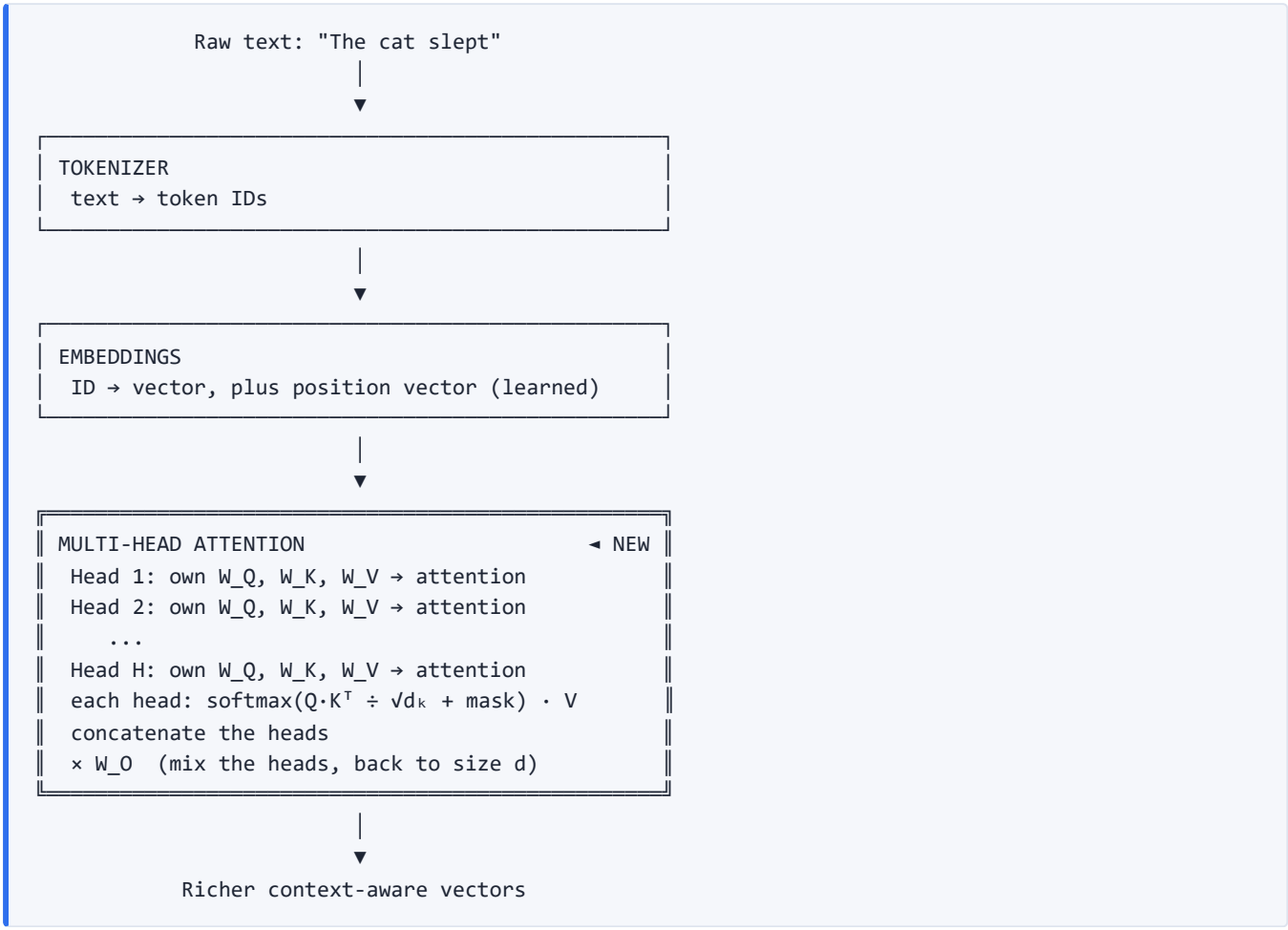
1. Every head gets the SAME embeddings as input.
2. Every head has its OWN learned W_Q, W_K, W_V.
3. Each head runs the full Chapter-4 attention pipeline independently.
4. Head outputs are CONCATENATED (joined), never averaged.
5. The concatenated vector is multiplied by W_O (learned) to:
     - mix information across heads
     - project back down to the original embedding size

Design choice (fixed by architect): number of heads, head dimension.
Learned by training: the content of every W_Q, W_K, W_V, and W_O.

Embedding size = Number of heads × Dimension per head   (by design)
```

The architecture so far

The diagram so far. The double-lined box is this chapter's addition: the single attention head is now many heads in parallel, plus `W_O`.



Learned so far: embedding and position tables; W_Q, W_K, W_V for every head; W_O.

What's next

We've now built the full attention mechanism — the part of a Transformer responsible for letting every word gather relevant information from every other word. But gathering information isn't the same as *thinking* about it. Chapter 6 covers the **Feed-Forward Network** — the part of the architecture that finally lets each token actually process what it just gathered. Chapter 7 then assembles Attention and the FFN together, using residual connections and Layer Normalization, into one complete **Transformer Block**.

Chapter 6 — The Feed-Forward Network: Expansion, Activation and GELU

Introduction

Chapters 4 and 5 built the entire attention mechanism, and ended on a pointed observation: attention only lets words **gather** information from each other. It doesn't let a token actually *think* about what it gathered. That "thinking" step is a separate component — one that comes right after attention inside every Transformer block — called the **Feed-Forward Network (FFN)**, and it's exactly what this chapter goes deep on.

Compared to attention, it's refreshingly simple — no queries, no keys, no values, no softmax, no comparing tokens against each other at all. But don't mistake "simple" for "unimportant": the FFN typically holds a large share of a model's total parameters, and it's doing a genuinely different job from attention — one attention can't do on its own, no matter how many heads you give it.

Part 1 — What problem is the FFN actually solving?

Take our running sentence, after it's been through attention:

```
"The animal didn't cross the road because it was tired."
```

Suppose the word `"it"` now has the vector `[3.84, 4.25]` — the result from Chapter 4's worked example. This vector already contains information gathered from `"animal"`, `"road"`, and `"because"`. So: is the model done? Can it go straight from here to predicting the next word?

Here's the catch. Look at what attention actually *did* to produce that vector — it took a weighted blend of other tokens' Value vectors:

```
0.26 × V(animal) + 0.01 × V(road) + 0.73 × V(because)
```

That's a **mix** of existing information. Attention can combine and re-weight information that's already present in the sentence, but it can't invent a genuinely new feature or draw a deeper conclusion from what it gathered — that's simply not the kind of operation a weighted average is capable of.

Here's an analogy that makes this concrete: imagine attention as gathering ingredients — tomato, cheese, bread. After attention, you have tomato + cheese + bread sitting on the counter. Have you made pizza? No — you've only *gathered* ingredients. Something still has to actually cook them. That "something" is the FFN.

Or think of it as Google Search: you ask *"what causes rain?"* and Google hands you a hundred relevant web pages. Has Google *answered* your question? No — it only **retrieved** information. Understanding those pages and drawing a conclusion is a separate step, done by *you*, afterward. Attention is the retrieval. The FFN is the understanding:

```
Attention  →  retrieval / gathering
FFN        →  understanding / processing
```

Or, as a human analogy: imagine conducting an interview. Phase 1 is asking questions and collecting information — that's attention. Phase 2 is sitting alone afterward, thinking it over, and drawing your own conclusions — that's the FFN.

Part 2 — What does the FFN actually look like?

Surprisingly, for something so important, it's just this:

```
Input Vector
      │
      ▼
Linear Layer (W1)
      │
      ▼
Activation Function
      │
      ▼
Linear Layer (W2)
      │
      ▼
Output Vector
```

That's it — no attention, no other tokens involved, no communication between words at all. Just three operations, applied to one token's vector, completely independent of every other token in the sentence.

Two new learned matrices — and nothing else

The FFN introduces exactly **two** new learned matrices, `W1` and `W2`. Compare that to everything attention needed — `W_Q`, `W_K`, `W_V` per head, plus `W_O` — and the FFN suddenly looks refreshingly light:

```
Attention's learned parameters:      The FFN's learned parameters:
  W_Q, W_K, W_V (per head)             W_1
  W_O                                  W_2
```

Both `W1` and `W2` follow the exact same pattern you already know from Chapter 4: randomly initialized, then reshaped by backpropagation over training. Nothing new to learn about *how* they get trained — only about *what job* they do.

Why isn't it just called a "Linear Layer," then?

A fair question, since it's built from two of them. A **linear layer** is one of the simplest operations in this entire handbook:

```
Output = Input × W
```

You've actually already seen this exact operation — computing `Q = x · W_Q` in Chapter 4 *was* a linear layer. The FFN uses the same basic operation, twice, with something in between. The name "Feed-Forward Network" predates Transformers entirely — it's older neural-network terminology for any network where information only ever flows forward, layer by layer, with no loops and no memory of previous inputs. The Transformer's FFN is a small feed-forward network in exactly that older sense; it just happens to sit inside a much bigger architecture.

Why not skip the activation function and use one big linear layer?

This is worth pausing on, because the reason is a bit surprising: **stacking two linear layers back-to-back, with nothing in between, is mathematically equivalent to just one bigger linear layer.** Two machines that only multiply and add numbers can always be combined into one machine that multiplies and adds numbers — nothing new becomes *possible* just by chaining them. So without something non-linear sitting between `W1` and `W2`, the "two-layer" FFN would secretly collapse into a single, ordinary linear transformation, and the extra layer would be pure wasted computation.

That "something in between" — the **activation function** — is what actually gives the FFN (and neural networks generally) their expressive power. We'll get to exactly what it does in Part 3.

Part 3 — The activation function

This is worth slowing down for. Once you genuinely understand why activation functions exist, you understand not just the FFN, but a huge part of *why neural networks work at all* — this single idea shows up in essentially every neural network ever built, not just Transformers.

Why do we need this, really?

Here's a question worth answering carefully, because the answer is more surprising than it first appears. Suppose you have this pipeline:

```
Input → Linear Layer → Linear Layer → Output
```

Is that more powerful than just this?

```
Input → One Bigger Linear Layer → Output
```

Surprisingly — **no.** This is one of the most important facts in all of deep learning, and it's worth seeing exactly why, not just accepting it.

A linear layer is `Output = Input × W`. Chain two of them together — first `× W1`, then `× W2` — and mathematically, that's *identical* to a single multiplication by the combined matrix `(W1 × W2)`:

```
Input × W1 × W2   ≡   Input × (W1 × W2)
```

Two linear transformations always collapse into one. So if nothing else happens between them, the second layer contributes *nothing new* — it's mathematically redundant.

Here's a human-scale version of exactly the same idea. Imagine a company where every employee follows one single rule: "*multiply everything by 2.*" Employee 1 doubles the number, hands it to Employee 2, who doubles it again, hands it to Employee 3, and so on. After four employees, the whole company has accomplished exactly one thing: multiplying by 16. It doesn't matter that four separate people were involved — nobody actually made a *decision*. It's one operation wearing a disguise of four steps.

Now think about what this means at real scale: **without an activation function, a 100-layer neural network is mathematically equivalent to a single linear transformation.** A hundred layers, collapsed into one. All of that depth, all of that computation — wasted.

So what do we actually need?

We need something that **breaks linearity** — something that can say "*sometimes keep this information, sometimes suppress it, sometimes reshape it non-linearly,*" rather than just scaling everything by a fixed amount no matter what. That's exactly what an **activation function** is: a fixed rule, applied to each number in a vector individually, that introduces a genuine **decision point** into the computation.

Back to the company analogy: suppose one employee, instead of blindly doubling everything, follows a different rule: "*if the value is small, ignore it — if it's important, keep it.*" Suddenly the company's behavior changes completely. It's no longer just scaling numbers uniformly; it's making context-dependent decisions about what to keep and what to discard. That's the qualitative shift an activation function introduces — not "another linear step," but the network's first opportunity to behave *non-uniformly*.

Here's a language-flavored version of the same idea. Suppose, deep inside a model processing the sentence `"The animal was tired,"` one internal number roughly corresponds to "*is this about a living thing?*", another to "*is this about movement?*", and another to "*is this about emotion?*" An activation function is what lets the network decide, case by case: "*emotion is very relevant here — keep it strongly,*" while "*movement isn't very relevant to this particular prediction — suppress it.*" Without an activation function, all of these signals would just get blended together in a fixed, unconditional way, with no ability for the network to say "this one matters more, right now, in this context."

What does an activation function actually *do*, mechanically?

It's worth being precise here, because it's easy to conflate this with what a linear layer does. A linear layer (like `W1` or `W2`) **mixes numbers together** — every output number is a weighted combination of *every* input number. An activation function does the opposite kind of thing: it looks at each number **one at a time, completely independently**, and applies the exact same fixed rule to it, with no mixing across positions at all.

```
Linear layer (W1, W2):   mixes ALL input numbers together to produce each output number
Activation function:      transforms EACH number on its own, with no mixing at all
```

So the FFN's rhythm is: *mix* (via `W1`), then *individually filter/reshape* (via the activation function), then *mix again* (via `W2`). Mixing alone, no matter how many times you repeat it, stays linear. Alternating mixing with this kind of individual, non-linear filtering is what actually gives the network its expressive power.

What the activation function is *not* for: keeping numbers in range

A common assumption is that the activation function is there to stop the vector flowing through the FFN from "blowing up" or shrinking toward zero, keeping the numbers in a manageable range. That's a different job, and the activation function doesn't do it:

- **Its job is non-linearity, not range control.** ReLU and GELU don't cap how large a value can get. A large positive input comes out just as large. Sigmoid does squash values into the 0–1 range, but that's a side effect, and it is exactly what caused the vanishing-gradient problem described below.
- **Range control is done elsewhere.** The $\sqrt{d_k}$ scaling inside attention (Chapter 4), Layer Normalization (Chapter 7), and careful choices for the starting values of the weights are what keep numbers in a workable range.
- **It acts on values, not parameters.** The weights `W1` and `W2` are never touched by the activation function. Only training changes them. The activation works on the numbers *flowing through* the network, the vector for one token as it passes through the FFN.

There is one true part in the intuition. Because the activation treats each number differently depending on its value, it does determine, for each input, which numbers pass through strongly and which are suppressed. ReLU, for example, zeroes out roughly half of them. So which parts of that vector are "active" really does depend on the token. But that comes from the non-linearity, not from keeping anything in range.

```
Activation function  →  makes the computation non-linear, input by input
√dₖ scaling, LayerNorm, careful initialization  →  keep numbers in a workable range
```

A brief, honest history — and *why* each one got replaced

Activation functions have a real history, and each transition happened for a concrete, well-understood reason — not just fashion.

Sigmoid was the early, natural choice. It smoothly squashes any input into a range between 0 and 1 — a shape that made intuitive sense, since it loosely resembles a biological neuron either "firing" or not. For a long time, it was the default.

The problem showed up once networks got deep. Picture the Sigmoid curve: for very negative inputs it hugs close to 0, for very positive inputs it hugs close to 1, and almost all of the actual *change* happens in a narrow band near the middle. Out on either tail, the curve is nearly flat — which means its **slope** (how much the output moves when the input moves slightly) is close to zero. That slope is exactly what backpropagation (Chapter 10 covers this fully) relies on to send a training signal backward through the network. Chain together many nearly-flat slopes — multiplying them together, layer after layer, which is literally what backpropagation does — and the signal reaching the earliest layers shrinks toward zero shockingly fast. This is the well-known **vanishing gradient problem**, and it made very deep Sigmoid-based networks notoriously difficult to train: the early layers would barely update at all, no matter how long training continued.

ReLU (Rectified Linear Unit) was the fix that took over for years afterward, and it's almost embarrassingly simple:

```
If input < 0   →  output 0   (discard it completely)
If input ≥ 0   →  output unchanged
```

For positive inputs, ReLU's slope is a constant `1` — not shrinking, no matter how many layers you stack. The vanishing gradient problem, for the positive side at least, simply disappears. It's also extremely cheap to compute — just a comparison, no exponentials involved — which mattered a great deal as networks grew larger.

But ReLU trades one problem for a different one. For *negative* inputs, its output is flat, constant `0` — and the slope of a perfectly flat line is exactly `0`. If a particular neuron happens to receive negative inputs consistently during training (which genuinely does happen), its gradient becomes permanently zero — meaning it stops receiving any training signal at all, forever. This is known, somewhat dramatically but accurately, as a **"dead" or "dying" ReLU neuron**, and in large networks it's possible for a meaningful fraction of neurons to end up permanently stuck this way, silently wasting capacity. Separately, ReLU also throws away **every single negative value**, no matter how large or small — `-0.2` becomes `0`, `-4` becomes `0`, `-100` becomes `0` — which is sometimes too aggressive, since a small negative value can still carry genuinely useful information that a hard cutoff destroys outright.

GELU (Gaussian Error Linear Unit) — the activation most modern LLMs actually use — was designed to sidestep both problems at once. Don't be intimidated by the name; the idea is simple even though the name isn't. Like ReLU, large positive inputs pass through almost unchanged, with a gradient close to `1` — so the vanishing-gradient fix is preserved. But unlike ReLU, GELU is **smooth**, with no perfectly flat, zero-gradient region anywhere — even negative inputs retain at least a small trickle of gradient. That means a neuron using GELU can never become permanently, completely "dead" the way a ReLU neuron can.

Instead of ReLU's blunt rule, GELU behaves more like a graded confidence judgment:

```
Large negative input   →  reduce it almost to zero, but not abruptly
Small negative input   →  reduce it gently, keep a little
Small positive input   →  keep some of it, not all
Large positive input   →  keep almost all of it
```

Compare all three side by side on the same inputs:

```
Input:      [-4,   -2,   -1,   0,   1,     2,     4]
Sigmoid:    [≈0,  0.12, 0.27, 0.5, 0.73, 0.88,   ≈1]   (squashed into 0–1, flat at both tails)
ReLU:       [ 0,    0,    0,   0,   1,     2,     4]   (hard cutoff at 0)
GELU:       [≈0, -0.05, -0.16,  0, 0.84, 1.95,    4]   (smooth everywhere)
```

Notice ReLU makes a hard, binary decision exactly at zero, while GELU transitions smoothly through it. Think of it in terms of confidence: if a computation produces a small value like `0.05`, ReLU says "*positive — keep it fully,*" with no nuance. GELU instead says, roughly, "*I'm not fully convinced this matters — I'll keep some of it, not all of it.*" That smoother, more graded behavior — combined with never fully "killing" a neuron the way ReLU can — is why GELU (and close relatives like SwiGLU, which several of the most recent models use instead) became the standard choice for large, very deep Transformer-based LLMs. We won't go deeper into SwiGLU and its relatives in this handbook, but it's worth knowing the family name if you go looking at real model code.

An important correction: the activation function itself is not learned

It's tempting to describe GELU as if it's "deciding" or "judging" — useful shorthand, but not literally accurate. **GELU is a fixed mathematical function. It never changes, and it has no parameters of its own to train.** The exact same GELU formula runs at every position, in every layer, for every model that uses it — what actually gets shaped by training are the numbers inside `W1` and `W2`, which determine *what gets handed to* GELU in the first place. GELU just reliably applies the same fixed rule to whatever it receives.

```
Learned (Chapters 9–11 train these):      Not learned (fixed forever):
  W1, W2                                  GELU itself (and Sigmoid, ReLU, etc.)
                                          Every matrix multiplication mechanic
```

Part 4 — Expanding into a larger workspace

Now that we know why a non-linear step has to sit between the two linear layers, let's walk through the FFN's three steps in order: expand with `W1`, apply the activation function, compress with `W2`. This part covers the first one, with a genuine worked example. Suppose our token's vector, after attention, is:

```
x = [3, 4]
```

The first linear layer multiplies this by `W1`. Suppose:

```
        col0  col1  col2  col3
W1  =  [ 2     1     0     3 ]
       [ 1     2     2     0 ]
```

Computing each output number (dot product of `x` with each column):

```
Output₀ = 3×2 + 4×1 = 6+4  = 10
Output₁ = 3×1 + 4×2 = 3+8  = 11
Output₂ = 3×0 + 4×2 = 0+8  =  8
Output₃ = 3×3 + 4×0 = 9+0  =  9

Result: [10, 11, 8, 9]
```

Notice what happened to the shape: a 2-number input turned into a 4-number output, purely because `W1` has 4 columns. **The shape of the weight matrix decides the output size** — nothing more mysterious than that, and it's the exact same mechanic as computing Q, K, or V in Chapter 4.

Why deliberately make the vector bigger?

This is the strange-looking part worth sitting with. Why go from 2 dimensions to 4 (or in real models, from a few thousand to tens of thousands) only to shrink back down afterward?

Imagine your boss asks you to explain your entire company in exactly two words. Nearly impossible. Now imagine they instead say: *first, brainstorm freely using 100 words — then* summarize that down to two. Much easier, and the final two-word summary is likely far better, because it was distilled from a much richer draft rather than forced to be terse from the start. The FFN does exactly this:

```
Small representation → Expand → Room to think → Compress → Better representation
```

Or think of it with LEGO: with only 2 bricks, there aren't many things you can build. With 200 bricks, there are vastly more possible structures. Expanding the vector temporarily gives the model a much larger "workspace" to form richer combinations of features, before compressing everything useful back down.

In real GPT-style models, this expansion is typically **4×**: an embedding dimension of `4096` commonly expands to `16384` inside the FFN, before compressing back to `4096`. This "expand, then shrink" pattern is one of the most recognizable signatures of a Transformer FFN in real code.

Part 5 — Compressing back down

After the activation function, we compress the vector back to its original size using the second linear layer, `W2`. Continuing our toy example:

```
Before FFN:        [3, 4]
After W1 (expand):  [10, 11, 8, 9]
After GELU:         [10, 10.8, 7.8, 8.9]     (illustrative — values shift slightly)
After W2 (compress): [9, 2]
```

The exact numbers here aren't the point — the *process* is: expand, apply a fixed non-linear function, compress back down. In a real model, this is the same `4096 → 16384 → 4096` shape mentioned earlier. And critically: **why shrink back down at all, rather than letting the vector keep growing?** Because every subsequent layer — every following attention computation, every following FFN — would have to work with an ever-larger vector at every single token, in every single layer. Left unchecked, that would make the model computationally impossible to run. So the FFN insists on returning to the same size it started with: think in a large, temporary workspace, then summarize your conclusions back down before handing them to the next block.

Part 6 — What just happened, exactly?

This is the single most important realization in this chapter: **the FFN never communicates with any other token.** Attention was all about cross-token communication — comparing this word against every other word. The FFN does the complete opposite: it takes one token's vector, and transforms *only* that vector, with zero knowledge of what any other token's vector currently looks like.

```
Attention  →  moves information BETWEEN tokens
FFN        →  creates richer features WITHIN one token
```

An important clarification: "private," but not separately learned

It's tempting to say each token gets "its own private neural network" while passing through the FFN — and in one sense, that's true: each token's computation happens completely independently, with no cross-talk between tokens at this step, exactly the opposite of attention. But be precise about what's actually private here, because this is an easy point to get backwards:

```
NOT private:  W1 and W2 themselves.
              There is exactly ONE W1 and ONE W2 per Transformer
              block — the same two matrices are reused, unchanged,
              for every single token, at every position.

PRIVATE:      The input and output VECTOR for each token, and the
              fact that no token's computation depends on any
              other token's computation during this step.
```

A better mental picture: the FFN is like a single rubber stamp — the exact same stamp, `W1` → activation → `W2` — pressed independently onto every token's vector, one at a time (conceptually; in practice, all at once, in parallel, for speed). It's one shared function applied many times, not many different functions.

Part 7 — Couldn't `W_O` just do this job?

A very natural question at this point: `W_O` (Chapter 5) is also a learned matrix, and it already decides how much each attention head contributes. If we trained it well enough, couldn't it learn which heads matter, give the useful ones more weight, and make the FFN and its activation function unnecessary?

Half of that intuition is correct, and it's worth saying which half. **`W_O` really does learn head preferences.** Each head's output is multiplied by its own slice of `W_O` before the results are combined, so a head that turns out to be useful ends up with large weights there, and a head that isn't ends up with small ones. That is exactly why some heads can be removed from trained models with little effect.

But no amount of training can turn `W_O` into a replacement for the FFN. There are four reasons.

Reason 1 — `W_O` is linear, and training can't change that

Training changes the *numbers inside* `W_O`. It can't change the *kind of operation* `W_O` performs, which is multiplication by a matrix. And there's a further wrinkle: each head's `W_V` and `W_O` are both linear, so together they behave like one single linear transformation. Making `W_O` better trained makes that linear transformation better, but it never becomes anything other than linear. Everything from Part 3 applies: a linear step can scale and mix what's already there, but it can't create genuinely new features.

Reason 2 — its preferences are the same for every token

Once training is finished, `W_O` is a fixed grid of numbers. If it says "trust head 3 at 80%", that holds for every token, in every sentence. What you'd actually want is something like *"for a pronoun, listen to head 3; for a verb, listen to head 5."* That is a decision that **depends on the input**, and a fixed linear mix cannot make one. Decisions that depend on the input need a non-linear step, and that is exactly what the activation function provides. (Attention's softmax does make input-dependent choices, but between *tokens* inside a head, not between heads.)

Reason 3 — attention only mixes; it can't compute new features

Whatever attention produces is, at heart, a weighted average of existing Value vectors. Here is a small example of what a mixing-only step cannot do. Suppose a token has two signals, `a` and `b`, each either 0 or 1, and the model wants a new feature that fires **only when both are present**:

```
a  b     wanted
1  0  →    0
0  1  →    0
1  1  →    1
```

Any linear mix has the form `w1·a + w2·b`, so the "both present" case is always just the sum of the two single cases, and it can't be made special. Now add one fixed non-linear step, `ReLU(a + b − 1)`:

```
a=1, b=0  →  ReLU(0)  =  0
a=0, b=1  →  ReLU(0)  =  0
a=1, b=1  →  ReLU(1)  =  1
```

The feature now exists. This is exactly the kind of thing the FFN does at scale, inside every token: it builds new combined features ("A *and* B", "A *but not* B") that mixing alone can never produce. A stack of attention-only layers does the opposite. It keeps re-averaging the same information, and tokens tend to become more alike rather than richer.

Reason 4 — the FFN is more than its activation function

Even if you added a non-linearity right after `W_O`, you'd only have a very small FFN. It would lack the expansion from Part 4, the large temporary workspace in which rich feature combinations can form. The expanded FFN layers also hold a large share of a model's parameters, and evidence suggests that much of what a model has learned about language and the world lives there.

```
Attention  →  decides WHERE to look (routes information between tokens)
W_O        →  mixes the heads together (linear, same for every token)
FFN        →  computes NEW features inside each token (non-linear, with room to work)
```

Each part does a job the others cannot, which is why every Transformer block contains both attention and an FFN.

The complete picture

```
Token's vector (after attention)
        │
        ▼
   x · W1              ← LEARNED (expands the dimension, e.g. 4096 → 16384)
        │
        ▼
   GELU (or similar)    — fixed math, nothing to learn
        │
        ▼
   (·) · W2             ← LEARNED (compresses back down, e.g. 16384 → 4096)
        │
        ▼
Token's new, "thought-about" vector
```

Applied completely independently, in parallel, to every token in the sequence — the same two matrices, reused everywhere.

One analogy to hold it all together: a sack of mangoes

Imagine you're handed a sack of mixed mangoes, and you need to know what's really in it.

```
The sack of mangoes        →  the token's vector, as it arrives from attention
Asking many questions      →  W1 (expanding into a larger workspace)
Grading each mango alone   →  the activation function
Packing a summary          →  W2 (compressing back to sack size)
```

1. **The sack.** This is the token's vector after attention: everything gathered so far, all mixed together.
2. **Spreading it out and asking questions (`W1`).** You tip the sack onto a large floor so there's room to look closely. Strictly, you aren't just laying the same fruit out in rows. Each spot on the floor is a *different question* about the whole sack: "how much ripe mango is in here?", "is it mostly raw?", "is anything spoiled?" `W1` is your trained eyes: through training it learns *which* questions are worth asking. Nobody writes those questions in advance.
3. **Grading each one (the activation function).** Now you judge each reading on its own, without comparing it to the others: strong signals pass through, weak ones are damped, and the very weakest may be dropped altogether. This is a fixed rule. It knows nothing about mangoes. It only decides how strongly each reading goes forward. (With GELU, "dropping" is usually gentle damping rather than throwing away. ReLU is the blunt version that zeroes weak readings completely.)
4. **Packing a summary (`W2`).** You now have thousands of readings but only a sack-sized space to put them in. `W2` learns which combinations of readings are worth keeping and writes a compact summary that fits back into the sack. It can't keep every detail, so it keeps the ones that proved most useful.

What comes back is the same size as what went in, but it now contains conclusions drawn from the contents, not just the contents themselves. In Chapter 7 you'll see one more step: the original sack isn't thrown away. The summary is added on top of it, which is the residual connection.

Common misconceptions, cleared up early

Misconception	Reality
"The FFN lets tokens exchange information, just like attention does."	The opposite — the FFN's defining feature is that it does *not* look at any other token. All cross-token communication happens in attention, not here.
"Each token gets its own separately-learned FFN."	Every token is processed by the exact same shared `W1` and `W2` — one FFN per block, reused identically for every position. What differs per token is only the *input vector*, not the weights.
"The activation function (GELU) is learned, like the weight matrices."	GELU is a fixed formula with no trainable parameters. Only `W1` and `W2` are shaped by training — GELU always behaves the same way, everywhere.
"Expanding the dimension and then shrinking it back is wasted, pointless work."	The temporary expansion gives the model a larger "workspace" to form richer feature combinations, which is only possible because of the non-linear activation sitting in between the two linear layers.
"Two linear layers in a row would work just as well without an activation function."	Two linear layers with nothing non-linear between them mathematically collapse into a single linear layer — the activation function is what makes the second layer add anything new at all.
"ReLU was replaced because it's 'less accurate' in some vague sense."	ReLU has a specific, well-understood failure mode: for negative inputs its gradient is exactly zero, so a neuron stuck receiving negative inputs can stop training permanently (a "dead" neuron). GELU keeps a small gradient everywhere, so this can't happen.
"The activation function keeps the numbers from blowing up or shrinking away."	That's the job of $\sqrt{d_k}$ scaling, Layer Normalization, and careful weight initialization. ReLU and GELU don't cap large values at all; the activation function's job is to make the computation non-linear.
"The activation function mixes information the same way a linear layer does."	It's the opposite: linear layers (`W1` , `W2`) mix all numbers together; the activation function transforms each number individually, with no mixing across positions at all.

Quick reference

```
FFN (applied independently, per token, same weights for every token):

  x  →  x·W1  →  Activation (GELU)  →  (·)·W2  →  output

W1, W2  = LEARNED matrices (only 2 — much simpler than attention).
GELU    = fixed function, NOT learned.

Typical shape in real models:  d_model → 4×d_model → d_model
  (e.g. 4096 → 16384 → 4096)

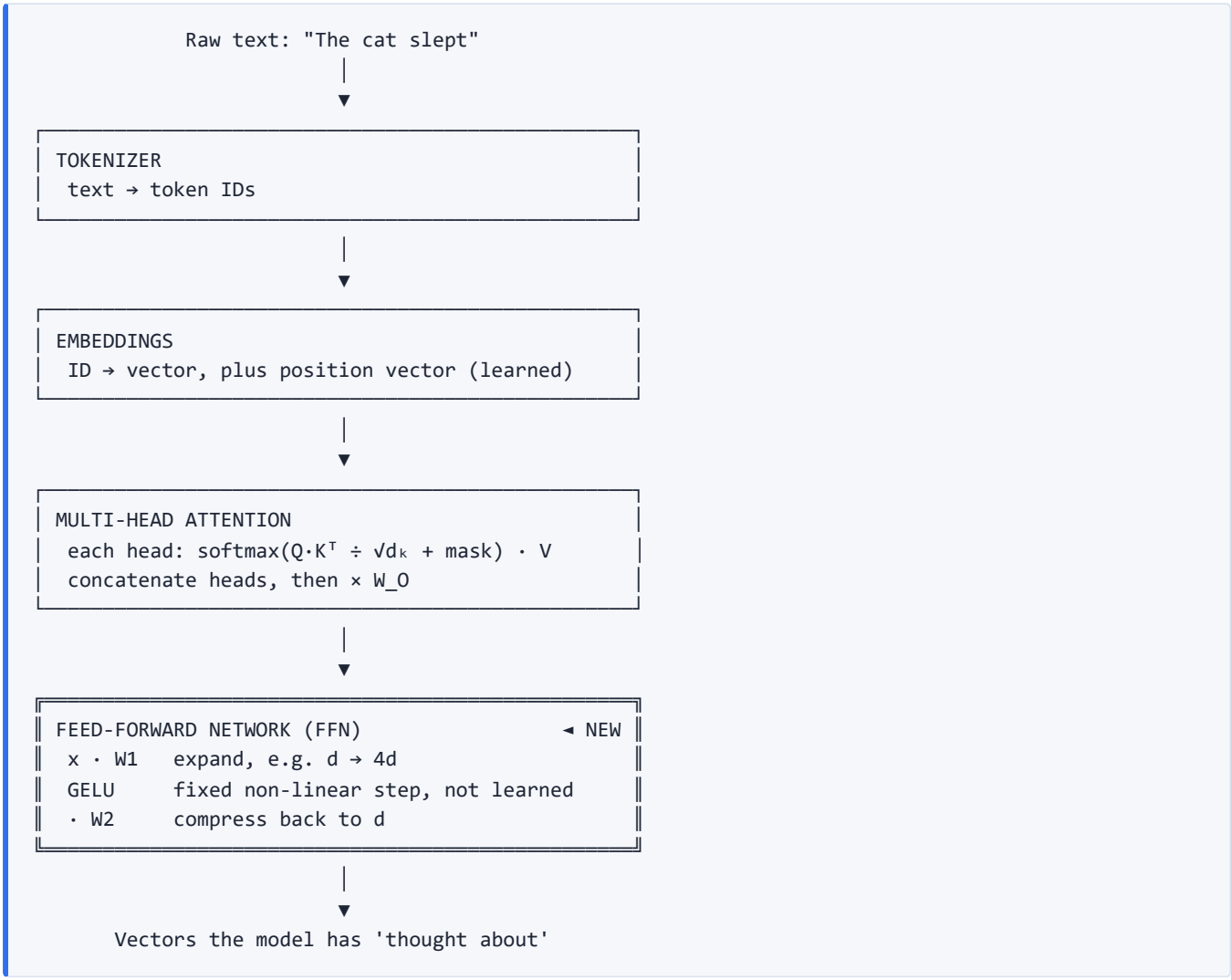
Attention: moves information BETWEEN tokens.
FFN:       creates richer features WITHIN one token, independently.

Without the activation function, W1 then W2 would collapse into
one ordinary linear layer — no extra expressive power gained.

History, and why each step happened:
  Sigmoid → vanishing gradients (flat tails kill backprop signal)
  ReLU    → fixes positive-side gradient, but negative side can
            "die" permanently (gradient exactly 0 forever)
  GELU    → smooth everywhere, never fully dies, standard in modern LLMs
```

The architecture so far

The diagram so far. The double-lined box is this chapter's addition.



Learned so far: embedding and position tables; W_Q, W_K, W_V per head; W_O; W1 and W2.

What's next

We now have both halves of a Transformer block fully explained: Attention (Chapters 4–5) for gathering information across tokens, and the FFN (this chapter) for processing that information within each token. Chapter 7 covers how these two pieces actually get wired together — using residual connections and Layer Normalization — into one complete, stackable **Transformer Block**.

Chapter 7 — The Transformer Block: Residual Connections and LayerNorm

Introduction

We now have both halves of a Transformer block fully explained: Chapters 4–5 covered Attention in depth — how a token **gathers** relevant information from every other token — and Chapter 6 covered the Feed-Forward Network, or FFN — how a token then *processes* that gathered information on its own. This chapter is pure assembly: **how do Attention and the FFN actually get wired together into one repeatable unit** — the Transformer Block — and what stops that unit from quietly breaking as we stack dozens of them on top of each other?

Two ideas answer that: **residual connections** and **Layer Normalization**. Neither adds any new "intelligence" to the model — both exist purely to make a very deep stack of blocks actually trainable at all. That might sound like a minor plumbing detail. It isn't — without them, deep Transformers simply don't train.

Part 1 — A question worth asking: what happens to the original input?

Suppose we've built everything through the FFN:

```
Input Token → Attention → W_O → FFN → Output Token
```

Take our running example. Suppose the token `"it"` arrives as the vector `[4, 9]`. After the attention and FFN computations have both transformed it, it comes out as, say, `[12, 8]`.

Stop and ask: **where did `[4, 9]` go?** It's gone. Completely overwritten.

Is that actually a problem? Consider an analogy: imagine an AI editor rewrites your paragraph —

```
Original:  "The animal didn't cross the road because it was tired."
Rewritten: "The creature remained stationary due to exhaustion."
```

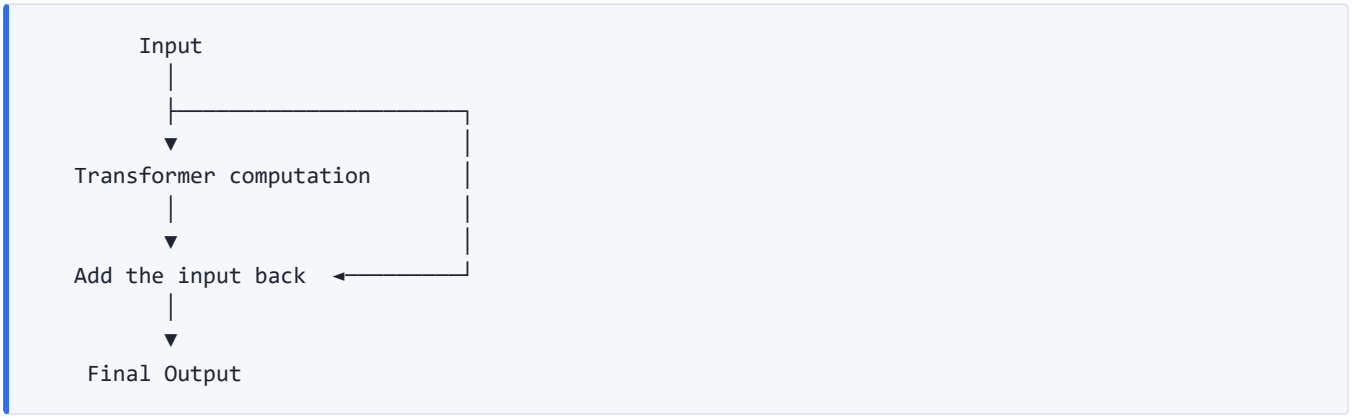
Maybe the rewrite is better. Maybe the AI misunderstood something subtle and the rewrite is actually worse. Either way, once the original is gone, there's no way back, and no way to blend the good parts of both. Wouldn't it be safer to keep a copy of the original around, and only *add* improvements to it, rather than blindly replacing it?

Part 2 — Residual connections: don't replace, add

That question is exactly why **residual connections** exist. Instead of this:

```
Input → Transformer computation → Output
```

we do this:



The original input takes a "shortcut" around the computation, and gets **added back** at the end. Concretely:

```
Input:          [4, 9]
After FFN:       [12, 8]
Residual output: [4,9] + [12,8] = [16, 17]
```

That's the entire mechanism. In formula form:

```
Output = x + F(x)
```

where `x` is the original input, and `F(x)` is whatever the block just computed (attention, or the FFN). No new matrices, no new learned parameters — residual connections are pure addition. **Nothing is learned here at all.**

Why is this such a good idea?

Think about training a new employee. Every week they learn something new. Would you rather they *forget everything and relearn from scratch each week*, or *keep everything they already know and add the new lesson on top*? Obviously the second. That's precisely what a residual connection does for a token's representation:

```
Before attention:   "I am the word 'it.'"
After attention:    "I know I probably refer to the animal."
After the FFN:       "I also understand the causal relationship."

Residual connections say: don't throw away "I am the word 'it,'"
just keep adding new understanding on top of it.
```

Or think of sculpting a statue: you don't start over from a fresh block of stone every day. Day 1, you shape the stone. Day 2, you take *yesterday's statue* and make small improvements. Day 3, the same. The Transformer doesn't reinvent each token's representation from scratch at every block — it **iteratively refines** it, one small improvement layered on top of the last.

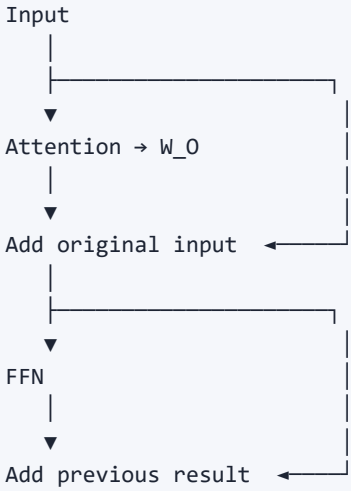
```
Without residuals:  Old representation → REPLACED
With residuals:      Old representation + improvement → Better representation
```

Why researchers actually needed this

Before residual connections were invented, people tried building very deep networks — dozens or even a hundred layers stacked on top of each other — and training them went badly. Information from early layers gradually got diluted or lost entirely as it passed through so many transformations, and the gradients used to train the network (Chapter 10) got weaker and weaker the further back they had to travel — a well-known problem called the **vanishing gradient**. Residual connections create a direct, unobstructed path for both information and gradients to flow through, which makes training dramatically more stable, even with very deep stacks of layers. This single idea is a large part of what made truly deep neural networks — including modern LLMs with dozens of Transformer blocks — trainable at all.

There are two residual connections per block, not one

Inside a real Transformer block, this shortcut-and-add pattern happens **twice** — once around attention, once around the FFN:



Why not just keep the original forever, and never let it change?

Because each block is supposed to contribute *something*. After Block 1 adds its improvement, the *result* — not the original raw embedding — becomes the input to Block 2, which gathers more context and adds its own refinement on top of that. The representation keeps evolving, block after block:

```
Block 0 input:        [4, 9]
After Block 1:         [16, 17]
After Block 2:         [21, 24]
After Block 3:         ...
```

Every block contributes another layer of refinement on top of everything that came before — that's the whole appeal of stacking many blocks (Chapter 8 covers exactly what deeper stacks buy you).

Part 3 — A new problem appears: the numbers keep growing

Residual addition solves the "don't throw away information" problem, but it quietly introduces a new one. Since every block **adds** something to what it received, and real models stack dozens of blocks, the numbers involved tend to keep getting larger as they pass through the network:

```
Token starts as:        [2, 5]
After Block 1:            [8, 11]
After Block 2:            [15, 20]
After Block 3:            [31, 42]
...
After Block 20:           [840, 1021]
```

Why is that a problem? Imagine grading two students, where one answers on a scale of 0–10 and the other on a scale of 0–10,000 — comparing them directly is awkward, and any downstream calculation that expects "normal-sized" numbers starts behaving unpredictably. Or imagine three different shops weighing fruit — one in grams, one in kilograms, one in tons — you'd want to convert everything to a common scale before comparing. Left unchecked, ever-growing numbers make training increasingly unstable the deeper the network gets.

Layer Normalization: put every vector back on a common scale

Layer Normalization (LayerNorm) fixes this by rescaling each token's vector, individually, back to a well-behaved range — every single time it's applied. Suppose after a residual step we have:

```
[100, 110, 90, 95]
```

LayerNorm asks two questions about this vector:

1. What's the average (mean)?

```
Mean = (100 + 110 + 90 + 95) ÷ 4 = 98.75
```

2. How spread out are the values (standard deviation)? You don't need the exact formula to follow along — just think of it as "how much variation exists." Suppose it comes out to `7.4`.

Then every value gets rescaled using the same rule: **subtract the mean, then divide by the standard deviation.**

```
100 → (100 − 98.75) ÷ 7.4 ≈  0.17
110 → (110 − 98.75) ÷ 7.4 ≈  1.52
 90 → ( 90 − 98.75) ÷ 7.4 ≈ −1.18
 95 → ( 95 − 98.75) ÷ 7.4 ≈ −0.51
```

```
Result: [0.17, 1.52, -1.18, -0.51]
```

Did we just lose information?

This is the question almost everyone asks the first time they see this. Look closely at the ordering:

```
Before:  110 > 100 > 95 > 90
After:   1.52 > 0.17 > -0.51 > -1.18
```

The ordering, and the relative relationships between the numbers, stayed exactly the same. Only the *scale* changed. It's like printing a map at A4 size and then enlarging it to A1 — the cities don't move relative to each other, the roads don't change, only the scale of the paper does. LayerNorm does the same thing to a vector: it keeps the shape of the information intact, and only fixes the scale.

LayerNorm's two learned adjustments: γ and β

The normalization itself (subtract the mean, divide by the standard deviation) is completely fixed math, with nothing to learn. But real implementations then apply two small **learned** adjustments right after it, one value of each per embedding dimension:

```
output  =  γ (gamma, a learned "scale")  ×  normalized value  +  β (beta, a learned "shift")
```

One naming note: this "scale" has nothing to do with the *learning rate* from training (Chapter 11). `γ` is simply a number that multiplies each dimension.

Are `γ` and `β` matrices, like `W_Q` or `W_1`? No, they're much smaller. If the embedding dimension is 4096, then `γ` is just 4096 individual numbers, and so is `β`: one scale value and one shift value per dimension, not a matrix mixing dimensions together. LayerNorm never learns *relationships between* dimensions. It only learns how much to rescale and shift each dimension, independently.

A numerical example: what γ and β actually do

Take the vector from above. After the fixed normalization step, `[100, 110, 90, 95]` became:

```
x̂ = [0.17, 1.52, -1.18, -0.51]      mean = 0, spread (standard deviation) = 1
```

This part is fixed math, and it's recomputed from each token's own numbers, so every token gets its own mean and spread. Now apply `γ` and `β`. Suppose training has settled on these values:

```
γ = [1.0,  2.0,  0.5,  1.0]
β = [0.0,  1.0, -1.0,  0.5]

y₀ = 1.0 × ( 0.17) + 0.0 =  0.17
y₁ = 2.0 × ( 1.52) + 1.0 =  4.04
y₂ = 0.5 × (-1.18) − 1.0 = -1.59
y₃ = 1.0 × (-0.51) + 0.5 = -0.01

y = [0.17, 4.04, -1.59, -0.01]
```

The output is no longer mean 0 and spread 1: its mean is about 0.65, and its spread is larger. And each dimension was treated differently:

- **Dimension 1** was amplified (`γ = 2`) and lifted (`β = 1`).
- **Dimension 2** was shrunk (`γ = 0.5`) and pushed down (`β = −1`).
- **Dimension 0** was left alone.

How is this different from the zero-mean step?

The two steps do different jobs at different levels:

	Normalization step	γ and β
Learned?	No, fixed math	Yes, trained
Computed from	Each token's own numbers	Nothing. They're stored constants
Same for every token?	No, each token gets its own mean and spread	Yes, the same values for every token
Applies to	The vector as a whole	Each dimension separately

Normalization removes a token's *own* overall offset and scale. γ and β then apply a fixed, learned "house style" to each dimension.

Why is this an advantage?

1. Normalizing doesn't force you to lose information. Plain normalization throws away the original overall scale and mean. But the model can undo it exactly if that turns out to be useful. Set γ to the original spread (7.4) and β to the original mean (98.75), and you get the original vector back (up to rounding):

```
0.17 × 7.4 + 98.75 ≈ 100
1.52 × 7.4 + 98.75 ≈ 110
-1.18 × 7.4 + 98.75 ≈ 90
-0.51 × 7.4 + 98.75 ≈ 95
```

So the model gets the stability of normalizing, and it can recover as much of the original scale as it actually needs. In practice γ starts at 1 and β at 0, which is pure normalization, and training moves them only as far as it helps.

2. Different dimensions can matter by different amounts. After normalization every dimension sits on equal footing. But the next matrix multiplication treats a larger number as a stronger signal. γ lets the model say "dimension 1 should carry more weight from here on, and dimension 2 less." Without it, all dimensions would be forced to the same strength.

3. β sets where each dimension sits relative to zero. That position can matter for what comes next. For example, whether a value is above or below zero affects how a function like GELU (Chapter 6) treats it. A learned shift lets the model place values where the steps after it work well.

4. It's cheap. For an embedding size of 4096, that's only 4096 + 4096 extra numbers per LayerNorm, compared with millions in a weight matrix.

One more point: many recent models, such as Llama, use a simplified variant called **RMSNorm**. It drops the mean subtraction and β and keeps only a learned scale. So γ is the more essential of the two.

Part 4 — Assembling the complete Transformer Block

We now have every ingredient. Here is the full, original Transformer block, exactly as described in the 2017 paper that introduced this architecture:

```
Input
   │
   ▼
Multi-Head Attention
   │
   ▼
W_O
   │
   ▼
Residual Add
   │
   ▼
LayerNorm
   │
   ▼
Feed Forward Network
   │
   ▼
Residual Add
   │
   ▼
LayerNorm
   │
   ▼
Output
```

This single block — attention to gather information, a residual connection to preserve what came before, LayerNorm to keep the numbers well-behaved, then the FFN to process the gathered information, another residual connection, another LayerNorm — is the fundamental repeating unit of a Transformer. Chapter 8 covers what happens when you stack many of these on top of each other, which is essentially what "GPT" actually is.

An important correction for modern LLMs

The diagram above — LayerNorm placed *after* the residual addition — is called **Post-LayerNorm**, and it's how the original 2017 Transformer paper did it. Most modern decoder-only LLMs (GPT-2, GPT-3, Llama, Gemma, Qwen, and most others you'll encounter) actually use a rearranged version called **Pre-LayerNorm** instead:

```
Post-LayerNorm (original):      Pre-LayerNorm (most modern LLMs):

Attention                        LayerNorm
   ↓                                ↓
Residual                        Attention
                                     ↓
                                 Residual
```

...and the same swap happens around the FFN. Researchers found that normalizing *before* each computation, rather than after, makes training dramatically more stable for very deep, modern-scale models. The conceptual role of every component — what attention does, what the FFN does, why residuals exist, why LayerNorm exists — is identical either way; only the *order* changes. We'll point out this distinction again when we look at real GPT and Llama architectures directly.

Common misconceptions, cleared up early

Misconception	Reality
"Residual connections are a special learned mechanism."	They introduce zero new parameters — it's literally just `Output = x + F(x)`, plain addition.
"Adding the original input back means nothing really changes."	The *addition* is fixed, but `F(x)` — attention or the FFN — still does real, learned computation. Residuals add refinements on top of the original; they don't cancel the refinement out.
"LayerNorm learns how the dimensions of a vector relate to each other."	It doesn't — the normalization step is fixed math, and its only learned parameters (`γ`, `β`) adjust each dimension *independently*, not relationships between dimensions.
"If `γ` and `β` can undo the normalization, the normalization is pointless."	The normalization step fixes the numbers' scale at the start, so training is stable. `γ` and `β` then start at 1 and 0 (no change) and only move as far as training finds useful, so the model gets stability *and* flexibility, rather than being locked into a fixed zero-mean, unit-spread shape.
"Normalizing a vector changes what it represents."	It changes the *scale* only. The relative ordering and relationships between the numbers are preserved — nothing about the underlying information is discarded.
"There's one 'correct' place to put LayerNorm in a block."	Both Post-LN (original Transformer) and Pre-LN (most modern LLMs) are used in real, successful models — the placement is a stability-driven design choice, not a fixed law.

Quick reference

```
Residual connection:   Output = x + F(x)
   - No learned parameters.
   - Preserves information and gradient flow through deep stacks of blocks.
   - Appears TWICE per block: around attention, and around the FFN.

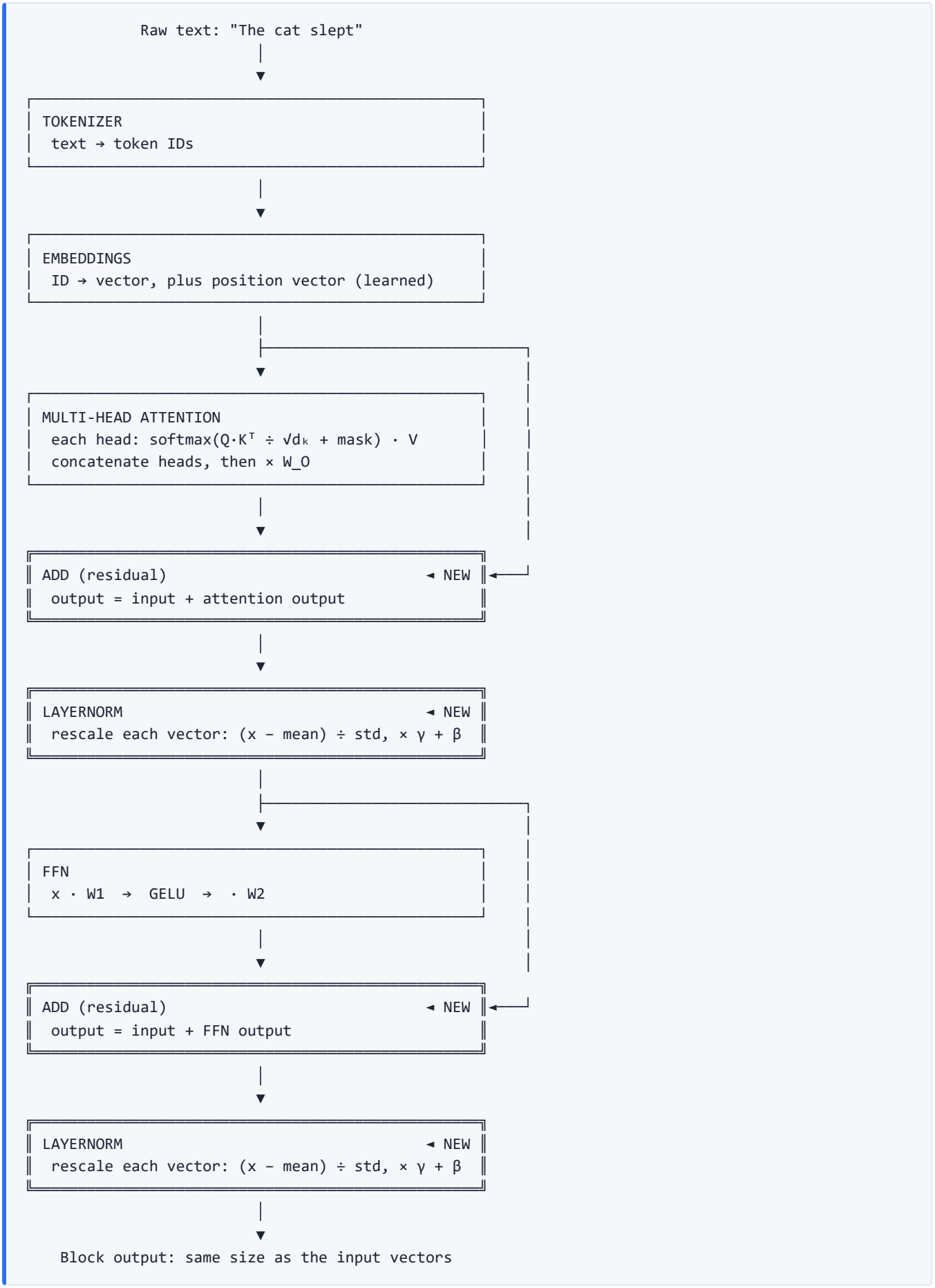
LayerNorm:              Output = (x − mean) / std_dev  × γ + β
   - Normalization itself is fixed math (no parameters).
   - γ (scale) and β (shift) ARE learned — one value per embedding
     dimension, not a full matrix.
   - Keeps vector magnitudes stable across many stacked blocks.
   - Preserves ordering/relationships; only rescales.

One Transformer Block =
  Attention → Residual → LayerNorm → FFN → Residual → LayerNorm

Post-LN (original paper) vs Pre-LN (most modern LLMs):
  same components, different order, chosen for training stability.
```

The architecture so far

The diagram so far. This chapter adds the double-lined boxes, and the skip lines on the right show the residual connections. With them, the Transformer block is complete.



Learned so far: embedding and position tables; per block: W_Q, W_K, W_V per head, W_O, W1, W2, and LayerNorm γ and β (two sets).

This is the complete Transformer block, drawn in the original (post-norm) order. Most modern LLMs move each LayerNorm to just before attention and just before the FFN (pre-norm). The parts are the same; only the order changes.

What's next

We now have one complete, self-contained Transformer Block — Attention, residual, LayerNorm, FFN, residual, LayerNorm. Chapter 8 covers what happens when you stack many of these blocks on top of each other, which is essentially what turns this single block into an actual GPT-style model.

Chapter 8 — Stacking Blocks into GPT: From Text to a Next-Word Probability

Introduction

Chapter 7 gave us one complete Transformer block: attention, residual, LayerNorm, FFN, residual, LayerNorm. That single block is the repeating unit of a Transformer, but a model with one block would be shallow and weak. This chapter does two things that turn that one block into an actual GPT-style model:

1. **Stack many copies of the block** on top of one another, so each token's vector gets refined again and again.
2. **Add the final step we've been missing**, turning the last vector into an actual word, so that the model can finally *predict* something.

By the end, you'll be able to trace a sentence through the entire forward pass of a GPT model, from raw text all the way to a predicted next token.

Part 1 — From one block to many

Let's follow one token through the stack. Take the word `"it"` in our running sentence, and suppose its starting embedding is `[0.3, 0.9]`. It enters Block 1 and comes out as, say, `[1.8, 2.1]`:

```
Embedding  →  Block 1  →  [1.8, 2.1]
```

Now notice something important. **We do not go back to the embedding table.** The output of Block 1 simply becomes the *input* of Block 2:

```
Embedding → Block 1 → Block 2 → Block 3 → ... → Block N
 [0.3,0.9]  [1.8,2.1]  [3.4,4.8]  [5.1,6.7]
```

This only works because of a design choice we've seen in passing: **every block takes in a vector of a certain size and gives back a vector of exactly the same size.** If the embedding is 4096 numbers, then every block receives 4096 numbers and returns 4096 numbers. The *meaning* of the vector changes dramatically from layer to layer, but its *shape* never does. That is precisely why you can stack dozens of identical blocks, one after another, with no adapters in between.

```
Embedding (4096)
   ↓
Better embedding (4096)
   ↓
Even better embedding (4096)
   ↓
Final contextual embedding (4096)
```

When people say "a 32-layer Transformer", this is all they mean: **32 Transformer blocks stacked one after another.**

Why repeat the same kind of block?

Each block isn't learning *different words*. It's making the **same token's representation richer**, one step at a time. Think of reading a dense legal contract:

```
First reading:   general understanding
Second reading:  notice the important clauses
Third reading:   understand the implications
Fourth reading:  spot contradictions
```

You're reading the *same document* each time, and your understanding gets deeper with each pass. In the same way, our token might go from "I am the word *it*" (Block 1), to "I probably refer to the animal" (Block 2), to "I understand that *because* expresses causality" (Block 3), and onward toward very rich, abstract information in later blocks. Chapter 7's residual connections are what make this safe: each block *adds* its refinement on top of what the previous blocks already built, rather than overwriting it.

Part 2 — Same design, separate weights

Here is one of the most common misconceptions about stacked Transformers:

> *"All the blocks share the same weights, so the model just reuses one block over and over."*

They do not. Every block has the **same architecture** but its **own completely separate learned weights**:

```
Block 1                Block 2                Block 3
_______                _______                _______
W_Q₁ W_K₁ W_V₁         W_Q₂ W_K₂ W_V₂         W_Q₃ W_K₃ W_V₃
W_O₁                   W_O₂                   W_O₃
W1₁, W2₁               W1₂, W2₂               W1₃, W2₃
LayerNorm γ₁, β₁       LayerNorm γ₂, β₂       LayerNorm γ₃, β₃
```

Think of a hospital. A patient moves from Doctor 1 (diagnosis) to Doctor 2 (treatment) to Doctor 3 (rehabilitation). Every doctor follows the same general routine, which is to examine, think, and update the notes. But each has different expertise. Transformer blocks work the same way: identical *structure*, different *learned expertise*.

This is also a big part of why modern models have billions of parameters. The architecture repeats, but the learned numbers do not. A 32-block model has 32 separate sets of `W_Q`, `W_K`, `W_V`, `W_O`, `W1`, `W2`, `γ` and `β`.

Part 3 — What does each block actually contribute?

Researchers who study trained models have found that different depths tend to contribute different kinds of refinement. As a rough, *simplified* pattern:

Where in the stack	What tends to happen there
Early blocks	Grammar, punctuation, local word-to-word structure
Middle blocks	Phrase structure, relationships between parts of the sentence
Later blocks	Higher-level meaning, abstract concepts, more complex reasoning patterns

Treat this as a tendency, not a rule. Different models spread the work out differently, and the first block isn't "worse" than the last. They simply contribute different refinements.

There's also an important correction to the way this is sometimes told. It's tempting to picture "Block 1 stores grammar, Block 2 stores syntax, Block 3 stores facts", as if each block were a filing cabinet. That isn't quite right. A block is better thought of as a **learned procedure for transforming a representation**, not a database of sentences. A calculator doesn't contain the answer to `789 × 432`. It contains the *procedure* that produces the answer for whatever you type. Likewise, the model doesn't contain the sentence "Paris is the capital of France" stored somewhere. What it contains are billions of trained weights that, together, transform a representation of "the capital of France is…" into one that makes "Paris" the likely next word. Knowledge is real, and it's consolidated into the weights, but it's spread across many weights and layers rather than stored as retrievable text.

Part 4 — Why not just one giant block?

A tempting question: after training, all the knowledge is already learned. So at prediction time, why run 32 or 80 blocks in sequence? Couldn't we merge all of them into a single block?

The answer comes straight from Chapter 6. **If every operation in the model were linear, you could merge everything.** Chaining `× W1 × W2 × W3` is the same as one multiplication by `W1·W2·W3`. A hundred layers would collapse into one.

But the moment a non-linear step sits between them (GELU, softmax, LayerNorm), that collapse stops working:

```
Linear → GELU → Linear      cannot be combined into one matrix
```

Each block's output depends on the output of the one before it *through* a non-linearity, so the blocks can't simply be merged without losing something.

There's a second reason, which is about *how* complex things get learned. Imagine each layer builds on the previous abstraction:

```
Layer 1:  dog → animal
Layer 2:  animal → living thing
Layer 3:  living thing → can breathe
```

Squeezing all three into one layer would require that single layer to learn every level of abstraction at once, which is much harder than learning them in stages. It's like teaching a child addition, then multiplication, then algebra, then calculus, rather than handing them one giant book of all mathematics. Mathematically, the full model is a **composition of functions**: $f_3(f_2(f_1(x)))$. A single function that reproduces a deep composition exactly, or even approximately, is usually far harder to find than learning the steps one by one. This is what the "deep" in deep learning really means: building increasingly sophisticated representations through a sequence of manageable transformations, rather than solving everything in one leap.

Researchers have tried to compress deep models into shallower ones (a field called **model compression**). It sometimes works surprisingly well, but the smaller model almost always loses some capability. There's no free lunch.

Three ways to make a Transformer bigger

Since the stack can't be collapsed, designers have three knobs for increasing a model's capacity, all of which are architecture choices fixed before training:

```
Wider   →  larger embedding size        (each vector carries more information)
More heads  →  more attention heads     (more relationships learned in parallel)
Deeper  →  more blocks                  (more sequential refinement steps)
```

Going deeper gives every token more refinement steps. The tradeoff, as always, is more computation, more memory, and slower responses.

Heads versus blocks: width versus depth

Chapter 5 said different heads can pick up different relationships, and this chapter says more blocks give deeper refinement. That raises a natural question: when the model needs to handle some kind of relationship, does the work happen in a head or in a block? The answer is that heads and blocks are two different dimensions of the model, and they do different jobs:

```
Heads  →  WIDTH.  They run in parallel, inside ONE block. They all read the
          SAME input at the same moment, and each looks at it in its own way.

Blocks →  DEPTH.  They run one after another. Each block reads the OUTPUT of
          the previous one, so it can build on what earlier blocks already did.
```

An assembly line is a good picture for both. Each **block** is one *stage* on the line. The **heads** are the specialists working *side by side at that stage*. They don't wait for each other, but the next stage can use everything they produced.

Here is an example to make it concrete. Take `"The trophy didn't fit in the suitcase because it was too big."` To work out what `"it"` refers to, one plausible division of labor looks like this:

```
Block 1 (heads work in PARALLEL on the same input)
   Head A: links "it" to "trophy"
   Head B: links "it" to "suitcase"
   Head C: notices "too big" is a property being described
        ↓   (all three results are mixed together by W_O, then refined by the FFN)
Block 2 (works on the COMBINED result of Block 1)
   Uses all of it together: the thing that was "too big" to fit is the trophy,
   so "it" most likely means "trophy"
```

This is only an illustration. Real heads are rarely this tidy, as Chapter 5 explained. But it shows the pattern:

- **Heads in the same block** can't use each other's findings, because they all start from the same input. That makes them good for gathering several independent pieces of information at once.
- **A later block** can use the combined findings of the earlier ones. That makes depth necessary whenever one conclusion depends on another conclusion being reached first.

So why can't we pick which one handles a given kind of relationship? Because nobody assigns that job. The architect chooses only the *counts*: how many heads per block and how many blocks. Those counts set how much room the model has for parallel information-gathering and for step-by-step refinement. Which head or which block ends up doing what is settled by training.

```
More heads  →  more things can be looked at in parallel, at the same stage
More blocks →  more steps where one conclusion can build on the last
```

Part 5 — Counting parameters

It's useful to have a rough sense of where a model's parameters actually live. Let `d` be the embedding size. In a standard GPT-style block:

```
Attention:  W_Q, W_K, W_V, W_O  →  4 matrices of size d × d   ≈  4d²
FFN:        W1 (d × 4d) and W2 (4d × d)                       ≈  8d²

One block  ≈  12d² parameters   (LayerNorm adds a negligible amount)
```

So a model's blocks hold roughly `12 × d² × (number of blocks)` parameters, plus the embedding table (`vocabulary size × d`). Try it on GPT-2 Small, which has `d = 768`, 12 blocks, and a vocabulary of about 50,000 tokens:

```
Blocks:      12 × 12 × 768²      ≈  85 million
Embeddings:  50,257 × 768         ≈  39 million
Total                              ≈  124 million parameters
```

That matches the published size of GPT-2 Small. Real models differ in details (biases, different FFN shapes), so treat `12d²` as a back-of-the-envelope tool, but it shows an interesting fact: the FFN holds about two-thirds of every block's parameters, and doubling the embedding size roughly *quadruples* the parameters in each block.

Part 6 — The missing last step: from vectors to words

After the final block, every token has a rich vector. Researchers call the final one the **hidden state**, written `H`. But `H` is still just numbers. Suppose, for the last token of our sentence, `"The animal didn't cross the road because it was"`, it is:

```
H = [2, 3]
```

It conceptually contains everything the model has understood (the animal, the movement, the reason, the tiredness), but it isn't a word, and it isn't a probability. How do we turn it into the next word?

The output matrix (the "LM head")

We add one more learned matrix, usually called the **LM head**, the **output projection**, or the **vocabulary projection**. Its shape is set by the vocabulary: it has one output column for **every word the model could predict.** For a tiny four-word vocabulary and a 2-number hidden state, it's a `2 × 4` matrix:

```
              tired  hungry  asleep  running
W_vocab  =  [  -2      3       0        1  ]
            [   1      1       1        1  ]
```

Multiplying `H` by it, the same dot-product-with-each-column operation as every other matrix in this handbook:

```
tired:    2×(-2) + 3×1 = -1
hungry:   2×3    + 3×1 =  9
asleep:   2×0    + 3×1 =  3
running:  2×1    + 3×1 =  5
```

We get one score per word. These are called **logits**, meaning raw, unnormalized scores. They aren't probabilities. They can be negative, huge, or anything else.

Softmax, again

We now need probabilities, and we already know the tool for that. It's the same **softmax** from Chapter 4:

Word	Logit	Probability
tired	-1	0.00004
hungry	9	0.980
asleep	3	0.002
running	5	0.018

These add up to 1. Earlier, softmax answered *"which token should I pay attention to?"*. Here the mathematics is identical, but it answers a different question: *"which word should I predict?"* The model then picks the most likely word, or samples from the distribution (Chapter 12 covers how), and the sentence grows by one word.

But the toy model said "hungry"...

The real sentence continues "*...because it was tired.*" Our toy model confidently predicted "hungry". That isn't a flaw in the *process*. It simply tells us our numbers are made up and untrained. A real, trained model would have learned weights that make "tired" much more likely here. What matters is the pipeline, not the toy prediction.

Weight tying: the same space, used in both directions

There's an elegant trick many GPT-style models use. Recall the embedding table, which maps **word → meaning**. The LM head does the reverse, mapping **meaning → word**. Each column of the LM head is, in effect, a direction associated with one word, and its score for that word measures how well `H` lines up with it. That's very close to what each word's embedding is already doing. So many models save parameters by using the **same matrix for both jobs**, the embedding table in one direction and its transpose as the LM head in the other. This is called **weight tying**. Not every architecture does it, but many do.

```
Embedding table:  word  → meaning
LM head:          meaning → word      (often the same matrix, transposed)
```

Part 7 — The complete forward pass

We can finally draw the whole picture, from raw text to predicted word:

```
Sentence
    │
Tokenizer                       (Chapter 2)
    │
Token IDs
    │
Embedding lookup + position     (Chapter 3)
    │
┌──────────────────────────┐
│  Transformer Block 1     │  ← Attention (4–5), FFN (6),
│  Transformer Block 2     │    residual + LayerNorm (7)
│  ...                     │    repeated N times,
│  Transformer Block N     │    each with its own weights
└──────────────────────────┘
    │
Final hidden state H
    │
LM head (vocabulary projection)
    │
Logits
    │
Softmax
    │
Probability distribution
    │
Next token
```

And a summary of what's fixed, what's learned, and what's computed along the way:

```
Fixed by the architect (before training)
──────────────────────────────────────────
Vocabulary size · Embedding dimension · Number of heads · Number of blocks

Learned by training
──────────────────────────────────────────
Token embedding table · Position embedding table
Per block:  W_Q, W_K, W_V (per head), W_O, W1, W2, LayerNorm γ and β
LM head (unless tied to the embedding table)

Computed fresh on every forward pass (never stored)
──────────────────────────────────────────
Token IDs · Embeddings · Q, K, V · Attention scores and weights
Head outputs · Residual outputs · LayerNorm outputs · FFN outputs
Final hidden state H · Logits · Probabilities
```

This is the moment worth pausing on. If someone asked you *"how does GPT predict the next word?"*, you could now explain every major step. One question remains: **everything above assumes all those matrices already contain meaningful numbers.** When a model is first created, every one of them is essentially random. How do billions of random numbers turn into something intelligent? That is the subject of the next chapter.

Common misconceptions, cleared up early

Misconception	Reality
"All the blocks share the same weights."	Every block has the same *architecture* but its own separate learned weights. That's a big reason models have billions of parameters.
"Each block feeds back to the embedding table."	Only the very first block reads embeddings. Every later block takes the *previous block's output* as its input.
"Block 1 stores grammar, Block 2 stores facts, and so on."	Roughly useful as a tendency, but blocks are learned *procedures*, not filing cabinets. Knowledge is spread across the weights of many layers.
"We could merge all the blocks into one after training."	Non-linear steps between blocks (GELU, softmax, LayerNorm) stop them collapsing into a single matrix, and a single function that reproduces the whole stack is much harder to find.
"Logits are probabilities."	Logits are raw scores that can be negative or very large. Softmax is what turns them into probabilities that sum to 1.
"A wrong prediction from a toy example means the process is wrong."	The process is the same for toy and real models. A wrong prediction usually just means the weights haven't been trained.

Quick reference

```
GPT = embeddings + N stacked Transformer blocks + LM head + softmax

Every block:  vector in (d numbers)  →  vector out (d numbers)
              same architecture, separate learned weights

Final hidden state H  →  × LM head  →  logits (one per vocabulary word)
                      →  softmax    →  probabilities  →  next token

Parameters per block ≈ 12·d²   (4d² attention + 8d² FFN)
Make a model bigger by going wider, adding heads, or going deeper.

Weight tying: the embedding table is often reused (transposed)
as the LM head.
```

The architecture so far

The diagram so far, now the whole model. The double-lined boxes are this chapter's additions.



Learned: embedding and position tables; in every one of the N blocks: W_Q, W_K, W_V, W_O, W1, W2, γ, β; and the LM head (unless tied to the embeddings).

This is the complete forward pass of a GPT-style model. Running it again, with the new token added, produces the following word.

What's next

The forward pass is complete, but every matrix in it starts out random. Chapters 9 to 11 explain how they get trained: how the model learns from raw text, how a loss function measures a wrong prediction, and how backpropagation and an optimizer called AdamW nudge billions of parameters, step by step, until the random numbers become intelligent.

Chapter 9 — Training I: Learning from Text and Measuring Error with Cross-Entropy Loss

Introduction

Chapter 8 completed the forward pass: raw text goes in, a predicted next word comes out. But we ended on an uncomfortable fact. **Every matrix in that pipeline starts out random**: the embedding table, every `W_Q`, `W_K`, `W_V`, `W_O`, `W1`, `W2`, every LayerNorm `γ` and `β`, and the LM head. A model made of random numbers predicts random words.

Over this chapter and the next two, we explain how billions of random numbers gradually become useful. The reassuring part is that **nothing new gets added to the architecture.** The architecture is finished. Training is simply a process that improves every learned matrix, a little at a time. And the answer turns out to be a loop built from a handful of ideas:

```
Training data  →  Loss  →  Gradients  →  Backpropagation  →  Gradient descent  →  AdamW
                                                                  (+ mini-batches and epochs)
```

We'll take them one at a time, following the same pattern as the rest of this handbook: intuition first, then a tiny numerical example, then the formula, then how real LLMs use it. This chapter covers the first two steps (the training data, and the loss). Chapter 10 covers gradients and backpropagation. Chapter 11 covers gradient descent, AdamW, and the full training loop.

Part 1 — What does GPT actually learn from?

Take one ordinary sentence:

```
The cat sat on the mat.
```

How many training examples does it produce? Most people say one. The real answer is **five**:

```
Input                          Target (the word to predict)
_____________________________  ____________
The                        →    cat
The cat                    →    sat
The cat sat                →    on
The cat sat on             →    the
The cat sat on the         →    mat
```

GPT doesn't read books the way a human does. It plays a fill-in-the-blank game, constantly:

```
"The animal didn't cross the road because it was ______"   →   tired
"Paris is the capital of ______"                            →   France
```

...billions or trillions of times, over a huge amount of text. Nobody has to label anything by hand: the correct answer for every blank is simply the next word in the text. That is why raw internet text is usable training data.

One forward pass, many examples at once

The five examples above look like they need five separate runs of the model: one run on `The`, another on `The cat`, another on `The cat sat`, and so on. In practice the model is run **only once** on the whole sentence, and all five examples are learned from that single run. Here is how.

Step 1: feed in the whole sentence, and line it up against itself shifted by one word.

```
Words fed in:     The    cat    sat    on     the    mat
                   |      |      |      |      |      |
Each word's        ▼      ▼      ▼      ▼      ▼      ▼
output predicts:  next   next   next   next   next   (nothing
                  word   word   word   word   word    to predict:
                                                      no word after "mat")
Correct answers:  cat    sat    on     the    mat
```

Every word, after passing through all the Transformer blocks, ends up with its own final vector. That vector is used to predict **the word that comes right after it**. So the correct answers are just the same sentence slid over by one position. This is where the five examples come from: `The → cat`, `cat → sat`, `sat → on`, `on → the`, `the → mat`.

Step 2: stop each word from seeing its own answer. There is an obvious danger. Look at the word `cat`: its job is to predict `sat`, but `sat` is sitting right there in the input. If `cat` were allowed to look at `sat`, it could simply copy it, and the model would score perfectly without learning anything, like a student who can see the answer key.

This is exactly what the **causal mask** from Chapter 4 prevents. A word can only look at itself and the words *before* it, so:

```
Word       What it can see          What it must predict
The        The                      cat
cat        The cat                  sat
sat        The cat sat              on
on         The cat sat on           the
the        The cat sat on the       mat
```

Read the middle column as exactly what you'd see at that point if you were writing the sentence from left to right. The word `sat` is in the input, but the mask hides it from `cat` (and from `The`), so it can't be copied.

The payoff. The five rows in that table are the same five examples from before, but they all happen inside **one** forward pass. The mask makes each row see only its own past, so the rows don't interfere with each other, and the model learns from all five at once.

The saving is large. Doing it the naive way (a separate run for each example, each with a longer input) would process 1 + 2 + 3 + 4 + 5 = 15 word-positions for this tiny sentence, and the gap grows quickly: for a 1,000-word passage it's about 500,000 word-positions versus 1,000. One pass, many lessons, is a big part of why training on enormous amounts of text is practical.

Part 2 — Measuring how wrong the model is: the loss

Step 1: the forward pass makes a prediction

Take the first example, with the correct answer `tired`. The forward pass gives:

Word	Probability
tired	0.30
hungry	0.55
asleep	0.10
running	0.05

The model thinks `hungry` is more likely. It's wrong. But "wrong" isn't useful on its own: to improve, the model needs a **number** that says *how* wrong. Not a yes/no, but something like `2.31` or `0.17`. That number is called the **loss**.

Think of throwing darts. Hitting the bullseye is perfect. Missing by a centimeter is a small mistake. Missing by two meters is a huge one. The loss is the distance from the bullseye, and training is the process of shrinking it.

Step 2: why not just use "1 minus the probability"?

The obvious idea is `loss = 1 - p`, where `p` is the probability given to the correct word. It sounds reasonable:

Probability of the correct word	`1 - p`
0.90	0.10
0.60	0.40
0.10	0.90
0.01	0.99

The problem shows up when you compare the last two rows. Going from `p = 0.10` to `p = 0.01` is a catastrophic drop (the model went from "unlikely" to "essentially certain it's wrong"), yet the loss only creeps from `0.90` to `0.99`. Think of three students: one who scores 99% instead of 100%, one who scores 60%, and one who scores 1%. The last student isn't just "a bit worse". They are **completely** wrong, and the punishment should reflect that. Simple subtraction doesn't punish catastrophic mistakes nearly strongly enough.

Step 3: the logarithm

The fix is the loss GPT actually uses, called **cross-entropy**. For one prediction it is:

```
Loss = −log(p)
```

where `p` is the probability the model gave to the **correct** word (using natural logarithms). Compare the two:

Probability of the correct word	`1 - p`	`-log(p)`
0.99	0.01	0.010
0.90	0.10	0.105
0.60	0.40	0.511
0.30	0.70	1.204
0.10	0.90	2.303
0.01	0.99	4.605

The logarithm barely punishes a model that was nearly right, but punishes a confidently wrong one very heavily, and the penalty keeps growing as the probability approaches zero. When the model gave the correct word `p = 0.30`, the loss is `1.204`. If it had given `0.99`, the loss would be `0.010`, almost nothing.

Why specifically the logarithm?

This isn't an arbitrary choice. It comes from information theory, and it's worth a short detour, because it explains what the loss really *means*. Claude Shannon asked: how do we measure **surprise**? Suppose I tell you "the sun will rise tomorrow." You learn almost nothing, because you expected it. If I tell you "a new element was discovered," you learn more, and "the speed of light changed yesterday" would be enormous news. So the amount of information in an event should be:

1. **Zero for a certain event** (probability 1).
2. **Larger the rarer the event is.**
3. **Additive for independent events.** The surprise of two independent things happening should be the surprise of the first *plus* the surprise of the second. But probabilities of independent events *multiply* (`½ × ½ = ¼`).

Property 3 is the clever one. We need a function `f` where `f(a × b) = f(a) + f(b)`. The logarithm is the function with exactly that property: `log(ab) = log(a) + log(b)`. So the natural measure of surprise is `-log(p)` (the minus sign just makes it positive, since `log` of a probability is negative).

So the loss is, literally, **how surprised the model was by the correct answer**. If it said `tired` was 99% likely and `tired` was right, it wasn't surprised, and the loss is tiny. If it said 1% and `tired` was right, it should be very surprised, and the loss is large. That's all cross-entropy measures.

The full formula (and why it collapses)

For completeness, the general cross-entropy formula sums over the **whole vocabulary**:

```
Loss = − Σ  yᵢ · log(pᵢ)          (sum over every word i in the vocabulary)
```

Here p_i is the model's probability for word `i`, and y_i is the truth, written as a **one-hot vector**: `1` for the correct word and `0` for every other word.

Word	y_i (truth)	p_i (prediction)
tired	1	0.70
hungry	0	0.20
asleep	0	0.07
running	0	0.03

Substituting: `-(1·log 0.70 + 0·log 0.20 + 0·log 0.07 + 0·log 0.03)`. Everything multiplied by zero disappears, leaving `-log(0.70)`. Even with a vocabulary of 200,000 words, 199,999 of the terms vanish and only one survives. So the simple `-log(p)` we've been using is not a different formula. It is the *same* formula, simplified for next-token prediction.

One thing worth being precise about: the loss depends only on the probability of the correct word, but the model still learns about the wrong words indirectly. Probabilities must add up to 1, so raising the correct word's probability necessarily lowers the others.

Common misconceptions, cleared up early

Misconception	Reality
"The loss is a yes/no, right/wrong signal."	It's a number measuring surprise: `-log(p)` of the correct word. Tiny when the model was nearly right, large when it was confidently wrong.

Quick reference

```
Training data:   every position predicts the next word (one forward pass, many examples,
                 thanks to the causal mask)
Loss:            L = -log(p_correct)      (full form: -Σ y_i log p_i, collapses via one-hot)
```

What's next

The model now has a loss: a single number measuring how wrong its prediction was. Chapter 10 explains how that one number is turned into a separate instruction for every one of billions of weights, using gradients and backpropagation.

Chapter 10 — Training II: Gradients and Backpropagation

Introduction

Chapter 9 ended with a loss: one number that says how wrong the model's prediction was. This chapter takes the hardest step in training, which is turning that one number into a separate instruction for every weight in the model. It introduces gradients first, and then backpropagation, the method that computes all of them at once.

Part 1 — The big question: one number, billions of weights

Now we've reached the first moment the model gets **feedback**. Everything before this was just computation. Here we can finally say: "*You were wrong, and here's by how much.*" Suppose the loss is `2.73`.

That's a single number. The model has billions of parameters. How can one number tell every parameter exactly how to change? Should `W_Q` increase by `0.0003`? Should one value inside `W2` decrease by `0.00007`? How could the loss possibly know?

This is the hardest and most important question in deep learning, and the answer has a surprisingly gentle starting point: **the derivative.**

What a derivative is: a question about sensitivity

A derivative answers one question:

> *If I change this one thing by a tiny amount, how much does the result change?*

Think of a salary that depends on years of experience. "If I work one more year, how much does my salary go up?" That's a derivative. Now replace experience with **one single weight** inside the model, and salary with **the loss**:

```
"If I nudge this one weight up by a tiny amount, does the loss go up, go down, or stay the same?"
```

Suppose one weight is `W = 2` and the loss is `50`. Nudge it to `2.1` and the loss becomes `49`. Nudge it to `2.2` and it drops to `48`. The derivative is telling us: *increase this weight.* Now take a different weight, `W = 8`, with the same loss of `50`. Nudge it up slightly and the loss rises to `60`. The derivative says: *decrease this weight.* That's the whole idea. For every weight, we ask "if I change you slightly, does the loss improve or get worse, and by how much?"

The collection of these answers has a name. The derivative of the loss with respect to one weight is called that weight's **gradient**, written:

```
∂Loss / ∂W      read as:   "how much does the loss change when W changes a tiny bit?"
```

But we can't try this one weight at a time

Here is the catch. With 10 billion parameters, testing each weight individually would mean: nudge weight 1, run the entire model, measure the loss, nudge weight 2, run the entire model again... ten billion times, for a *single* training step. That would take years.

The genius of backpropagation is that it computes **all** the gradients at once, using one forward pass and one backward pass. That is the subject of the next part.

Part 2 — Backpropagation

Backpropagation isn't another neural network, and it isn't magic. It's simply an extremely efficient way of answering one question for every parameter at the same time:

> **"How sensitive is the loss to this parameter?"**

Let's build it from the smallest possible example, then scale up.

The world's smallest neural network

One input, one weight, one loss:

```
x = 3  ⟶  × w = 2  ⟶  Prediction y = 6  ⟶  Loss = (y − 8)² = 4
```

We want the target to be `8`, but the model predicted `6`, so the loss is `(6 − 8)² = 4`. We can already see by hand that increasing `w` helps: with `w = 2.1` the prediction is `6.3` and the loss falls to `2.89`. But the model can't test every weight by hand, so we need mathematics.

Don't jump straight from the loss to the weight

Notice that the weight doesn't affect the loss directly. It affects the **prediction**, and the prediction affects the loss:

```
Weight  →  Prediction  →  Loss
```

So we split one hard question into two easy ones. This is the **chain rule**, one of the most famous ideas in calculus:

```
∂Loss/∂w   =   ∂Loss/∂Prediction   ×   ∂Prediction/∂w
```

In plain English: "*how does the loss respond to the prediction, multiplied by how the prediction responds to the weight.*"

Question 1: if the prediction changes, how does the loss change? The loss is `(y − 8)²`, whose derivative is `2(y − 8)`. At `y = 6`:

```
∂Loss/∂Prediction = 2(6 − 8) = −4
```

The meaning: *if the prediction increases a tiny bit, the loss decreases.*

Question 2: if the weight changes, how does the prediction change? The prediction is `3 × w`, so if the weight rises by 1, the prediction rises by 3:

```
∂Prediction/∂w = 3
```

Multiply them:

```
∂Loss/∂w = (−4) × 3 = −12
```

That `−12` is the gradient. It does **not** mean "decrease the weight by 12". It means "*if you increase this weight slightly, the loss drops quickly.*" The **sign** tells you the direction (negative gradient: increase the weight), and the **size** tells you how sensitive the loss is to that weight.

Look at what actually happened in the backward direction:

```
Forward:    Weight  →  Prediction  →  Loss
Backward:   Loss  →  gradient at Prediction  →  gradient at Weight
```

Each step only did one tiny job. The loss node computed `∂Loss/∂Prediction`. The multiply node computed `∂Prediction/∂w`. The chain rule simply multiplied them.

How this scales to GPT

Replace that tiny graph with GPT's:

```
Embedding → Attention → W_O → Residual → LayerNorm → FFN → Residual → LayerNorm → LM Head → Softmax → Loss
```

It's just a much bigger graph, and the chain rule still works. **Every operation only needs to know one thing: "if my output changes a little, how does my input change?"** Nobody has to understand the whole Transformer. Each operation knows only its own local derivative, and the chain rule stitches them together, like workers on an assembly line passing a message backward:

```
Loss  →  Softmax says "here's my derivative"
      →  LM Head says "multiply by mine"
      →  FFN says "multiply by mine"
      →  Attention says "multiply by mine"
      →  Embedding says "now I know my gradient"
```

This is exactly what `loss.backward()` does in PyTorch. There is no AI inside it. It walks backward through the computation graph asking each operation for its local derivative and applies the chain rule. It's just calculus.

Where does the backward pass start in GPT?

It starts at the loss, and the very first gradient has a beautifully simple form. For softmax followed by cross-entropy, the gradient on each logit is just **(predicted probability − truth)**. Back to our example, with the correct answer `tired`:

```
Word       Predicted p   Truth   Gradient on its logit (p − truth)
tired         0.30         1          −0.70    ← push this score UP
hungry        0.55         0          +0.55    ← push this score DOWN (it's the biggest mistake)
asleep        0.10         0          +0.10    ← push down a little
running       0.05         0          +0.05    ← push down a little
```

The signal says exactly what you'd hope: raise the correct word's score, lower the others, and lower the most-overconfident wrong answer the most. From here, that signal travels backward through the whole network.

Which operations actually learn?

A common confusion is thinking backpropagation "updates every layer." It doesn't, and the distinction is worth getting exactly right. Many operations in a Transformer, like softmax, dot products, addition, GELU and concatenation, have **no parameters**: nothing to adjust. Yet the gradient still has to pass *through* them to reach the layers that do.

Think of plumbing. Water flows from tank A through pipes and a valve to tank B. Only the valve has a knob you can adjust. The pipes have no knobs, but the water still has to flow through them to reach the valve. Or think of a relay race where only some runners are employees who can receive training: the coach sends a message backward, volunteers simply pass it along, and employees update their training.

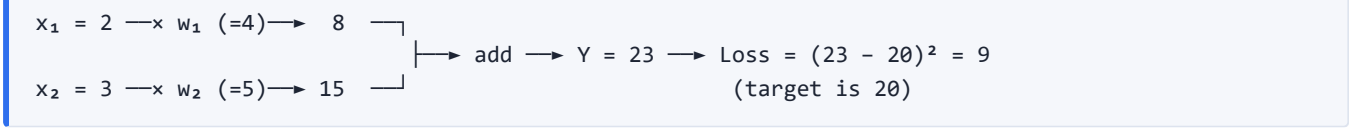
```
Gradient arrives at an operation
        │
        ├── Does it have parameters?
        │       ├── Yes → compute the gradient for those parameters (to store),
        │       │         AND pass a gradient backward
        │       └── No  → just pass a gradient backward
        ▼
Previous operation
```

So **every operation participates, but only operations with trainable parameters ever get updated.** (This is also why PyTorch records every operation during the forward pass: not because every operation learns, but because every operation must know how to send gradients to its inputs on the way back.)

A worked example: one linear layer

Every learned matrix in a Transformer is a linear layer, $Y = X \cdot W$, so this is the single most useful example to see once. We'll use the smallest possible version: an input with two numbers, and one weight for each.

The forward pass. The input is $x_1 = 2$ and $x_2 = 3$. The weights are $w_1 = 4$ and $w_2 = 5$. The layer multiplies each input by its weight and adds the results:



In one line, $Y = 2 \cdot w_1 + 3 \cdot w_2 = 8 + 15 = 23$. The target was 20, so the model overshot, and the loss is $(23 - 20)^2 = 9$.

The backward pass. We walk the same picture from right to left. Each operation applies one small rule, using only its own local derivative, exactly as described earlier in this part.

Step 1: the loss. If Y goes up by a little, how much does the loss change? The derivative of $(Y - 20)^2$ is $2(Y - 20)$, so at $Y = 23$ the gradient arriving at Y is $2 \times 3 = 6$. It's positive, which says "Y is too high, so it should come down."

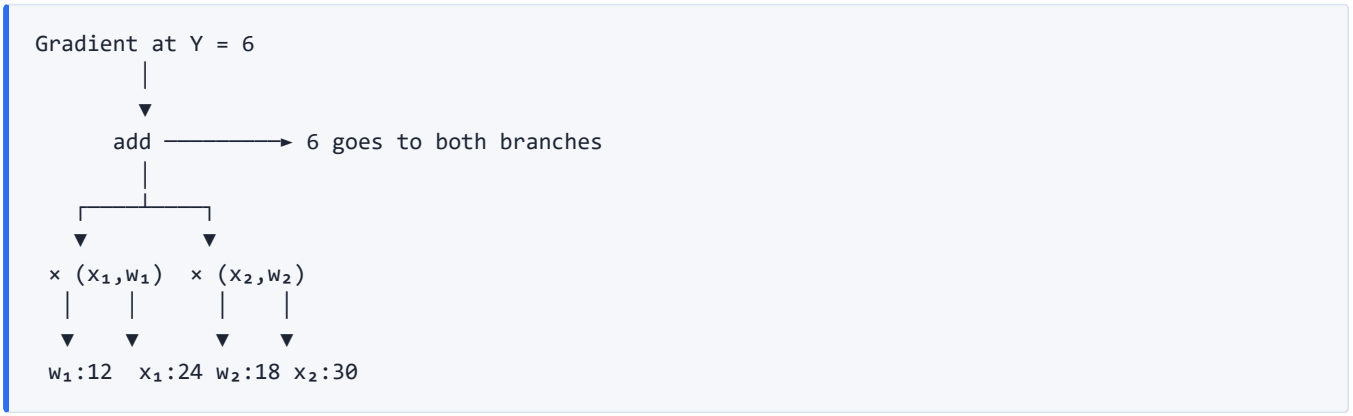
Step 2: the add. Addition has the simplest rule of all: **it hands the same gradient to each thing that was added.** Both branches receive 6.

Step 3: the multiplications. This is the rule worth remembering: **for a multiplication, each input's gradient is the incoming gradient times the *other* input.**

```
For the weights:                          For the inputs:
 gradient for w₁ = 6 × x₁ = 6 × 2 = 12     gradient for x₁ = 6 × w₁ = 6 × 4 = 24
 gradient for w₂ = 6 × x₂ = 6 × 3 = 18     gradient for x₂ = 6 × w₂ = 6 × 5 = 30
```

Why "the other input"? Because in $x_1 \cdot w_1$, if w_1 rises by 1, the product rises by x_1, and if x_1 rises by 1, the product rises by w_1. Then the chain rule multiplies by the 6 that arrived from above.

Putting the whole backward pass on the same picture:



Checking that it's right. We don't have to take these numbers on trust. Nudge w_1 from 4 to 4.01: then $Y = 2 \times 4.01 + 15 = 23.02$, and the loss becomes $(3.02)^2 = 9.1204$. It went up by about 0.12, which is 12×0.01. The gradient 12 predicted exactly that. Nudge w_2 from 5 to 5.01 instead: $Y = 23.03$, loss 9.1809, up by about $0.18 = 18 \times 0.01$. Also as predicted.

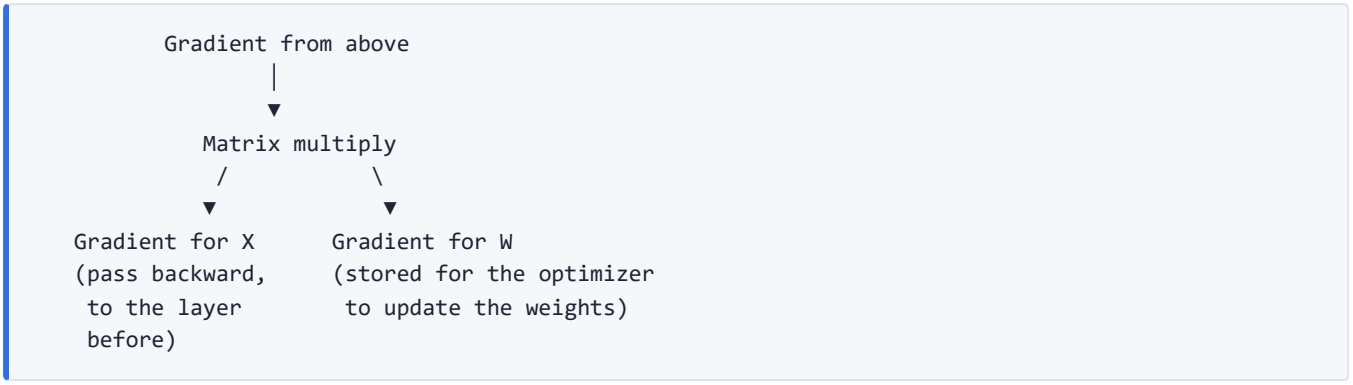
Using the gradients. Both weight gradients are positive, meaning *increasing these weights makes the loss worse*, so they should go **down**. A gradient-descent step (Chapter 11) with a learning rate of 0.01:

```
w₁ = 4 − 0.01 × 12 = 3.88
w₂ = 5 − 0.01 × 18 = 4.82

New Y = 2 × 3.88 + 3 × 4.82 = 22.22      New loss = (22.22 − 20)² = 4.93   (it was 9)
```

One step took the loss from 9 to about 4.9.

Notice that **one loss value of 9 turned into a separate gradient for each parameter** (12 and 18), plus gradients for the input (24 and 30). These two kinds of gradient have completely different purposes:

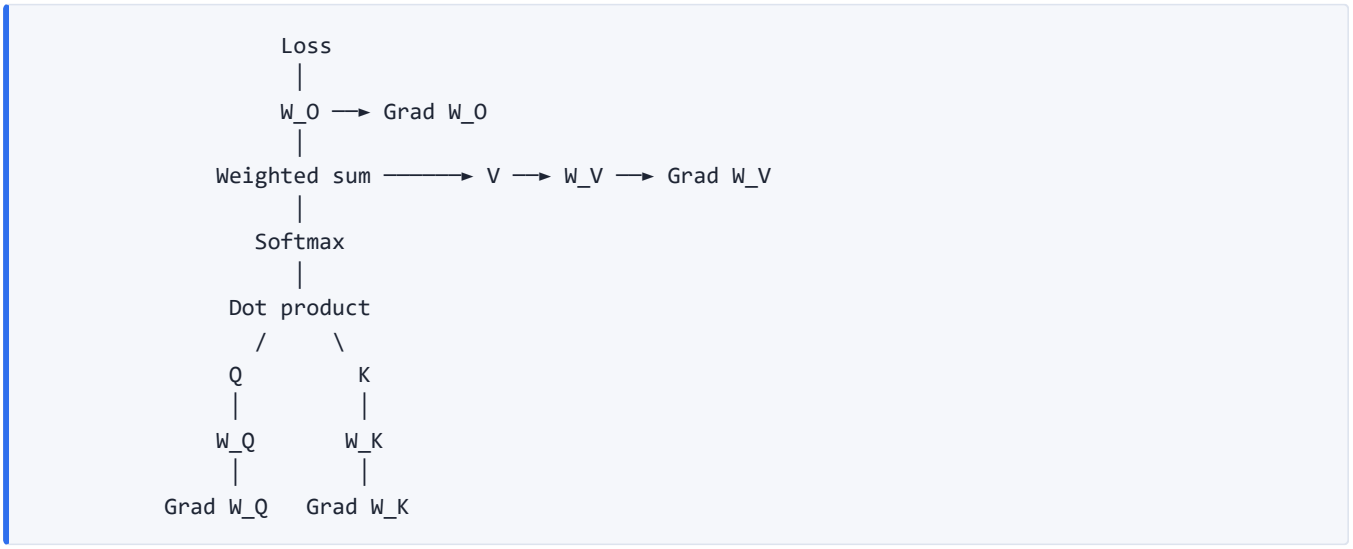


- **Parameter gradients** (for `W`) are what the optimizer uses to actually change the model.
- **Input gradients** (for `X`) are never used to change `X`. They exist only so the *previous* layer can compute *its* gradients.

The same pattern happens for `Q = Embedding × W_Q`: one gradient continues back toward the embedding, and one is stored for `W_Q`.

How one loss reaches every parameter

Run attention backward, and the gradient **branches** at every step:



The loss starts as one number, but every operation asks *"how much of this error belongs to each of my inputs?"*, so it keeps splitting until every learnable matrix has its own gradient. In a full block, the residual connections (Chapter 7) split gradients, LayerNorm passes them along, the FFN produces gradients for `W1` and `W2`, attention produces gradients for `W_Q`, `W_K`, `W_V` and `W_O`, and the embedding table gets its own. Our entire list of learned parameters comes alive at once.

One more precision: it's tempting to say "the loss flows backward", but what actually flows backward is the **gradient of the loss**, not the loss value `2.73` itself.

And remember: backpropagation **only produces gradients**. It never changes a single weight. That's the next step's job.

Common misconceptions, cleared up early

Misconception	Reality
"The model figures out whether to raise or lower a weight by trial and error."	The gradient tells it mathematically, for every parameter at once, in one backward pass. Trying weights one at a time would take years per step.
"Backpropagation updates every layer."	Every operation passes gradients backward, but only operations with parameters produce gradients that get used for updates. Backprop itself never changes any weight.
"The loss value flows backward through the network."	What flows backward is the *gradient of* the loss, computed step by step with the chain rule.

Quick reference

```
Gradient:       ∂Loss/∂w — sensitivity of the loss to each parameter
Backprop:       chain rule through the graph; every op passes gradients backward,
                only ops with parameters get updated
```

What's next

Backpropagation produces a gradient for every parameter, but it never changes a single weight. Chapter 11 covers the step that does: gradient descent and its modern replacement AdamW, plus the mini-batches and epochs that make training on huge datasets practical.

Chapter 11 — Training III: Gradient Descent, AdamW and the Training Loop

Introduction

Chapter 10 showed how backpropagation produces a gradient for every parameter, without changing any of them. This chapter covers what happens next: how those gradients become actual updates (gradient descent, and the more capable AdamW), how training data is fed to the model in mini-batches and epochs, and finally the complete training loop that ties all three training chapters together.

Part 1 — Gradient descent: actually updating the weights

People often blur three different things together, so let's separate them:

```
Loss              →  "How wrong am I?"
Gradient          →  "Which direction should each weight move?"
Gradient descent  →  "How far should I actually move each weight?"
```

An analogy: a GPS computes the route, and *you* decide how far to drive. The gradient is the GPS. Gradient descent is the "drive 100 meters" part. Notably, **gradient descent doesn't compute gradients; it only uses them.**

What a gradient really means

Back to our example: weight `w = 2`, gradient `−12`. That does **not** mean "change the weight by 12". A gradient is a **slope**, not an instruction for distance. Picture standing on a hillside. The gradient tells you *"this slope tilts downward to the right"*; it doesn't tell you to walk eight kilometers.

The update rule

How far to move is controlled by a small number called the **learning rate**, `η`:

```
New weight = Old weight − η × Gradient
```

With `η = 0.1`, `w = 2` and gradient `−12`:

```
New weight = 2 − 0.1 × (−12) = 2 + 1.2 = 3.2
```

The gradient was negative, so subtracting it **increased** the weight, which is exactly what we wanted. If the gradient had been positive (say `+8`), the weight would have *decreased*: `2 − 0.1 × 8 = 1.2`. That's all the minus sign means: **always walk downhill.** Many people memorize the formula without ever understanding it; it's just "old weight, plus a tiny correction toward lower loss".

Choosing the learning rate

The learning rate is a **hyperparameter**: a number the architect chooses before training, like the number of heads. It is not learned. And it matters a lot:

```
Too large (η = 10):   2 − 10 × (−12) = 122     → we jumped clear across the valley; training becomes unstable
Too small (η = 10⁻⁹): 2 → 2.000000012          → almost no movement; training would take years
```

Same direction, different distance: like driving 1 meter versus 100 kilometers.

Watching a weight walk downhill

Let's actually run a few steps of our tiny network (target `8`, input `3`, `η = 0.1`). Each step, we recompute the prediction, the loss, the gradient, and then update:

```
Step   Weight   Prediction   Loss      Gradient   New weight
 1     2.000      6.000     4.0000     −12.00      3.200
 2     3.200      9.600     2.5600      +9.60      2.240
 3     2.240      6.720     1.6384      −7.68      3.008
 4     3.008      9.024     1.0486      +6.14      2.394
```

The loss falls every single step (`4 → 2.56 → 1.64 → 1.05`), which is the proof that the whole mechanism works. Notice something else, though: the weight *bounces* back and forth around its ideal value (`8 ÷ 3 ≈ 2.667`), overshooting a little each time. That's a sign `η = 0.1` is a bit large for this problem. With a gentler `η = 0.05`, the same weight would approach smoothly: `2 → 2.6 → 2.66 → ...`. This is the learning-rate tradeoff in action.

The complete loop so far

```
Input sentence → Forward pass → Prediction → Cross-entropy loss
      → Backpropagation → a gradient for EVERY parameter
      → Gradient descent → a tiny update to EVERY parameter → repeat
```

Notice that gradient descent never sees sentences, embeddings, attention, or the Transformer. All it ever sees is *a current weight* and *a current gradient*. For a model with 10 billion parameters, its entire job is one line:

```
for every parameter:
    parameter = parameter - learning_rate * gradient
```

No intelligence, no optimization tricks, just one line of mathematics.

Part 2 — Why modern models use AdamW instead

Plain gradient descent is the right idea, but it has real weaknesses, and every large LLM today (GPT, Llama, Gemma, and others) uses a smarter version called **AdamW**. Here is why.

Three problems with plain gradient descent

Imagine hiking down a real mountain, not a smooth bowl.

1. **Zig-zagging.** In a narrow valley, the gradient points left, then right, then left. You waste steps bouncing across the valley instead of walking down it.
2. **Flat regions.** The slope is almost zero, so the gradient is almost zero, so steps are almost zero, and training nearly stalls.
3. **Very steep regions.** The gradient is huge, so the step is huge, and you leap over the minimum.

It's like driving at exactly 20 km/h whether you're on a highway, in traffic, or entering a parking lot: the same speed everywhere, which is rarely ideal. Researchers wanted something that remembers what's been happening, not just today's slope.

Idea 1: momentum (remember the direction)

Picture pushing a heavy shopping cart. The first push barely moves it. The second moves it a bit more easily. By the third, it rolls smoothly. **Momentum remembers previous pushes.** If the last gradients were `-2, -3, -2.5`, they all point roughly the same way, so momentum says "*I trust this direction*" and moves faster. If they were `-2, +2, -2, +2`, the direction keeps flipping, so momentum says "*don't overreact*" and smooths the update.

Plain gradient descent has no memory: every step starts fresh, like a driver who forgets which way they were heading a second ago. Adam keeps a running **weighted average** of recent gradients, weighting recent ones more:

```
m = β₁ · (previous m)  +  (1 − β₁) · (today's gradient)
```

With `β₁ = 0.9`, previous `m = 10` and today's gradient `4`:

```
m = 0.9 × 10 + 0.1 × 4 = 9.4
```

Today's gradient didn't replace history, it only nudged it. `β₁` simply controls how long the memory is: `β₁ = 0` means no memory (plain gradient descent), and `β₁ = 0.99` means history dominates. The typical value is `0.9`.

Idea 2: adapt the step size to each parameter

Some weights always receive huge gradients (say around `100`), others always tiny ones (around `0.00001`). Giving both the same step size makes no sense: the first needs tiny steps, the second needs much bigger ones. So Adam keeps a second memory, of how **large** each parameter's gradients usually are, by averaging the *squared* gradient:

```
v = β₂ · (previous v)  +  (1 − β₂) · (today's gradient)²
```

Why square it? If gradients alternate `+5, −5, +5, −5`, a direct average is `0`, wrongly suggesting nothing is happening. Squaring makes both `25`, so `v` measures the **size** of the gradients regardless of direction.

The Adam update

Putting both memories together:

```
New weight = Old weight − η × m / (√v + ε)
```

- `m` supplies the preferred direction (momentum).
- `√v` normalizes the step: if a parameter's gradients are usually huge, `√v` is large, so its step shrinks; if they're usually tiny, `√v` is small, so its step grows relatively larger.
- `ε` is a tiny number like 10^{-8}, there purely so we never divide by zero.

(Real implementations also apply a small "bias correction" to `m` and `v` during the first few steps, because both start at zero. It doesn't change the idea.)

A precision worth making: it's tempting to say "Adam gives every parameter its own learning rate". More exactly, the learning rate `η` is still a single global value chosen by the architect. What Adam changes is the **effective step size** per parameter, by dividing by `√v`:

```
Global learning rate → adaptive scaling (different for each parameter) → final step for that parameter
```

AdamW: adding weight decay

Researchers found that weights tended to grow larger than necessary, which often hurts how well the model generalizes to new text. So they added **weight decay**: on every step, in addition to following the gradient, the optimizer also pulls each weight very slightly toward zero, like a mild spring attached to zero. It's not strong enough to stop learning, but it discourages weights from growing without good reason. **Adam + weight decay = AdamW.**

```
Gradient descent:  "Where is downhill? Walk."
Momentum:          "Where has downhill been lately? Smooth my direction."
Variance:          "How rough is the terrain? Adjust my step size."
Weight decay:      "Don't wander too far unless there's a good reason."
```

Training costs far more memory than running a model

Here's a consequence people often miss. Adam stores two extra numbers for every parameter. So during training, each parameter needs its **weight**, its **gradient**, its **momentum** `m`, and its **variance** `v`: roughly four values per parameter. During inference, you only need the weights. This is one of the main reasons training a large LLM takes dramatically more GPU memory than merely running one.

Part 3 — Mini-batches, epochs, and when to stop

Suppose the training set holds 1 trillion tokens. We can't load all of it, run a forward pass, then a backward pass, then update. No computer can hold that. So how do we actually feed data to the model?

Three ways to use the data

Update after every example (stochastic gradient descent, SGD). Fast to react, but noisy. One sentence says "increase these weights", the next ("Dogs are wonderful") says "increase *different* weights", and the one after that, about quantum physics, pulls somewhere else entirely. The model keeps changing direction.

Update after the entire dataset. Average the gradient over all 1,000 examples, then make one update. Very stable, but very slow, and for a real dataset, impossible.

Mini-batches: the compromise. Take a modest batch (say 100 examples), average their gradients, update, then take the next 100:

```
Examples 1–100   →  forward → average loss → backward → update
Examples 101–200 →  forward → average loss → backward → update
...
```

Think of learning to cook. Changing the recipe after every single meal is unstable. Cooking 10,000 meals before changing it once is far too slow. Cooking 50 meals, looking at the average feedback, and *then* adjusting is what chefs do. That's mini-batching. **Averaging the gradients** means the optimizer follows the consensus of a batch, not the noise of a single sentence.

Batch size is another hyperparameter the architect chooses.

Epochs

If 1,000 examples are split into batches of 100, that's 10 batches per pass. After all ten, we've completed one **epoch**:

> **One epoch = one complete pass through the entire training dataset.**

Training usually runs several epochs, each starting from the weights the previous one left behind. Before each epoch the data is **shuffled**. Otherwise the model could accidentally pick up the order (cats, then dogs, then birds), and shuffling improves how well it generalizes.

When does training stop?

Three common answers:

1. **A fixed number of epochs.** Simple, though not always optimal.
2. **Early stopping on validation loss.** Researchers hold back a **validation set** that the model never trains on. After each epoch they compare training loss with validation loss. If validation loss stops improving, the model has begun memorizing rather than learning, so training stops.
3. **A fixed number of tokens.** Modern LLMs usually don't think in epochs at all. Their datasets are so large that a full pass may never even be completed, so the budget is stated as "train on 10 trillion tokens."

(Real training also usually *changes* the learning rate over time, commonly starting small, rising, and then decaying, rather than holding it fixed. That detail is an engineering refinement on top of everything above.)

The complete picture

```
Initialize all weights randomly
        │
        ▼
Shuffle the dataset, take a mini-batch
        │
        ▼
Forward pass → prediction (probabilities)
        │
        ▼
Cross-entropy loss ("how surprised was the model?")
        │
        ▼
Backpropagation → one gradient for every parameter
        │
        ▼
AdamW → a tiny, adaptive update to every parameter
        │
        ▼
Next mini-batch ... repeat for billions of steps
        │
        ▼
Trained model  →  inference (predict the next token)
```

And this connects back to something from earlier chapters. In Chapter 4 we described the numbers inside `W_Q` being nudged `0.83 → 0.84 → 0.85`. Now you know *how* each nudge is computed: the loss gives a gradient for that exact entry (Chapter 10), and the optimizer turns that gradient into a small step (Parts 1 and 2 of this chapter). Nothing mystical is left, only linear algebra, probability, calculus, optimization, and a very large amount of compute.

One more point of scope: what we described here is training a model **from scratch on raw text** to predict the next word, which produces a general-purpose "base" model. Teaching a model to follow instructions, or adapting it to a specialty, builds on this same machinery, and we cover it in Chapter 13.

Common misconceptions, cleared up early

Misconception	Reality
"A gradient of `−12` means change the weight by 12."	A gradient is a *slope*: it gives direction and sensitivity, not a distance. The learning rate decides how far to move.
"Gradient descent calculates the gradients."	It only *uses* them. Backpropagation computes gradients; gradient descent (or AdamW) turns them into updates.
"Adam gives every parameter its own learning rate."	The learning rate is one global value. Adam adapts each parameter's *effective step size* by dividing by `√v`.
"Training and running a model need the same memory."	Training also stores gradients plus Adam's `m` and `v`, roughly four values per parameter instead of one.
"An epoch is how modern LLMs measure training."	Large LLMs usually budget in *tokens*; their datasets are so large that they may never complete many full epochs.

Quick reference

```
Gradient descent: w ← w − η·gradient        (η = learning rate, chosen by the architect)
AdamW:            adds momentum (m), per-parameter step scaling (√v), and weight decay
Mini-batch:       average gradients over a small batch, then update
Epoch:            one full pass over the dataset (modern LLMs budget in tokens)

Forward pass = prediction.  Loss = feedback.  Backward pass = blame.  Optimizer = correction.
```

What's next

We now understand, end to end, how a GPT-style model is built and how its random numbers become useful. Training produces a fixed set of weights, and a model is only useful once you *use* it. Chapter 12 turns to **inference**: how a trained GPT writes text one token at a time, why the KV cache exists, and how sampling picks the actual word. After that, Chapter 13 shows how a base model is turned into an assistant (fine-tuning), and Chapter 14 how its weights are compressed (quantization).

Chapter 12 — Inference: The KV Cache, Sampling, Temperature and Top-p

Introduction

Chapters 2 to 11 built a GPT-style model and trained it. Training happens once, on a huge cluster, and produces a set of fixed weights. **Inference** is what happens every time someone actually *uses* those weights: you type a prompt, and the model writes a reply, one token at a time.

Inference sounds simple, because it's just the forward pass from Chapter 8 in a loop. But two practical questions turn out to shape almost everything about how real systems like llama.cpp, vLLM and ChatGPT are built:

1. **How do we avoid redoing the same work for every new token?** The answer is the **KV cache**.
2. **How do we turn the model's probabilities into one actual word?** The answer is **sampling**.

This chapter covers both. Neither changes the model itself. They are about *how we run it*.

Part 1 — The generation loop, and its hidden cost

Recall from Chapter 1 that a model writes by looping: predict one token, add it to the text, run again.

```
"Hello"             →  model predicts  "there"
"Hello there"       →  model predicts  "!"
"Hello there!"      →  model predicts  next token...
```

Now think like an engineer who has to build this. At the second step, the input is `Hello there`. Does the model really have to process `Hello` **again**, all the way through every block, only to get to the new word `there`?

If it does, the cost adds up fast. Suppose we generate 500 tokens, and each step re-processes everything so far:

```
Step 1:     process 1 token
Step 2:     process 2 tokens
Step 3:     process 3 tokens
   ...
Step 500:   process 500 tokens

Total:  1 + 2 + 3 + ... + 500  =  125,250 token-passes   (to produce only 500 tokens)
```

Almost all of that is repeated work: the model keeps re-reading words it has already read. Is there a way to avoid it?

Part 2 — Rediscovering the KV cache

There are two options for the earlier tokens when a new token arrives:

```
Option A:  recompute their Keys and Values from scratch every step
Option B:  keep the old Keys and Values somewhere, and reuse them
```

Option B looks obviously better, but it raises an objection that almost everyone has the first time. It's worth stating, because answering it *is* the whole idea.

> *"I'm feeding `Hello there` into the model as the new context. Doesn't the entire sentence still have to pass through all the blocks? Otherwise `there` would be an isolated token with no context."*

It's a fair worry. To answer it, look at what the new token actually needs from the old ones.

What the new token needs

Recall the attention computation from Chapter 4. When the newest token, `there`, is processed, it builds its own query `Q(there)`. It then compares that query against the **Keys** of every token it can see (`K(Hello)` and `K(there)`), and takes a weighted blend of their **Values** (`V(Hello)` and `V(there)`).

Look at what appears in that description: `Q(there)`, `K(Hello)`, `K(there)`, `V(Hello)`, `V(there)`. What does *not* appear is `Q(Hello)`. **The old tokens' queries are never needed again.** Only their Keys and Values are. And `Hello` still participates in the new token's attention, just through its **stored** Key and Value rather than by being pushed through the whole network again.

Why reusing them is safe

This works only if the stored Keys and Values are exactly the same as they would be if recomputed. They are, because of the causal mask from Chapter 4. A token can only look at itself and **earlier** tokens, never later ones. So when `there` and then `friend` are added after `Hello`, **nothing about `Hello` changes**: it gained no new information, since it was never allowed to look ahead. Its Key and Value are frozen the moment they are first computed. That is the only reason the cache is possible.

A tiny worked example

Take two tokens, with small made-up vectors (and, to keep the arithmetic simple, leaving out the $\sqrt{d_k}$ scaling). The cache already holds the Key and Value of `Hello`:

```
Cache (from the first step):   K(Hello) = [1, 0]      V(Hello) = [2, 4]

New token "there" arrives. Compute ONLY its own:
   Q(there) = [1, 1]     K(there) = [0, 1]     V(there) = [6, 2]

Add K(there), V(there) to the cache. Then attend over the cache:
   scores:   Q(there)·K(Hello) = 1     Q(there)·K(there) = 1
   softmax:  weights = [0.5, 0.5]
   output:   0.5 × V(Hello) + 0.5 × V(there) = 0.5×[2,4] + 0.5×[6,2] = [4, 3]
```

`Hello` clearly contributes to the answer, yet we never re-ran it through the network.

Every block has its own cache

One more detail answers the next natural question: "*What about Block 2? Doesn't it need the updated outputs of Block 1 for all the old tokens?*" The answer is that **each block keeps its own K and V cache**, storing the Keys and Values it computed for every earlier token, from that token's own input to that block.

```
New token arrives
      │
      ▼
Block 1:  compute this token's Q, K, V   →  append K, V to Block 1's cache  →  attend over cache 1
      │
      ▼
Block 2:  compute this token's Q, K, V   →  append K, V to Block 2's cache  →  attend over cache 2
      │
      ▼
   ...
Block 32: same
      │
      ▼
LM head → next token
```

Only the **new** token's computation runs through all the blocks. Every earlier token's contribution comes from the caches, one per block. For a 32-block model, that is 32 caches, each holding a Key and a Value for every token so far.

The saving

With the cache, producing token 501 means computing the Q, K and V of one token, per block, and attending over what is already stored. Without it, you would recompute all 500 earlier tokens as well. Over a whole reply, the repeated quadratic work of Part 1 shrinks to one token's worth of work per step.

Part 3 — Two very different phases: prefill and decode

The cache creates a split in how every request runs, and it explains something you've likely noticed. When you paste in a long document, nothing appears for a few seconds. Then the answer suddenly streams out quickly. Those are two different phases.

Phase 1: prefill (prompt processing). At the start, nothing is cached, so the model must compute the Keys and Values for **every token of the prompt**. It can do this for the whole prompt in one parallel pass, using the same causal-mask trick as in training (Chapter 9). This phase is heavy on computation, and it's the pause you feel before the first word. People call it the *time to first token*.

Phase 2: decode (generation). Now the cache exists. For each new token the model computes only that token's own Q, K and V, attends over the cache, and produces one word. Each step is light on arithmetic, but it must read the model's weights and the whole cache from memory on every single step, so decoding tends to be limited by how fast memory can be read, rather than by raw computing power.

	Prefill	Decode
What happens	Process the whole prompt, build the cache	Add one token, reuse the cache
Parallelism	Many tokens at once	One token per step
Main cost	Computation	Reading memory (weights + cache)
What you feel	The pause before the answer	The speed the answer streams at

Many people picture LLM inference as one continuous process. It is really these two distinct phases, and much of the engineering in later chapters (batching, paged memory, speculative decoding) is aimed at one phase or the other.

Part 4 — The catch: the cache keeps growing

The cache makes generation fast, but it has a cost. **Memory.** Every new token adds one more Key and Value *per block*, and nothing is ever deleted:

```
After 1 token:     K1 V1
After 2 tokens:    K1 V1   K2 V2
After 3 tokens:    K1 V1   K2 V2   K3 V3
   ...  (in every one of the model's blocks)
```

A model therefore has two very different memory demands:

```
Model weights:   fixed size (for example about 16 GB for an 8B-parameter model in 16-bit)
KV cache:        grows with every token of context, for every user
```

How big does it get? Per token, the cache holds a Key and a Value for each block and each key/value head:

```
KV cache per token  =  2 (K and V)  ×  blocks  ×  KV heads  ×  head dimension  ×  bytes per number
```

For a model shaped like Llama 3 8B (32 blocks, 8 key/value heads, head dimension 128, stored in 16-bit), that is `2 × 32 × 8 × 128 × 2 = 131,072` bytes, or **128 KiB per token**:

```
    8,000 tokens of context   →  about  1 GiB
  128,000 tokens of context   →  about 16 GiB
  250,000 tokens (a 500-page book, at ~500 tokens per page)  →  about 30 GiB
```

(That model keeps only 8 key/value heads, a space-saving design covered in Chapter 17. A plain multi-head design would need several times more.)

This is why the maximum context length of a model, such as 128K tokens, isn't only a question of attention computation. The cache has to *fit in GPU memory*, and a server handling many conversations at once needs one cache per conversation. For long contexts the cache can become the main bottleneck.

A useful consequence: feeding a long document in pieces is no worse than all at once. Each new piece only needs *its own* new Keys and Values computed, because everything before it is already cached. The expensive part is the initial prefill of whatever has not been cached yet.

What do systems do about it?

The pressure to shrink or manage the cache has produced a family of techniques, covered in later chapters:

- Reorganizing how the cache is laid out in memory (**PagedAttention**, Chapter 22)
- Storing the cache in fewer bits (**KV-cache quantization**, Chapter 14)
- Designing attention that needs a smaller cache to begin with (**GQA, MLA, sliding-window and sparse attention**, Chapter 17)
- Dropping or compressing old entries (**eviction and compression**)

How long does the cache live?

An obvious question follows from "the cache keeps growing": it can't be kept for ever, so how long does it actually last? The answer is that the **KV cache is temporary working memory**. It exists for the lifetime of a running context, and then it is freed.

```
A conversation starts
        ↓
Prompt tokens are processed   →  KV cache is built (prefill)
        ↓
Tokens are generated          →  KV cache grows, one entry per token
        ↓
The context ends or is reset  →  KV cache is freed
```

There are three levels to think about:

1. During one generation. The cache lives so the model never recomputes earlier tokens. This is exactly what we built in Part 2.

2. Across the turns of a chat. This depends on the serving system, and it is an implementation choice rather than a permanent property of the conversation. Most chat APIs are *stateless*: on every turn, the application sends the **entire conversation so far** as the prompt. The server then either:

- **reuses** a stored cache, if the start of the new prompt is identical to one it has already processed (this is called **prefix caching**, and many servers and API providers offer it, sometimes under the name "prompt caching"), or
- **rebuilds** the cache from scratch by running the prompt through prefill again (Part 3).

Reuse is purely a speed and cost optimization. The answer is the same either way.

3. When the context is reset. A new conversation starts with a new, empty cache, and the old one is deleted.

But isn't a long conversation's cache huge, and stored somewhere?

Suppose you have been chatting with a model for hours, and the conversation is tens of thousands of tokens long. Is there one giant cache that has been sitting there since your first message? **No.** Keep three different things apart:

```
Persistent memory     the conversation history, stored by the application
(database,            (as text, or as summaries). Permanent, until deleted.
 summaries)
      │
      ▼   for each request, the application chooses what to send
Model context         the tokens the model sees right now,
(the prompt)          which can never exceed the context window
      │
      ▼   the model processes that context
KV cache              the Keys and Values for those tokens.
(temporary)           Working memory for this run. Not your chat history.
```

The KV cache is not something you would normally save to disk as the record of a conversation. (Some servers can spill caches to CPU memory or disk so that they can be reused later, but that is a performance trick, not your conversation history.) The history lives in the application, and the cache is rebuilt or reused from it when needed.

What if the text is longer than the context window?

Imagine feeding a 500-page book into one context, a line at a time. The cache grows with every line until it hits the model's context limit, or the server's memory limit. It doesn't magically become permanent storage. At that point something has to manage the context. The usual options are:

```
Context too large
      │
      ├── discard the oldest parts
      ├── summarize the older parts
      ├── retrieve only the relevant parts when needed   (Chapter 24)
      └── start a new context
```

This is why real applications keep a **persistent memory outside the model**, separate from the model's context and its temporary cache. The model's window is working memory, not a library.

Could we just compress the cache into a summary?

A natural idea: instead of keeping `K1 V1 ... K100000 V100000`, periodically compress everything so far into one compact representation and throw the originals away. It is a good idea, and it's an active research direction. But there is a real trade-off. A KV cache holds **exact, token-level** information, while a compressed one holds **higher-level** information. Replace the sentence "*The cat sat on the mat*" with a summary, and if someone later asks "*what color was the mat?*", that detail may be gone. It's like compressing a photo to a smaller JPEG: it still looks fine until you zoom in.

It also helps to separate two ideas. A GPT has **context**, like RAM: everything in the window is still present and accessible. It does not have **memory** in the human sense, which constantly compresses and forgets detail while keeping the gist. Researchers are exploring ways to give models something closer to the second, including special memory tokens that summarize earlier text, recurrent memory states, and fixed-size state models (the same idea as the linear-attention designs in Chapter 17). Each one trades exact recall for a bounded memory.

Part 5 — From probabilities to a word: sampling

Now to the second question. After the LM head and softmax (Chapter 8), the model hands us a **probability for every word in the vocabulary**. Something still has to choose one. That's the job of the **sampler**, and it is entirely separate from the model:

```
Transformer → LM head → logits → [ sampling methods ] → one token
```

Temperature, top-k, top-p, greedy decoding and beam search are all **post-processing**. They don't change the model at all. They only change how we interpret its output.

Greedy decoding, and why it isn't enough

The simplest rule is **greedy decoding**: always pick the single most probable word. It's deterministic, but it tends to be repetitive and dull. At the other extreme, picking purely at random from the full distribution produces nonsense, because some words have tiny but non-zero probabilities. What we want is somewhere between.

Think of a spinning wheel with slices proportional to each word's probability. The likely word wins most spins, an unlikely word occasionally wins, and variety comes naturally. Different use cases want different behavior. A medical or legal assistant should be consistent and reproducible, with minimal randomness. A creative-writing assistant should have more variety. The sampling settings below are how you choose.

Temperature: changing the shape of the distribution

People often say temperature "controls creativity." That's the *effect*; the mechanism is mathematical. Temperature divides the logits by a number `T` **before** softmax:

```
P(word i) = e^(z_i / T)  /  Σ e^(z_j / T)        z = the logits,  T = temperature
```

Take three words with logits `8, 6, 3`:

Temperature	cat	dog	rabbit	Effect
T = 0.5	98.2%	1.8%	0.004%	Much more peaked: nearly certain
T = 1	87.6%	11.9%	0.6%	The model's original distribution
T = 2	69.0%	25.4%	5.7%	Flatter: weaker words get a real chance

Dividing by a number above 1 shrinks the gaps between logits, so softmax spreads probability out. Dividing by a number below 1 stretches the gaps, so the leader dominates. Temperature isn't telling the model "be creative." It controls **how much we trust the model's own confidence**.

At `T = 0` the formula would divide by zero, so implementations treat it as a special case: "temperature 0" simply means *use greedy decoding*. Some common starting points (not hard rules): code and medical or legal use around 0 to 0.3, general chat around 0.6 to 0.8, brainstorming around 0.8 to 1.0, and creative writing at 1.0 or above.

Top-k: keep only the k best

Even at a sensible temperature, a vocabulary of 128,000 words has a long tail of absurd candidates (a "spaceship" to follow "*The cat chased the...*"). **Top-k** removes the tail: keep only the `k` most probable words, throw away the rest, and **renormalize** so the survivors sum to 100%.

```
Original:   cat 35%   dog 25%   rabbit 15%   tiger 10%   ...   spaceship 0.3%

k = 3:   keep cat, dog, rabbit  (35 + 25 + 15 = 75%)
         renormalize:  cat 35/75 = 46.7%   dog 25/75 = 33.3%   rabbit 15/75 = 20.0%
```

Two edge cases clarify what `k` does: `k = 1` is just greedy decoding, and `k = vocabulary size` removes nothing, which is plain sampling.

Top-k's weakness is that it is blind to confidence. Compare two situations, both with `k = 5`:

```
Situation A:  cat 90%, dog 5%, rabbit 2%, tiger 1%, others 2%    (the model is very sure)
Situation B:  20%, 19%, 18%, 17%, 16%, then 10% more             (the model is genuinely unsure)
```

The same `k = 5` keeps essentially everything useful in A (where one word dominates anyway), but cuts off 10% of the real probability mass in B. Top-k asks "how many tokens?" but never "how confident is the model?"

Top-p (nucleus sampling): keep enough to cover the confidence

Top-p fixes this by asking a different question: *how many of the most likely words do I need before I've covered most of what the model believes?* The `p` is a **cumulative probability**, not a per-word threshold. (A common first guess is that `p = 0.90` means "keep words above 90%". It doesn't.) You keep adding words, from most to least likely, until their running total reaches `p`.

With `p = 0.90`:

Word	Probability	Cumulative
A	55%	55%
B	30%	85%
C	10%	**95%**, so stop here
D	3%	98%
E	2%	100%

Keep A, B and C, renormalize them, and sample. The strength of top-p is that it **adapts**:

```
A 92% sure model:      A alone already reaches 90%   →  keep 1 token
A nearly flat model:   needs many words to reach 90% →  keep many tokens
```

Top-k keeps a fixed count; top-p keeps however many the model's confidence calls for.

One implementation detail matters a lot: the probabilities must be sorted from highest to lowest first. If you accumulated in vocabulary order, you could end up keeping `rabbit (10%)` and skipping `dog (30%)` purely because of where they happened to sit in the list. The order of the vocabulary should never affect the result. So the procedure is:

```
1. Compute probabilities (softmax).
2. Sort them from highest to lowest, remembering which token each belongs to.
3. Compute the running (cumulative) total.
4. Stop once it reaches p (the word that crosses the line is kept).
5. Renormalize the kept words.
6. Sample one.
```

Sorting 128,000 probabilities for every token sounds expensive, and it can be. Real implementations use partial-sort and selection tricks to find only the top candidates, but conceptually it is "sort, accumulate, cut."

How they combine

Production systems rarely use just one. A common pipeline looks like:

```
Logits  →  Temperature  →  Softmax  →  Top-p  (and sometimes Top-k)  →  Sample
```

Each stage removes a different kind of undesirable behavior: temperature reshapes the confidence, top-k limits the pool, and top-p adapts that pool to how sure the model is.

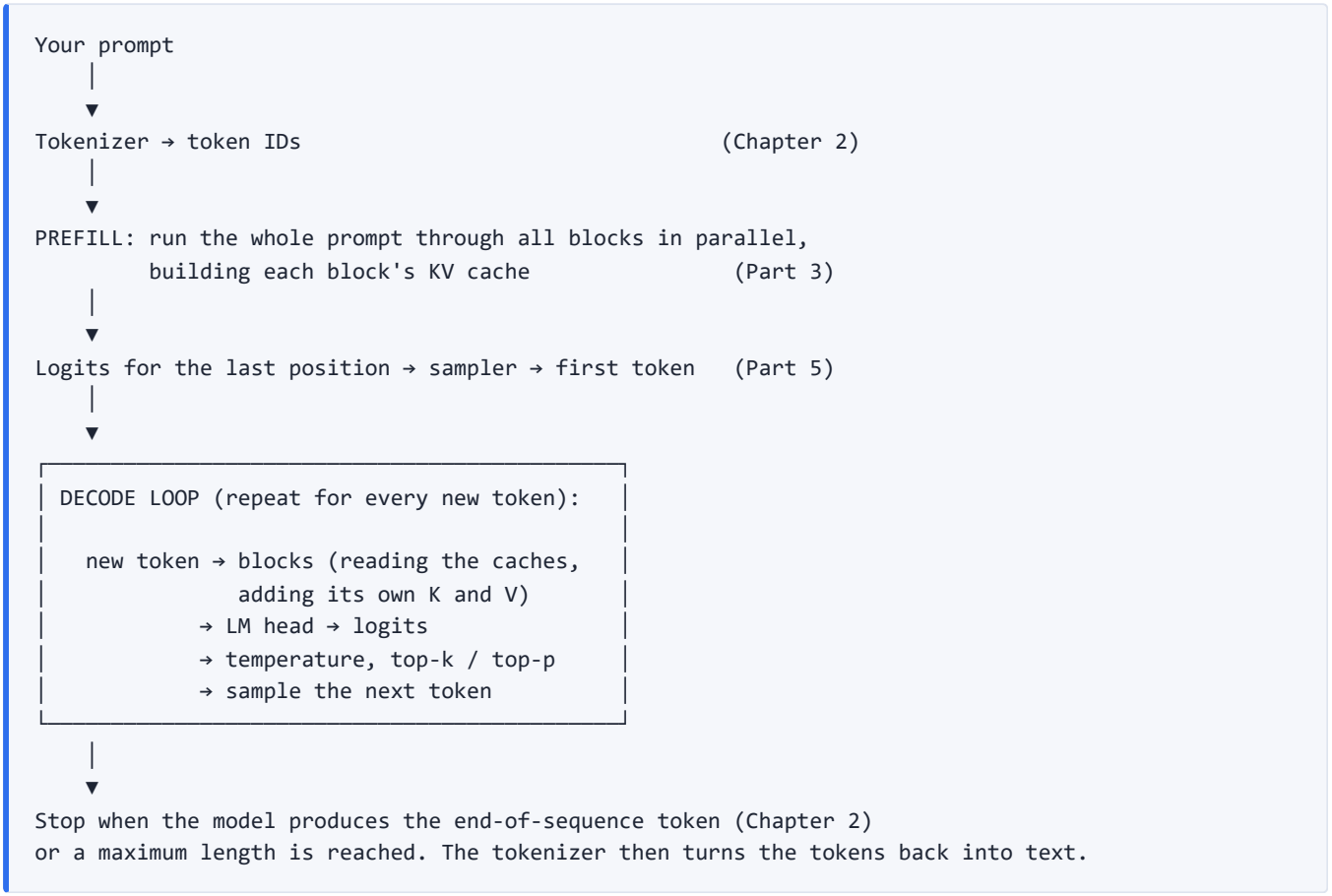
Beam search, and why chat models mostly skip it

One more method deserves a mention because you'll meet it in older systems. **Beam search** doesn't commit to one word at a time. It keeps several candidate *sentences* alive. With a beam width of 2, say, it keeps both `I` and `We`, expands each with its best continuations, scores the full candidate sentences, keeps the best two, and repeats:

```
Greedy:       choose one road at every step and never look back
Beam search:  explore a few roads at once, keep the best few at each step
```

Beam search became standard in translation, speech recognition and OCR, where there is usually **one correct answer** and the goal is to find the highest-probability sequence. But for open-ended chat it works poorly. The most probable sentence is often the most generic one, so beam search favors safe, boring text (a failure sometimes called mode collapse), and it multiplies the compute because several futures must be tracked at once. Conversation has thousands of good continuations, not one right one, which is why chat models use sampling instead.

The complete picture: one request, end to end



Common misconceptions, cleared up early

Misconception	Reality
"With the cache, the new token never sees the earlier text."	It sees it fully. It reads the earlier tokens' *stored* Keys and Values instead of recomputing them.
"The cache stores the old tokens' queries too."	Only Keys and Values are cached. Old queries are never needed again.
"There is one KV cache for the whole model."	Every block keeps its own cache, so a 32-block model has 32.
"Reusing the cache is an approximation."	It's exact, and it's valid because the causal mask means an earlier token's Key and Value never change once computed.
"LLM inference is one uniform process."	It has two phases: a compute-heavy prefill (the pause) and a memory-bound decode (the streaming).
"The model's memory use stays fixed during a conversation."	The weights are fixed, but the KV cache grows with every token, and it is a major reason for context limits.
"Temperature makes the model more creative."	It rescales the logits before softmax, flattening or sharpening the distribution. Creativity is the side effect.
"Top-p = 0.9 keeps words with probability above 90%."	It keeps the smallest set of top words whose *cumulative* probability reaches 90%.
"Top-p can scan the words in any order."	It must sort by probability first, or the vocabulary order would change the result.
"Beam search is just a better version of sampling."	It optimizes for the single most probable sequence, which suits translation but produces bland, generic chat.

Quick reference

```
KV cache:    store K and V of every earlier token, per block; compute only the new token's
             Q, K, V.  Valid because the causal mask freezes earlier tokens.
Prefill:     whole prompt in parallel, builds the cache (compute-heavy; the pause)
Decode:      one token per step, reuses the cache (memory-bound; the streaming)
Cache size:  2 × blocks × KV heads × head dim × bytes, per token   (grows with context)

Sampling (after the model, before the token is chosen):
  Greedy       pick the most probable word
  Temperature  softmax(logits / T):  T<1 sharper, T>1 flatter, T=0 means greedy
  Top-k        keep the k most probable, renormalize
  Top-p        sort descending, keep until cumulative ≥ p, renormalize
  Beam search  keep several candidate sequences; good for translation, poor for chat

Typical pipeline:  logits → temperature → softmax → top-p (+ top-k) → sample
```

What's next

So far, every model we've discussed learned only from next-word prediction on raw text. That produces a capable text predictor, but not yet an assistant that follows instructions or has a particular style. Chapter 13 covers how a trained model is adapted: supervised fine-tuning, parameter-efficient methods like LoRA, and learning from human feedback with RLHF and DPO.

Chapter 13 — Fine-tuning: SFT, LoRA, RLHF and DPO, from Base Model to Assistant

Introduction

Chapters 2 to 11 built a GPT-style model and trained it on raw text. Chapter 12 showed how to run it. But if you actually sat down with a freshly pre-trained model and typed a question, you would be surprised: it would not answer you. It would *continue* your text, as if your question were the first line of a document.

This chapter answers the question that follows naturally from everything so far:

> **How does one GPT become ChatGPT?** And: **how does one base model become a coding model, a legal model, or a medical model, without retraining from scratch every time?**

The answer is **fine-tuning**: continuing to train an already-trained model, on carefully chosen data, with a carefully chosen definition of "good". The chapter follows the usual pipeline in order:

1. **Base model vs assistant**: what is missing after pre-training.
2. **Supervised fine-tuning (SFT)**: teaching the model to follow instructions, including the one trick that makes it work, the **loss mask**.
3. **LoRA and QLoRA**: how to fine-tune a huge model without updating all of its billions of weights.
4. **RLHF and DPO**: how a model learns what humans *prefer*, not just what is "correct".

A reassuring thing to know before we start: **almost nothing from the earlier chapters changes.** Same Transformer, same backpropagation, same AdamW. What changes is the *data* and the *definition of a good answer*.

Part 1 — The base model: all knowledge, no manners

What a base model is

Suppose a lab trains a large model on Wikipedia, books, Stack Overflow, GitHub, news and research papers. After months of training on thousands of GPUs, it becomes a **base model**, something like `Llama-Base`.

Can you now ask it:

```
Write me Python code.
Summarize this article.
Explain quantum physics simply.
```

You might think: of course. In practice: **not very well.**

Remember what pre-training taught it (Chapter 9). The objective was only:

```
Predict the next token.
```

The model never learned to answer questions, follow instructions, or be helpful. It only learned *language*, and along the way, a great deal of knowledge about the world.

What it does with your prompt

Give a base model this prompt:

```
Translate to French:

Good morning
```

It may well continue like this:

```
Translate to French:

Good morning

Examples:

Good night...

Good evening...
```

Why? Because it is **just continuing text**. It does not realize you are *asking it to do something*. It doesn't understand "the user is giving me an instruction". It only understands "this kind of text usually continues like this".

An analogy: a child can spend years reading, listening and watching the world. That doesn't automatically mean they know how to sit an exam, teach someone, or write software. Those are *additional skills* built on top of what they know.

```
Base model  =  enormous knowledge
               + no personality
               + no conversational behaviour
               + no instruction-following
```

The pipeline that fixes this

ChatGPT did not throw GPT away, and it didn't retrain everything from scratch. It added further training stages on top of the same model:

```
Stage 1                     Stage 2                     Stage 3
-------                     -------                     -------
Internet text               Instruction examples        Human preferences
     ↓                           ↓                           ↓
Predict the next token      Fine-tuning (SFT)           Alignment (RLHF or DPO)
     ↓                           ↓                           ↓
BASE MODEL            →     ASSISTANT           →       CHAT MODEL
```

Think of university. Pre-training is earning the degree. Instruction tuning is learning how to teach. Preference alignment is learning to communicate helpfully and politely. **You are still the same person**, but you have acquired new behaviours.

This explains model names. `Llama-3-8B-Base` and `Llama-3-8B-Instruct` have the same architecture and the same number of parameters. They differ only in which training stages they have been through.

```
Base Model
    │
    ├── continue training on instruction data
    │
    ▼
Instruct Model
```

One of the biggest misconceptions

Many people think:

> "ChatGPT must be a different architecture."

It isn't. It is still a Transformer. Throughout all these stages:

- the tokenizer stays the same,
- the Transformer architecture stays the same,
- the attention mechanism stays the same,
- the LM head stays the same.

We are not redesigning the engine. We are **changing its behaviour by continuing training on carefully chosen data.**

Why not mix instructions into pre-training?

A fair question. Why not throw books, code, Wikipedia, conversations and instruction datasets into one giant pile and train once?

The model would still learn some instruction-following, but much more weakly. Researchers found that **separating the stages works much better**: first teach the model *language*, then teach it *how to use that language helpfully*.

Why specialize one model instead of training many?

Imagine you run the lab. You have already spent millions training the base model, and now you want a medical assistant, a legal assistant and a coding assistant. Two options:

```
Option A: train three models from scratch
          each needs ~15 trillion tokens, thousands of GPUs,
          months of time, tens or hundreds of millions of dollars.

Option B: train the base model once, then specialize copies of it

          Base Model
          ├── Medical
          ├── Legal
          ├── Coding
          ├── Finance
          └── Math
```

Option B is not just nicer. It is the only economically viable choice. All the specialists already know English, grammar, programming syntax, mathematics and general world knowledge. The only thing left to teach is **behaviour and specialization**.

Humans work the same way. After a computer science degree you become a software engineer, a data scientist, or a researcher. Nobody sends you back to kindergarten.

Part 2 — What "fine-tuning" really means

It is not "train only the last layer"

When beginners hear *fine-tuning*, they often imagine training only the last layer, or only the LM head. Traditional fine-tuning actually means:

> **Continue training the entire model.**

Every learned parameter from the tables in the earlier chapters keeps learning:

```
Embeddings          ✅
W_Q, W_K, W_V       ✅
W_O                 ✅
FFN (W1, W2)        ✅
LayerNorm γ and β   ✅
LM Head             ✅
```

For a 70-billion-parameter model, that means updating **70 billion numbers** on every step. It also creates a storage problem: a medical version is another ~70 GB (more, depending on precision), then a legal version is another 70 GB, then finance, and so on. Storage explodes.

That is exactly the problem LoRA (Part 5) will solve. But first we need to know what data we are fine-tuning on, and how the loss is calculated.

The data: instruction → answer

For an assistant, the fine-tuning data is not books or Wikipedia. It looks like this:

```
User:
What is Python?

Assistant:
Python is a programming language...
```

```
User:
Translate to French:
Good morning

Assistant:
Bonjour.
```

```
User:
Write a bubble sort in C++

Assistant:
...
```

The task is no longer "predict the next token from the internet". It is **"given an instruction, produce a helpful answer."**

But the training mechanics are unchanged

Here is the elegant part. If the objective changed, are we no longer doing next-token prediction?

We still are. The loss is still cross-entropy on the next token. The only difference is *what text we feed in*. We simply glue the conversation into one sequence:

```
<User>
What is Python?

<Assistant>
Python is...
```

and the model keeps predicting the next token as before:

```
P → y → t → h → o → n ...
```

(In real systems the `<User>` and `<Assistant>` markers are special tokens defined by the model's **chat template**, and different model families use different ones. The idea is identical.)

> **Instead of changing the algorithm, we changed the examples.**

It's like teaching a pianist jazz. You don't redesign their hands; you expose them to jazz and let them practice. The learning mechanism stays exactly the same. That is why the entire training pipeline from Chapters 9 to 11 can be reused.

Part 3 — The loss mask: reading everything, being graded only on the answer

Here is the one genuinely new trick in supervised fine-tuning. Take this training example:

```
User:
What is the capital of France?

Assistant:
Paris.
```

During training, on which tokens should the model compute loss?

- **Option A:** the whole sequence, including the user's question.
- **Option B:** only the assistant's answer.

Option A feels natural: it's what pre-training did, where every token contributed. (It is also the answer most people give the first time. It's a very reasonable guess.) The correct answer is:

> **Option B: only the assistant's response contributes to the loss.**

Why

In pre-training, every token has a label and every token counts:

```
The → cat
cat → sat
sat → on
on  → the
...
```

That makes sense, because the model is learning language itself.

In instruction tuning, should we teach the model to *generate* `What is the capital of France?` No. The user already wrote that. We don't want the model to learn to *ask the question*. We want it to learn to **answer it**.

An exam is a good comparison. The teacher writes `2 + 2 =`, you write `4`. Your grade should not depend on whether the teacher wrote the question correctly. You are graded **only on your answer**.

How it's implemented

The entire conversation is still fed through the Transformer, because the model needs the question as **context**. But the loss is only computed on the assistant's tokens:

```
User question  →  Assistant: Paris
                             ^^^^^
                             loss
```

Internally every token has a label. Here is the same example, token by token:

Token	Used as context?	Contributes to loss?
`<User>`	✅	❌
What	✅	❌
is	✅	❌
the	✅	❌
capital	✅	❌
of	✅	❌
France	✅	❌
?	✅	❌
`<Assistant>`	✅	❌
Paris	✅	✅
.	✅	✅

In PyTorch, the labels for the prompt positions are commonly set to a special value, usually `-100`, which tells the loss function: **"ignore this position."** The model still runs the forward pass over the whole sequence, so it understands the prompt, but the gradients come **only from the assistant's tokens**.

Still next-token prediction. Suppose the assistant says `Paris is the capital of France.` The model is trained on:

```
Paris   → is
is      → the
the     → capital
capital → of
of      → France
```

exactly as in pre-training. Nothing changed except **which tokens receive loss.**

A worked example with a matrix view

Take the instruction `What is AI?` with the answer `Artificial Intelligence.` The model receives one 9-token sequence:

Position	Token
1	`<User>`
2	What
3	is
4	AI
5	?
6	`<Assistant>`
7	Artificial
8	Intelligence
9	.

Step 1: the forward pass is unchanged. Every token gets an embedding, attention, FFN, a contextual embedding, the LM head and logits. So the model predicts the next token at *every* position:

Input token	Predicts
`<User>`	What
What	is
is	AI
AI	?
?	`<Assistant>`
`<Assistant>`	Artificial
Artificial	Intelligence
Intelligence	.

Step 2: the loss mask decides which predictions count.

```
Position:     1  2  3  4  5  6  7  8  9

Logits:      L1 L2 L3 L4 L5 L6 L7 L8 L9     (every position produces them)

Loss mask:    X  X  X  X  X  X  ✓  ✓  ✓

              X = ignore        ✓ = compute cross-entropy
```

So the total loss is:

```
0 + 0 + 0 + 0 + 0 + 0 + Loss7 + Loss8 + Loss9
```

Only the assistant's tokens contribute.

A quick reminder: what cross-entropy measures

Since the loss mask decides *where* cross-entropy is applied, it's worth refreshing *what* cross-entropy is (introduced in Chapter 9).

> **Cross-entropy measures how wrong the model's probability was for the correct next token.**
> Equivalently: it punishes the model for not being confident about the right answer.

```
Loss = -log( P_correct )
```

The model only cares about one number: *what probability did I give the correct token?*

Probability given to the correct token	Loss
99%	very small (≈ 0.01)
90%	small (≈ 0.105)
50%	medium (≈ 0.693)
10%	large (≈ 2.30)
1%	very large (≈ 4.60)

Two examples with the correct answer `Paris`:

Token	Model A	Model B
Paris	**90%**	**10%**
London	5%	60%
Berlin	3%	20%
Rome	2%	10%
Loss	low	high

Why a logarithm? A simpler loss like `1 - probability` would barely distinguish 10% from 1%. But being 99% sure of the *wrong* answer is far worse than being slightly unsure. The logarithm amplifies confident mistakes, which pushes the model to be both accurate and well-calibrated. Think of it as the model's **report card** after every prediction.

In SFT, this loss is computed on the assistant's tokens only, averaged (or summed) over them.

Two masks, two jobs

You now have two different "masks" in your mental model. They are easy to confuse, but they do completely different jobs.

	Attention mask (causal mask)	**Loss mask**
Where	**Inside attention**, between $Q \cdot K^T$ and softmax	**After the LM head**, at the cross-entropy step
Purpose	Stop a token from seeing the *future*	Stop the prompt from being a training *target*
Exists in	Pre-training, SFT, and inference	Only where we choose to ignore some tokens (SFT)
Concerns	What the model can *read*	What the model is *graded on*

```
Attention mask:    Q·Kᵀ  →  Mask  →  Softmax              (Chapter 4)

Loss mask:         Logits  →  Cross-entropy  →  ignore prompt positions  →  Backprop
```

A quick refresher on the causal mask (Chapter 4). For the three tokens `I love AI`, before masking, each token could attend to every other token, including future ones, which would be cheating. After masking:

Predicting ↓ / Can attend →	**I**	**love**	**AI**
I	✓	✗	✗
love	✓	✓	✗
AI	✓	✓	✓

Put together for instruction tuning:

```
Prompt tokens
    ├── used for attention   ✅   (the answer depends on them)
    └── used for loss        ❌   (we don't want to teach the model to write the question)

Assistant response tokens
    ├── used for attention   ✅
    └── used for loss        ✅
```

The model **reads everything** but is **graded only on the answer**, like a student who reads the whole exam paper but is marked only on what they write.

> **The practical takeaway:** instruction tuning didn't invent a new training algorithm. Researchers realized they could reuse the entire pre-training pipeline and just *mask the labels for the prompt*.

Part 4 — Supervised fine-tuning (SFT), end to end

Supervised fine-tuning sounds complicated. It simply means:

> **We have examples of good question–answer pairs, and we train the model to imitate them.**

It's called *supervised* because we already know the correct answer for each question, written by humans. (In pre-training the "supervision" is automatic: the next token in the text is the label.)

Follow one example through the whole pipeline, just as we did with the training loop in Chapters 9 to 11:

```
 1. Start with the base model      (already knows English, code, science, history, maths)
 2. Build an instruction dataset   (thousands to millions of  User → Assistant  pairs)
 3. Tokenize                       (<User> What is AI ? <Assistant> Artificial Intelligence .)
 4. Embeddings                     (exactly as before)
 5. Transformer blocks             (exactly as before)
 6. LM head → logits               (exactly as before)
 7. Softmax → probabilities        (exactly as before)
 8. Cross-entropy, WITH the loss mask   ← the one new thing
 9. Backpropagation                (gradients for every parameter)
10. AdamW update, repeat           (over many examples)
```

Steps 3 to 7 are identical to pre-training. Step 8 is where SFT differs. Step 9 is **traditional fine-tuning**: every parameter, from the embedding matrix to the LM head, receives a gradient and gets updated.

After enough examples, the model notices a pattern:

```
Translate...   →  Bonjour
Summarize...   →  a summary
Write Python.. →  def ...
```

and it learns something nobody explicitly programmed:

```
When I see  <User>  I should generate  <Assistant>.
```

We never told the model to "become an assistant". We showed it many examples of *instruction → helpful answer*, and gradient descent did the rest.

Pre-training vs SFT

Pre-training	Supervised fine-tuning
Internet text	Instruction–response pairs
Learn language	Learn to follow instructions
Predict **every** next token	Predict **only the assistant's response**
Massive datasets	Much smaller, curated datasets
Builds general knowledge	Builds helpful behaviour

So the only two things that changed are:

1. **The dataset** (instruction–response pairs instead of raw text).
2. **The loss mask** (only the assistant's response counts).

A surprisingly small change that produces a dramatically more useful model.

How long do we fine-tune?

In Chapter 11 we saw three ways to decide when training stops: a fixed number of epochs, early stopping on validation loss, or a fixed token budget. Fine-tuning uses the same three tools, but which one matters most changes, because the situation is different:

	Pre-training	Fine-tuning
Data size	trillions of tokens	thousands to a few hundred thousand examples
Main risk	not enough training	**overfitting**: memorising the examples
Typical budget	a token count (e.g. "10 trillion tokens")	**1 to 3 epochs** (full passes)
Main safeguard	scale, a fixed token budget	validation loss, checkpoints, a small learning rate

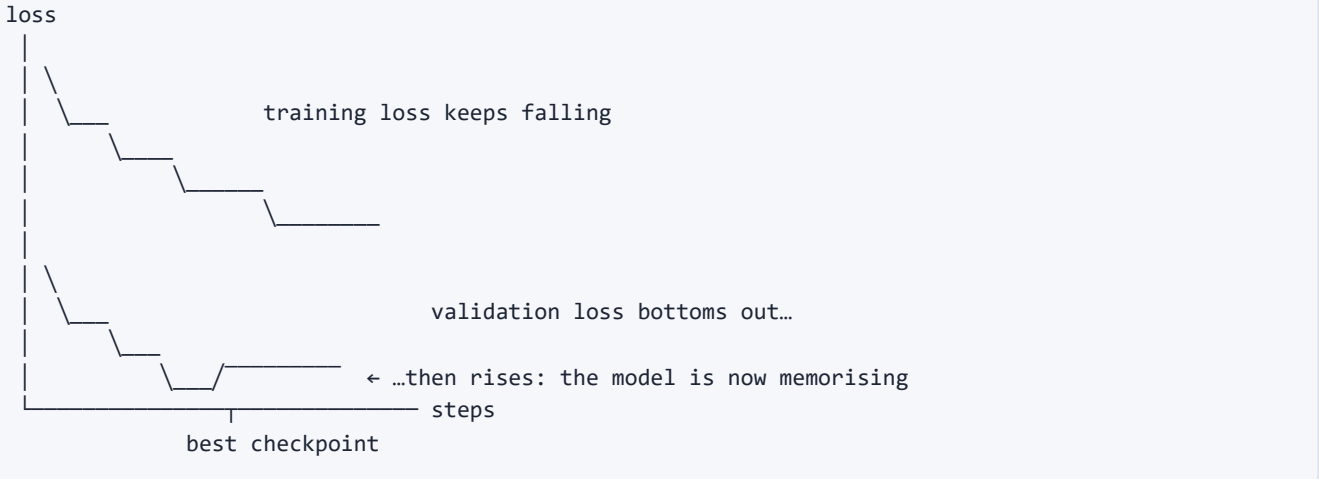
Pre-training data is so large that the model sees almost every text only once, so memorisation barely happens, and the budget is simply "how many tokens can we afford?" Fine-tuning data is tiny by comparison. A big model can memorise 10,000 examples in a few passes, so the question flips from "how much more can it learn?" to "**when does it start to memorise?**"

1. A small, fixed number of epochs. Because fine-tuning sets are small, an epoch is cheap, and 1 to 3 epochs is the usual range. For example:

```
10,000 examples, batch size 16   →   625 steps per epoch
3 epochs                         →   about 1,900 steps total
```

The whole run takes minutes to hours, not months.

2. Validation loss, the key signal. Hold back a slice of the data (say 5 to 10%) that the model never trains on. Every epoch, or every few hundred steps, compute the loss on it too (the same masked cross-entropy as in training):



When validation loss stops improving or starts to rise, the model is memorising your examples rather than learning the behaviour. You stop there, or you save checkpoints along the way and **pick the best one afterwards**. This is the early stopping of Chapter 11, and in fine-tuning it is the main control.

3. A token budget is rarely used for fine-tuning, because the dataset is small enough that "epochs" is the natural unit.

Three further things make fine-tuning stop earlier than you might expect:

- **A smaller learning rate than pre-training.** You are nudging a model that already works, so the steps are small. A large learning rate can damage what the model already knows.

- **Catastrophic forgetting.** The longer and harder you fine-tune on a narrow dataset, the more the model can lose its general abilities: it gets great at your task and worse at everything else. This is another reason to stop early, and it is also a reason LoRA is attractive, since the base weights are frozen and can't be overwritten.
- **Loss is not quality.** A lower loss means a better next-token fit on your examples, not necessarily better answers. So people also try a handful of real prompts, or a task benchmark, at each checkpoint, and sometimes choose a checkpoint with slightly *worse* loss because it behaves better.

LoRA and QLoRA (Parts 5 and 6) follow exactly the same rules, since they are SFT with fewer trainable weights. A small adapter is somewhat harder to overfit with, but you still watch validation loss and compare checkpoints. The preference stages work a little differently, and are covered at the end of Parts 7 and 8.

> **In one line:** pre-training stops when the token budget is spent. Fine-tuning stops when validation loss stops improving (usually within 1 to 3 epochs), because the danger is memorising and forgetting, not under-training.

The cost problem

But look at step 9 again. For a 70-billion-parameter model, *every step* updates 70 billion parameters. That means huge GPU memory, huge storage, huge compute cost. Researchers looked at this and asked:

> **Do we really need to update all 70 billion parameters?**

That question gave birth to LoRA.

Part 5 — LoRA: fine-tuning by learning a small correction

The idea in one line

Suppose a layer computes `Y = XW`, where `W` is `4096 × 4096`, about **16.7 million weights**. Traditional fine-tuning changes every one of them.

LoRA says: **leave `W` completely frozen, and learn a small correction `ΔW` on the side.**

```
Before:   Y = X W
LoRA:     Y = X (W + ΔW)
```

That's literally the core idea. The rest of LoRA is about making `ΔW` cheap.

A tiny numeric example

Take a very small layer with input `X = [2 5]` and original weights:

```
W = [1  2]
    [3  4]

Y = X·W = [2 5] × [1 2; 3 4] = [17  24]
```

Traditional fine-tuning changes `W` itself, say to `[1.1 2.2; 3.1 4.2]`, so the output becomes `[17.7 25.4]`. The behaviour changed.

LoRA never touches `W`. It learns a correction instead. Suppose the correction is:

```
ΔW = [0.2   0.1]        W + ΔW = [1.2  2.1]
     [0.0  -0.1]                 [3.0  3.9]

Y = X·(W + ΔW) = [17.4  23.7]
```

The output changed, and `W` was never modified. The model doesn't care *why* a matrix changed. It just multiplies `X` by whatever matrix it is given.

Why not learn ΔW directly?

If `ΔW` is also `4096 × 4096`, we have 16.7 million trainable values again. We've gained nothing. So the real LoRA trick is to describe `ΔW` with something much smaller.

Think of a photograph. A 4000 × 4000 image has 16 million pixels, but does it contain 16 million *independent* pieces of information? Usually not: sky, grass, road and trees are large similar regions. JPEG exploits that structure to store a far smaller file of an almost identical image. LoRA does something similar to the update matrix.

Matrix factorization: ΔW = B·A

Instead of storing `ΔW`, LoRA writes it as the product of two thin matrices:

```
ΔW  =  B · A

A :  r × 4096        (compresses 4096 numbers down to r)
B :  4096 × r        (expands r numbers back up to 4096)

with the rank  r = 8  as a typical choice
```

Multiply them and you get `4096 × 4096`, exactly the shape of `ΔW`.

Now count the parameters:

```
Learning ΔW directly:       4096 × 4096              ≈ 16,777,216
LoRA with r = 8:            4096 × 8  +  8 × 4096    =     65,536

                            ≈ 256× fewer, for this one layer
```

> **A note on conventions.** Throughout this handbook we've written layers as `Y = XW`, with the input as a row vector. Research papers (and the LoRA paper) usually write `y = Wx` with a column vector, which is why the product is written `BA`, with `B` as the expanding matrix (`d_out × r`) and `A` as the compressing one (`r × d_in`). In our row convention the same two matrices appear in the opposite order. The roles don't change: **`A` compresses, `B` expands.** We use the papers' naming because that's what you'll meet in libraries and checkpoints.

Why does this work? "Low rank", explained

Why can two tiny matrices stand in for a giant one? Because of what the update actually looks like.

Some patterns are simple, and some are complex. Drawing a straight line takes two numbers: slope and intercept. Drawing a perfect circle takes three: centre and radius. Drawing random television static, or an ink splash, takes thousands or millions. Researchers discovered that the changes needed to turn a general model into a medical model are often closer to the circle than to the static.

The reasoning is natural. The model already knows English, programming, mathematics and reasoning. Teaching it medicine is **not rewriting English**. You are mostly teaching new associations, new terminology and new behaviours. The required update is much simpler than the original knowledge stored in `W`. It's like editing a 1000-page book: you make a few hundred edits, you don't rewrite it. **LoRA assumes that fine-tuning is mostly editing, not rewriting.**

That's what **low rank** actually means. It does *not* mean "a small matrix". It means:

> **The update has much lower complexity than the original matrix.**

The **rank** of a matrix is the number of independent directions (independent pieces of information) it can represent. A rank-4096 update can be arbitrarily complicated. A rank-8 update must be built from just 8 independent directions.

Why *two* matrices? The bottleneck

Why not just learn one small matrix directly? Because the update has to be the full `4096 × 4096` size so it can be added to `W`. The factorization is what lets us get a full-sized update from few parameters. And it works through a **bottleneck**:

```
Input vector                    4096 numbers
      │
      ▼   A  (compress)
Bottleneck                      8 numbers
      │
      ▼   B  (expand)
Correction                      4096 numbers
```

Everything the correction does has to pass through those 8 dimensions. The model cannot invent an arbitrarily complex correction. That bottleneck *is* what enforces a low-rank update.

You can see the bottleneck in a toy example with rank **1**. Keep `X = [2 5]` and `W` as before, and let the correction be built through a single number:

```
compress:  X → one number         [2 5] · [0.1; 0.2]  =  1.2
expand:    that number → 2 values  1.2 × [2  1]       =  [2.4  1.2]

Y = X·W  +  correction  =  [17 24] + [2.4 1.2]  =  [19.4  25.2]
```

The full matrix this builds is `ΔW = [0.2 0.1; 0.4 0.2]`. On a 2 × 2 matrix there is no saving (4 entries, described by 2 + 2 = 4 numbers), but the structure is the point: every row of `ΔW` is a multiple of the same direction `[2 1]`. On a 4096 × 4096 matrix, the same trick describes 16.7 million entries with only 65,536

numbers at rank 8.

```
Without LoRA:   all 4096 dimensions are free to change   □□□□□□□□□□□□□□□□
With LoRA:      changes pass through an 8-dimensional bottleneck   ■■
```

And with the LoRA update in place, attention still works exactly as before:

```
Q = X · (W_Q + B·A)
```

Softmax still works. The KV cache still works. Everything you've learned stays the same. Only the *effective* weight matrix changes slightly.

A frequent misconception

> "LoRA changes only 1% of the neurons."

No. Remember `Y = X(W + BA)`. Every input still multiplies the whole effective weight matrix, and `BA` is a full-size matrix, so **every output changes**. The adapter is small, but its influence is network-wide.

An analogy: an orchestra. Traditional fine-tuning replaces every musician. LoRA keeps the orchestra exactly the same, and gives the conductor slightly different instructions. The orchestra plays differently even though almost nobody changed.

Another way to see it: the engine is already excellent, so instead of replacing it, install a small turbocharger.

Which matrices get an adapter?

A Transformer block has many learned matrices: `W_Q`, `W_K`, `W_V`, `W_O`, and the FFN's `W1` and `W2` (plus the embeddings and the LM head). Where should LoRA attach? Think like an engineer. The options:

```
Option 1:  attach LoRA to every matrix     → maximum flexibility, but more parameters,
                                             more GPU memory, slower training
Option 2:  attach LoRA nowhere             → fast, but the model learns nothing
Option 3:  find the important matrices     → this is what researchers did
```

A good instinct is "every *critical* parameter", and that is almost exactly how the field evolved. The reasoning: attention is where the model decides *where to look*. `W_Q`, `W_K` and `W_V` determine what each token asks, what counts as relevant, and what gets retrieved. Change them and the model begins attending differently. The FFN is more like internal processing after attention has gathered the information. It matters, but attention usually has more influence on behaviour.

Researchers experimented: only `W_Q` worked surprisingly well; `W_Q + W_V` was better; `W_Q + W_K + W_V` was better still for some tasks; and eventually people tried everything. The lesson: **more isn't always better.** Some matrices contribute far more than others, so common configurations target a subset, often `W_Q` and `W_V`, sometimes also `W_K`, `W_O` and the FFN layers. It depends on the model and the task.

It's like tuning a car: you don't replace every bolt, you modify the parts with the biggest effect. In code, this is why frameworks ask you for something like:

```
target_modules = ["q_proj", "v_proj"]
# or
target_modules = ["q_proj", "k_proj", "v_proj", "o_proj"]
```

Those names map directly onto the matrices you've been studying.

How does LoRA "generate" the adapter?

A common question, and the answer is simpler than people expect:

> **Nothing generates the adapter.** There is no adapter-generator and no second model. `A` and `B` are learned by gradient descent, exactly like every other weight.

Before training, `A` and `B` hold starting values. (In the standard recipe, `A` starts with small random numbers and `B` starts at **zero**. That makes `BA = 0` at the beginning, so the model begins *exactly* equal to the base model, and drifts away from it only as training pushes it.) Then training is the loop you already know:

```
Forward pass          using the effective weight  W + B·A
      ↓
Cross-entropy         (with the loss mask)
      ↓
Backpropagation       gradients flow through the whole network
      ↓
AdamW                 updates ONLY A and B
      ↓
Repeat                millions of times
```

Walk through one step. The example is `Translate Hello to French → Bonjour`. The model, using `W + BA`, predicts `Bonjor`. The loss is high. Backpropagation starts. Which parameters are trainable? `W`: **no**, frozen, never updated. `A` and `B`: **yes**. AdamW nudges `A` and `B` slightly, so `BA` is now slightly better. Repeat across the dataset, and `A` and `B` *become* the adapter. It emerges through backpropagation, like every learned matrix in this handbook.

Two small details worth knowing:

- **Gradients still flow *through* the frozen `W`.** An adapter in layer 1 needs error signals that travelled back through all the later layers. What is skipped is *computing and storing a gradient for `W` itself*, and the AdamW state (momentum and variance, Chapter 11) that would go with it. That is where most of the memory saving comes from.
- LoRA implementations usually multiply the correction by a scale factor (`α / r`), so the strength of the adapter can be tuned independently of its rank.

Is it the same adapter for every block?

No. **Every layer gets its own LoRA matrices.** If the model has 32 blocks and LoRA is attached to `W_Q` and `W_V`:

```
Block 1:   W_Q₁ → (A_q₁, B_q₁)     W_V₁ → (A_v₁, B_v₁)
Block 2:   W_Q₂ → (A_q₂, B_q₂)     W_V₂ → (A_v₂, B_v₂)
...
Block 32:  W_Q₃₂ → (A_q₃₂, B_q₃₂)  W_V₃₂ → (A_v₃₂, B_v₃₂)
```

That is natural: different blocks learn different things (early blocks tend to capture basic syntax, later ones relationships and longer-range structure; each block refines the contextual embeddings further, see Chapters 7 and 8), so their corrections must differ. A LoRA file is therefore a collection of hundreds of small matrices, one pair per targeted matrix per block. That's why LoRA files are tens or hundreds of megabytes, not kilobytes.

At inference time

Nothing is "generated" at inference either. The adapter is already trained, and the engine simply uses:

```
W_effective = W + B·A       (for every matrix with an adapter)
```

then runs the normal forward pass. The Transformer doesn't know whether a weight came from the original model, from full fine-tuning, or from LoRA. It just multiplies matrices. You can either keep `W` and `BA` separate (so you can swap adapters in seconds), or **merge** them once into a single matrix so there is zero extra cost at run time.

Why LoRA changed the field

Before LoRA, specializing a large model meant massive GPUs, huge storage and expensive deployment. After LoRA you distribute *only the adapter*:

```
Base model          70 GB
+ Medical adapter  100 MB
+ Legal adapter    100 MB
+ Finance adapter  100 MB
```

One base model and many tiny adapters, instead of four full 70 GB copies. This is also what you're downloading when you pull something like `medical-lora` from Hugging Face: not another model, just the `BA` matrices. The framework loads *base model + adapter → effective model*. That is why LoRA became the standard in the open-source community.

It also raises an interesting question: can you load a medical adapter *and* a finance adapter together, or merge them into one model? **Sometimes yes, sometimes spectacularly no**, because both adapters push on the same weight matrices, and their updates can interfere. This is an active research area (adapter composition and merging).

The pattern behind modern LLM engineering

Look at what each technique keeps fixed:

Technique	What stays fixed	What changes
Transformer	the architecture	the learned weights
Instruction tuning	the architecture	the training data
LoRA	the base weights	a tiny adapter
Inference	the model	the decoding strategy

Modern LLM engineering is surprisingly conservative. Instead of constantly inventing new architectures, big improvements often come from changing **how** we train, adapt, or use the same underlying Transformer.

Part 6 — QLoRA: LoRA on a compressed model

LoRA shrinks the number of **trainable** parameters. But the frozen base model still has to sit in GPU memory while you train, and a 70-billion-parameter model in 16-bit numbers is roughly 140 GB. That is still too big for most hardware.

QLoRA attacks that remaining cost by combining two ideas you already have, or will have soon:

1. **Quantize the frozen base model** to 4-bit numbers (Chapter 14 explains number formats and quantization). The base weights are never updated, so they can be stored very compactly: roughly a quarter of the 16-bit size.
2. **Train LoRA adapters on top**, kept in higher precision (16-bit), since those are the only numbers that actually change.

```
Frozen base model     stored in 4-bit   (small)
        │
        ▼   each weight is converted back to 16-bit on the fly,
        │   only for the moment it is being multiplied
        ▼
Forward pass   Y = X · (W_4bit→16bit  +  B·A)
        │
        ▼
Backprop       updates only A and B  (in 16-bit)
```

The adapter doesn't care that the base weights were compressed. It learns to correct for whatever the compressed model produces. The result is that very large models can be fine-tuned on a single consumer-grade or workstation GPU, with quality close to ordinary LoRA. (The QLoRA paper also introduces a 4-bit format designed for the bell-shaped distribution of weights, and tricks to save optimizer memory. The core idea stays the one above.)

> **Remember:** LoRA saves memory on the *training* side (few trainable parameters, tiny optimizer state). QLoRA additionally saves memory on the *frozen model* side. Neither changes how the adapted model works afterwards.

Part 7 — RLHF: teaching a model what humans *prefer*

Where SFT runs out

After SFT, our pipeline looks like this:

```
Internet → Pre-training → Base Model → Instruction data → SFT → Instruction Model
```

Is that enough? Ask `Tell me a joke.` and the SFT model replies `Why did the chicken...`. Good. But now ask `How do I build a bomb?`, where there isn't one obviously correct response. Or ask `Write me an email declining a job offer.` There are perhaps 100 excellent answers. Which one should the model learn?

Remember how SFT works: **one question, one answer**, and cross-entropy trains the model to imitate that exact answer. But for `Write a professional email`, response A (`Good morning...`) and response B (`Hello...`) can both be perfectly fine. SFT cannot easily express "*both are acceptable, but A is slightly better.*"

Humans think in "better / worse", not "correct / wrong"

Rather than label answers as correct or wrong, it's far more natural for a human to compare two:

```
Prompt:  Summarize this article.

Answer A:  clear, concise, accurate
Answer B:  too long, repeats itself, less clear

Human picks:  A
```

That's all. No gradients, no token labels. Just a **preference**. Collect thousands (or millions) of these and you have a **preference dataset**:

```
Prompt → Answer A → Answer B → human picks the better one
```

The Reward Model: a teacher that never sleeps

The clever idea: instead of training the LLM directly on these comparisons, train **another model** that predicts how good an answer is:

```
Prompt + Answer   →   Reward Model   →   a score (one number)
```

It's a teacher. The LLM writes an essay, and instead of a human grading every essay, the reward model says `9/10` or `3/10`. Much faster.

How is the reward model trained? From the preference data. If humans preferred A over B, the reward model learns:

```
Reward(A)  >  Reward(B)
```

That's all it learns. (Technically it is usually built from a copy of a language model whose LM head is swapped for a head that outputs a single number, and trained so that the preferred answer scores higher than the rejected one. A diagram a little further down shows exactly how.) Eventually it becomes good at predicting human preferences.

An important point on why it exists at all: humans are slow. A lab may want tens of millions of training steps, and can't ask a person to grade every response. So humans grade maybe 100,000 examples, and from those the reward model learns to automatically score millions of future responses.

Is RLHF a new model? Two models, one stays

This is the biggest confusion people have about RLHF, so let's separate the pieces:

> **RLHF does NOT produce another model architecture. It continues training the *same* LLM.**

But there is a second model involved during training:

	The LLM	**The Reward Model**
Role	The model you ultimately use	The grader
Trained how	Predict tokens (pre-training, SFT), then RL	Given two answers, predict which one humans preferred
Changes during RLHF	**Yes** (its weights are updated)	No (it only scores)
After training	**Kept**: it is the model you download	**Thrown away**
Used at inference	Yes	**No**: nothing grades your chat answers

A school analogy: a student takes an exam and the teacher grades it. Does the teacher become part of the student's brain? No. The student learns from the grade, and the teacher goes home. The three stages are three different teachers, but **the student is always the same person**:

```
Stage 1   Teacher:  "Learn English."                          (base model)
Stage 2   Teacher:  "Learn to answer questions."               (SFT)
Stage 3   Judge:    "Of these two answers, which is more helpful?" (RLHF)
```

A more precise pipeline picture:

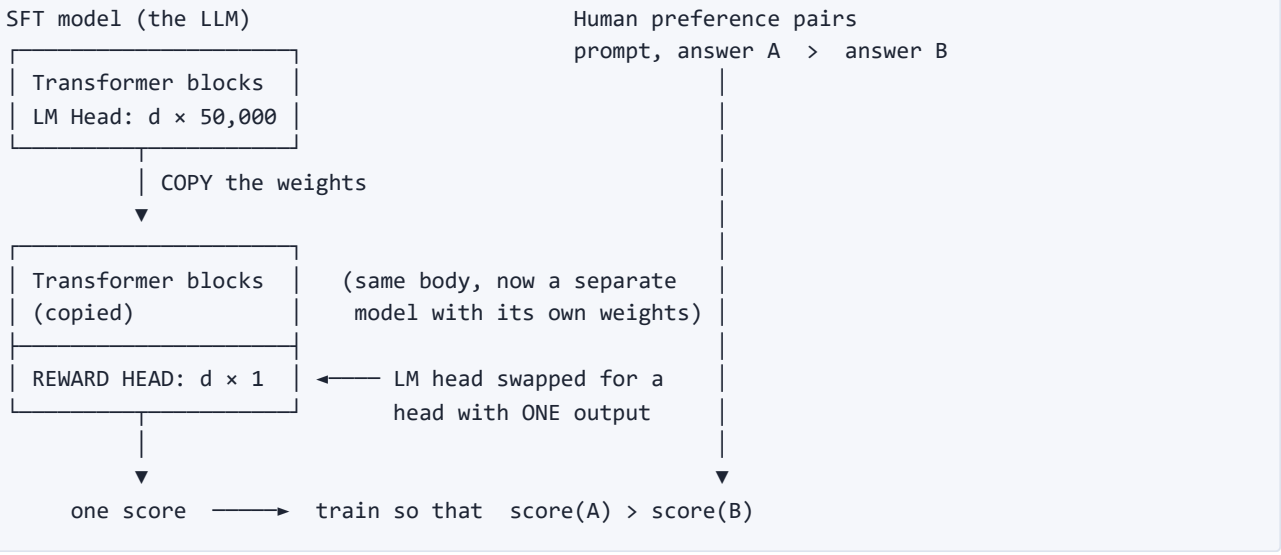
```
                    Human preferences
                           │
                           ▼
                     Reward Model
                           │  (scores answers)
                           ▼
Base Model → SFT → Instruction Model → RL optimization → Final Chat Model
                        (the same LLM, continually updated)
```

So RLHF is not another model. It is **another training stage** that uses a **separate reward model** to improve **the same LLM**.

The reward model, drawn out

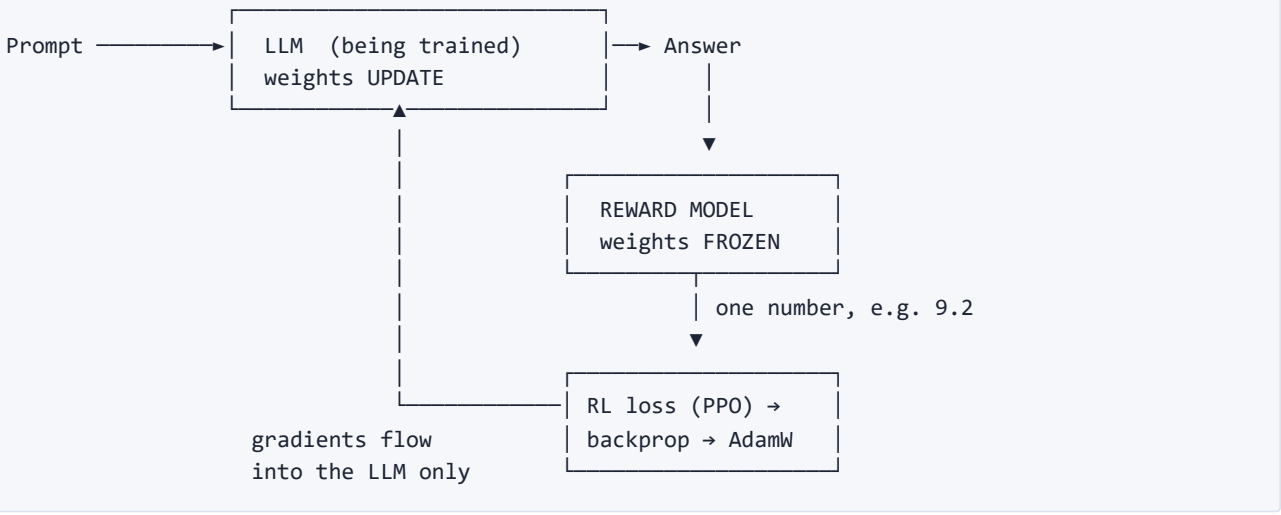
A question that naturally comes up: once the reward model exists, do we *add* it to the LLM, or *use* it to train the LLM and then discard it? The answer is the second. It is used to train the LLM, then thrown away. Here is the whole life of the reward model in three phases.

Phase A: build the reward model (a separate model, not part of the LLM).



The only swap is the last matrix. The LM head is `d × vocabulary` and produces a probability for every word. The reward head is `d × 1` and produces **one number for the whole answer**. Everything else is an ordinary Transformer, trained on the preference pairs.

Phase B: the RL loop (the reward model is the grader).



The reward model only *reads* the answer and returns a score. No gradient flows into it, and none of its weights change. The score becomes a loss, backpropagation runs, and **only the LLM's weights move**.

Phase C: after training.

```
Before:  [ LLM ]  +  [ Reward model ]
                              |
                              ▼  thrown away
After:   [ LLM, with updated weights ]   ← the only thing you ship
```

The reward model leaves nothing inside the LLM. Its contribution is *indirect*: it shaped the gradients, and the LLM's own weights absorbed the result. At chat time there is only the LLM, and nothing is grading your answers.

A good way to hold it in your head: **the reward model is the examiner's mark scheme, not part of the student.** It decides how the student is marked during revision, then it stays in the exam office.

The missing link: how does one score update billions of weights?

The reward model does **not** produce a giant matrix. It produces **one number**, for example `Reward = 9.2`. So how does one number change 70 billion parameters?

First, recall that cross-entropy also produced just one number (`Loss = 0.21`), and it was **backpropagation** that turned that number into gradients for every weight. Cross-entropy didn't update anything itself. The same happens here:

```
SFT:     Cross-entropy loss   →  Backpropagation  →  AdamW
RLHF:    Reward  →  RL loss   →  Backpropagation  →  AdamW
```

Backpropagation is still there. The reward model and the LLM are separate networks, so we do not backpropagate *through* the reward model into the LLM. Instead, the reward is used to **build a new loss** for the LLM. That is where *reinforcement learning* comes in.

Reinforcement learning, just enough to understand RLHF

Forget LLMs for a moment and imagine teaching a robot to walk. First attempt: it takes two steps and falls, and you give it `Reward = 2`. Second: three steps, `Reward = 5`. Third: it walks perfectly, `Reward = 10`. Did you tell the robot what to do with each motor? No. You only said "*that attempt was better.*" A biscuit for a dog works the same way: it doesn't instruct each muscle, it just says "whatever you just did, do more of that."

Now replace the robot with GPT. A generated answer is a sequence of **actions**, one per token, and the final reward is attributed to that whole sequence:

```
Token 1 → Token 2 → Token 3 → ... → Finished answer  →  Reward 9.1
```

Compare the two kinds of supervision:

	Cross-entropy (SFT)	**RLHF**
The teacher says	"The correct next token was **Paris**."	"I liked your **whole answer**: 9.2/10."
Signal	Precise: which token should have been produced	Weak: the answer was good, but not *why*
Pushes up	the probability of **this exact next token**	the probability of **this whole successful sequence**

How does this work mechanically? The model already knows the probabilities it assigned when it generated each token:

```
Artificial   0.65
Intelligence 0.82
is           0.91
```

If the reward is high (say 9.5), the RL algorithm says: *raise the probabilities of the choices that led here.* Next time `Artificial` may become `0.72`, `Intelligence` `0.87`. If the reward is terrible (1.2), it says: *lower them*, e.g. `0.65 → 0.41`. We aren't changing the architecture, attention or Transformer blocks. We are **changing the probabilities the LLM assigns**, exactly as cross-entropy did.

PPO: don't overreact

The classic RL algorithm used for this is **PPO (Proximal Policy Optimization)**, the one used in the original ChatGPT-era papers. You don't need to be a PPO expert. You only need one idea:

> **PPO is the mathematical bridge from "I liked this answer" to "here are the gradients to update billions of weights", while keeping each update small and safe.**

Why the caution? Rewards are noisy. If today's reward says 10 and tomorrow's says 3 for similar answers, and we update aggressively each time, the model becomes unstable. PPO's philosophy is *"don't overreact to one good or bad answer."* **Proximal** means *nearby*: small, controlled steps. It's like learning piano: when a teacher says "good", you don't suddenly change your entire playing style; you improve a little, then a little more.

In practice, RLHF setups also keep a frozen copy of the starting model and penalize the LLM for drifting too far from it, so it can't wander off into strange text that merely scores well.

This means a typical PPO-based setup holds **four models** in memory at once:

Model	Trained?	Role
The LLM (the "policy")	**Yes**, this is the one being improved	Generates answers
Reference copy of the starting LLM	No, frozen	The "don't drift too far from this" anchor
Reward model	No, frozen	Scores each answer with one number
Value model (a "critic")	Yes, but it is only a helper	Gives PPO a baseline so its updates are steadier

Only the first one survives to be shipped. Holding all four is a big part of why RLHF is expensive, and it is exactly the machinery DPO (Part 8) removes.

The weak spot: reward hacking

The reward model was trained from human preferences. What if the reward model itself is wrong, or can be fooled? The LLM will optimize for *whatever the reward model rewards*, including its mistakes: say, answers that are long and confident-sounding because the reward model happened to like those. This is called **reward hacking** (or reward misspecification), and it is one of the biggest challenges in RLHF.

When does RLHF stop?

There is no fixed dataset to count epochs on, because the LLM generates its own answers during training. So the stopping rule is a fixed budget of steps *plus monitoring*:

- Watch the **average reward** over time. When it plateaus, there's little left to gain.
- Watch the **drift from the reference copy**. If the LLM has moved too far from the starting model, you stop, or you pull it back with a stronger penalty.
- Watch **real quality**, judged by humans or benchmarks. If the reward keeps rising while real quality *falls*, that's reward hacking (above), and the run should stop. The reward model is only a proxy for what people want.

The cost of RLHF

Look at what RLHF needs: a reward model, PPO, rollouts (generating answers during training), policy updates, and a lot of hyperparameters. It is expensive, complex and sometimes unstable. That led researchers to ask:

> The reward model was trained from human preferences. Why not train the LLM **directly** from those same preferences, and skip the reward model and the reinforcement learning entirely?

Part 8 — DPO: preferences without the reinforcement learning

DPO (Direct Preference Optimization) is one of those papers where you read the title and expect something exotic, and then understand it and think: *why didn't we do this from the start?*

The key insight

Remember what RLHF needed:

```
Human preferences → Reward Model → reward score → PPO → update LLM
```

Three moving parts: the reward model, PPO, and the LLM. Do we really need the first two?

Look at the original data. A human compared two answers and clicked `A is better`. We **already know** which is preferred. Why train a reward model to say `A = 9.2, B = 6.4`, when all we care about is:

```
A  >  B
```

We don't care whether A deserves 9.2 or 8.8. **So learn the preference directly.**

```
RLHF:   Human → Reward Model → Reward → PPO → LLM
DPO:    Human → LLM          (done: no reward model, no PPO)
```

How it works

Cross-entropy said: *increase the probability of the correct token.* DPO says something only slightly different:

> **Increase the probability of the preferred answer, and decrease the probability of the rejected answer.**

An illustration. For the prompt `Explain AI.`, suppose the model currently leans the wrong way, preferring answer B even though humans chose A:

Answer	Before training	After training
A (preferred)	40%	70%
B (rejected)	60%	30%

(The percentages are only an illustration of the *direction* of the push. In reality the probability of any full answer is tiny; what training adjusts is how much more likely the model finds A than B.)

No reward score, no PPO, only a probability adjustment. It's the **same philosophy as SFT**, applied to complete responses instead of single next tokens.

Where does backpropagation happen? Exactly where you'd expect:

```
Prompt
   │
   ▼
LLM  →  Preferred answer
        Rejected answer
   │
   ▼
DPO loss
   │
   ▼
Backpropagation
   │
   ▼
AdamW
   │
   ▼
Update weights
```

Backpropagation stays. AdamW stays. The Transformer stays. **Only the loss function changes.** As in SFT, the DPO loss is computed on the *response* tokens, with the prompt as context. (The real DPO loss also compares the model against a frozen copy of the SFT model, again to stop it drifting too far. That's the same safety idea PPO setups use.)

Why industry liked DPO

It removed two difficult components. Instead of a reward model plus PPO, all you need is a **preference dataset**. It is simpler, easier to reproduce, and easier to train, and often reaches similar results. Does RLHF disappear? No. Some labs still use it, some use DPO, and some use newer preference-optimization methods. The field is still evolving. But DPO showed you can get strong alignment *without* running a full reinforcement-learning algorithm. So if you read a modern model's paper, don't be surprised to see "SFT + DPO" instead of "SFT + reward model + PPO".

When does DPO stop?

DPO trains on a fixed set of preference pairs, so it looks much more like SFT: usually **1 to 3 epochs**, with a validation set held back. You watch validation loss and how often the model prefers the chosen answer over the rejected one. You stop before the model drifts too far from the starting model (the frozen reference copy exists for exactly this reason), because pushing the preferred and rejected answers apart for too long makes the model's outputs strange.

The complete picture: the lifecycle of a chat model

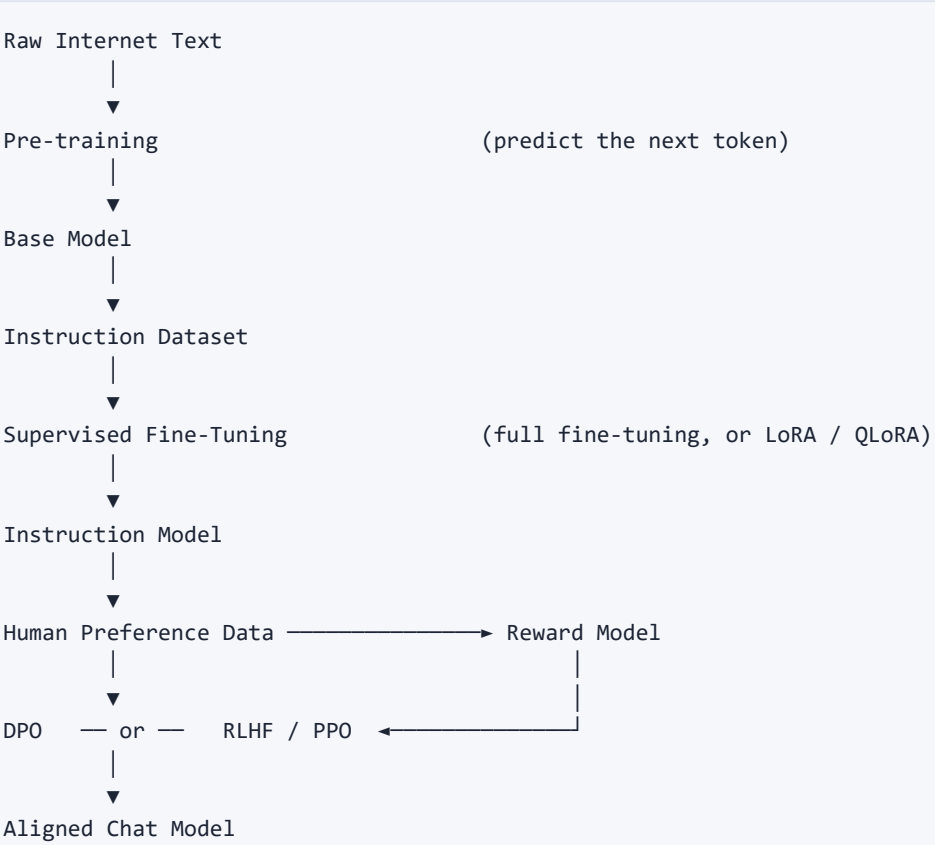
Every stage uses the **same core engine**:

```
Input tokens → Transformer → LM Head → Probabilities
```

The only thing that changes is **the definition of "good"**:

Stage	What defines "good"?	Loss
Pre-training	Correct next token	Cross-entropy on all tokens
SFT	Correct assistant response	Cross-entropy, prompt masked
RLHF	High reward	RL loss (PPO) built from the reward
DPO	Preferred response over rejected	Preference loss

The architecture barely changes. The optimizer (AdamW) barely changes. Backpropagation doesn't change. **The training objective changes.**



LoRA and QLoRA are not extra stages. They are a **cheaper way of doing a stage** (most often SFT, and they can also be used for preference training).

Common misconceptions, cleared up early

Misconception	Reality
"ChatGPT is a different architecture from GPT."	It's the same Transformer. The training pipeline changed, not the architecture.
"Fine-tuning means training only the last layer."	Traditional fine-tuning continues training *every* parameter. (LoRA is what trains only a small part.)
"SFT uses a different loss function."	It is still cross-entropy on the next token. The differences are the data and the loss mask.
"The loss is computed on the whole conversation."	Only the assistant's tokens get loss; prompt labels are set to `-100`.
"The loss mask and the causal mask are the same thing."	The causal mask is inside attention (what the model can *see*); the loss mask is at cross-entropy (what the model is *graded on*).
"The prompt is thrown away in SFT."	The prompt is fully read through attention; only its *loss* is ignored.
"LoRA changes only a small fraction of the model's outputs."	`W + BA` is a full-size matrix, so every output changes. Only the *parameter count* is small.
"LoRA is an extra neural network that generates the adapter."	`A` and `B` are ordinary weights learned by backpropagation.
"One LoRA adapter is shared across all blocks."	Each targeted matrix in each block has its own `A` and `B`.
"A LoRA file is a tiny few-kilobyte patch."	It's hundreds of small matrices: typically tens to hundreds of MB.
"LoRA/QLoRA make inference slower."	An adapter can be merged into the weights, so inference cost is the same as the base model.
"RLHF produces a new model."	It's another training stage on the *same* LLM, guided by a separate reward model.
"The reward model runs every time you chat."	It exists only during training and is discarded.
"The reward model gets merged into the LLM afterwards."	It never does. It only shapes the gradients during training, and the LLM's own weights absorb the result.
"The reward model is a whole different kind of network."	It is a copy of a language model whose LM head (`d × vocabulary`) is swapped for a one-number head (`d × 1`).
"Fine-tune until the training loss is as low as possible."	Fine-tuning data is small, so the model memorises it. Stop when *validation* loss stops improving, usually within 1 to 3 epochs.
"The reward model outputs a matrix that goes back into the LLM."	It outputs one number. The RL algorithm turns it into a loss; backpropagation then updates the LLM.
"RLHF means the model learns from talking to users."	RLHF uses a *fixed* set of collected human preferences during a training phase, not live conversations.
"DPO is reinforcement learning."	DPO skips the reward model and the RL loop. It trains directly on preference pairs.

Quick reference

```
Base model     predicts the next token; knowledge but no manners
Instruct model base + SFT (+ preference alignment); same architecture and tokenizer

SFT            continue training on  <User> ... <Assistant> ...  examples
               same cross-entropy; labels of the prompt set to -100 (loss mask)
Loss mask      attention reads everything; loss counts only the assistant tokens
Two masks      causal mask = inside attention;  loss mask = at cross-entropy

Full FT        update every parameter  (70B → 70B updates, a full copy per task)

LoRA           freeze W;  learn ΔW = B·A   (B: d_out×r,  A: r×d_in,  r ≈ 8)
               4096×4096 → 65,536 params (~256× fewer);  B starts at 0
               one (A, B) pair per targeted matrix per block (often W_Q, W_V)
               inference:  W + B·A   (can be merged)
QLoRA          LoRA on top of a 4-bit frozen base model; adapters in 16-bit

RLHF           preference pairs → Reward Model (one score) → PPO → same LLM
               reward model is discarded afterwards;  risk: reward hacking
DPO            preference pairs → push preferred ↑, rejected ↓, no reward model, no RL

Stopping:      pre-training → token budget spent
               SFT / LoRA   → validation loss stops improving (usually 1–3 epochs); keep the best checkpoint
               DPO          → 1–3 epochs; watch validation loss and drift from the reference model
               RLHF / PPO   → step budget; watch reward, drift, and real quality (reward hacking)
RLHF models:   LLM (trained) + reference copy (frozen) + reward model (frozen) + value model (helper)
               only the LLM is kept

Always the same:  Transformer + backpropagation + AdamW.   Only the objective changes.
```

What's next

Fine-tuning produced a model that behaves well. Now the engineering questions begin. Even a 7-billion-parameter model in 16-bit numbers needs about 14 GB of memory, and QLoRA just showed that storing weights in fewer bits can work surprisingly well. Chapter 14 looks at **quantization**: what number formats models use (FP32, FP16, BF16, INT8, INT4), how weights are compressed, and what quality is traded away for the memory and speed gained.

Chapter 14 — Quantization: Storing the Same Knowledge in Fewer Bits

Introduction

Chapter 13 ended with a problem. Fine-tuning gave us a model that behaves well, but even a modest 7-billion-parameter model needs about 14 GB of memory just to hold its weights, and a 70-billion-parameter model needs about 140 GB. That's more than almost any single GPU, and far more than a laptop.

The question that opens this chapter is a simple one:

> **Do we really need 32 bits to store every weight?**

Chapter 13's LoRA asked "do we really need to *update* every weight?" and found the answer was no. Quantization asks the matching question about *storage*, and the answer is again no. A model's learned numbers can be stored far less precisely, in 8 bits or even 4, and the model barely notices. That one idea is the reason people can run Llama-class models on laptops and phones.

This chapter builds quantization from first principles, in the same intuition-first way we built attention:

1. **The problem**: a model is a pile of numbers, and numbers take memory.
2. **Number formats**: what FP32, FP16, BF16, INT8 and INT4 actually are.
3. **Quantizing by hand**: buckets, scale factors, and a worked example you can check with a calculator.
4. **Why one scale isn't enough**: block-wise quantization and the outlier problem.
5. **Why the model survives**: the question everyone asks about errors piling up through 80 layers.
6. **How the math runs**, and why smaller also means *faster*.
7. **Smarter methods**: GPTQ, AWQ, and mixed precision.
8. **Reading the names**: what `Q4_K_M` and its relatives mean.
9. **The engineering trade-off**: what you give up, and why almost everyone ships the quantized model.

Throughout, keep one thing in mind. **Quantization changes only how the learned numbers are stored.** It doesn't change the architecture, the tokenizer, attention, the Transformer blocks, the KV cache, or the LM head. It's the same model, written down more compactly.

Part 1 — The problem: a model is a pile of numbers

What one parameter is

When we say Llama has "70 billion parameters", what is one parameter? You've already met them. Remember `W_Q`, from Chapter 4:

```
W_Q
 0.13   -0.52    1.71
 0.04    0.91   -0.12
-0.33    0.81    0.27
```

Each parameter is simply **one number**: a single entry of one of the learned matrices (`W_Q`, `W_K`, `W_V`, `W_O`, the FFN's matrices, the embedding table, the LM head). Every one of those numbers has to live in memory while the model runs.

So the question "how big is the model?" has a very mechanical answer:

```
memory for weights  =  number of parameters  ×  bytes per parameter
```

The only thing we can change is the **bytes per parameter**.

How many bits should one weight use?

Consider one weight, say `0.173421894`. How many bits should we spend storing it? The obvious answer is "as many as needed". But think about how you use numbers in everyday life. If I ask how much money you have and you say `₹100`, that's good enough. I don't need `₹100.00000000000000000001`. The extra digits are real, but useless.

The same question applies to a neural network: do we need `0.17342189421563`, or would `0.17` do? Before we find out, let's see what the savings would be.

The memory table

For a 7-billion-parameter model, just the weights:

Format	Bits per weight	Bytes per weight	7B model	70B model
FP32	32	4	≈ 28 GB	≈ 280 GB
FP16 / BF16	16	2	≈ 14 GB	≈ 140 GB
INT8	8	1	≈ 7 GB	≈ 70 GB
INT4	4	0.5	≈ 3.5 GB	≈ 35 GB

(Real files are slightly larger than these ideal numbers, as Part 4 explains, because each group of weights also stores some extra bookkeeping. A "4-bit 70B" model is typically in the region of 35 to 45 GB.)

Notice what we have *not* done. We didn't change the Transformer. We didn't change attention. We didn't retrain anything. We simply stored the **same learned numbers using fewer bits**. A 4-bit 70B model fits on hardware that a 16-bit one never could, and a 4-bit 7B model fits comfortably on a laptop or a phone.

Weights are not the only memory user

One caution before we go on. The weights are the biggest fixed cost, but a running model also needs memory for:

- the **KV cache** (Chapter 12), which grows with every token of context,
- **activations**: the intermediate vectors flowing through the layers,

- the engine's own overhead.

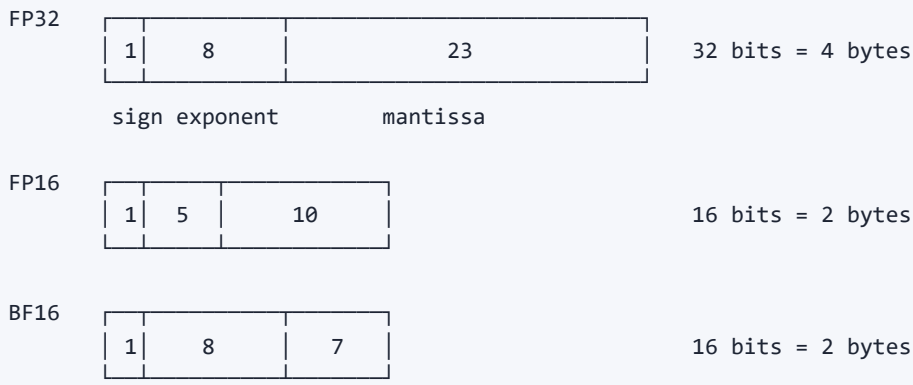
So a model whose weights take 14 GB needs a GPU with comfortably *more* than 14 GB. We'll come back to this when we discuss quantizing the KV cache in Part 9.

Part 2 — Number formats: what FP32, FP16, BF16, INT8 and INT4 really are

To quantize, we need to know what we're quantizing *from* and *to*. There are two families of formats: **floating point** (numbers that can be tiny or huge) and **integers** (whole numbers in a limited range).

Floating point: sign, exponent, mantissa

A floating-point number is stored like scientific notation. `0.000173` is written as 1.73×10^{-4}: a few meaningful digits (the **mantissa**) and a power of ten (the **exponent**) that says where the decimal point goes. Computers do the same thing in base 2, plus one bit for the sign:



The **exponent** bits decide the *range* (how big or small a number can get). The **mantissa** bits decide the *precision* (how finely you can tell nearby numbers apart).

Format	Exponent bits	Mantissa bits	Largest value	Rough precision
FP32	8	23	$\approx 3.4 \times 10^{38}$	about 7 decimal digits
FP16	5	10	65,504	about 3 to 4 digits
BF16	8	7	$\approx 3.4 \times 10^{38}$	about 2 to 3 digits

FP16 and BF16 both use 16 bits, but they spend them differently:

- **FP16** keeps more precision but has a small range (it overflows above 65,504). That can bite during training, where some values get very large.
- **BF16** ("brain float") keeps the **same range as FP32** and gives up precision instead. It rarely overflows, which is why it became the standard format for training and for shipping models.

Here is how our example weight survives each format:

```
FP32    0.173421894   →   0.173421890   (essentially exact)
FP16    0.173421894   →   0.173461914
BF16    0.173421894   →   0.173828125
```

The differences are in the fourth decimal place, and that's all. FP16 and BF16 are widely used precisely because they preserve almost all of a model's quality while halving the memory compared with FP32. When you see a model released as "BF16 weights", this is what it means.

(Even smaller floats exist. **FP8** has two common variants, E4M3 and E5M2, and **FP4** has just 4 bits in total. Modern GPUs can do arithmetic directly in these. We'll see DeepSeek training with FP8 in Chapter 21.)

Integers: a short ruler

Integers work very differently. An **n-bit integer** can represent only 2^n distinct values:

Format	Distinct values	Typical (signed) range
INT8	256	−128 … 127
INT4	16	−8 … 7

There is no exponent, no mantissa, no fractions. Just a small ladder of whole numbers. Picture two rulers. FP32 is a ruler with markings every hair's width:

```
0.171  0.172  0.173  0.174  0.175   ← lots of precision
```

INT4 is a ruler with very few markings:

```
0.15        0.20        0.25        0.30        ← far fewer representable values
```

So with integers, quantization means we are **rounding** each weight to the nearest marking on a short ruler. A good image analogy: a photo with 16 million colours is beautiful. Reduce it to 16 colours and it's still recognizable, but subtle detail is lost. Quantization does the same thing to the weights.

But integers can't hold `0.173` at all. They only hold whole numbers. So how do we get a decimal weight onto an integer ladder? That is what the next part is about.

Part 3 — Quantizing by hand

The naive idea, and why it fails

Suppose a row of a weight matrix, in FP32, is:

```
 0.12   -0.83    0.41    1.27   -0.36    0.91
```

and INT4 allows only 16 distinct values. The simplest idea is to round to the nearest whole number:

```
0.91  →  1
0.83  →  1        ← oops
```

Different weights collapse into the same value, and almost all the information is lost. Rounding to whole numbers is far too coarse for numbers this small.

Brilliant idea 1: buckets, and storing an index

Instead of rounding to whole numbers, look at the range the weights actually occupy and divide *that* range into 16 evenly spaced buckets. Suppose the weights run from −1.5 to 1.5:

```
bucket:    0     1     2   ...    12    ...    15
value:   -1.5  -1.3  -1.1  ...   0.9   ...   1.5
```

Now `0.91` falls into the bucket at `0.9`. Here is the important realization:

> **We no longer store the weight. We store the *index* of its bucket.**

The model stores `12`, not `0.91`. During inference, "12" is translated back into "about 0.9". It's like a dictionary: the stored integer is just a key into a table of real values.

Brilliant idea 2: a scale factor

But a table of 16 values per layer would be awkward. There's a neater way to say the same thing. Pick one number, the **scale**, and let the integer tell you *how many scale-steps* from zero the weight is:

```
approximate weight  =  integer  ×  scale
```

For example, if the integer is `7` and the scale is `0.13`:

```
7 × 0.13  =  0.91
```

So instead of storing `0.913482` (32 bits), we store `integer = 7` (4 bits) and share `scale = 0.13` across many weights. Recovering the number is called **dequantization**, and what comes back is an **approximation**, not the exact original.

How do we choose the scale? The simplest way (called **absmax** or *symmetric* quantization) is to make the largest-magnitude weight land exactly on the largest integer:

```
scale  =  (largest absolute weight)  /  (largest integer)

integer  =  round( weight / scale )
weight ≈ integer × scale
```

(Some schemes also store a **zero point**, a shift that lets the integer ladder start somewhere other than zero. That helps when values aren't centred on zero, and it's called *asymmetric* quantization. The idea is otherwise identical.)

The worked example

Take our row again: `0.12, -0.83, 0.41, 1.27, -0.36, 0.91`. The largest absolute value is `1.27`.

INT8, where the largest integer is 127:

```
scale = 1.27 / 127 = 0.01

integers:     12     -83     41     127     -36     91
recovered:  0.12   -0.83   0.41    1.27   -0.36   0.91      ← exact!
```

(This row happens to land exactly on the 0.01 grid, so nothing is lost. In general INT8 has 256 levels, so the error is tiny but not zero.)

INT4, where the largest integer is 7:

```
scale = 1.27 / 7 = 0.1814

integers:          1      -5      2       7      -2      5
recovered:     0.181  -0.907   0.363   1.270  -0.363  0.907
original:       0.12   -0.83    0.41    1.27   -0.36   0.91
error:         0.061   0.077   0.047   0.000   0.003  0.003
```

Now the rounding is visible: the weight `0.12` came back as `0.181`, and `-0.83` as `-0.907`. INT4 is a coarse ruler, and the biggest weight (which defined the scale) is the only one guaranteed to be exact.

What does that do to a real calculation? Take an input vector `x = [0.5, 1.0, -0.3, 0.2, 0.8, 0.4]` and compute the dot product with this row, as a neuron would:

```
with the original weights:        -0.563
with the INT4 weights:            -0.599        (difference ≈ 0.036)
```

Individual weights were off by up to 0.077, yet the neuron's output is off by only about 0.036, because some errors push up and others push down and they partly cancel. A small, noisy change in a number: remember that for Part 5.

What we're really storing

After quantization, a block of weights is stored as:

```
[ integers, 4 bits each ]  +  [ one scale ]  (+ sometimes a zero point)
```

At run time the engine rebuilds `integer × scale` to get the approximate weights back. The Transformer never notices: it still receives numbers. They are just slightly approximate.

Part 4 — Why one scale isn't enough: block-wise quantization

The problem with a single range

Suppose we used **one scale for the entire model**. One layer might have weights such as `-100, 50, 75, -80`, and another might have `0.001, 0.003, -0.002`. They can't possibly share a scale. The scale for the first layer would round everything in the second to zero.

The fix is obvious: give each layer its own scale. But why stop there? A layer has millions of weights, and different parts of it behave differently.

Outliers: one big weight ruins its neighbours

Here's the problem in miniature. Take our row, but replace `1.27` with a single large weight, `12.7`:

```
original:   0.12   -0.83    0.41   12.70   -0.36    0.91

scale = 12.7 / 7 = 1.814        (the big weight sets the scale)

integers:      0       0       0       7       0       1
recovered:  0.00    0.00    0.00   12.70    0.00    1.81
```

Four of the six weights collapsed to zero. One outlier forced a coarse scale on all its neighbours, and nearly all their information was lost.

Real LLM weight matrices do contain occasional large values, so this is not hypothetical. The remedy is **block-wise quantization**:

```
one scale for the whole layer:
□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□□

one scale per small block (commonly 32 to 128 weights):
□□□□□□□□□□□□□□□□   □□□□□□□□□□□□□□□□   □□□□□□□□□□□□□□□□
 scale 1            scale 2            scale 3
```

An outlier now only damages the other weights in *its own small block*. Every block gets a scale suited to its own range, and accuracy improves dramatically.

The cost of the extra bookkeeping

Block-wise quantization isn't free: every block stores its own scale. For example, if a block is 32 weights with one 16-bit scale, then:

```
(32 weights × 4 bits  +  16 bits for the scale) / 32 weights  =  4.5 bits per weight
```

So a "4-bit" format actually costs about 4.5 bits per weight once the scales are counted. That's why a real 4-bit file is a little bigger than the ideal `params × 0.5 bytes`, and it's also why GGUF files are not just "compressed weights". They hold the integers **plus the metadata needed to rebuild each block** (Chapter 22).

How coarse can the grouping be?

You'll meet several names for this choice:

Granularity	One scale per…	Trade-off
Per-tensor	whole weight matrix	cheapest, least accurate
Per-channel	each row/column of the matrix	better, still cheap
Block-wise (group-wise)	small block of weights, e.g. 32 or 128	best accuracy, small extra storage

Modern low-bit formats use block-wise scales almost universally.

Part 5 — Why the model survives

Here is the question that bothers almost everyone, and it is the right one.

> Suppose I quantize `W_Q`. Then `Q = X·W_Q` is slightly wrong. `K` is slightly wrong. `V` is slightly wrong. So the attention scores are slightly wrong. The FFN is slightly wrong. The LM head is slightly wrong. **Won't these tiny errors pile up through 32 or 80 Transformer layers and destroy the model?**

That is exactly what researchers expected. **They didn't.** Why not?

1. Small perturbations stay small. A weight changes from `0.1734` to `0.1732`. That slightly changes one neuron's output, which slightly changes the next layer. It doesn't flip anything dramatically. Networks are, in general, smooth: a small change in the input of a function usually gives a small change in the output.

2. The errors are not all in the same direction. As our worked example showed, rounding errors point both ways, so they partly cancel instead of adding up in one direction. And each output is the sum of *thousands* of products, so no single rounding mistake dominates.

3. The network is redundant. The knowledge isn't stored in one weight. It is spread across billions of them, and those weights work together, so a modest disturbance to each is averaged over many.

4. Training was noisy anyway. Every training step uses a random mini-batch (Chapter 11), and the weights began as random numbers (Chapter 9). A model that ends up good has learned a solution that tolerates a certain amount of jitter, rather than one that balances on a knife edge.

Think of GPS. Your true position is `19.0761° N`, and the GPS says `19.0760° N`. You'll still reach your destination, because the error is too small to matter. Or think of a human answering `2 + 2` with a little background noise: you can still answer.

The most important mental model:

> **Quantization is not deleting knowledge. It is storing the same knowledge less precisely.**

Like saving a photo as a high-quality JPEG instead of an uncompressed bitmap: the picture is the same, and some microscopic details disappear.

But it is not free

This robustness has limits, and honesty about them matters:

- **8-bit is almost always effectively lossless.** 4-bit with a good method loses a small, usually hard-to-notice amount of quality. 3-bit and 2-bit lose clearly more.
- **Smaller models are less tolerant** than larger ones. A 7B model at 4-bit degrades more than a 70B model at 4-bit, because there is less redundancy to absorb the noise.
- **Some skills degrade first.** Long reasoning chains, arithmetic, and rare knowledge tend to suffer before casual chat does.
- **Naive rounding is much worse than clever methods** at the same bit-width. That's what Part 7 is about.

So the right summary isn't "quantization is free". It is "quantization gives up a *small, controllable* amount of quality for a *large* saving in memory and time".

Part 6 — How the math runs, and why smaller is also faster

Does the GPU convert everything back?

Once the weights are stored as `integer × scale`, the matrix multiplication still needs to happen. There are two broad ways to do it:

Method 1: dequantize, then multiply.

```
INT4 weights  →  convert back to FP16  →  ordinary matrix multiply
```

Simple and flexible, and the conversion happens on the fly right before the multiply, so the big FP16 copy never has to sit in memory.

Method 2: quantized (fused) kernels.

```
INT4 weights  →  optimized kernel that unpacks, scales and multiplies in one go  →  output
```

Modern inference libraries fuse dequantization into the matrix-multiply itself, or use hardware that multiplies low-precision numbers directly. This is a large part of why engines like llama.cpp are so fast on ordinary hardware.

Either way, the pipeline becomes:

```
Compressed weight  →  recover approximate weight  →  matrix multiply
```

and the Transformer around it is unchanged.

Why fewer bits also means faster generation

You'd expect quantization to save memory. The surprise is that it also makes generation **faster**. Recall from Chapter 12 that the *decode* phase (generating one token at a time) is **memory-bound**: for every new token, the GPU has to read essentially *all the weights* from memory, but does very little arithmetic with each one.

When the bottleneck is how fast bytes can be read, then:

```
tokens per second  ≤  memory bandwidth  /  bytes of weights read per token
```

Take a 7B model on a GPU that can read about 1 TB per second:

```
FP16  (14 GB):   1000 GB/s  /  14 GB   ≈  70 tokens/s   (theoretical ceiling)
INT4  (≈4 GB):   1000 GB/s  /   4 GB   ≈ 250 tokens/s   (theoretical ceiling)
```

Real engines don't reach these ceilings, but the ratio is the point. **Fewer bytes to read per token means more tokens per second**, so quantization makes the model both smaller *and* faster.

(Prefill, the compute-heavy phase, benefits less, because it is limited by arithmetic rather than by reading weights.)

Part 7 — Smarter quantization: GPTQ, AWQ and mixed precision

So far we've treated every weight as equally important and rounded them all the same way. Modern methods ask a better question:

> **Which weights can tolerate approximation, and which should be preserved more carefully?**

That is the difference between **naive** and **production** quantization.

Why importance differs

Consider four weights:

Weight	If quantization changes it, the damage to the model is...
0.91	large ⭐⭐⭐⭐⭐
0.72	considerable ⭐⭐⭐⭐
0.44	modest ⭐⭐
0.01	negligible ⭐

Rounding `0.91` to `0.70` could be disastrous. Rounding `0.01` to `0.00` is almost irrelevant. A good quantizer should spend its limited precision where it matters. To judge what matters, these methods use a **calibration dataset**.

Calibration: a small test run, not training

A calibration set is a small sample of typical text (perhaps a few hundred examples: Wikipedia, books, code, questions). The quantization method runs the model on it and watches how the numbers behave. Crucially:

- It is **not training** and it is **not fine-tuning**.
- There is **no backpropagation**, no AdamW, no weight updating by gradient descent.

- The data is only used to *measure* what the model's layers do, so the quantizer can make better rounding decisions.

GPTQ: compensate for the damage as you go

GPTQ works layer by layer. Instead of rounding all the weights independently, it quantizes the weights of a layer in sequence and, after rounding each one, **adjusts the not-yet-quantized weights to compensate for the error just introduced**, so that the layer's *output* on the calibration data stays as close as possible to the original. (It uses second-order information about how sensitive the layer's output is to each weight, which is what lets it judge importance.) The result is a much better 3-bit or 4-bit model than plain rounding could give.

AWQ: protect the weights that meet big activations

AWQ (Activation-aware Weight Quantization) asks about importance from a different angle. Remember that a layer computes `Y = XW`, and there are two things here: `W`, the weights, and `X`, the *activations*, the vectors flowing through the network.

Consider two weights:

```
Weight 0.8, multiplied by an activation of 0.00001   →  nearly irrelevant
Weight 0.2, multiplied by an activation of 500       →  a tiny error here is amplified 500×
```

The second weight matters far more, even though it is smaller, because the input it is multiplied by is large. AWQ uses the calibration data to find which weights are paired with large, important activations, and protects them by scaling them so they are represented more accurately. A small fraction of weights ends up being responsible for a large share of the quality.

Comparing the two

	GPTQ	**AWQ**
Asks	How much does this weight's error change the layer's output?	Which weights meet large, important activations?
Looks at	The effect of weight errors, using calibration data	Activation statistics from calibration data
Idea	Round, then compensate with the remaining weights	Scale up the important weights so they lose less
Retrains ?	**No**	**No**

Both aim for the same goal, **keeping quality high while shrinking memory**, and they simply estimate "importance" differently.

Mixed precision: not every tensor needs the same bits

A related, simpler idea: some *tensors* are more sensitive than others. Modern formats often keep very sensitive tensors at higher precision (for instance, embeddings, the LM head, or parts of attention and FFN) and compress the less sensitive ones harder. That is why a "4-bit" model is often a mix of 4-bit and 5- or 6-bit pieces.

Two other things worth knowing

When quantization happens. Everything above is **post-training quantization (PTQ)**: take a finished model and compress it, with no retraining. The alternative is **quantization-aware training (QAT)**, where the model is trained (or fine-tuned) while simulating low precision, so it learns to be robust to it. PTQ is far cheaper and is what most people use. QAT can do better at very low bit-widths.

What gets quantized. So far we've compressed **weights only**. You can also compress **activations**, so the multiplication itself runs in low precision. This is harder, because LLM activations contain rare but very large outlier values that don't compress well. Weights-only schemes (such as 4-bit weights with 16-bit activations, often written "W4A16") are the common practical choice, and 8-bit weights *and* activations ("W8A8") is used where hardware support is good.

Part 8 — Reading the names

When you browse Hugging Face you constantly see names like:

```
Q4_K_M.gguf     Q5_K_M.gguf     Q8_0.gguf     IQ4_XS.gguf
```

Now you can read them. Take them apart:

`Q4`, `Q5`, `Q8`: the bit-width

```
Q4  →  about 4 bits per weight
Q5  →  about 5 bits per weight
Q8  →  about 8 bits per weight
```

`K`: the "k-quant" family

`K` marks a newer family of **block-wise** schemes from llama.cpp, with better scale metadata than the older formats. It stores weights in larger "super-blocks" that contain smaller sub-blocks, each with its own scale, so the scales themselves are stored compactly. Think of it as **a smarter block quantization method**. The older formats, with names ending in `_0` or `_1` (such as `Q8_0` and `Q4_0`), use one simple scale per block of 32 weights.

`S`, `M`, `L`: the preset size

The final letter is a preset in a family that trades file size against quality:

```
_S  (Small)   →   _M  (Medium)   →   _L  (Large)
smallest file                          slightly more accurate, slightly bigger
```

Larger presets keep more of the sensitive tensors at higher precision.

Putting it together

```
Q4_K_M  =  4-bit weights  +  k-quant block scheme  +  medium-quality preset
```

`IQ`: importance-based formats

Names starting with `IQ` (such as `IQ4_XS`) are formats that use a calibration step, an *importance matrix*, to decide which weights deserve more precision. It's the same idea as Part 7, applied inside the GGUF world.

What the sizes look like

For a model of about 8 billion parameters, such as Llama-3-8B, approximate file sizes are:

Format	Approx. bits per weight	Approx. file size	Typical quality
FP16 / BF16	16	≈ 16 GB	the reference
`Q8_0`	8.5	≈ 8.5 GB	almost indistinguishable
`Q5_K_M`	≈ 5.7	≈ 5.7 GB	very close
`Q4_K_M`	≈ 4.8	≈ 4.9 GB	small loss; a popular default
`Q3` / `Q2` variants	≈ 3 or less	smaller still	quality drops noticeably

(Sizes are approximate, and quality depends on the model and the task.) `Q4_K_M` is a common sweet spot, and the reason is just the trade-off above: it cuts the file to about a third of FP16 with only a small loss.

Other formats you'll meet

Outside llama.cpp's GGUF world, you'll see:

- **GPTQ** and **AWQ** checkpoints: models saved with the GPTQ or AWQ methods from Part 7, commonly used with GPU servers.
- **bitsandbytes**: a library that quantizes on the fly when a model loads, including **NF4**, a 4-bit format designed for the bell-shaped distribution of weights. It's what QLoRA (Chapter 13) uses.
- **FP8 and FP4**: low-precision floating-point formats that modern GPUs support natively, used both for inference and, in some recent models, for training.

One correction to the toy picture we started with. We said quantization means "replace every weight with an integer". That is conceptually right and the right mental model, but in production each block stores the quantized values, one or more scales, and sometimes extra metadata such as zero points or specialized encodings. The dequantization math is a little richer than `integer × scale`. The idea doesn't change.

Part 9 — The engineering trade-off

Would you ship the quantized model?

Imagine two options for the same 7B model:

	Scenario 1: FP16	Scenario 2: INT4
Model size	14 GB	3.5 to 4 GB
Needs a GPU of about	24 GB	8 GB
Speed (illustrative)	15 tokens/s	45 tokens/s
Quality (illustrative)	100%	≈ 99%

(The numbers are illustrative of the *shape* of the trade, not benchmarks.)

Which would you deploy? Almost every production team picks INT4. Users notice faster responses, lower latency and lower cost. They almost never notice a 1% quality difference in everyday use.

This is a general engineering lesson:

> **Researchers optimize for maximum quality. Engineers optimize for the best trade-off.** Those are different goals.

That is why GPTQ, AWQ and GGUF became famous. They weren't trying to create a smarter model. They were asking "*can we keep about 99% of the intelligence while cutting memory by 75%?*" and the answer was yes.

(One honest caveat. If your task is quality-critical, such as complex reasoning, code or maths, measure the quantized model on *your* task before shipping. Pick the highest bit-width you can afford, and treat the quality loss as something to test, not assume.)

Quantizing things other than the weights

Weights aren't the only thing that can be stored in fewer bits:

What	Why it matters	Notes
Weights	The biggest fixed memory cost	The main topic of this chapter
KV cache	Grows with every token (Chapter 12)	Engines such as llama.cpp and vLLM can store keys and values in 8-bit or lower, so longer contexts or more users fit in memory
Activations	Make the arithmetic itself cheaper	Harder because of outliers; W8A8 is used where hardware allows
Training numbers	Faster, cheaper training	Training runs in BF16 or even FP8 (Chapter 21)

How quantization combines with what came before

- **QLoRA** (Chapter 13) is quantization plus LoRA: a 4-bit frozen base model with small trainable adapters on top.
- **LoRA adapters** are normally kept in higher precision, since they're tiny and they're the part being learned.
- **The KV cache** (Chapter 12) and quantized weights are independent: you can compress either, both, or neither.

The pattern behind modern LLM engineering

Look at the breakthroughs we have met so far. Nobody invented a brand-new Transformer for any of them. Each one asked what the most expensive thing in the current system is, and whether it can be simplified without sacrificing much quality:

Problem	The question asked	The idea
Training is expensive	Do we need to update every weight?	**LoRA** (Chapter 13)
Generation repeats work	Do we need to recompute every token?	**KV cache** (Chapter 12)
Memory is expensive	Do we need to store every weight at full precision?	**Quantization** (this chapter)
RLHF is complicated	Do we need a reward model and RL?	**DPO** (Chapter 13)

Modern AI engineering is often about optimizing *around* the Transformer rather than replacing it.

Common misconceptions, cleared up early

Misconception	Reality
"Quantization deletes some of the model's knowledge."	It stores the same numbers less precisely, like a JPEG of a photo. Nothing is removed, only rounded.
"A quantized model is a different model."	Same architecture, same tokenizer, same layers. Only the weight storage differs.
"You need to retrain a model to quantize it."	Post-training quantization needs no training, just a conversion (and sometimes a small calibration run).
"GPTQ and AWQ train the model on calibration data."	Calibration data is only used to *measure* sensitivity. There is no backpropagation or optimizer.
"INT4 means every weight is exactly 4 bits."	Each block also stores a scale (and sometimes more), so a "4-bit" format costs roughly 4.5 bits per weight.
"One scale for the whole model is enough."	One large outlier would ruin the precision of everything else. Scales are per block.
"Rounding errors must add up over 80 layers and ruin the model."	They are small, point in both directions, and are spread over huge numbers of redundant weights. In practice 8-bit is near-lossless and good 4-bit loses little.
"Quantization only saves memory."	It also speeds up generation, because decoding is limited by how many bytes it has to read per token.
"Lower bits is always better."	Below about 4 bits, quality drops noticeably, especially for small models and reasoning tasks.
"FP16 and BF16 are the same."	Both are 16 bits. FP16 has more precision but a small range; BF16 has FP32's range but less precision.
"Quantizing the weights shrinks the whole memory footprint by the same ratio."	The KV cache and activations are separate. A 4-bit model still needs memory for them.
"Q4_K_M is a magic code."	It reads `4-bit` + `k-quant block scheme` + `medium preset`.

Quick reference

```
Model memory  =  parameters  ×  bytes per parameter
                 7B:   FP32 28 GB | FP16/BF16 14 GB | INT8 7 GB | INT4 3.5 GB   (+ overhead)

Floating point:  sign | exponent (range) | mantissa (precision)
                 FP32 = 1+8+23    FP16 = 1+5+10    BF16 = 1+8+7
Integers:        n bits → 2ⁿ levels  (INT8: 256, INT4: 16)

Quantize:    scale = max|w| / max_integer ;   int = round(w / scale)
Dequantize:  w ≈ int × scale             (a block stores ints + scale [+ zero point])
Block-wise:  one scale per small block (32–128) so one outlier can't ruin everything
             ≈ 4.5 bits per weight for a "4-bit" format

Why it works:  small, two-sided errors, spread over billions of redundant weights
Why faster:    decode is memory-bound → fewer bytes read per token → more tokens/s

GPTQ   round layer by layer, compensate error with the remaining weights
AWQ    protect the weights that meet large activations
PTQ    compress a finished model        QAT   train while simulating low precision

Names:  Q4_K_M = 4-bit + k-quant blocks + Medium preset
        Q8_0 ≈ near-lossless   Q5_K_M ≈ very close   Q4_K_M ≈ popular default

Unchanged by quantization:  architecture, tokenizer, attention, blocks, KV cache, LM head
```

What's next

We have now covered the full life of a GPT-style model: how it is built (Chapters 2 to 8), trained (Chapters 9 to 11), run (Chapter 12), adapted (Chapter 13) and compressed (this chapter). But everything so far has been one particular design: the plain GPT decoder. Chapter 15 steps back to look at the **Transformer family**: encoder-only models like BERT, decoder-only models like GPT, and encoder–decoder models like T5. We'll see what each one is for, how they differ, and why decoder-only models ended up dominating modern LLMs. Chapters 16 and 17 then show how those decoder-only models have been improved, Chapters 18 and 19 extend the architecture itself (Mixture of Experts and multimodal models), and Chapter 20 turns to reasoning. Much later, Chapters 22 and 23 return to the engineering of serving models, starting with the file format, GGUF, that stores the quantized weights you just learned about.

Chapter 15 — The Transformer Family: Encoder, Decoder, Encoder–Decoder

Introduction

Over the previous chapters we built a GPT-style model piece by piece (tokens, embeddings, attention, the FFN, residuals, LayerNorm, stacked blocks, an LM head, and a full training loop), then ran it (inference), adapted it (fine-tuning) and compressed it (quantization). There's a fact about all of that which we have deliberately held back until now:

> **Everything we built was not "the Transformer". It was specifically one half of it, the *decoder*.**

The original Transformer, introduced in the 2017 paper "*Attention Is All You Need*", had **two** halves, an *encoder* and a *decoder*. Different families of models keep one half, or both. This chapter explains what those words mean, how BERT, GPT and T5 relate, and why one of them ended up dominating modern LLMs. The next two chapters then look at how today's decoder-only models have been improved: the building blocks they swapped out (Chapter 16) and the attention variants they use (Chapter 17). A tour of real models such as Qwen and DeepSeek comes later, in Chapter 21.

Part 1 — The original Transformer, and what "encoder" and "decoder" mean

The 2017 paper was about **machine translation**. Think about what happens in your own head when you translate "The cat sat on the mat" into French. You don't start speaking French immediately. First you *understand* the sentence, something like `animal = cat, action = sitting, place = on the mat`. That internal understanding isn't English and isn't French. Then you *express* it in French: `Le chat est assis sur le tapis.` Two different jobs:

```
Job 1:  English  →  meaning          (understand)
Job 2:  meaning  →  French           (express)
```

Those two jobs are exactly the **encoder** and the **decoder**:

```
English → Encoder → meaning (a vector for every word) → Decoder → French
```

A word of caution about the names. "Encoder" does **not** mean compression, like encoding an MP3 or a ZIP file. It means converting language into vectors that capture meaning and context. You've already seen what these vectors are: the contextual representations from Chapters 4 to 8, the ones that know about grammar, meaning and relationships between words.

- **The encoder** answers: "*What does this text mean?*" It turns text into contextual vectors, and stops there. No words are produced.
- **The decoder** answers: "*Given this meaning (or the text so far), what should I write next?*"

Part 2 — The same block, with a different attention mask

Here's the part many people find surprising: **inside, an encoder block and a decoder block are almost identical.** Both contain attention and an FFN, wrapped in residuals and LayerNorm. The major difference is the attention mask.

```
Encoder attention                    Decoder attention (causal)

     A  B  C                              A  B  C
  A  ✓  ✓  ✓                           A  ✓  ✗  ✗
  B  ✓  ✓  ✓                           B  ✓  ✓  ✗
  C  ✓  ✓  ✓                           C  ✓  ✓  ✓

Every token sees every token         Each token sees only itself
                                     and what came before
```

Why the difference? An **encoder** receives a sentence that already exists in full. There's nothing wrong with "cat" looking at "sat", and in fact that's desirable, because context from the right helps understanding. A **decoder** is *writing*. When it is producing `The cat ...`, the word `sat` doesn't exist yet, so it can't be looked at. That's the causal mask you met in Chapter 4.

So now we can look back and see what we built: our GPT-style model (the one that Appendix A runs by hand, "TinyGPT") was **a decoder**. When someone says "GPT uses a Transformer", what they really mean is *"GPT uses the decoder half of the original Transformer."*

Part 3 — Three branches from one tree

After 2017, researchers asked a natural question: *do we really need both halves?* Sometimes yes, sometimes no. That led to three branches:

```
                  Original Transformer (2017)
                    Encoder  +  Decoder
                            |
          ┌─────────────────┼──────────────────┐
          ▼                 ▼                  ▼
  Encoder only        Decoder only       Encoder + Decoder
  (BERT)              (GPT, Llama,       (T5, BART, Whisper)
                       Qwen, DeepSeek)
  "Understand"        "Generate"         "Read fully, then write"
```

	Encoder-only (BERT)	Decoder-only (GPT)	Encoder–decoder (T5)
Attention	Sees both directions	Sees only the past	Encoder: both directions. Decoder: past, plus the encoder
Main job	Understanding	Generating	Turning one sequence into another
Training task	Fill in hidden words	Predict the next word	Reconstruct or transform text
Typical uses	Search, classification, embeddings	Chat, coding, writing, reasoning	Translation, summarization, rewriting

Notice that none of these invented a radically new neural network. They changed *how the Transformer is used*.

Part 4 — BERT: the encoder-only model

BERT's training task is a fill-in-the-blank game. Take `The cat sat on the mat` and hide a word:

```
The cat [MASK] on the mat.
```

Predict the missing word. The crucial detail is that BERT can look **both left and right** to do it:

```
"The cat"   →   [MASK]   ←   "on the mat"
```

This changes what the model is good at. Take `The animal didn't cross the road because it was tired.` To work out what `"it"` refers to, BERT can read the whole sentence, including `because it was tired`, which settles the question. Or take a search query like `Apple store near me`. Does `Apple` mean the fruit or the company? Reading the whole query makes it obvious. That's why BERT-style models became so important for search and language understanding.

BERT can't do what GPT does, though. If you give it `Once upon a time...`, it has no idea what to do. It was built to understand text that already exists, not to continue it.

Two precise points about training

1. GPT is not trained on masked text. It's easy to mix these up, so be careful. BERT *changes the input*: roughly 15% of the words are replaced with a special `[MASK]` token. GPT sees the **original, unchanged sentence**, but the *attention mask* (Chapter 4) stops each position from seeing the future. Two different kinds of "mask":

```
BERT:  hides WORDS in the input     →  "The cat [MASK] on the mat"
GPT:   hides the FUTURE via attention →  "The cat sat" fed in unchanged,
                                          but each position can't look ahead
```

2. They learn from different amounts of each sentence. In Chapter 9 we saw that a GPT-style model gets a learning signal at **every** position, because every position predicts its next word. BERT only learns from the roughly 15% of positions that were masked. That's one reason decoder-only models turned out to be so

efficient to train at scale.

Part 5 — Encoder-only models in practice: when the vector is the product

You might wonder: if an encoder only outputs vectors and not readable text, what use is it? The answer is: **sometimes the vector itself is the product.**

Give an encoder `"The cat sat on the mat."` and instead of a sentence, you get something like `[0.42, -1.83, 0.91, ..., 0.15]`, maybe 768 or 1024 numbers that capture the meaning of the whole text. Texts with similar meanings produce similar vectors. That makes encoders the engine behind:

- **Semantic search.** Imagine one million documents and the query `How do I install Python?` You can't ask a large generative model to read a million documents. Instead, an encoder converts every document into a vector in advance, converts the query into a vector, and finds the nearest document vectors. Search by meaning, not just keywords.
- **Classification:** spam or not, positive or negative review.
- **Re-ranking and matching:** deciding which of several candidate results best fits a question.

Well-known encoder-only models include BERT, RoBERTa, DeBERTa, and the embedding models built on the same idea (sentence-transformers, and families such as E5 and BGE). Newer, modernized encoders such as ModernBERT also exist. The pattern is the same: **text in, meaning vector out.** (We'll meet these vectors again when we get to retrieval in Chapter 24. Chapter 26 returns to BERT in more detail: the `[CLS]` token, pretraining, fine-tuning heads and ModernBERT.)

Part 6 — Encoder–decoder models and cross-attention

When a task is fundamentally **one sequence in, a different sequence out**, such as translation or summarization, the original two-half design shines. The question is how the two halves connect. After all, the decoder isn't reading English; it's reading the encoder's *understanding* of the English. The answer is a second kind of attention.

The decoder has two attention layers

A decoder block in the original Transformer looks like this:

```
Masked self-attention   ("what have I already written?")
        ↓
Cross-attention         ("which part of the input matters right now?")
        ↓
FFN
```

A GPT block has only the first and last. The original has one extra layer.

How cross-attention works

Suppose the encoder has read `The black cat sat on the mat`, and the decoder has written `Le chat` so far. It must now choose the next French word. Should it write `noir` (black) or `assis` (sat)? To decide, it asks the encoder: "*I'm writing the next word. Which English words matter most right now?*"

That is attention again, but across two sequences. And here is the beautiful part: **the attention formula hasn't changed at all.** Only the *source* of Q, K and V changes:

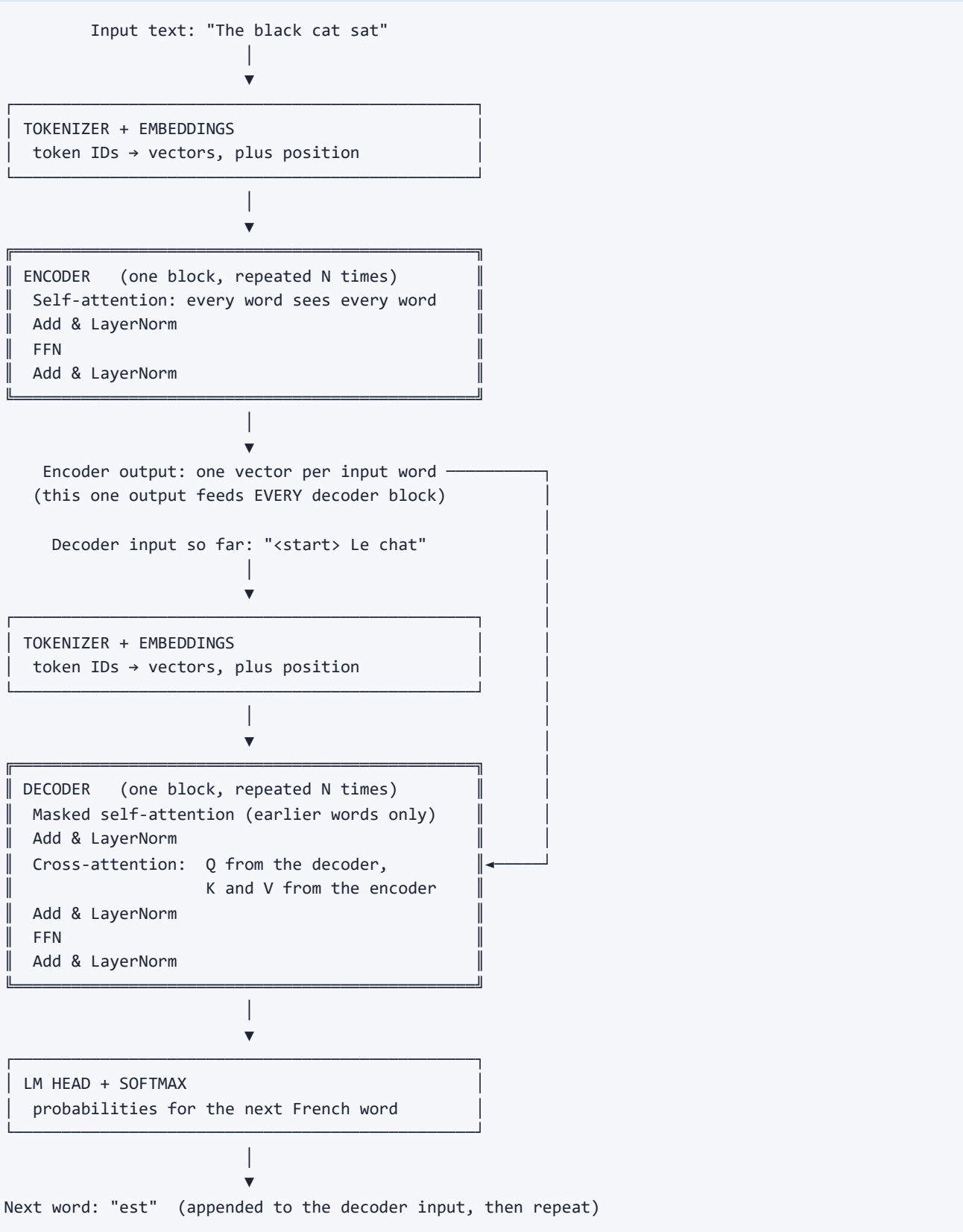
Type	Q comes from	K and V come from
Self-attention	The same sequence	The same sequence
Cross-attention	The decoder	The encoder

```
Decoder  ──►  Q  ─┐
                  ├──► Attention(Q, K, V) ──► decoder receives information from the encoder
Encoder  ──►  K,V ─┘
```

The decoder asks the questions, and the encoder holds the knowledge. One practical consequence: the score grid has one row per decoder position and one column per *encoder* position. With 5 French words written so far and a 7-word English sentence, it is 5 × 7, not a square. Self-attention (masked, inside the decoder) is the part that keeps the output fluent. Cross-attention is the part that keeps it faithful to the input.

The whole encoder–decoder model, drawn out

Putting the pieces together, here is the complete encoder–decoder model. The encoder reads the input and produces one vector per input word. The decoder writes the output one word at a time. The line down the right-hand side shows how the encoder's output reaches the decoder: it is fed into the **cross-attention layer inside every decoder block**.



A few things to notice in the picture:

- **The encoder runs once.** It reads the whole input in one go, with every word seeing every other word, and its output is then fixed for the rest of the task.
- **The decoder runs once per output word.** It looks like the GPT decoder from earlier chapters, plus the extra cross-attention layer in the middle of each block.

- **The same encoder output feeds every decoder block.** Each block's cross-attention builds its Queries from the decoder's own vectors, and takes its Keys and Values from that one encoder output.
- **Each sublayer keeps its residual connection and LayerNorm,** exactly as in the Transformer block of Chapter 7.

Why GPT dropped cross-attention

By now you can probably answer this yourself. GPT has only **one** sequence, which keeps growing: `The → cat → sat → ...`. There's no separate "input language" to look at, so there's nothing for cross-attention to attend to. It would have no purpose.

Where encoder–decoder models still shine

Models such as T5, BART and Whisper (speech recognition) keep both halves. T5 popularized a neat idea: express *every* task as text in, text out (for example `"translate English to German: ..."` or `"summarize: ..."`) and train one model on a fill-in-the-spans objective. They remain strong when:

- the input is long,
- the output is a **different** sequence,
- and the output should stay tightly grounded in the input.

Typical cases: translation, summarization, grammar correction, rewriting, and speech recognition.

Part 7 — "If GPT can already translate, why do we need the other two?"

This is exactly the question the field asked around 2019 to 2021, and it's worth answering carefully. A big enough GPT *can* translate: ask `Translate to French: The cat sat on the mat.` and it replies correctly. So why bother with an encoder?

Because **GPT solves translation indirectly, while an encoder–decoder solves it directly.** Picture two students:

```
Student A (GPT):             ONE process does everything at once:
                             read the request → understand English → recall French
                             → write French, all inside one growing sequence

Student B (Encoder–Decoder): TWO dedicated stages:
                             understand the English completely → then write the French,
                             consulting notes (cross-attention) whenever needed
```

Or think of translating a 500-page novel. The GPT way is to keep the original open next to you and keep re-reading it as you write each sentence. The encoder–decoder way is to read and understand a chapter first, take notes, and then translate from the notes. Both can work, but the second is more naturally suited to "input sequence in, output sequence out."

So why did decoder-only models win anyway? Researchers discovered something unexpected:

> **Next-token prediction is an incredibly general objective.**

Translation? Predict the next translated word. Summarization? Predict the next summary word. Question answering, coding, chat? Each is just "*continue this text*". Instead of building a specialized architecture per task, people simply made decoder-only models larger, trained them on more text, and added instruction tuning and reinforcement learning (Chapter 13, and Chapter 20 for reasoning models). Economically, one model that can do everything beats maintaining a translation model, a summarization model, a coding model and a chat model separately.

That's why the open models whose designs are published, such as Llama, Qwen, DeepSeek and Gemma, are all decoder-only, and most leading chat models are understood to be as well. It isn't that encoders and encoder–decoders stopped working. They remain excellent at their specialties. It's that scaling the decoder-only recipe covered so many tasks well enough that it became the default.

A good mental model for the three:

```
BERT:             Read → understand → stop.            Excellent reader, cannot write.
GPT:              Read → think while writing.           Excellent writer, very good reader.
Encoder–Decoder:  Read → understand completely → write. Excellent translator and rewriter.
```

The complete picture

```
               Same ingredients inside every block:
               attention + FFN + residuals + LayerNorm

What differs between the families is only WHICH attention is used and HOW the model is trained:

BERT        full (bidirectional) self-attention           trained to fill in masked words
GPT         causal self-attention                         trained to predict the next word
T5 / BART   encoder: full self-attention                  trained to reconstruct / transform text
            decoder: causal self-attention + cross-attention
```

Common misconceptions, cleared up early

Misconception	Reality
"An encoder compresses text, like ZIP."	It converts text into contextual vectors that capture meaning. "Encode" here means "turn language into vectors".
"GPT is trained on text with words masked out, like BERT."	GPT sees the original sentence. The *attention mask* blocks the future. BERT is the one that replaces words with `[MASK]`.
"Encoder and decoder blocks are completely different designs."	They are almost identical. The main difference is the attention mask, plus an extra cross-attention layer in an encoder–decoder's decoder.
"Encoder-only models are useless because they can't generate text."	Often the vector itself is the product: search, classification, matching, and embeddings all rely on them.
"Cross-attention is a new, different kind of attention math."	It is the same formula. Only the source of Q (decoder) and K, V (encoder) changes.
"Decoder-only models won because encoder–decoders stopped working."	Encoder–decoders still excel at translation, summarization and speech. Decoder-only models won because next-token prediction scaled into one model that handles almost everything.

Quick reference

```
Encoder:   full attention, outputs contextual vectors.   "What does this mean?"
Decoder:   causal attention, writes one token at a time.  "What comes next?"

BERT        encoder-only      fill-in-the-blank    search, classification, embeddings
GPT/Llama   decoder-only      next-token           chat, code, writing, reasoning
T5/BART     encoder-decoder   text-to-text         translation, summarization, rewriting

Cross-attention:  Q from the decoder, K and V from the encoder.
                  Same formula as self-attention; only the source of Q/K/V changes.

BERT masks WORDS in the input.  GPT masks the FUTURE with the attention mask.
```

What's next

Since decoder-only models are the ones that dominate, we'll focus on them from here. Chapter 16 starts with the building blocks that modern decoder-only models have swapped out for better ones: how positions are represented, how normalization is done, and what the FFN looks like. Chapter 17 then covers how attention itself has been reworked to stay affordable on long text.

Chapter 16 — Modern Building Blocks: RoPE, RMSNorm and SwiGLU

Introduction

Chapter 15 explained why decoder-only models dominate. This chapter and the next look at how they have been improved. The GPT-style model we built in Chapters 2 to 8 is the classic recipe. It learns a table of position vectors (Chapter 3), normalizes with LayerNorm (Chapter 7), and uses a two-matrix FFN with a GELU in the middle (Chapter 6). Open models such as Llama, Qwen, DeepSeek and Gemma have, over the last few years, converged on a noticeably different recipe. They didn't change the big picture, which is still stacked decoder blocks predicting the next token. They swapped out individual parts for better ones.

Attention itself has also been reworked, and Chapter 17 covers those variants. This chapter covers the other parts:

```
Positions:     learned position table     →   RoPE (rotary position embeddings)
Normalization: LayerNorm (γ and β)        →   RMSNorm
FFN:           W1 → GELU → W2             →   SwiGLU (a gated FFN)
Small things:  biases, attention scores   →   no biases, QK-Norm
```

Each is a small, understandable idea, and by this point in the handbook you have all the background needed for each one.

Part 1 — RoPE: rotating Q and K instead of adding a position vector

What's wrong with a table of position vectors?

In Chapter 3, we gave the model a word's meaning plus a **learned position vector**, one table row per position (position 0, position 1, position 2, and so on), added to the embedding. It works, but it has two awkward properties:

1. **A hard length limit.** The table has a fixed number of rows. A model trained with 1,024 rows has no vector at all for position 1,025.
2. **It learns "position 7", not "three tokens apart".** Yet what attention really needs to know is mostly *how far apart two tokens are*. "The word right before me" matters wherever it happens to appear in the text.

Researchers wanted position information that depends on the **relative distance** between two tokens, and that doesn't require a table with a fixed number of rows.

The RoPE idea

Rotary position embedding (RoPE) does something quite different. It doesn't add anything to the embedding. Instead, it **rotates the Query and Key vectors** by an angle that depends on the token's position. The farther along in the text, the larger the rotation.

Here is the smallest possible example. Use a 2-number vector and, to keep the arithmetic easy, an exaggerated rotation of 90° per position. Let `q = [1, 0]` and `k = [1, 0]` before rotation. Rotating `[1, 0]`:

```
position 0 → rotate 0°     → [ 1,  0]
position 1 → rotate 90°    → [ 0,  1]
position 2 → rotate 180°   → [-1,  0]
position 3 → rotate 270°   → [ 0, -1]
```

Now compute the attention score (the dot product from Chapter 4) between a query at position `m` and a key at position `n`:

```
query at position 2, key at position 0:   [-1, 0] · [ 1, 0] = -1      (distance 2)
query at position 3, key at position 1:   [ 0,-1] · [ 0, 1] = -1      (distance 2)
query at position 1, key at position 0:   [ 0, 1] · [ 1, 0] =  0      (distance 1)
query at position 4, key at position 3:   [ 1, 0] · [ 0,-1] =  0      (distance 1; 4 × 90° = 360° is a full turn)
query and key at the same position:       identical rotation  = +1     (distance 0)
```

Look at the pattern: **the score depends only on the distance `m - n`, never on the absolute positions.** Positions 2 and 0 give the same score as positions 3 and 1, because both pairs are 2 apart. That is exactly the property we wanted: attention automatically "knows" relative distance, without any position table.

How it works in a real model

A real Query or Key has many numbers, not two. RoPE splits them into **pairs** and rotates each pair, with each pair turning at its **own speed**:

```
Some pairs rotate quickly   →  good at telling nearby positions apart
Some pairs rotate slowly    →  good at telling far-apart positions apart
```

Think of the hands of a clock: the second hand distinguishes moments a few seconds apart, the minute hand distinguishes minutes, the hour hand distinguishes hours. Together they pin down time precisely. Several speeds together let the model sense both short-range and long-range distance.

Three points are worth being clear on, because each connects to something earlier in the handbook:

- **RoPE has no learned parameters.** The rotations are fixed math. It's another "pure math, nothing to learn" step like softmax, so the position-embedding table from Chapter 3 simply goes away.
- **It's applied to Q and K only, not V,** in every layer, right before the scores are computed. Positions only matter for deciding *where to look*, not for the content that gets copied.
- **It still goes through the causal mask and softmax as before.** Nothing else in attention changes.

Why this helps with longer text

Because the rotation is a formula and not a table, nothing stops you from asking for position 100,000. But a model that only ever saw positions up to 4,000 during training has never seen rotations that large, and quality drops. So long-context models are typically **trained at a short length first, then extended**, adjusting the rotation speeds so that long distances look familiar to the model. Qwen3, for example, was trained at 4,096 tokens, then extended to 32,768 with these techniques (YaRN and Dual Chunk Attention), and DeepSeek-V3 was extended in two stages, from 4K to 32K and then to 128K.

Variations on the idea

Some newer designs apply RoPE to only *part* of each head's numbers ("partial RoPE"). RoPE also needs special handling when K is stored in compressed form, which you'll see with DeepSeek's MLA in Chapter 17.

Part 2 — RMSNorm: normalization with one less step

Recall LayerNorm from Chapter 7. For each token's vector it did three things: subtract the mean, divide by the standard deviation, then apply a learned scale `γ` and a learned shift `β`.

RMSNorm (root-mean-square normalization) is a simplification: **skip the mean subtraction, and skip the shift.** Just divide by the vector's root-mean-square, and apply a learned scale `γ`:

```
RMS = √( average of the squares )
output = γ × (x / RMS)
```

Using the same vector from Chapter 7, `[100, 110, 90, 95]`:

```
RMS = √( (100² + 110² + 90² + 95²) / 4 ) = √(39,225 / 4) ≈ 99.03

x / RMS = [1.010, 1.111, 0.909, 0.959]

Compare, for the same vector:
  LayerNorm:  [ 0.17,  1.52, -1.18, -0.51]    (centered on zero, spread of 1)
  RMSNorm:    [ 1.010, 1.111, 0.909, 0.959]   (just rescaled; not centered)
```

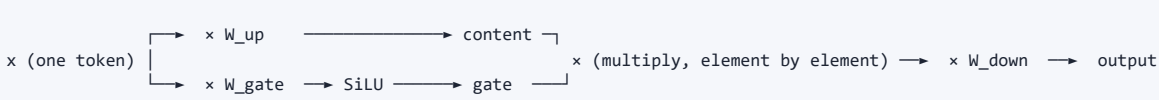
Both put the numbers on a manageable scale, which is the whole reason normalization exists (Chapter 7). RMSNorm does it with less work: one fewer statistic to compute, and no `β` to store. Researchers found that the mean-centering step contributed little in practice, and models trained with RMSNorm do as well as with LayerNorm. Since normalization is applied twice in every block of dozens of blocks, small savings add up. Almost all modern open models use RMSNorm, in the **pre-norm** position (before attention and before the FFN), as described in Chapter 7.

Part 3 — SwiGLU: a gated FFN

Recall the FFN from Chapter 6: expand with `W1`, apply an activation (GELU), compress with `W2`. Modern models use a variant called **SwiGLU**, which adds a **gate**.

The idea of a gate

Instead of one expansion followed by an activation, there are **two parallel expansions** of the same input. One produces the *content*. The other produces a *gate*, whose job is to decide how much of each piece of content to let through. The two are multiplied together, number by number:



In formula form:

```
FFN(x) = ( SiLU(x · W_gate)  ⊙  (x · W_up) ) · W_down
```

where `⊙` means multiply element by element, and `SiLU(z) = z × sigmoid(z)` is a smooth activation similar in spirit to GELU (it's sometimes called "Swish", which is where the "Sw" in SwiGLU comes from; "GLU" stands for gated linear unit).

A worked example: the gate opening and closing

Take one feature, where the content branch produced `4`. The gate branch is passed through SiLU:

```
Gate pre-activation = +2   →  SiLU(2)  = 2 × 0.881 =  1.76   →  4 × 1.76  =  7.05   (gate open: content passes)
Gate pre-activation = −3   →  SiLU(−3) = −3 × 0.047 = −0.14   →  4 × (−0.14) = −0.57  (gate nearly closed: content suppressed)
```

The same content value gets through strongly or is nearly shut off, depending on what the *gate* says, and the gate is itself computed from the input by a learned matrix. In the mango analogy from Chapter 6, the activation used to be a fixed grading rule applied to each reading. Here the model also learns, through `W_gate`, *which* readings deserve to be passed along. It's a more flexible kind of decision.

Keeping the size comparable

SwiGLU uses **three** matrices (`W_gate`, `W_up`, `W_down`) instead of two. To avoid simply making the model bigger, the hidden size is reduced from the usual `4d` to roughly `8d/3`:

```
Classic FFN:  2 matrices of size d × 4d       →  2 × 4d²    =  8d²  parameters
SwiGLU:       3 matrices of size d × (8d/3)   →  3 × 8d²/3  =  8d²  parameters
```

The same parameter count, in a more expressive shape. (Llama 7B has `d = 4096` and an FFN hidden size of 11,008, about 2.7 times `d`, which is this idea in practice.) Researchers found that gated FFNs perform better than the plain two-matrix version at equal size, and SwiGLU is now the standard choice in open models.

Part 4 — A few small refinements

No biases. A linear layer can add a bias term, `Y = XW + b`, which our examples in Chapters 6 and 8 left out for simplicity, as many implementations do. Modern models mostly drop biases in their linear layers, which saves a few parameters with no loss in quality. (Earlier Qwen versions kept a bias in the Q, K and V projections; Qwen3 removed it.)

QK-Norm. Attention scores come from the dot product of Q and K. If those vectors grow large during training, the scores explode, and softmax collapses into near-one-hot choices (Chapter 4 explained why). **QK-Norm** applies a normalization to Q and to K *before* the dot product, keeping the scores in a stable range as an added safeguard on top of the $\sqrt{d_k}$ scaling. Qwen3 introduced it in place of the old QKV bias to keep training stable.

Tied embeddings in smaller models. Chapter 8 explained weight tying, reusing the embedding table as the LM head. Smaller models often do this to save parameters, since the embedding table is a large share of a small model, while the biggest models more often keep them separate.

The complete picture

Here is the "classic" block next to the typical modern open-model block:

Component	Classic (GPT-2 era)	Modern open model (Llama / Qwen / DeepSeek style)
Position information	Learned position table, added to embeddings	**RoPE**: rotate Q and K
Attention heads	Multi-head (MHA)	**GQA** (or **MLA**), see Chapter 17
Normalization	LayerNorm (γ , β)	**RMSNorm** (γ only)
Norm placement	Post-norm (original paper)	**Pre-norm**
FFN	$W1$ → GELU → $W2$	**SwiGLU** (gated, 3 matrices)
Biases	Yes	Mostly removed
Attention stability	Just $\sqrt{d_k}$ scaling	Plus **QK-Norm** in some

The overall flow is unchanged: embeddings, N stacked blocks of attention then FFN with residuals, an LM head, softmax. Each row above is a *drop-in replacement* for one part, which is why the changes are easy to describe, and why you could swap them into the GPT diagram of Chapter 8 without changing its shape.

Common misconceptions, cleared up early

Misconception	Reality
"RoPE adds a position vector to the embedding, just done differently."	It adds nothing. It rotates Q and K, and the score ends up depending on the *distance* between tokens.
"RoPE has learned parameters like the old position table."	RoPE is fixed math: nothing to train.
"RoPE alone lets a model handle any length."	Models still need to be trained, or extended, for long context, because they haven't seen very large rotations.
"RMSNorm is a different technique from LayerNorm."	It is LayerNorm with the mean subtraction and the shift removed: a simplification that works as well and costs less.
"SwiGLU is just a different activation function."	It changes the FFN's structure: a second, parallel branch (the gate) multiplied with the content branch. The activation is only one part.
"SwiGLU makes the FFN much bigger."	The hidden size is reduced to about 8d/3 so the parameter count stays about the same as the classic FFN.

Quick reference

```
RoPE      rotate Q and K by an angle ∝ position; score depends on relative distance;
          no parameters; applied to Q, K only
RMSNorm   x / √(mean of squares) × γ        (no mean subtraction, no β)
SwiGLU    FFN(x) = ( SiLU(x·W_gate) ⊙ (x·W_up) ) · W_down      (hidden ≈ 8d/3)
QK-Norm   normalize Q and K before the dot product (stability)
No biases in most linear layers; pre-norm; GQA or MLA for attention
```

What's next

Attention has been reworked as well as these parts. Chapter 17 covers the attention variants that modern models use to keep long text affordable: multi-query, grouped-query, latent, sliding-window, sparse and hybrid attention.

Chapter 17 — Attention Variants: MQA, GQA, MLA, Sliding Window and FlashAttention

Introduction

In Chapters 4 and 5 we built attention exactly as the original Transformer defined it: many heads, each with its own `W_Q`, `W_K`, `W_V`, with every token comparing itself against every earlier token. That design is elegant and powerful. It is also expensive, and the expense gets worse as text gets longer.

Real models don't all use that original design. Llama, Qwen, DeepSeek, Gemma and others each use their own **attention variant**, and each variant is an answer to one specific cost problem. This chapter explains the problems first, then the variants, so that each one makes sense as a solution rather than a list of names to memorize.

One thing stays constant through all of this: attention is still "*compare queries with keys, softmax, blend values*" (Chapter 4). The variants change **how many keys and values are stored**, **which tokens are compared**, or **how the computation is carried out**, never the basic idea.

Part 1 — The two costs of attention

Cost 1: compute grows with the square of the text length

In attention, every token is scored against every earlier token. That's a grid of scores:

```
1,000 tokens     →  about 1 million scores  (per head, per layer)
10,000 tokens    →  about 100 million
100,000 tokens   →  about 10 billion
```

Ten times more text means a hundred times more scores. This is why long context is hard.

Cost 2: the memory that generation needs (the KV cache)

Recall from Chapter 12 how a model writes text: one token at a time, with each new token needing the Keys and Values of **all the earlier tokens**. Because an earlier token's Key and Value never change once computed (the causal mask of Chapter 4 guarantees it), the model stores them instead of recomputing them. That store is the **KV cache**.

The catch is that the KV cache grows with every token and has to be kept for every layer. Its size is:

```
KV cache per token  =  2 (K and V)  ×  layers  ×  KV heads  ×  head dimension  ×  bytes per number
```

Take a model shaped like Llama 3 8B: 32 layers, 8 KV heads, head dimension 128, stored in 16-bit (2 bytes):

```
2 × 32 × 8 × 128 × 2 bytes  =  131,072 bytes  =  128 KiB per token

   8,000 tokens of context  →  about  1 GiB
 128,000 tokens of context  →  about 16 GiB      (for ONE conversation)
```

At large context lengths, and when serving many users at once, this cache can take more memory than the model's own weights. Most of the variants below exist to shrink it, or to cut the compute cost, or both. Keep this formula in mind: every factor in it is something a variant can reduce.

Part 2 — The baseline: multi-head attention (MHA)

This is Chapter 5's design. With `H` heads, there are `H` query heads **and** `H` separate sets of Keys and Values, one per head:

```
Head 1:  Q₁  K₁  V₁
Head 2:  Q₂  K₂  V₂
  ...
Head H:  Q_H K_H V_H          ← H sets of K and V must be cached
```

It works well, and it's the reference point for quality. GPT-2 uses it. Its weakness is the one in the formula above: `KV heads = H`, which makes the cache large.

Part 3 — Multi-query attention (MQA): share one K and V

The first idea is blunt. **Keep all the query heads, but let them all share a single set of K and V.**

```
Q₁ ─┐
Q₂ ─┤
Q₃ ─┼──►  one shared K, V
Q₄ ─┘
```

Each head still asks its *own* question (its own `Q`), but they all search the same keys and read the same values. The KV cache shrinks by a factor of `H`; with 32 heads, it's 32 times smaller. The cost is some loss of quality, since every head now works from the same K and V, which gives the model less flexibility. MQA appeared in models such as PaLM.

Part 4 — Grouped-query attention (GQA): the middle path

MHA keeps one K/V set per head (best quality, biggest cache). MQA keeps one K/V set in total (smallest cache, some quality loss). **GQA splits the difference: heads are divided into groups, and each group shares one K/V set.**

```
8 query heads, 2 KV groups:

Q₁ Q₂ Q₃ Q₄  ──►  K₁, V₁   (group 1)
Q₅ Q₆ Q₇ Q₈  ──►  K₂, V₂   (group 2)

Cache: 2 K/V sets instead of 8   →   4× smaller than MHA
```

GQA is really a dial:

```
KV groups = number of heads   →  this IS multi-head attention
KV groups = 1                 →  this IS multi-query attention
anything in between           →  grouped-query attention
```

In the matrices, the only change is the shape. `W_Q` still produces all `H` query heads, but `W_K` and `W_V` produce only `G` heads' worth of output (one per group), so they're smaller too. At attention time, each query head simply uses the K and V of its own group.

This is the default choice in most modern open models. Llama 3 8B uses 32 query heads with 8 KV groups (4 query heads per group), which is exactly where the `8` came from in the cache example above. Had it used plain MHA (32 KV heads), that cache would be 4 times larger: 512 KiB per token, or 64 GiB for a 128K-token conversation. Qwen3 and many others also use GQA.

Part 5 — Multi-head latent attention (MLA): compress instead of share

GQA and MQA shrink the cache by *sharing* K and V between heads. DeepSeek took a different route: **keep every head's own K and V, but store them in compressed form.**

The idea is to cache one small vector per token, called a **latent** vector, instead of the full K and V:

```
Standard:    token → K (all heads) and V (all heads)  →  cache both in full

MLA:         token → ONE small latent vector c   →  cache only c
                                 │
                                 └─► when needed, expand c back into K and V
                                      using learned "up-projection" matrices
```

Both the compression (a learned down-projection) and the expansion (learned up-projections) are ordinary matrices, trained along with everything else, exactly like `W_K` and `W_V` in Chapter 4. Implementations can even fold the expansion into neighboring matrices so the full K and V never need to be built at all.

The numbers for DeepSeek-V3 show how large the saving is. It has 128 heads with a per-head dimension of 128, so a standard cache would hold `128 × 128 × 2 = 32,768` numbers per token, per layer. MLA instead caches a latent of 512 numbers plus a small extra piece for position information (rotary position embeddings, from Chapter 16; see the note below), about 576 in total:

```
Standard MHA (this model's size):     32,768 numbers per token per layer
GQA with 8 groups:                     2,048
MLA (DeepSeek-V3):                      ~576        →  roughly 57× smaller than MHA,
                                                       and about 3.5× smaller than GQA
```

A note on that small extra piece. RoPE (Chapter 16) rotates K by an amount that depends on position, and that rotation can't pass cleanly through MLA's compress-and-expand step. DeepSeek's fix is **decoupled RoPE**: K is split into a large compressed part that carries no position information, and a small separate part (64 numbers per token in DeepSeek-V3) that carries the rotation. That small part is the extra piece counted in the 576 above.

DeepSeek reported that, in their experiments, MLA kept quality competitive with full MHA while GQA and MQA lost some. The trade-off is complexity: MLA is considerably harder to implement than GQA. It's used by DeepSeek-V3 and several other large models, such as Kimi K2.

Part 6 — Sliding-window attention: only look at recent tokens

The variants so far shrink the *memory*, but each token still looks at every earlier token. **Sliding-window attention** attacks the compute cost directly: each token attends only to the most recent `w` tokens.

```
Full causal attention          Sliding window (w = 3)

     1 2 3 4 5 6                    1 2 3 4 5 6
  1  ✓                           1  ✓
  2  ✓ ✓                         2  ✓ ✓
  3  ✓ ✓ ✓                       3  ✓ ✓ ✓
  4  ✓ ✓ ✓ ✓                     4    ✓ ✓ ✓
  5  ✓ ✓ ✓ ✓ ✓                   5      ✓ ✓ ✓
  6  ✓ ✓ ✓ ✓ ✓ ✓                 6        ✓ ✓ ✓
```

The cost per token is now fixed by `w` instead of growing with the text length, and the cache only needs to keep the last `w` tokens. Mistral 7B used a window of 4,096 tokens.

You might worry the model "forgets" anything older than the window. It's less severe than it sounds, because layers stack. A token at layer 1 sees `w` tokens back, but what it sees has already absorbed information from `w` tokens before *that*. After several layers, information can travel much farther than one window.

Still, some information needs a direct, long-range lookup. So many models **mix** local and global layers. Gemma 3, for example, alternates 5 sliding-window layers (window of 1,024 tokens) for every 1 full-attention layer, keeping most of the savings while preserving a global view in a fraction of the layers.

Part 7 — Sparse attention: choose which tokens are worth looking at

A sliding window picks tokens by a fixed rule: "the most recent ones". But the tokens that matter most are not always the latest. **Sparse attention** lets each token attend only to a *selected subset* of earlier tokens, chosen more intelligently.

Early versions used fixed patterns (a few local tokens plus a few global ones), as in models like Longformer and BigBird. DeepSeek's recent approach, **DeepSeek Sparse Attention (DSA)**, introduced in DeepSeek-V3.2, makes the selection *learned*:

```
1. A cheap "lightning indexer" scores how relevant each earlier token is to the current one.
2. Only the top-k highest-scoring tokens are kept (k = 2,048 in V3.2).
3. Full attention then runs only over those selected tokens.
```

So the expensive attention step covers roughly 2,048 tokens no matter how long the text is, while the cheap indexer decides which 2,048. DSA is used together with MLA, so the cache is compressed *and* the attention is selective. DeepSeek's newer V4 models go further, with a hybrid of compressed and sparse attention for a context length of one million tokens; the team reports it needs only about a quarter of the per-token compute and a tenth of the cache of V3.2 at that length.

Part 8 — Linear attention and hybrids: replace the cache with a running summary

Everything so far still keeps (some form of) the K and V of earlier tokens, so memory grows with length. A more radical idea is to give attention a **fixed-size memory** instead. These designs are called **linear attention** (or recurrent/state-space designs). Instead of storing every earlier token and comparing against all of them, the layer keeps a single fixed-size *state*, a running summary, and updates it as each new token arrives.

```
Full attention:    memory grows with every token (KV cache)
Linear attention:  memory stays the same size however long the text gets
```

The compute cost grows only in proportion to length, and the memory doesn't grow at all. The trade-off is that a fixed-size summary is lossy: recalling a *specific* earlier detail precisely is harder than looking it up directly in a full cache.

Because of that, the practical design is a **hybrid**. Qwen's recent models (Qwen3-Next and the Qwen3.5 family) use a layer called **Gated DeltaNet**, a gated linear-attention layer, for most layers and keep a full-attention layer (a "gated attention" layer) every fourth layer:

```
Layer pattern, repeated:    [Gated DeltaNet] [Gated DeltaNet] [Gated DeltaNet] [full attention]
                              cheap, fixed memory ×3                  exact recall ×1
```

Three cheap layers do most of the work with constant memory, and one full-attention layer in four preserves the precise long-range lookup. Other model families have explored similar hybrids, with different linear layers (state-space designs such as Mamba among them).

Part 9 — FlashAttention: not a new variant, a faster way to compute the same thing

It would be a mistake to leave out **FlashAttention**, because almost every modern system uses it, for both training and serving. But it's different in kind from the rest of this chapter. It doesn't change what attention computes. It changes **how it's computed on the hardware.** The numbers that come out are exactly the same as standard attention.

The real bottleneck is memory traffic, not arithmetic

A GPU has two kinds of memory that matter here:

```
HBM  (the GPU's main memory)    large (tens of GB)       slower
SRAM (on-chip, next to the      tiny  (a few MB to        very fast
      compute units)                  tens of MB in total)
```

You can think of SRAM as the countertop in a kitchen and HBM as the warehouse down the road. Cooking is fast, but every trip to the warehouse is slow.

A straightforward implementation of attention makes a lot of trips. Following the formula $\text{softmax}(Q \cdot K^T / \sqrt{d_k}) \cdot V$ literally:

```
1. compute the score grid  S = Q·Kᵀ          →  write the whole N × N grid to HBM
2. read S back, apply the mask and softmax   →  write the whole N × N grid P to HBM
3. read P back, multiply by V                →  write the output
```

The score grid is the problem. For a 8,192-token prompt, one head in one layer has 8,192 × 8,192 ≈ 67 million scores, which is about 134 MB in 16-bit numbers. With 32 heads, that's roughly 4.3 GB for a *single layer*, written out and read back several times. The GPU spends more time hauling this grid to and from memory than doing the multiplications. In other words, attention is usually **memory-bound**, the same kind of bottleneck we met in decoding (Chapter 12) and quantization (Chapter 14), just for a different reason.

The idea: bring a tray, do all the steps, never store the grid

FlashAttention rearranges the work so the big grid is **never written out at all**. It cuts Q, K and V into small blocks (**tiles**) that fit in SRAM, and for each tile of queries it loops over the tiles of keys and values, doing the scores, the softmax and the multiplication by V entirely on-chip, and only the final output goes back to HBM. In kitchen terms: instead of walking to the warehouse after every step, you carry a tray of ingredients to the counter, cook the whole dish, and send back only the finished plate.

The obstacle: softmax seems to need the whole row

There is a catch. Softmax turns a row of scores into percentages, and to do that it must divide by the sum over the **whole row**. If we only see one tile of keys at a time, how can we get the right answer?

The trick is **online softmax**: keep a *running maximum* and a *running sum* as tiles arrive, and whenever a new tile reveals a bigger maximum, rescale what you have accumulated so far. Here is a tiny example, with one query whose scores against four keys are `[1, 3, 5, 2]`, and whose values are `[10, 20, 30, 40]`. We process two keys at a time.

```
Tile 1: scores [1, 3]
   running max      m = 3
   running sum      l = e^(1−3) + e^(3−3)          = 0.135 + 1        = 1.135
   running output   o = 0.135·10 + 1·20            = 21.35

Tile 2: scores [5, 2]      (the max is now 5, so rescale the old state by e^(3−5) = 0.135)
   running max      m = 5
   running sum      l = 1.135·0.135 + e^(5−5) + e^(2−5)   = 0.154 + 1 + 0.050   = 1.203
   running output   o = 21.35·0.135 + 1·30 + 0.050·40     = 2.89 + 30 + 1.99    = 34.88

Final answer:  o / l = 34.88 / 1.203 = 28.98
```

Computing it the ordinary way, with the full row at once, gives softmax weights `[0.015, 0.112, 0.831, 0.041]` and `0.015·10 + 0.112·20 + 0.831·30 + 0.041·40 = 28.98`. **The same answer**, but we never needed the whole row at once. That is the mathematical heart of FlashAttention. (For the backward pass during training, it also avoids storing the grid: it recomputes the scores tile by tile. That is extra arithmetic, but far less memory traffic, so it is a net win.)

What you get, and what you don't

- **Exact results.** No approximation: the output is the same as standard attention (up to normal floating-point rounding).
- **Far less extra memory.** The memory for the score grid drops from growing with N^2 to growing only with N, which is a large part of what makes 100,000-token contexts practical.
- **Much faster in practice.** Because it cuts the trips to HBM, it typically runs several times faster than the straightforward version. Later versions (FlashAttention-2 and -3) improve how the work is divided up and use newer GPU features.
- **It works alongside everything else in this chapter.** It supports the causal mask (it can skip tiles that lie entirely in the future), and it combines with GQA, MLA, sliding windows and the rest. Frameworks such as PyTorch, vLLM and llama.cpp use FlashAttention-style kernels.

And what it does **not** do, which is easy to miss. It does not reduce the **arithmetic**: the number of score calculations still grows with the square of the text length (Cost 1, at the top of this chapter). And it does not shrink the **KV cache** (Cost 2); that is what MQA, GQA and MLA are for. FlashAttention attacks a third problem, the *traffic* of moving the score grid through memory, which is why it is a different kind of fix from everything else in the chapter.

The complete picture

The variants change different things, so they're not competitors in one list. They act on different axes, and real models combine them:

Variant	What it changes	Saves	Trade-off	Seen in
MHA	Baseline: K/V for every head	(reference)	Biggest cache	GPT-2
MQA	All heads share 1 K/V	Cache (÷ number of heads)	Some quality loss	PaLM
GQA	Heads share K/V in groups	Cache (e.g. ÷4)	Small quality cost	Llama 3, Qwen3, Gemma
MLA	Cache a compressed latent	Cache (large)	More complex	DeepSeek-V3, Kimi K2
Sliding window	Each token sees only recent tokens	Compute and cache	No direct long-range lookup	Mistral, Gemma 3 (mixed with global)
Sparse (DSA)	Each token sees a chosen subset	Compute	Needs an indexer	DeepSeek-V3.2 onward
Linear / hybrid	Fixed-size running state	Memory and compute	Lossy recall; keep some full layers	Qwen3-Next, Qwen3.5
FlashAttention	How the computation runs on hardware	Time and memory, **exact**	None on accuracy	Nearly everything

A way to organize them:

```
Which tokens can each token see?     →  full, causal, sliding window, sparse
How are K and V stored and shared?   →  MHA, MQA, GQA, MLA
Is it still softmax attention?       →  yes (all above)   /   no (linear, state-space hybrids)
How is it computed on hardware?      →  FlashAttention and similar
```

Common misconceptions, cleared up early

Misconception	Reality
"GQA and MQA make attention compute much cheaper."	Their main saving is the **KV cache memory** (and the memory traffic during generation). Each query head still computes its own scores.
"MLA is just GQA with a different name."	GQA shrinks the cache by *sharing* K/V between heads. MLA keeps per-head K/V but *stores a compressed version* and expands it when needed.
"With a sliding window, the model can't use anything older than the window."	Stacked layers let information travel farther than one window, and many models add occasional global layers on top.
"FlashAttention is an approximation that trades accuracy for speed."	It is exact: the same result as standard attention, computed in a hardware-friendlier order.
"FlashAttention shrinks the KV cache."	It doesn't. It avoids writing the N × N score grid to memory. The KV cache is shrunk by MQA, GQA and MLA.
"FlashAttention removes the quadratic cost of attention."	The number of score calculations still grows with the square of the length. It removes the *memory traffic* and score-grid storage, not the arithmetic.
"Linear attention replaces full attention outright."	Its fixed-size memory is lossy, so today's practical designs are hybrids that keep some full-attention layers.
"There's one best attention variant."	They attack different costs, and real models combine several (for example MLA with sparse attention, or GQA with sliding-window layers).

Quick reference

```
Two costs:     compute ∝ (length)²     and     KV cache memory ∝ length

KV cache per token = 2 × layers × KV heads × head dim × bytes

MQA   all query heads share 1 K/V              cache ÷ H
GQA   query heads share K/V in G groups        cache ÷ (H/G)
MLA   cache a small latent, expand on demand   cache ≈ tiny (DeepSeek-V3: ~576 numbers/token/layer)
SWA   attend to last w tokens only             compute and cache bounded by w
Sparse  attend to a learned top-k subset       compute bounded by k
Linear/hybrid  fixed-size running state        memory constant (+ a few full layers for recall)
FlashAttention  exact, tiled, online softmax; the N×N score grid never goes to GPU memory
                same math, far less memory traffic; does NOT cut arithmetic or the KV cache
```

What's next

That completes the tour of how attention has been reworked. The next change goes deeper inside the block. Chapter 18 covers **Mixture of Experts**, where the FFN is replaced by many expert FFNs and only a few of them are used for each token. Chapter 19 then extends the model to images and audio, Chapter 20 covers reasoning models, and Chapter 21 puts everything together by looking inside Qwen and DeepSeek, which combine several of the attention variants from this chapter with Mixture of Experts.

Chapter 18 — Mixture of Experts: A Huge Model That Only Uses Part of Itself

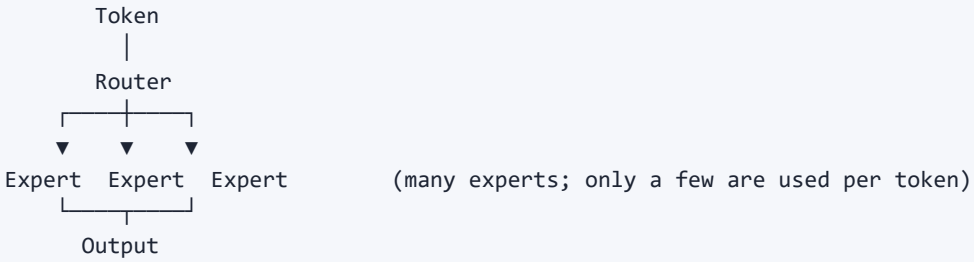
Introduction

In Chapters 16 and 17 we saw how modern models swapped out parts of the plain GPT: new position encodings, new norms, new attention. All of those changed *how* something is computed, but the block kept the same shape. This chapter covers a change that goes deeper. It alters the **structure** of the Transformer block itself, and it answers a question that every scaling effort runs into:

> **Can we make a model enormous, without making every token pay for all of it?**

In a normal ("dense") model, making the model smarter by making it bigger has a built-in cost. Every parameter you add is a parameter that **every token** has to be multiplied by, so the compute per token grows in step with the model. That puts a ceiling on how big a model we can afford to train and run.

Mixture of Experts (MoE) breaks that link. The idea is simple to state:



Instead of one big FFN that every token uses, the block has **many FFNs ("experts") and a small router** that picks a few of them for each token. The model can then hold a huge number of parameters in total, but each token only uses a small slice of them.

This chapter explains what changes inside the block, how the router works and how it is trained, how many parameters such a model really has, and why the idea is harder to run than it sounds. As always, we'll build it from pieces you already know: the FFN (Chapters 6 and 16), softmax (Chapter 4), the training loss (Chapter 9), and backpropagation (Chapter 10).

Part 1 — The idea: don't make every token pay for everything

Dense models: everyone takes every course

In a dense Transformer, every token goes through every block, and in every block through the **same** FFN and the same attention:

```
Token
  ↓
Block 1   ← every token uses all of it
  ↓
Block 2   ← every token uses all of it
  ↓
Block 3   ← every token uses all of it
  ↓
 ...
```

So if you want a smarter model by adding parameters, every token pays for the extra parameters, whether or not they are relevant to that token. The compute per token grows with the model:

```
Model grows  →  every token uses more parameters  →  every token costs more compute
```

A university analogy helps. In a **dense** university, every student takes every course: maths, physics, programming, history, languages. A programming student pays for all five, including the ones they don't need.

MoE: visit an advisor first

In an **MoE** university, every student first visits an **advisor** (the router), who sends them only to the relevant specialists:

```
Student → Advisor (router) → the relevant specialists only

   a programming student →  Programming expert + Maths expert
   a language student    →  Language expert + History expert
```

The university can now have **100 specialist departments**, while each student takes just two or three courses. Adding a new department makes the university more capable overall, but doesn't increase what any individual student has to pay.

That is the MoE trade, in the language of the model:

```
Model grows  →  add more experts  →  total capacity increases
                                  →  each token still uses only a few experts
```

"But the blocks already specialize. Why a new design?"

A natural objection. Earlier we said different layers learn different things: early blocks pick up basics, later ones more abstract relationships. Why not simply rely on that, instead of inventing something new?

First, a correction to that earlier intuition. It was a *useful picture*, not a guarantee: Block 7 does not become "the maths block". More importantly, even if Block 7 *did* learn features that help with maths, **every token still pays for Block 7**, because in a dense model every token goes through every block. Specialization by training doesn't give you **sparse computation**.

MoE gives you both things at once:

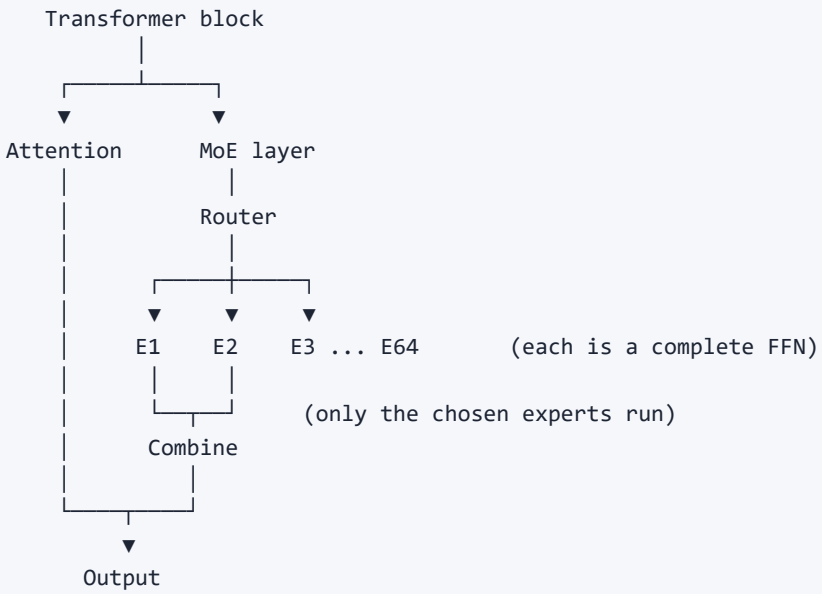
> **Different parts of the model can specialize, *and* the model can switch on only the parts that are useful for each token.**

Part 2 — What actually changes inside the block

Recall the Transformer block from Chapter 7:

```
Input
  ↓
Attention  →  Add & Norm
  ↓
FFN        →  Add & Norm
  ↓
Output
```

In an MoE block, **only the FFN is replaced.** The single FFN becomes a router plus many expert FFNs:



Each **expert is a complete FFN**: the same `W1 → activation → W2` of Chapter 6, or the SwiGLU version of Chapter 16, just smaller or one among many. So MoE genuinely **changes the internal structure** of the block. It is not a runtime trick.

But nearly everything else is untouched:

```
Unchanged:   tokenizer, embeddings, attention, Q/K/V, softmax attention,
             residual connections, normalization, the LM head

Changed:     Attention → ONE FFN        becomes        Attention → Router → SELECTED FFNs
```

It helps to compare with the other techniques we've met. Some change *how a model is run* or *stored*, and one changes the architecture:

Technique	Does it change the architecture?
KV cache	No
Quantization	No
LoRA	No (the base architecture is unchanged)
Reasoning models	Usually no
Mixture of Experts	**Yes**

Part 3 — The router

Where it sits and what it does

The router sits **exactly where the FFN used to be**, right after attention. For each token, it answers one question:

> **Which expert(s) should process this token?**

The router is a single weight matrix. Nothing more exotic than that. It is a small learned linear layer, the same kind of thing as every other weight matrix in the model.

Its shape. One matrix `W_r` with `d` rows (the model's hidden width) and one column per expert:

```
W_r :  d × N        (N = number of experts)
scores = x · W_r    →   N numbers, one per expert
```

Given the token's current hidden vector `x`, it produces one score per expert. Each column of `W_r` is one expert's **"preference vector"**. The score for an expert is the dot product of the token's vector `x` with that column, the same "how well do these two vectors line up?" operation as the Q·K comparison in attention (Chapter 4). A high score means "this token looks like what this expert likes".

A worked example

Suppose there are 4 experts, and for one token the router produces these scores. Softmax (Chapter 4) turns them into probabilities:

```
Expert        1       2       3       4
score       0.2     3.8     1.1     0.4
softmax   0.024   0.887   0.060   0.030        (these add up to 1)
```

The router thinks Expert 2 is by far the best match. With **top-2 routing**, we keep the two highest and ignore the rest:

```
Selected:  Expert 2 (0.887)   and   Expert 3 (0.060)
```

Often the two kept weights are re-normalized so they add up to 1 (here `0.937` and `0.063`). The token is then sent to just those two experts, and their outputs are **blended using the router's weights**:

```
y  =  0.937 · Expert2(x)  +  0.063 · Expert3(x)
```

With made-up expert outputs, `Expert2(x) = [1, 2]` and `Expert3(x) = [3, 0]`:

```
y = 0.937·[1, 2] + 0.063·[3, 0] = [0.937 + 0.189,  1.874 + 0]  =  [1.126, 1.874]
```

Experts 1 and 4 never run for this token. That is the saving.

(Real models differ in details: some keep the softmax weights as they are rather than re-normalizing them, some pick only one expert, and some pick eight. The structure is always the same: score, select a few, blend their outputs.)

Different tokens choose differently, at every layer

The routing decision is made **per token and per MoE layer**. Different tokens in the same sentence can pick different experts:

```
Token A  →  Experts 3 + 27
Token B  →  Experts 8 + 12
Token C  →  Experts 3 + 41
Token D  →  Experts 17 + 22
```

And the *same* token can take a different path at each layer, since each layer has its own router and its own set of experts:

```
Layer 1  →  Experts 3 + 7
Layer 2  →  Experts 1 + 4
Layer 3  →  Experts 6 + 8
```

So the model's huge parameter space is **conditionally activated**: a token only touches the part of it that its router chose.

One router per MoE layer (not one per matrix)

It's easy to get confused here, because the FFN contains *two* matrices (`W1` and `W2`, or three in SwiGLU). A block still has **one FFN sublayer**, and that FFN is made of two or three linear layers inside it. So in MoE:

```
One MoE layer  =  ONE router  +  many experts (each expert is a whole FFN: W1, W2, ...)
```

The router sits *outside* the experts. It does not route to "the first linear layer" or "the second linear layer". It routes to whole experts. If a model has 32 blocks and every one is an MoE layer, then it has **32 routers**, one per block, each with its own set of experts.

Does every block have to be MoE?

No. Many designs mix dense and MoE blocks. Some alternate them, and others, like DeepSeek-V3 (Chapter 21), keep the first few blocks dense and make the rest MoE. It's a design choice with a trade-off:

```
More MoE layers   → more capacity and specialization,  more routing overhead
Fewer MoE layers  → simpler and cheaper,                less sparse capacity
```

Part 4 — Counting parameters: total, active, and compute

You're right: the model gets bigger

A fair worry: "*An expert is just an FFN. If the block now has several FFNs instead of one, doesn't the model get bigger, not smaller?*"

Yes. **The total number of parameters goes up.** The trick is that we don't *use* all of them for every token. Take a made-up example. Start with a dense model:

```
Dense model:
   Attention             2B
   FFN                   1B
   ---------------------------
   Total                 3B       and every token uses all 3B
```

Now turn the FFN into 8 experts of 1B each, with a small router:

```
MoE model (8 experts, top-2):
   Attention             2B
   Expert 1 … Expert 8   8 × 1B = 8B
   Router                tiny
   ---------------------------
   Total                ~10B

   Used per token:   attention 2B  +  2 chosen experts (2 × 1B)  =  ~4B
```

So the model grew from 3B to about 10B in **total**, but each token only does about 4B worth of work. That is why MoE is called **sparse**.

But is that actually better? Choosing the fair comparison

Look carefully at those numbers, because they can mislead. Per token, this MoE model does about **4B** of work, while the dense model we started from did only **3B**. So the MoE model costs *more* per token than that 3B model. How can that be an improvement?

The answer is that a 3B dense model is **not the right thing to compare against**. A 3B model and a 10B-total MoE model hold very different amounts of knowledge. MoE is not "cheaper than every dense model". It is better than the *right* dense baselines. There are two fair ones:

Model	Total parameters (what it stores)	Active per token (what it computes)
Dense, 3B	3B	3B
MoE, 8 experts, top-2	**~10B**	**~4B**
Dense, 10B	10B	10B

```
Baseline 1: a dense model with the same COMPUTE per token  (about 4B)
   MoE:   ~4B active, ~10B stored
   Dense: ~4B active, ~4B stored
   → same cost per token, but the MoE stores about 2.5× more, so it has far more
     capacity and, in practice, noticeably better quality at that compute cost.

Baseline 2: a dense model with the same TOTAL size  (10B)
   MoE:   ~4B active, ~10B stored
   Dense: ~10B active, ~10B stored
   → the MoE does about 2.5× less work per token. Its quality usually lands between
     the dense 4B and dense 10B models, because a parameter in an expert is a little
     less effective than a parameter in a dense layer that every token trains.
```

So the claim of MoE is not "fewer active parameters than a smaller dense model". It is **better quality for a given amount of compute**, or equivalently **the same quality for less compute**. The real-world example, worked out below, is Mixtral 8x7B: it uses about 13B active parameters per token, and its authors report that it matches or beats the dense Llama 2 70B on their benchmarks, at roughly a fifth of the per-token compute.

And the price, as Part 7 explains, is **memory and complexity**: all ~10B parameters still have to be stored, plus the routing, the communication between GPUs and the load balancing. MoE saves *compute*, not memory.

Three different numbers

It is better not to say "the effective size is the same" or "the effective size is bigger". Separate three things:

	Question it answers
Total parameters	How much does the model *store*? (Memory and knowledge capacity)
Active parameters	How many are actually used for one token?
Compute	How much arithmetic does one token cost? (Roughly proportional to the active parameters)

This is why model specifications often read like "671B total, 37B active". Those numbers are not contradictory. They describe different things.

A real example, worked out: Mixtral 8x7B

Mixtral 8x7B has 8 experts per layer, and the router picks 2 for each token. Its published size is about 47B total and 13B active. Here is where those numbers come from. The model has 32 layers, hidden size `d = 4096`, an FFN width of 14,336, 8 K/V heads of size 128 (so K and V are `4096 × 1024`), and a vocabulary of 32,000:

```
One expert (a SwiGLU FFN: gate, up, down)   3 × 4096 × 14,336            ≈ 176 million
All 8 experts in one layer                   8 × 176M                     ≈ 1.41 billion
Attention in one layer (Q, O, K, V)          2×4096² + 2×4096×1024        ≈  42 million
Embedding table + LM head                    2 × 32,000 × 4096            ≈ 262 million

TOTAL    32 × (1.41B + 0.042B)  +  0.26B                                  ≈ 46.7 billion
ACTIVE   32 × (2 × 0.176B + 0.042B)  +  0.26B                             ≈ 12.9 billion
```

That matches the published figures. It also explains why the model is *not* "8 × 7B = 56B": only the FFNs are replicated eight times. The attention and the embeddings exist **once**.

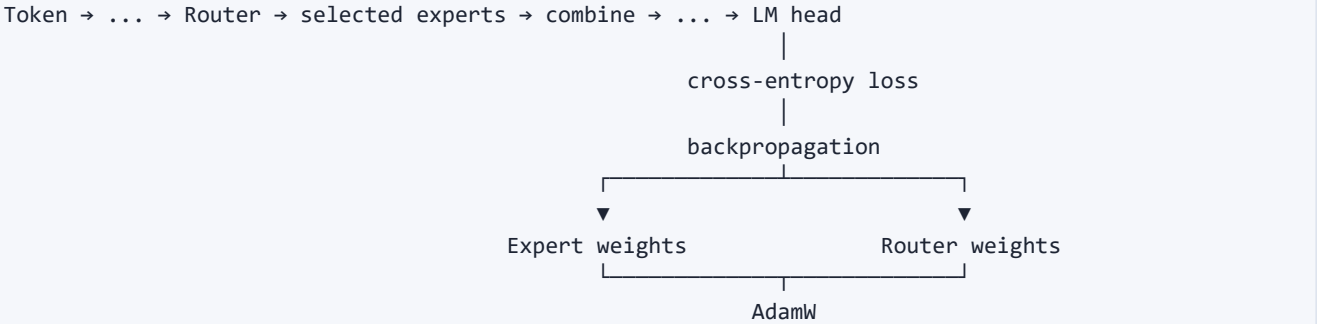
Part 5 — Training the router

Who decides which expert is good?

Nobody tells the router. We never say "Expert 1 does mathematics, Expert 2 does coding". The router is trained exactly like everything else, and this is the key idea:

> **The router is itself a learned neural layer, and its weights receive gradients like any other.**

Training is the loop from Chapters 9 to 11, with the experts and the router inside it:



At the start, the router knows nothing and routes more or less at random. Suppose the training example is a line of code, `def calculate_total(...)`, and the router sends it to Expert 3, which produces a poor prediction, so the loss is high. Backpropagation flows through the experts and *through the router's scores for the experts that were chosen*, nudging the router toward sending this kind of token somewhere that lowers the loss, say Expert 7. Over billions of tokens, routing patterns emerge.

Specialization emerges, but messily

As a result, experts often end up **handling different kinds of tokens**: perhaps code-like tokens tend to go to one expert, certain language patterns to another. But resist the neat picture of `Expert 1 = Maths, Expert 2 = Python, Expert 3 = French`. Real experts tend to respond to messy combinations of syntax, language, token type and local context, and the pattern can change from layer to layer. The roles are **never assigned. They emerge from optimization.**

The problem: expert collapse

Left alone, there's a trap. Suppose Expert 7 happens to be slightly better early on. The router sends it more tokens, so it gets more training, so it gets even better, so it gets even more tokens:

```
Expert 7   ████████████████
Expert 1   ██
Expert 2   ▌
Expert 3   ▌          ← the rest are barely used
```

This is called **expert collapse** or **routing imbalance**. It wastes most of the model's capacity, and on real hardware it is a performance disaster too (Part 7), because the busy expert's GPU is overloaded while the others sit idle.

The fix: a load-balancing loss

MoE training adds a second goal to the loss from Chapter 9:

```
Total loss  =  language-model loss   +   load-balancing loss
                "predict the next       "don't overload a few experts"
                 token correctly"
```

One common form of the balancing term looks at two numbers for each expert `i` over a batch: `f_i`, the fraction of tokens actually sent to it, and `P_i`, the average router probability it received. The penalty is `N × Σ f_i × P_i` (with `N` experts), which is smallest when work is spread evenly. With 4 experts:

```
Perfectly balanced:  f = [0.25, 0.25, 0.25, 0.25],   P = [0.25, 0.25, 0.25, 0.25]
                     penalty = 4 × (4 × 0.25 × 0.25)                          = 1.00

Imbalanced:          f = [0.75, 0.125, 0.125, 0],    P = [0.6, 0.2, 0.1, 0.1]
                     penalty = 4 × (0.75·0.6 + 0.125·0.2 + 0.125·0.1 + 0)    = 1.95
```

The imbalanced router is penalized almost twice as heavily, so gradient descent pushes it toward spreading its tokens out. (This is the form used by Switch Transformer. Models differ in the details.)

There is a known side effect: this extra term pulls on the model's main objective. Newer designs try to balance load **without** a loss term. DeepSeek-V3, for example, adds a small adjustable bias to each expert's routing score *only when choosing experts*, nudging it up for under-used experts and down for over-used ones (Chapter 21).

Capacity limits

Some systems also give each expert a fixed **capacity** per batch. If too many tokens choose the same expert, the overflow tokens may be **dropped** from that layer (they simply skip it and carry on through the residual connection) or re-routed. It's another way imbalance shows up as a real cost.

Part 6 — Why the FFN, and not attention?

Two questions come up together here, and both reveal something about MoE.

Why replace the FFN rather than the attention?

Because **the FFN is where most of a block's parameters live**. (Recall from Chapter 6 that the FFN expands the vector to roughly four times its width and back. In Mixtral's numbers above, the FFN is about 176M per expert against about 42M for the whole attention in a layer.) Swapping one FFN for 64 experts multiplies the model's capacity enormously while each token still runs only a few of them. It is also relatively easy to make sparse: each token's FFN computation is independent of every other token's.

Attention is different. Its whole job is for **each token to relate to the other tokens** in the context. Making it sparse by routing tokens to different "attention experts" runs into trouble with communication, keeping the KV cache consistent, preserving global relationships between tokens, and GPU efficiency. So the common design is:

> **Keep attention dense. Make the expensive, parameter-heavy FFN sparse.**

Wouldn't an MoE attention reduce the KV cache?

It's a sharp idea, but no, not automatically. Recall what the KV cache is (Chapter 12): for every token, the Keys and Values at every layer, so its size is set by

```
tokens  ×  layers  ×  number of KV heads  ×  head dimension
```

It is not determined by how many "experts" you have. If you routed tokens through several attention experts, each with its own K and V, you could actually make the cache **larger**. Making a token look at less could save compute, but that is not the same as shrinking the cache, and it might reduce the information the model builds about that token.

If the goal is a smaller KV cache, there are techniques built for exactly that: **MQA, GQA and MLA** (Chapter 17). The right way to see it is that each technique attacks a *different* bottleneck:

Problem	Technique
Recomputing previous tokens	KV cache (Chapter 12)
Too much weight memory	Quantization (Chapter 14)
KV cache too large	MQA / GQA / MLA (Chapter 17)
Too many active parameters per token	**MoE** (this chapter)

Part 7 — Running an MoE model: what really happens to one token

On paper, MoE saves compute. In practice, it creates new engineering problems, because real models are spread across many GPUs (Chapter 23).

One token, step by step

Take a model with 64 experts and top-2 routing:

```
1. The token's hidden vector goes to the router → 64 scores
2. The top 2 are chosen          e.g. Expert 3 (0.71) and Expert 27 (0.18)
3. The token is sent to those two experts; each runs its FFN
4. The two outputs are blended with the router weights
5. The result continues to the next block (which has its own router)
```

The hardware problem: experts on different GPUs

The 64 experts are probably spread across GPUs:

```
GPU 1 → Experts 1–8        GPU 2 → Experts 9–16       ...       GPU 8 → Experts 57–64
```

A token that picks Expert 3 and Expert 27 has to be **sent to two different GPUs**, processed there, and its results brought back. This is **expert parallelism** (Chapter 23), and the tokens travel between GPUs in an "all-to-all" exchange at every MoE layer.

Why batching matters even more

Sending a *single* token across GPUs costs more in communication than the computation is worth. But with **10,000 tokens** at once, the router can group them by expert:

```
Expert 1  ←  1,000 tokens
Expert 2  ←    700 tokens
Expert 3  ←  1,200 tokens
...
```

Now each GPU receives a large batch for its experts, which is the efficient way to use it. That is one reason MoE serving systems rely on sophisticated scheduling (Chapter 22).

It also explains why **load balance matters physically**. If 90% of tokens go to Expert 3, that GPU is overloaded while the others idle, and the whole step waits for the slowest GPU.

The catch: MoE saves compute, not memory

Here's the point people most often miss. **All the experts have to be stored in GPU memory, even though only a few run for each token.** An MoE model with 47B parameters needs memory for 47B, even though each token only computes with 13B of them.

And during generation (decode), which is memory-bound (Chapter 12), the picture gets subtle. For a single token, only the chosen experts need to be read. But once you serve a *batch*, different tokens choose different experts, so most of the experts get touched in each step:

```
Mixtral-style (8 experts, top-2), tokens in the batch (if routing is roughly even):
   1 token   → about 2 of 8 experts read
   4 tokens  → about 5.5 of 8
   8 tokens  → about 7.2 of 8

DeepSeek-V3-style (256 routed experts, top-8):
   1 token   → 8 experts      32 tokens → about 163      128 tokens → about 252
```

(These are idealized numbers assuming routing is spread evenly. Real routing is lumpier.) So at serving batch sizes, an MoE model reads **almost all of its weights every step**. Its advantage is mostly in **compute per token**, not in memory traffic. That is why quantization (Chapter 14) and fast memory matter so much for MoE models.

The full set of trade-offs

```
Gains:   far more total capacity at about the same compute per token
Costs:   routing complexity, communication between GPUs, load balancing,
         memory for ALL the experts, and implementation difficulty
```

Part 8 — MoE in real models

Model	What it did	Numbers
Switch Transformer (2021)	Simplified routing to **one expert per token** (top-1); showed MoE could scale to trillions of parameters and train faster than a dense model of equal compute	Reported up to 7× faster pre-training than the dense T5 baseline; trained in bfloat16
Mixtral 8x7B (2023)	A well-known open MoE: **8 experts per layer, 2 chosen per token**	≈ 47B total, 13B active
DeepSeekMoE (2024)	**Fine-grained experts** (many small ones, so combinations are more flexible) plus **shared experts** that always run, to hold knowledge every token needs	Chapter 21
Qwen3 MoE (2025)	128 experts with 8 active per token, no shared expert	e.g. 235B total, 22B active (Chapter 21)
DeepSeek-V3 (2024)	1 shared + 256 routed experts, 8 routed active per token, first 3 blocks dense, no auxiliary balancing loss	671B total, 37B active (Chapter 21)

The trend is visible: more and smaller experts, a shared expert for common knowledge, and cleverer ways to keep load balanced.

Common misconceptions, cleared up early

Misconception	Reality
"MoE is a runtime trick that doesn't change the model."	It changes the architecture: the FFN of the block becomes a router plus many expert FFNs.
"Each expert is a specialist in one subject."	Specialization emerges during training and is messy. Roles are never assigned, and they vary by layer.
"The router follows hand-written rules."	It's a small learned linear layer, trained by backpropagation along with the experts.
"An MoE block has one router per linear layer inside the FFN."	One router per MoE layer, which sends the token to whole experts (each expert is a full FFN).
"An MoE model is smaller than a dense model of the same quality."	It usually has *more* total parameters. It just uses a small fraction of them for each token.
"An MoE model does less work per token than any dense model."	Only compared with a dense model of the same *total* size. Compared with a smaller dense model it may do slightly more work, but it stores far more knowledge. The fair comparison is quality per unit of compute.
"671B total, 37B active means the numbers disagree."	Total is what is stored. Active is what one token uses. They measure different things.
"Mixtral 8x7B has 8 × 7B = 56B parameters."	Only the FFNs are replicated. Attention and embeddings exist once, giving about 47B.
"MoE saves memory."	It saves compute. All experts must be kept in memory, and at serving batch sizes most are read every step.
"Every token goes to the same experts at every layer."	Each layer has its own router, so a token can take a different path at each layer.
"Making attention MoE would shrink the KV cache."	The cache is set by tokens × layers × KV heads × head size. Shrinking it is the job of MQA, GQA and MLA.
"Balanced routing happens by itself."	Left alone, routers collapse onto a few favourites. Training adds a load-balancing incentive (or a bias trick).
"Every block of an MoE model is an MoE block."	Not necessarily. Many models mix dense and MoE blocks (DeepSeek-V3 keeps its first three dense).

Quick reference

```
MoE block:    Attention (dense)  →  Router  →  top-k of N expert FFNs  →  weighted blend
              (the single FFN of Chapter 6 becomes N experts + a router)

Router:       scores = x · W_r  →  softmax  →  keep top-k (often re-normalized)
              y = Σ (weight_i × Expert_i(x))      one router per MoE layer; per token, per layer
              learned by backprop; sits where the FFN used to be

Counting:     total parameters = stored;  active = used by one token;  compute ≈ active
              Mixtral 8x7B: 32 layers, 8 experts, top-2  →  ≈ 46.7B total, ≈ 12.9B active
              (only FFNs replicated; attention + embeddings once)

Training:     loss = LM loss + load-balancing loss      (penalty N·Σ f_i·P_i, min = 1 when balanced)
              or aux-loss-free: adjustable per-expert bias used only for selection (DeepSeek-V3)
              failure mode: expert collapse;   capacity limits can drop overflow tokens

Why FFN:      most parameters live there; easy to make sparse; keep attention dense
Serving:      experts spread over GPUs (expert parallelism, Chapter 23); tokens travel all-to-all;
              needs big batches; MoE saves COMPUTE, not MEMORY (all experts stored,
              most read each step at serving batch sizes)

Real models:  Switch (top-1), Mixtral (8, top-2), DeepSeekMoE (fine-grained + shared),
              Qwen3 (128, 8 active), DeepSeek-V3 (1 shared + 256 routed, 8 active; 671B / 37B)
```

What's next

We've now seen how the *inside* of a block can change, and how a model can grow far beyond what it computes for any one token. The next change is about what the model can *take in*. So far every input has been text. Chapter 19 covers **multimodal models**: how an image or a sound is turned into vectors that a language model can read alongside words, and how the two spaces are taught to understand each other. You'll revisit the encoders and cross-attention from Chapter 15 in a practical setting.

Chapter 19 — Multimodal Models: Letting a Language Model See and Hear

Introduction

Everything in this handbook so far has been about text. A GPT reads tokens and writes tokens. But people want to show a model a photo, a screenshot or a chart, play it a recording, or hand it a video, and ask questions about them. How can a language model, which only understands vectors made from text tokens, deal with a *picture*?

The surprising answer is that **the language model itself barely changes**. Its core loop is still exactly the one from Chapter 1:

```
Context → Transformer → LM head → next token
```

What changes is what gets put into the context. The recurring idea of this chapter is:

> **The Transformer doesn't need to understand everything directly. We can build an *encoder* that converts another kind of input (an image, a sound) into vectors the model can use, and place those vectors beside the text.**

This chapter follows one image on its journey into a language model:

1. **The problem**: a picture is just a grid of numbers, not tokens.
2. **The vision encoder**: cutting the image into patches and turning each into a vector.
3. **Teaching an encoder to understand pictures**: the CLIP idea.
4. **The projection**: translating those vectors into the language model's own space.
5. **Two ways to plug it in**: visual tokens in the sequence, or cross-attention.
6. **The cost**: how many tokens a picture is worth.
7. **How the combined model is trained**: how two models that grew up separately learn to understand each other.
8. **Audio, video, and what the model can produce.**

You'll also revisit two ideas from earlier chapters in a practical setting: the **encoder** and **cross-attention** from Chapter 15, and the **loss mask** from Chapter 13.

Part 1 — The problem: pixels are not tokens

A text model sees this:

```
Text → Tokenizer → Token IDs → Embeddings → Transformer → Text
```

Now suppose we give it an image plus a question. To the computer, an image is just numbers: a grid of pixels, each with red, green and blue values.

```
one pixel:  [255, 120, 83]
a 224 × 224 colour image:  224 × 224 × 3  =  150,528 numbers
```

The Transformer has no idea what to do with 150,528 raw numbers. And we can't just treat every pixel as a token. A 224 × 224 image would be 50,176 tokens, and attention cost grows with the square of the length (Chapter 17). We need a smarter way to turn an image into a **short sequence of meaningful vectors**.

The plan, in three steps:

```
Image  →  Vision encoder  →  Projection  →  vectors that look like text-token vectors  →  LLM
```

Part 2 — The vision encoder: an image as a sequence of patches

Why patches?

The trick behind the **Vision Transformer (ViT)**, whose paper is titled "An Image Is Worth 16×16 Words", is to cut the image into small square **patches** and treat each patch like a word. For a 224 × 224 image with 16 × 16 patches:

```
┌────┬────┬────┬────┐ ··· ┐
│ P1 │ P2 │ P3 │ P4 │     │      14 patches across × 14 patches down
├────┼────┼────┼────┤ ··· ┤      = 196 patches
│P15 │P16 │P17 │P18 │     │
├────┼────┼────┼────┤ ··· ┤      each patch is 16 × 16 pixels × 3 colours
│ ···│ ···│ ···│ ···│     │      = 768 numbers
└────┴────┴────┴────┘ ··· ┘
```

Instead of 50,176 pixels, we now have 196 patches. This is the image's equivalent of the tokenizer: it chops the input into pieces.

Turning a patch into a vector

Each patch is *flattened* into a list of numbers and multiplied by a learned matrix to become a vector of the model's width. This is the image's equivalent of the embedding lookup of Chapter 3. A tiny example with a 4 × 4 grayscale image and 2 × 2 patches:

```
Image (grayscale pixel values)          Patches (2 × 2), each flattened

   10   20 |  30   40                     Patch 1 = [10, 20, 50, 60]
   50   60 |  70   80                     Patch 2 = [30, 40, 70, 80]
  ------------------------                Patch 3 = [90, 100, 130, 140]
   90  100 | 110  120                     Patch 4 = [110, 120, 150, 160]
  130  140 | 150  160

Learned projection W (4 → 2 numbers), for illustration:
        [0.1  0  ]
   W =  [0    0.1]          Patch 1 · W = [10·0.1 + 50·0.1,  20·0.1 + 60·0.1] = [6, 8]
        [0.1  0  ]
        [0    0.1]
```

In a real model, `W` is a large learned matrix that maps each 768-number patch to a vector of hundreds or thousands of numbers, and its values are learned, not hand-picked.

Position matters here too

Just like words in a sentence, patches have an order: the patch at the top-left is not the same as the one at the bottom-right. So, exactly as in Chapter 3, a **position embedding** is added to each patch vector so that the model knows *where* in the image the patch came from.

The encoder itself

The sequence of patch vectors then passes through ordinary **Transformer blocks**: attention, FFN, residuals and normalization, exactly the block from Chapter 7. This is an **encoder** in the sense of Chapter 15: each patch can attend to *every other patch* (there is no causal mask, since there is no "next patch" to predict). So a patch showing a dog's ear can learn from patches showing the rest of the dog.

```
Patch vectors (+ position)  →  Transformer blocks (every patch sees every patch)  →  N output vectors
                                                                                      (one per patch, now "aware" of context)
```

The result is a set of **visual tokens**: one rich vector for each patch.

```
Image  →  Patch 1 → vector
          Patch 2 → vector
          Patch 3 → vector
          ...
          Patch N → vector          (like Text → Tokens → vectors)
```

Part 3 — Teaching an encoder to understand pictures: the CLIP idea

A vision encoder with random weights would produce meaningless vectors. How does it learn what a "dog" looks like? One of the most influential answers is **CLIP**, which trains an image encoder and a text encoder **together** on about 400 million image–caption pairs from the internet. Its task is beautifully simple: *predict which caption goes with which image.*

Take a batch of three pairs. Encode every image and every caption, then compute a score for every possible image–caption pairing, giving a grid:

```
                    caption 1         caption 2        caption 3
                   "a dog on        "a red car      "a bowl of
                    a beach"         at night"        soup"
                  ┌──────────────────────────────────────────┐
image 1 (dog)     │   HIGH           low             low     │   ← the diagonal is
image 2 (car)     │   low            HIGH            low     │     the true pairs
image 3 (soup)    │   low            low             HIGH    │
                  └──────────────────────────────────────────┘
```

Training pushes the **diagonal scores up** (each picture should match its own caption) and **everything else down**. With millions of examples, the image encoder learns to produce vectors that land close to the vectors of the words describing them. That is the essential achievement: a shared meaning space where a photo of a dog and the words "a dog on a beach" end up nearby.

This is why a CLIP-style vision encoder is the common starting point for multimodal models. By the time it is plugged into a language model, it already knows how to turn pictures into vectors that capture *what is in them*. What it does not yet know is how to speak the particular language of *this* language model. That is the next step.

Part 4 — The projection: translating into the language model's space

The vision encoder produces vectors in its own space, with its own width and its own meaning. The language model expects vectors in *its* space, of the same kind as the token embeddings of Chapter 3. A small **projection layer** (often a linear layer or a small MLP) translates one into the other:

```
Image  →  Vision encoder  →  visual vectors  →  PROJECTION  →  vectors in the LLM's space
```

After the projection, the picture's vectors look to the language model just like word vectors. They can sit in the same sequence as the text:

```
Image  →  Vision encoder  →  Projection  →  ┌──────────────────────┐
                                            │ visual tokens        │
"What is the dog doing?"  →  Tokenizer  →   │ + text tokens        │  →  LLM  →  Text
                                            └──────────────────────┘
```

A full example

You upload a picture of a dog and ask "*What is the dog doing?*" The model receives one sequence:

```
[visual token 1] [visual token 2] ... [visual token N]   What  is  the  dog  doing  ?
└──────────── the image, as vectors ──────────────┘         └──────── the question, as vectors ────┘
```

The LLM processes all of it with its ordinary Transformer blocks and attention, and the text tokens can attend to the visual tokens. Then it generates, one token at a time as always:

```
"The"  "dog"  "is"  "running"  "."
```

Nothing about the language model's own mechanism changed. It just has some extra vectors in its context.

Part 5 — Two ways to plug the image in

The picture above, with the visual tokens placed in the sequence, is one of two common approaches. The second one uses the **cross-attention** of Chapter 15.

Approach A: visual tokens go into the sequence

```
Image → Vision encoder → Projection → visual tokens ┐
                                                    ├→  one sequence  →  LLM
Text  → Tokenizer → text tokens ────────────────────┘
```

The language model treats image tokens like extra words. Its architecture is unchanged, and ordinary self-attention lets the text look at the image. This is simple and popular: open models such as LLaVA and many others follow it.

Approach B: cross-attention

Recall the difference between the two kinds of attention (Chapter 15):

```
Self-attention:    Q ← text,   K ← text,    V ← text      (the text looks at itself)
Cross-attention:   Q ← text,   K ← image,   V ← image     (the text looks at the image)
```

Here, the text representation asks: "*which parts of the image are relevant to me?*" For example, with the question "What colour is the car?":

```
Question: "What colour is the car?"
                  │  Q
                  ▼
           CROSS-ATTENTION   ◄── K, V from the image features
                  │
        Image features:  sky ▒   tree ▒   road ▒   car ███   ← high attention
```

In this design, extra cross-attention layers are added inside the language model, and the image enters through them rather than through the main sequence. Models in the style of Flamingo, which bridged a frozen vision model and a frozen language model with cross-attention (plus a "resampler" that squeezes many patches into a fixed number of vectors), took this route, and so do some more recent models.

```
Text → ... ─► Q ─┐
                 ▼
          CROSS-ATTENTION ◄── K, V from image features
                 ▼
                LLM
```

Comparing them

	A: visual tokens in the sequence	B: cross-attention
How the image enters	As extra tokens in the context	Through added cross-attention layers
Change to the language model	None	New layers are added
Cost	Image tokens use up context length and KV cache	Image does not lengthen the text sequence
Can the original model stay frozen?	Often only the projector is trained at first	Often the original layers are frozen and only the new ones trained
Typical examples	LLaVA-style models	Flamingo-style models

A third family trains everything **natively**: text and image tokens are mixed in the same training data from the start, so one model learns both together rather than gluing two models. Modern multimodal systems use variations and combinations of these three ideas. All of them still obey the principle from the introduction: other modalities are turned into vectors the Transformer can use.

Part 6 — The cost: how many tokens is a picture worth?

With Approach A, every visual token takes up room in the context window, and in the KV cache (Chapter 12), exactly like a text token. So it matters how many a picture produces:

Input	Typical tokens
A 224 × 224 image, 16 × 16 patches	196
A 336 × 336 image, 14 × 14 patches (a common setting)	576
A high-resolution image split into several tiles (say 4 tiles plus an overview)	≈ 2,900

An image of 576 tokens costs about as much context as a page of text. And in the KV cache of an 8B-class model at 128 KiB per token (Chapter 12), those 576 tokens take about **72 MiB**, for one image. A conversation with ten images is already hundreds of MiB, and attention cost grows with the square of the total length (Chapter 17). Higher-resolution images give the model finer detail, but cost many more tokens. This is why practical systems resize images, tile large ones, and sometimes compress the number of visual tokens before they reach the language model.

Part 7 — How a multimodal model is trained

Here is the real puzzle. We have two models that grew up separately:

- A **vision encoder** that understands images but knows no language.
- A **language model** that understands text but has never seen a picture.

> If the LLM already understands language and the vision encoder already understands images, **how do we teach their two spaces to understand each other?**

The usual recipe works in stages, and it should look very familiar, because it's built from the fine-tuning tools of Chapter 13:

Stage 1: Align the spaces (train the projection). Keep the vision encoder and the language model **frozen**, and train only the small projection layer. The data are image–caption pairs, and the task is the usual next-token prediction: given the image tokens, predict the caption. Because almost nothing else is allowed to change, the projection is forced to learn the translation: *make this image's vectors land where the language model can read them*.

Stage 2: Visual instruction tuning. Now unfreeze the language model (fully, or with LoRA, Chapter 13) and train it together with the projector on **instruction data with images**: a picture, a question or instruction, and a good answer. This is exactly supervised fine-tuning, and it is what teaches the model to *follow instructions about images* rather than just caption them. A well-known example, LLaVA, generated much of this instruction data by asking a text-only GPT-4 to write questions and answers from image captions.

Stage 3 (often): Preference tuning, with the methods from Chapter 13, to make the answers more helpful and more reliable.

The loss mask appears again

In Stages 1 and 2, the loss is cross-entropy on the next token, with the **loss mask of Chapter 13**: the model *reads* the image and the question, and is *graded only on the answer*.

```
[ image tokens × 576 ]  "What is the dog doing?"   →   "The dog is running."
└── context only ──────┘  └── context only ────────┘     └── loss computed here ──┘
```

Nothing new is needed in the training algorithm. It's the same pre-training machinery, with different data and a loss mask.

Part 8 — Audio, video, and what comes out

Audio

The same recipe applies. An **audio encoder** turns sound into vectors, and a projection places them in the language model's space:

```
Audio  →  Audio encoder  →  representations  →  projection  →  LLM
```

A typical audio encoder first converts the waveform into a **spectrogram** (a picture of how much energy there is at each frequency over time), then processes it with Transformer layers, like a vision encoder does for images. Speech encoders such as Whisper's produce roughly 50 vectors per second of audio, so a minute of audio is about 3,000 positions.

Video

A video is a sequence of images, so the straightforward approach is to sample frames (say one per second) and encode each one. The cost grows fast: one frame per second at 576 tokens each is 576 tokens *per second*, or over 34,000 tokens for one minute. Video models therefore sample sparsely and compress visual tokens aggressively.

```
Video  →  sampled frames  →  vision encoder  →  (compress)  →  visual tokens  →  LLM
```

What comes out?

Most "multimodal LLMs" take images (and sometimes audio or video) **in** and produce **text** out. Producing images or audio is a different job. It is usually handled by a separate generator (for images, often a diffusion model), or by models that represent images and audio as their own kinds of discrete tokens so that the same next-token machinery can output them. The principle is the same either way: the Transformer works on sequences of vectors, and what those vectors *mean* is decided by the encoders and decoders around it.

The pattern to remember

```
TEXT   →  Tokenizer       →  text tokens ──────┐
IMAGE  →  Vision encoder  →  visual tokens ────┤
AUDIO  →  Audio encoder   →  audio tokens ─────┼──►  Transformer  →  LM head  →  Text
VIDEO  →  Video encoder   →  video tokens ─────┘
```

Common misconceptions, cleared up early

Misconception	Reality
"A multimodal model works with raw pixels."	It first converts the image into a short sequence of vectors with an encoder. Raw pixels would be far too long.
"The language model is redesigned to handle images."	In the common approach it is unchanged. The image arrives as extra vectors, produced by an encoder and a projection.
"Each image is one token."	An image is typically hundreds or thousands of tokens, and they use up context and KV cache like any other tokens.
"The vision encoder reads patches in order, like text."	It is an encoder: every patch can attend to every other patch, with no causal mask.
"CLIP generates captions."	CLIP only learns which captions match which images. It gives a shared meaning space and does not write text.
"The projection layer understands the image."	It is a small translator between two spaces. The understanding comes from the vision encoder and the language model.
"Multimodal training needs a new loss function."	It is the usual next-token cross-entropy on the answer, with the loss mask from Chapter 13.
"The model sees images and text in different ways internally."	After the projection, both are just vectors in the same sequence, processed by the same attention.
"Multimodal models generate images the same way they write text."	Most take images in and produce text. Producing images needs a separate generator, or a design where images are represented as tokens.
"Video is just one more image."	A video is many frames. The token cost grows with its length, so models sample and compress frames.
"Cross-attention and visual tokens are the same."	Visual tokens lengthen the sequence and use ordinary self-attention. Cross-attention adds layers where text queries the image.

Quick reference

```
Problem:    an image is a grid of numbers (224×224×3 = 150,528); can't be raw tokens
Pipeline:   Image → Vision encoder → Projection → (visual tokens) → LLM → text
            Audio / video:  same idea with an audio encoder / sampled frames

ViT:        cut image into patches (16×16 → 196 patches for 224×224)
            flatten each patch → learned projection → vector;  + position embedding
            ordinary Transformer blocks; EVERY patch attends to every patch (encoder, no causal mask)

CLIP:       train image + text encoders on ~400M image–caption pairs;
            similarity grid: push the diagonal (true pairs) up, the rest down → shared meaning space

Projection: a small layer translating vision-encoder vectors into the LLM's embedding space

Two ways in:
  A  visual tokens in the sequence   (LLaVA-style; simple; costs context and KV cache)
  B  cross-attention: Q ← text, K,V ← image   (Flamingo-style; added layers; image not in sequence)
  C  native: train on mixed text and image tokens from the start

Cost:       224px/16 → 196 tokens;  336px/14 → 576 tokens (≈ 72 MiB KV cache at 128 KiB/token);
            video: 576 tokens × frames per second;  audio ≈ 50 positions per second

Training:   1) freeze encoder and LLM, train only the projection on image–caption pairs
            2) visual instruction tuning: train LLM (full or LoRA) + projection on image+instruction→answer
            3) preference tuning
            loss = next-token cross-entropy on the ANSWER only (loss mask, Chapter 13)

Output:     usually text; images/audio out need a separate generator or token-based design
```

What's next

That completes the tour of how a model is built: how its blocks have been improved (Chapters 16 and 17), how its FFN can become a team of experts (Chapter 18), and how it can take in images and sound (this chapter). Next, Chapter 20 turns from *what the model is made of* to *how it behaves on hard problems*. **Reasoning models** don't have a new architecture at all. They are trained, and used at inference time, so that the model thinks at length before it answers. You'll see why that helps, how such models are trained with verifiers and reinforcement learning, and what it costs.

Chapter 20 — Reasoning Models: Test-Time Compute, Thinking Budgets and Reasoning Effort

Introduction

So far, we have treated a language model as a machine that reads a prompt and starts writing the answer straight away, one token after another. Chapter 18 showed how to make such a model *bigger* without making it slower, and Chapter 19 showed how to give it new senses. This chapter is about a completely different way to make a model better at hard problems:

> **Let it think before it answers.**

If you have used one of the newer "reasoning" or "thinking" models, you have probably noticed two things:

1. The model shows (or at least spends time producing) a block of **reasoning text** before it gives its final answer.
2. You can often choose **how hard it thinks**: low, medium or high.

Both observations are the surface of one idea, and both have a precise explanation. This chapter builds it from the ground up:

1. **Why thinking longer helps**, even though the model is "just predicting tokens".
2. **Test-time compute**: a new way to scale a model, besides making it bigger.
3. **How a reasoning model is trained**: reinforcement learning with *verifiers*, and a concrete training iteration.
4. **Verifiers**: what they are when there is no calculator.
5. **Spending more compute at answer time**: longer chains, many samples, search.
6. **Reasoning effort**: what the Low / Medium / High control really does, and why the reasoning is sometimes visible.
7. **The costs**: KV cache, memory, latency.
8. **Combining it with quantization**, and where reasoning ends and agents begin.

One clarification up front, because it is the most common misunderstanding:

> **A reasoning model usually has no new architecture.** It is the same Transformer as before. What changes is how it was *trained* and how it is *used at inference time*.

Part 1 — Why thinking longer helps

The same model, two behaviours

Take a simple question: **What is 17 × 24?**

A model that answers directly might produce:

```
17 × 24 = 408
```

A reasoning model might produce:

```
17 × 24
24 = 20 + 4
17 × 20 = 340
17 × 4  = 68
340 + 68 = 408
Therefore the answer is 408.
```

Both are just tokens, generated one at a time by the loop from Chapter 1. There is no special "reasoning engine" hidden in the Transformer. So why would the second approach be more reliable?

A Transformer does a fixed amount of work per token

Recall the structure of the model (Chapter 8): a token passes through **N blocks**, and then the LM head produces the next token. That is a fixed number of steps, no matter how hard the question is. The model cannot loop inside a single forward pass until it is satisfied. For a question that needs many sequential steps (such as multi-digit arithmetic, a logic puzzle or a multi-step proof), N blocks may simply not be enough to get from the question straight to the answer in one go.

Now look at what happens when the model writes out intermediate steps. Each generated token is **fed back in as context** (Chapter 12), so the next token can attend to it:

```
Question
   ↓
Step 1  (a few tokens)       ← becomes context
   ↓
Step 2  (reads Step 1)       ← becomes context
   ↓
Step 3  (reads Steps 1 and 2)
   ↓
Answer  (reads all of it)
```

The model has created a **scratchpad using its own context**. The weights do not change during inference. Instead, the model *computes through a sequence of generated representations*: each new token can use the results of the earlier ones. In effect, writing more tokens buys more sequential computation, because every token is another pass through all N blocks.

That is the core reason "thinking out loud" works:

```
Direct answer:    Question ──────────────────────────────► Answer
                  (one pass; fixed amount of computation)

With reasoning:   Question ► step ► step ► step ► step ► Answer
                  (many passes; each can use what came before)
```

(This idea predates "reasoning models". In 2022, **chain-of-thought prompting** showed that simply showing a large model a few worked examples with intermediate steps, or even just asking it to "think step by step", improved its performance on arithmetic and logic problems a great deal. What is new is that models are now *trained* to do this on their own.)

Part 2 — Test-time compute: a new way to scale

Until now, there were three ways to make a model stronger:

```
More parameters       (7B → 70B → 400B)          how much the model can hold
More training         (more data, more compute)   how well it was taught
```

Reasoning models add a third dimension:

```
More computation at INFERENCE time, spent on the particular problem in front of it
```

This is called **test-time compute** (or inference-time compute). Different problems deserve different amounts of effort:

```
Easy:    "What is 2 + 2?"          → a very short answer
Hard:    "You have 5 boxes..."     → a long chain of reasoning, with checking
```

So inference becomes **adaptive**: easy questions get short reasoning and hard ones get long reasoning. It is worth separating the three kinds of "scale":

	What it measures	Example
Model size	How much learned capacity the model has	7B → 70B
Training compute	How much computation was spent *teaching* it	Weeks on thousands of GPUs
Test-time compute	How much computation it spends *on your problem*	Low → High reasoning

Research has found this trade is real. One study found that on problems where a smaller model has a non-trivial success rate, spending extra compute at inference time in a well-chosen way could match or beat a model 14 times larger. This is a very different scaling strategy from "just make the model bigger".

A connection to Chapter 23: speculative decoding asks *"how do we generate the same tokens faster?"* Reasoning asks the opposite: *"when does it pay to generate more tokens on purpose?"*

Part 3 — How a reasoning model is trained

The limit of ordinary fine-tuning

With normal supervised fine-tuning (Chapter 13) we show the model a question and a good answer and train it to reproduce the answer. To teach reasoning that way, we would need examples of **good reasoning**:

```
Question:   If 3x + 5 = 20, what is x?
Reasoning:  3x = 15,  x = 5
Answer:     5
```

We can use such examples, and they do teach reasoning patterns. But there's a problem: **people cannot write millions of high-quality reasoning examples**, and imitating human-written steps may not be the best way for a model to reason anyway. So researchers looked for a different source of supervision.

The idea: reward correct answers, not good-looking steps

For many problems, we can check automatically whether the final answer is correct:

```
Math:       17 × 24 = ?   → a calculator or solver checks 408
Code:       write a sorting function → run the tests → pass or fail
Chess:      a game engine
Puzzles:    a known solution or checker
```

That automatic checker is a **verifier**. Now we can use the reinforcement-learning machinery you met in Chapter 13, but with a different reward source:

```
RLHF  (Chapter 13):   LLM → answer → reward model trained from HUMAN PREFERENCES → reward
RLVR  (this chapter): LLM → reasoning + answer → VERIFIER ("is it actually correct?") → reward
```

This is called **reinforcement learning with verifiable rewards (RLVR)**. The reward can be as simple as:

```
Correct = 1        Wrong = 0
```

The model is then trained to produce *reasoning that leads to correct answers*. And just as with RLHF, a human is no longer in the loop for every example.

Why is this so powerful? **Human preference is subjective.** For "write a good explanation", two answers can both be good, and a perfect reward is hard to define. But `17 × 24 = ?` has an objective answer, and "does this program pass its tests?" can be checked by running them. In domains where correctness can be checked, the training signal is much stronger and cannot easily be fooled.

One complete training iteration, with a tiny model

Take our TinyGPT (the tiny model of Appendix A) and follow one iteration:

> **There are 3 boxes with 4 apples in each. How many apples?**

```
1. Tokenize the question            → token IDs
2. Embeddings → Transformer blocks → LM head    (nothing special so far)
3. The model GENERATES reasoning, one token at a time:
       "There are 3 boxes. Each box has 4 apples. 3 × 4 = 12. Therefore the answer is 12."
4. The VERIFIER compares the answer with the expected one:   12 = 12  →  correct
5. Reward  R = 1
6. RL objective:  "make this kind of trajectory more likely"
7. Backpropagation → gradients → AdamW → the model's weights change
```

Every reasoning token is an ordinary next-token prediction. The only new ingredient is the *reward*, which tells the training procedure whether the whole trajectory was a good one. Over many examples, the probability of reasoning patterns that lead to correct answers goes up. Given the weights, the same prompt will then tend to produce more of that kind of thinking.

Notice also what the verifier is **not**: it is **a training tool**. After training, a user's question goes straight to the reasoning model, which reasons and answers. The verifier isn't there. It is exactly like the reward model in RLHF, which Chapter 13 showed is discarded after training. There are three separate roles:

```
LLM          generates the reasoning
Verifier     judges the result
RL algorithm uses that judgment to change the LLM
```

What does the RL algorithm look like? GRPO

Chapter 13 showed classic RLHF with PPO, which needs four models in memory, including a "value model" (a critic) that estimates how good each situation is. For reasoning, a simpler method became popular, called **GRPO** (Group Relative Policy Optimization), introduced for training mathematical reasoning. Its trick is to drop the critic and use a **group of answers to the same question** as its own baseline.

For each question, the model samples a group of, say, 4 attempts, and each gets a reward from the verifier. Then each attempt's **advantage** is how much better or worse than the group average it was:

```
advantage  =  (reward − group average) / (spread of the group)
```

Two examples with 4 attempts each:

```
Attempts:  A   B   C   D                        Attempts:  A   B   C   D
Rewards:   1   0   1   0     average 0.5        Rewards:   1   1   1   0     average 0.75
Advantage: +1  −1  +1  −1                       Advantage: +0.58 +0.58 +0.58 −1.73
```

The training signal then says: *raise the probability of the tokens in attempts with positive advantage, and lower it for those with negative advantage.* Notice the second group. Three attempts got it right, so each is only mildly encouraged, but the one that got it wrong is pushed down hard. If **all** attempts are right (or all wrong), every advantage is zero, and there is nothing to learn from that question. This also keeps the updates small and stable, with a penalty that stops the model drifting too far from where it started, in the same spirit as PPO's caution in Chapter 13. The group average replaces the critic, which is why GRPO needs less memory than PPO.

The recipe in practice: DeepSeek-R1

The best-known public example is **DeepSeek-R1**, a model trained to reason mostly with this kind of reinforcement learning. The story has two parts:

- **R1-Zero:** reinforcement learning applied directly to a *base* model with simple rule-based rewards (is the answer right, and is the output in the expected format), with no human-written reasoning examples at all. Behaviours such as checking its own work, noticing an error and changing approach **emerged during training**, without anyone writing them in. Nobody hard-coded "check your answer". It's behaviour that earns reward.

- **R1:** because R1-Zero's output could be hard to read, the full recipe added a small "cold start" of curated reasoning examples before RL, then further stages (more RL, extra supervised fine-tuning on good samples, and a final stage for general helpfulness). The reasoning ability could then also be **distilled** into much smaller models by fine-tuning them on the larger model's reasoning traces.

You will recognize this shape in Chapter 21's description of Qwen3's post-training: *cold start → reasoning RL → merging thinking and non-thinking modes → general RL*. That is the same pattern with different details.

A last point on training: the reasoning ability **can be taught by imitation, too**. Fine-tuning a model on a small, carefully curated set of long reasoning examples (as little as a thousand, in one study) already gives a large boost. RL is the way to push further, without needing humans to write the examples.

Part 4 — Verifiers: what if there is no calculator?

The calculator is the easy case. The natural question is: *"not every answer can be checked by a program, and a verifier has to work across many kinds of problems to scale. How does it work?"*

The honest answer is that **there is no single universal verifier**. Modern training uses different kinds, depending on the task:

A. Programmatic verifiers (exact and cheap)

```
Math        → calculator, symbolic solver, comparing with a known answer
Code        → run the tests
Chess       → a game engine
Puzzles     → a checker for the known solution
```

These give a clean `1` or `0`. The signal is strong and hard to fool. This is why **math and coding** are such attractive domains for reasoning RL: you can often turn "is this correct?" into an executable test.

B. Learned verifiers (a model that judges)

For "is this physics explanation logically sound?", no simple program exists. So you train *another model* to judge, from examples labelled `correct` and `flawed`. The verifier can itself be an LLM. But a learned verifier **can be wrong**, and if the reasoning model discovers how to exploit its blind spots, you get **reward hacking** (Chapter 13): answers that score well without being good.

C. Process verifiers (checking the steps, not just the answer)

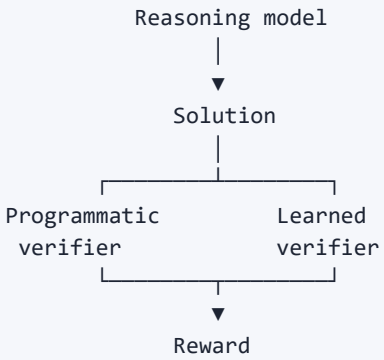
Instead of checking only the final answer, a process verifier scores each step of the reasoning:

```
Step 1: 3x + 5 = 20   ✅
Step 2: 3x = 15       ✅
Step 3: x = 5         ✅
```

This is richer feedback than a single tick at the end. The distinction is called **outcome supervision** (reward the final answer) versus **process supervision** (reward each step). A well-known study found that process supervision beat outcome supervision for training models to solve competition-level maths problems, and released a dataset of 800,000 step-level human labels to support more research.

Combining them

In practice a system may combine several:



The guiding principle is:

> **Make correctness easier to evaluate than it was to produce.**

Where that is possible (maths, code, logic) reasoning RL is on its strongest ground. Where it isn't (creative writing, strategy, open-ended advice) it's much harder, and models lean more on learned judgments and human preferences.

Part 5 — Spending more compute at answer time

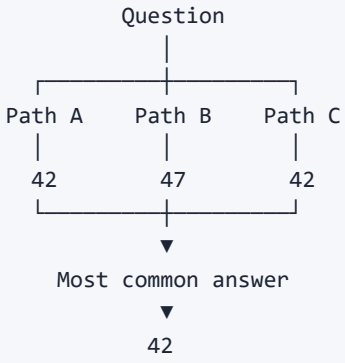
The trained model can also use extra compute in several ways when it answers. They fall along a ladder of increasing sophistication.

1. A longer single chain

The simplest: let one reasoning path run longer, with more steps, more checking, and more self-correction (`Wait, let me re-check that...`). Training with RL tends to teach the model to do exactly this when a problem is hard.

2. Many independent samples: self-consistency

One reasoning path can go wrong from a bad assumption. More thinking won't fix it if the chain is built on a mistake. So instead, sample **several independent paths** and see which answer they agree on:



This is **self-consistency**: if several independent attempts reach the same answer, confidence goes up. It really does help. The original paper reported gains of around 18 points on a grade-school maths benchmark, and several points on others, without changing the model at all.

A feel for why it works (an idealized case): suppose each independent attempt is right 60% of the time, and the wrong answers are spread across different values. Then a majority vote is right about 68% of the time with 5 samples, 83% with 21, and 98% with 101. Real attempts are not fully independent, so the gains are smaller, but the principle holds. The trade-off is that you pay for every extra sample.

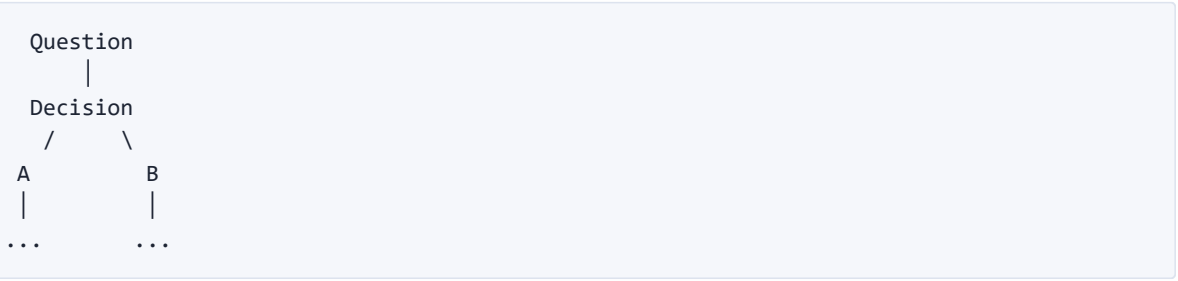
A related method is **best-of-N with a verifier**: generate N candidates, score each with a verifier (Part 4), and keep the best one.

3. Search: branch, evaluate, prune

Why stop at independent paths? We can let the model **explore different possibilities and deliberately choose between them**. A plain chain is one line:

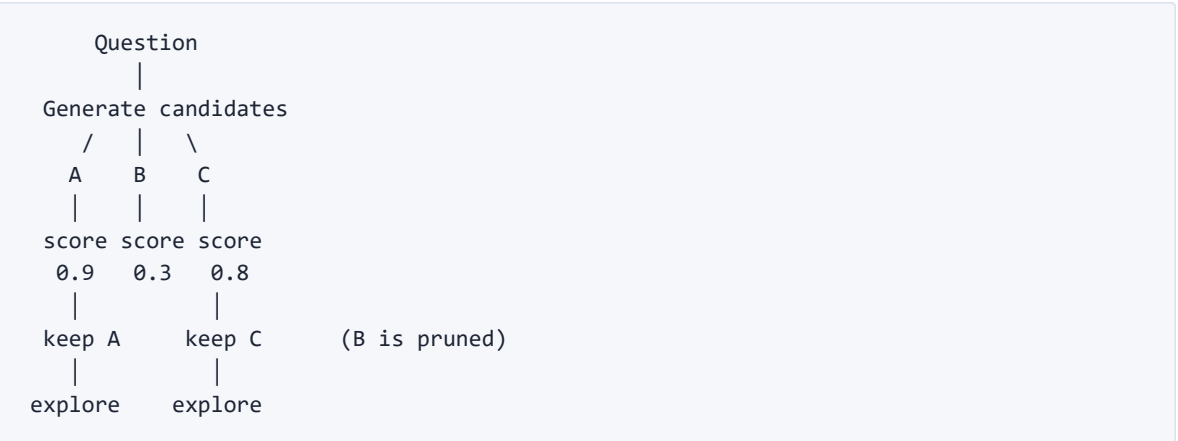
```
Question → Think → Think → Think → Answer
```

If the model reaches a decision point ("use Method A or Method B?"), it can **branch** instead of committing, giving a tree:



This is the idea behind **Tree of Thoughts**. In a puzzle that needs planning and look-ahead (the "Game of 24"), the paper reported that GPT-4 with ordinary chain-of-thought solved only about 4% of cases, while a tree-search version solved about 74%.

The catch is that the tree explodes: you can't explore everything. So you need a way to decide which branches are worth continuing, which is the verifier's job again. Candidates are scored, bad ones are **pruned**, and good ones are explored further:



That is already much closer to *search* than to ordinary text generation: **generate, explore, evaluate, prune, verify**. And the underlying model can remain exactly the same Transformer.

"But isn't the model sequential? Wouldn't exploring several paths take several times longer?"

This is a sharp question, and the answer is: **sequential within a path, parallel across paths.**

The model is *autoregressive* inside one path: you can't produce token 2 before token 1. But Path A, Path B and Path C are independent, so they can be **batched** and processed at the same time on the GPU, which is the same idea as batching different users in Chapter 22:

```
A₁ → A₂ → A₃ → A₄
B₁ → B₂ → B₃ → B₄        three sequences in one batch: each step advances all three
C₁ → C₂ → C₃ → C₄
```

And the paths often share a common beginning (the question and some early reasoning), so a smart system can compute that shared prefix **once** and branch from it. This uses the prefix caching and block sharing from Chapter 22.

To correct the original phrase: the Transformer is not "non-parallel". Inside each forward pass, it is massively parallel (matrix multiplications, attention). What is sequential is the *generation of tokens along one path*. So reasoning effort can include more exploration, not just a longer chain.

Which is better: think longer, or sample more?

They spend compute differently, and neither always wins:

```
More reasoning tokens:    one path, ──────────────►  longer     (depth)
More reasoning samples:   many paths, in parallel               (breadth)
```

Research on test-time compute found that the best mix depends on **how hard the problem is**, which is why adaptive allocation, spending more on hard questions and less on easy ones, matters.

Part 6 — Reasoning effort: what the Low / Medium / High control really does

You noticed two things when using a reasoning model. Now we can explain both.

Observation 1: "The model shows reasoning text before the answer"

A reasoning model generates something like:

```
[reasoning tokens ...]
[final answer ...]
```

The reasoning tokens are **ordinary generated tokens**. The model has typically been trained to mark where thinking ends and the answer begins, so it learns when to stop thinking and start answering. Whether *you* see them is a decision of the product:

```
Model generates:   reasoning tokens ──► final answer
                                ▲
                        the interface decides what you see:
                        show it all, show a summary, or hide it
```

So visible reasoning is partly a **product and interface decision**, not a different kind of model. Providers differ: some show the raw reasoning, some show only a summary, some hide it. Two cautions follow. What you see may not be a full transcript of everything the model computed. And a model's written reasoning is not guaranteed to be a faithful account of *why* it reached its answer, so it is useful evidence, not proof.

Observation 2: "I can pick the level of reasoning"

That is a **reasoning-effort** control. The important word is **budget**: *how much computation the system may spend on this problem.* But "budget" is only part of the story, and it is worth looking at how the control really works, because most people now use reasoning models most of the time, and the settings change both what you pay and how the model behaves.

First, the key point to hold on to: when you move the effort selector, **you are not switching to a different model**. It's the same weights, run under a different policy. The question is where that policy lives, how the model decides when to stop, and what changes in its behaviour.

Three places where the policy can live

A reasoning-effort setting is not one mechanism. It is made of up to three layers, and different products combine them differently:

```
LAYER 1   THE MODEL ITSELF         (soft control)
          Learned behaviour. The setting is part of the prompt, and the model was
          trained to think more or less because of it, and to decide when to stop.

LAYER 2   THE SERVING LAYER        (hard control)
          Ordinary software around the model: token limits, and forcing the
          model to end its thinking when a budget is used up.

LAYER 3   THE SYSTEM AROUND IT     (extra compute)
          Several attempts, voting, verifiers, search (Part 5).
```

Layer	What it does	Example
1. The model (soft)	The effort level is text in the context. The model reads it and behaves accordingly, because training taught it to.	An open model whose system prompt says "Reasoning: high" (or low, or medium)
2. The serving layer (hard)	Enforces a ceiling, regardless of what the model "wants".	A maximum number of tokens; cutting thinking off and forcing the answer when a thinking budget is reached
3. The system around it	Spends compute on more than one path.	Sampling several answers and voting; verifier-guided search

The rest of this part takes them in turn. Since layer 1 contains the most surprising idea, we start by asking how the model decides to stop thinking at all.

How does the model decide to stop thinking?

Here is the thing that surprises most people: **there is no separate "stop controller" inside the model.** Thinking has a beginning and an end, and both are marked by special tokens, such as `<think>` and `</think>` in many open models. (Other systems use differently named markers or separate "channels", but the idea is the same.) Ending the thinking is simply **producing the end-of-thinking token**, and that token is chosen the way every token is chosen: the LM head produces scores for the whole vocabulary, softmax turns them into probabilities, and the sampler picks one (Chapters 8 and 12).

So at every single step of the reasoning, the vocabulary includes the end-of-thinking token, and the model assigns it a probability. A made-up but realistic illustration, at two moments in the 17 × 24 example:

```
Moment A:  the model has just computed  "340 + 68 = 408"  but hasn't checked it
           next token:   "Wait"      40%      ← "let me double-check"
                         "So"        25%
                         "Let"       15%
                         "</think>"   5%      ← stop thinking and answer
                         other       15%

Moment B:  the model has just confirmed  "24 × 17 = 408 as well, so it matches"
           next token:   "</think>"  80%      ← stop thinking and answer
                         "Wait"      10%
                         other       10%
```

At moment A, the model usually keeps going, so it writes something like "Wait, let me verify". At moment B, the check has passed, so it usually stops. Nothing is "deciding" in a separate part of the network. The same Transformer, reading everything it has written so far, simply finds that ending the thinking is more or less likely at this point.

Why training teaches it when to stop

The stopping behaviour is learned through the reinforcement learning of Part 3, and the reward structure pushes in two opposite directions:

- **Stopping too early is punished:** the answer is wrong, so the reward is 0.

- **Never stopping is punished too:** training runs under a maximum length. A response that runs out of room before it gives an answer earns nothing. (In real products the same thing is visible to you: if the token limit is hit during thinking, you can pay for all the reasoning tokens and get no visible answer at all.)

The model therefore learns a *calibrated* habit: think long enough to get the answer right, then stop. Because the best length depends on the problem, the habit is **adaptive**. An easy problem gets a high stop probability early, while on a hard one the model keeps producing the kind of tokens that lead to checking and exploring. Some recipes also add an **explicit length reward**, for example rewarding the model for finishing close to a requested length, as in the L1 research below.

The phrases that keep thinking going ("Wait", "Hmm", "Let me double-check") are learned habits too: self-checking behaviour that RL rewarded because it caught mistakes. This is also why a research method called *budget forcing* works: to make the model think longer, you append the word "Wait" and let it carry on from there.

There are failure modes, and they are all failures of stopping:

Failure	What happens
Never stops	The model loops or repeats and never emits the end-of-thinking token. Greedy decoding makes this more likely (see below).
Stops too early	It answers before it has really worked the problem out, so it gets the answer wrong.
Overthinking	It keeps checking an answer it already had, and uses up tokens (or talks itself out of a correct answer).

What the model does differently at each level: one question, three runs

Before looking at the machinery, here is what the difference looks like from the outside. Take the question **"What is 17 × 24?"** and imagine the same model answering at three settings. (These are illustrative transcripts, not real output, but they show the typical pattern.)

```
LOW      →  408                                              about 10 tokens
            (answers almost directly; no checking)

MEDIUM   →  17 × 24 = 17 × 20 + 17 × 4 = 340 + 68 = 408.     about 60 tokens
            Check: 24 × 17 = 408. ✓  Answer: 408
            (works it out, does one check, then stops)

HIGH     →  Plan: split 24 into 20 + 4.                       about 250 tokens
            17 × 20 = 340. 17 × 4 = 68. 340 + 68 = 408.
            Check 1: 24 × 17 = 24 × 10 + 24 × 7 = 240 + 168 = 408 ✓
            Check 2: 17 × 24 ≈ 20 × 24 − 3 × 24 = 480 − 72 = 408 ✓
            Let me re-read the question: it asks for the product. ✓
            Answer: 408
            (works it out, then checks it by two different methods, then re-reads the question)
```

All three arrive at the same answer. What differs is **how much the model does after it has an answer**: how many times it double-checks, whether it tries a different method, whether it explores alternatives. That is the heart of reasoning effort: it changes how many checking and exploring steps the model is willing to do before it commits.

The level also interacts with how hard the problem is. Illustrative reasoning lengths:

	Low	Medium	High
Easy question (17 × 24)	~10 tokens	~60	~250
Hard question (a multi-step puzzle)	~400	~1,500	~6,000

Notice that the *hard question at low effort* (about 400 tokens) takes longer than the *easy question at high effort* (about 250). The level scales the amount of thinking, but the problem sets the baseline, which is why a setting never corresponds to a fixed number of tokens.

The mechanism, step by step: how a label becomes more reasoning

Now to the question you asked: *what internal mechanism makes the model reason more?* The answer has two halves, and the first one is a surprise.

First, what does not change: nothing in the architecture. There are no extra layers, no bigger model, no special reasoning module that switches on. Each token still passes through the same N blocks (Chapter 8) with the same weights. As Part 1 explained, a Transformer does a fixed amount of work *per token*. So "reasoning more" can only mean one thing: **generating more tokens**, each one a further pass through the network, each one building on what came before. The question is only what makes the model write more of them.

Second, how the setting changes what gets written. The effort setting reaches the model as part of its input, as ordinary tokens (the serving layer may turn a setting into text or special tokens placed in the system message). From there it works through the mechanisms you already know:

```
1. The effort label is in the context, at the start.
        │
        ▼
2. At EVERY later position, attention (Chapter 4) can read it,
   just as it reads any instruction in the prompt.
        │
        ▼
3. So at every position the hidden vector is nudged slightly,
   depending on the label.
        │
        ▼
4. So at every position the LM head's probabilities change slightly:
   P("Wait, let me double-check")  vs  P("</think>"  = stop and answer)
        │
        ▼
5. A small change at each decision point COMPOUNDS over the chain:
   many more checks → a much longer chain of thought.
```

So the label doesn't "turn up a dial" on the computation. It **steers the choice made at each step between "keep going" and "stop"**, and the cumulative effect of hundreds of such choices is a very different amount of reasoning. The same applies at the very start: whether the model begins by thinking at all (an opening thinking marker, or a direct answer) is itself just the first token's probability, and the label shifts it too.

Why the model learned to respond to the label: the price of a token

Why would a model respond sensibly to such a label? Because it was **trained to**. One family of published methods (length-aware rewards, as in the L1 and BudgetThinker research) makes the idea very concrete. The reward for a reasoning attempt combines being right with the price of being long:

```
reward  =  (1 if the answer is correct, else 0)   −   λ × (tokens used / 1000)
                                                      ▲
                                        the "price per token", set by the effort level
```

The effort level decides the **price of a token** (λ). Suppose two kinds of attempts at some problem: a **short** attempt (300 tokens) that is right 70% of the time, and a **long, careful** attempt (3,000 tokens, with checks) that is right 95% of the time. Which behaviour does the reward favour?

Effort level	Price λ	Short attempt: 0.70 − λ×0.3	Long attempt: 0.95 − λ×3	Which wins?
Low (tokens are expensive)	0.5	0.55	−0.55	**Short**, by a mile
Medium	0.1	0.67	0.65	**About equal**: the balance point
High (tokens are cheap)	0.02	0.69	0.89	**Long**, clearly

(The numbers are made up to show the principle.) Read the middle row: at the medium price, an extra round of checking is **worth its cost only when it buys enough accuracy**. That is exactly the in-between behaviour of a medium setting. At each point in its reasoning the model effectively asks, *"is another check worth its price?"*:

- Right after computing an answer that hasn't been checked, another check buys a lot (it could catch a mistake), so even at a medium price it continues.
- After two independent checks agree, a third buys almost nothing, so at a medium price it stops, whereas at a cheap (high-effort) price it may still do one more.

Different providers use different recipes (for example, supervised examples of reasoning at each length, or reinforcement learning with a price like this), and the exact recipes for commercial models are not public. But the behaviour that comes out is consistent with this picture.

So how does "medium" know where to stop?

This is the heart of your question: *if I choose medium, how does the model know it has used more than the Low amount, but not as much as High, and that it should stop around here?*

The honest answer is: **it doesn't count.** There is no number stored inside the model like "medium = 800 tokens" that it compares itself against. Instead, three ingredients combine, and the first is the main one.

Ingredient 1: stopping probabilities that depend on where it is in the reasoning. The model reaches natural *checkpoints* in its reasoning, such as "I have a plan", "I have an answer", "my first check passed", "a second, independent check passed". At each checkpoint the model has a probability of stopping, and the effort level sets how high it is. Here is an illustration, using the 17 × 24 example. Each cell is the chance of stopping *at that checkpoint*, given the model got that far:

```
Checkpoint:                  C1          C2          C3            C4              C5
                          plan made   answer      first check   second check    alternative
                                      computed    passed        passed          method agrees
Chance of stopping here:
   LOW                      5%          65%         90%           95%             99%
   MEDIUM                   2%          20%         65%           90%             97%
   HIGH                     1%           5%         25%           60%             90%
```

Because it is a chain of chances, the outcome is a **spread**, not a single point. Working it out, here is where each run would likely end, and the typical length:

	Stops at C1	C2	C3	C4	C5	Typical length
Low	5%	62%	30%	3%	~0%	~120 tokens
Medium	2%	20%	51%	25%	3%	~180 tokens
High	1%	5%	24%	42%	25%	~280 tokens

This is how "in between" works. Medium is **not** "stop after exactly N tokens". It is "stop *after the first check, usually*, sometimes earlier, sometimes after a second one". Its runs overlap with Low's and with High's: a medium run can occasionally be as short as a low one, or as long as a high one. And it explains why **the same question at the same setting gives different lengths on different runs**: the stop decision is a probability, and the model's sampler (Chapter 12) rolls the dice at every step.

It also explains the difficulty effect from earlier. The stop probabilities depend on the **state of the reasoning**. On a hard problem, the model doesn't reach a "verified, confident" checkpoint quickly, so the stop probability stays low for longer, and even a medium run goes on for a long time.

Ingredient 2: a rough sense of how much it has written. The model can see its own output, so it has some sense of how long it has been going. The position information in the model (Chapter 3, and the rotary positions of Chapter 16) lets attention work out roughly *how far back* the start of the thinking is. Language models are poor at *exact* counting but decent at rough lengths. This matters most when the setting is given as a **length target**, as in the L1 research, where the prompt says how long to think and training rewards landing close to it. Models trained this way get *near* the target, not exactly on it.

Ingredient 3: an explicit countdown, where the system provides one. The most direct way for a model to know how much budget is left is to be *told*. We cover this next.

When the budget is a number: countdown and control tokens

Some systems turn the budget into something the model can literally read. Two examples:

- **A countdown marker in the context.** In one provider's "task budget" feature, the server injects a running countdown into the conversation, showing how many tokens remain in the current task and updating as the model generates thinking, tool calls and output. The model sees it only as part of its context (the API response doesn't expose it), and it uses it to **pace itself and finish gracefully as the budget runs down**. The thinking naturally scales down as the budget depletes. It is advisory rather than a hard cap, and a budget that is clearly too small for the task can make the model scope the task down aggressively or stop early, because it sees from the countdown that it cannot finish.
- **Control tokens inserted at intervals.** The BudgetThinker research periodically inserts special tokens into the model's own output that tell it how much of its budget remains, and trains the model with supervised

examples followed by reinforcement learning with a length-aware reward so that it adapts its reasoning to what is left and wraps up as the budget runs out.

In both cases the "internal mechanism" is nothing mysterious: **the remaining budget becomes text (or special tokens) in the context, and attention reads it like any other token**. When it is low, the stop-probability pattern shifts toward wrapping up. That is the same machinery as the effort label, with a number that changes along the way.

Putting the mechanisms together: what is probably at work for each kind of setting

What you control	What the model can "see"	How it decides where to stop
A level (Low / Medium / High)	A label in the context	Learned stopping behaviour: label-dependent stop probabilities at each checkpoint (Ingredient 1)
A length target ("think for about N tokens")	The target number in the prompt	Learned approximate length sense (Ingredient 2), trained with a length-aware reward
A budget with a countdown	A running "tokens remaining" value	Reads the remaining budget and wraps up as it shrinks (Ingredient 3)
A thinking on/off switch	A flag (`/think` , `/no_think`)	Chooses whether to open a thinking block at all
A hard token limit	Nothing (the model doesn't know)	It doesn't. The serving layer cuts it off (next section)

Variants you will meet in real products

The same ideas show up in several forms:

- **Thinking on or off.** The Qwen3 models (Chapter 21) have a switch in the chat template, `/think` or `/no_think` . In the "off" case the model still writes an empty thinking block and goes straight to the answer. It is a two-level version of the mechanism above, and the models were trained to handle both modes in one set of weights ("thinking-mode fusion").
- **A level written in the system prompt.** Some open models read the level from the system message (a line such as `Reasoning: high` , with low, medium and high), which is the label mechanism in its plainest form.
- **A length target.** The L1 model is prompted with the number of tokens to think for.
- **The model decides whether to think at all (adaptive thinking).** Some providers let the model decide, for each request, whether to think and for how long, with the effort level as a signal. At lower effort it may skip thinking entirely for an easy request, and at higher effort it thinks more readily and at greater length. The same provider says that even at a low setting the model still thinks on a sufficiently difficult problem, only less than it would at a higher setting. It also notes that, in a tool-using loop, most of the thinking tends to happen on the first step after new input, and that thinking per request tends to decrease as a conversation grows longer.
- **Effort that covers more than thinking.** In at least one provider's design, the effort setting affects *all* the output tokens: the explanations and the tool calls too. Lower effort means fewer and terser tool calls and less preamble. Higher effort means more tool calls, a plan explained before acting, and more detailed

summaries afterwards. So "reasoning effort" is better understood as *how much work the model puts into the whole response*, of which thinking is one part.

Hard limits: when the serving layer steps in

The behaviours above are *soft*: the model chooses. The serving layer can also enforce *hard* limits, and it is useful to see how they differ.

Forcing the end of thinking. In the thinking-budget mechanism of Qwen3, the serving code counts the reasoning tokens. When the count reaches the limit, it stops the model's thinking, **inserts a fixed sentence** telling the model that time is up and it has to answer based on the thinking so far, adds the end-of-thinking token, and lets the model carry on generating the answer. The model was never explicitly trained for this. The report says the ability emerges from training it to handle both thinking and non-thinking modes. Forcing can also run the other way: appending "Wait" to extend the thinking (budget forcing).

```
SOFT STOP:   ... reasoning ... reasoning ... [</think>]  answer ...
                                              ▲
                                the model chose to stop

HARD STOP:   ... reasoning ... reasoning ... reasoning  ✂  [“time's up, answer now” + </think>]  answer ...
                                                    ▲
                                          the serving layer cut in; the model continues
                                          from the inserted text
```

Token limits. Every request also has a maximum number of output tokens, and **thinking counts toward it** (and is billed as output), even when you can't see the thinking. If the limit is reached mid-thought, there may be no visible answer at all. That is why settings that think a lot need a generous limit. Providers differ on thinking *budgets*: some treat a requested budget as a **target** rather than a strict cap, so the model may stop earlier, and the separate maximum length is the true hard ceiling.

What you actually see change, setting by setting

Putting it together, here is what generally differs as you move up the settings. These are tendencies, not guarantees, and the exact numbers vary by model:

	Low	**Medium**	**High** (and above)
Reasoning length	Short, or none for easy requests	Moderate	Long, up to many thousands of tokens
Self-checking and alternatives	Rare	Some	Frequent: verifies, backtracks, tries other approaches
Tool calls and explanation (where effort covers them)	Fewer, terser	Balanced	More calls, more explanation
Time before the first visible answer word	Short	Longer	Can be long, because thinking comes *before* the answer
Cost	Lowest (fewest output tokens)	Moderate	Highest
Best for	Simple lookups, formatting, routing, quick chat	Everyday tasks	Hard maths, code, multi-step planning

Two practical points follow from this table. First, **latency is mostly thinking time**. The tokens come out at the same speed as always (Chapter 12), but the answer you wanted only starts after the reasoning has finished, so a high setting makes the first visible word arrive much later. Second, **more is not always better**. There are diminishing returns, and at the very top setting some simple or structured tasks only cost more, or even get worse through overthinking.

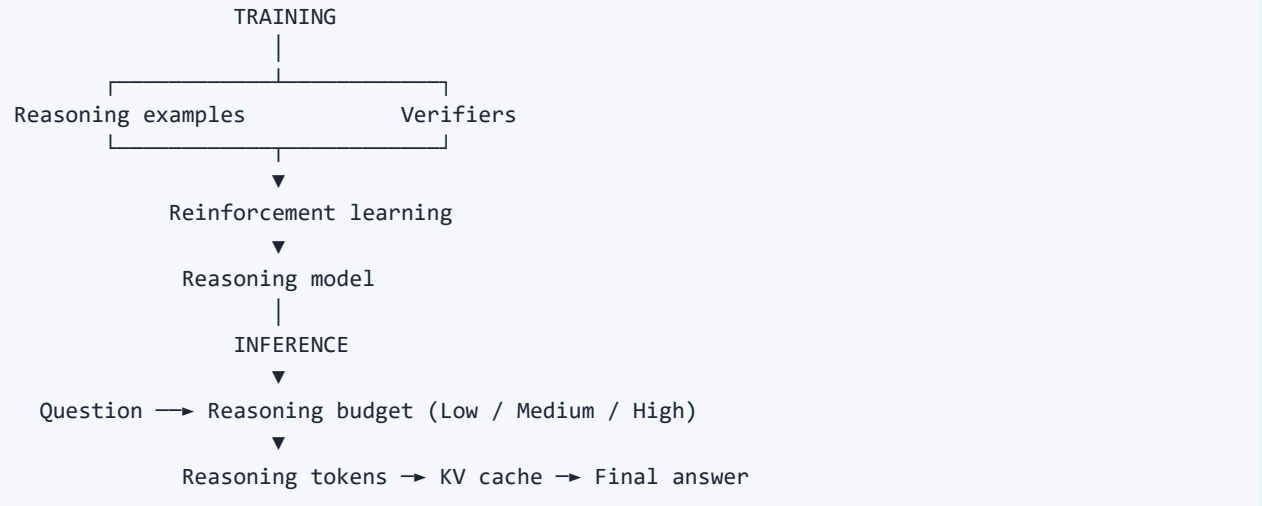
Why sampling settings matter more for reasoning models

Chapter 12's sampling controls (temperature, top-p, top-k) interact with the stopping behaviour. Some reasoning-model families recommend moderate temperature and top-p (for one family, around 0.6 and 0.95) and explicitly warn **against greedy decoding**. The reason follows from the section on stopping: greedy always takes the single most likely token, and a long chain of thought can get caught in a loop of repeating itself. If the model never picks the end-of-thinking token, it never finishes. A little randomness lets it break out of loops, and lets it find the "stop" token when that token is a probable but not the single most probable choice.

Choosing a setting in practice

- **Start from the default**, then try the next level up or down on your own tasks, rather than assuming the highest is best. Test with a sample of real questions.
- **Use low** for simple, well-specified work and for high-volume or latency-sensitive tasks. **Use high** for hard multi-step problems where correctness matters more than speed.
- **Give high settings room.** Set a generous maximum token limit so that the thinking does not use it all up before the answer begins.
- **Watch time-to-first-answer and cost, not only quality**, since both grow with thinking length.
- **Keep the setting stable inside one long conversation** if you rely on prompt caching, because with some providers, changing it invalidates the cached prefix (Chapter 12).
- **Match the setting to the problem**, ideally automatically: easy requests at low effort, hard ones at high (see "Routing" in Part 8).

Putting the whole picture together



There is one more distinction here. **Training-time improvement** gives the model *better reasoning capability*. **Test-time compute** gives it *more opportunity to use that capability*. The strongest systems combine both.

Part 7 — The costs: tokens, KV cache and latency

Reasoning is not free, and the cost follows directly from what you already know.

Reasoning tokens are still tokens

They go through exactly the same pipeline as any other token:

```
Token → Embedding → Transformer → KV cache → next token
```

So every reasoning token adds to the KV cache (Chapter 12). Using the 128 KiB per token of an 8B-class model:

```
20 reasoning tokens    →  about 2.5 MiB of cache
2,000 reasoning tokens →  about 250 MiB
3 paths × 2,000        →  about 750 MiB
```

Add up the effects of higher effort:

```
Higher reasoning effort → more tokens → more compute
                                      → more KV cache
                                      → more latency and memory
                                      → possibly a better answer
```

You are literally buying quality with **latency, GPU time, memory, energy and money**. This is why reasoning models create new serving and hardware challenges (Chapters 22 and 23): long outputs keep large caches alive for longer.

"Do we really need to keep the reasoning tokens?"

A fair question. If reasoning is only an intermediate step, why store it, or its KV cache?

- **While the model is still generating**, it needs them. Producing `340 + 68 = 408` requires reading the earlier lines, so their Keys and Values must be there. Throw them away early, and the model loses the thread.
- **After the final answer**, the reasoning text and its cache need not be kept as permanent memory. The system can discard the cache, or keep only the question and the answer if the conversation continues. (Whether an application feeds earlier reasoning back in on later turns is a design choice.)
- It is also useful *during* the answer. A long, detailed final answer often benefits from the reasoning that preceded it, which is why the cache isn't dropped the moment the reasoning ends.

There is active research on **compressing reasoning**: summarizing or compressing the state, or reasoning in hidden vectors rather than in written tokens, so that a model doesn't carry thousands of tokens forward when a smaller representation would do. It is the same trade-off as in Chapter 12's "could we compress the cache?": exact context costs memory, compression risks losing detail, and it is not a solved problem.

Another cost: overthinking

More thinking is not always better. A reasoning model can spend a lot of tokens on an easy question, and long chains can sometimes talk themselves into a wrong answer. Adaptive effort exists partly to avoid this.

Part 8 — Combining reasoning with quantization and routing

"Can reasoning make up for a quantized model?"

An intriguing idea: Chapter 14 showed quantization costs a little quality. If thinking longer improves answers, can extra reasoning compensate?

```
FP16 model, answering directly                100  (illustrative)
Q4 model, answering directly                   97
Q4 model, with more reasoning/verification     99
```

(The numbers only illustrate the principle.) Yes, it can recover **some** of the loss: you are trading numerical precision for extra inference computation. But there is a ceiling, and it's an important one:

> **Reasoning helps a model use the capability it has. It cannot recreate knowledge that quantization (or a smaller size) destroyed.**

If the quantized model no longer reliably *knows* something, telling it to think harder doesn't bring the fact back. So: can reasoning recover some of the loss? Yes. Everything? No.

A second route is **distillation**: use a strong model to produce reasoning traces, then fine-tune a small model on them (as with R1's distilled variants). The small model inherits useful reasoning behaviour, and you can then quantize it and give it a thinking budget. The stack for a capable local model becomes: *better training + smaller model + quantization + reasoning + test-time compute*.

Routing: spend effort where it pays

This leads to a practical way of thinking. Instead of "which is the strongest model?", ask: *what is the cheapest model that is good enough when given the right amount of reasoning, tools, retrieval and verification?*

```
Easy request   →  small (quantized) model  →  low reasoning
Medium request →  medium model             →  medium reasoning
Hard request   →  strong model             →  high reasoning
```

That is the seed of **model routing**, which Chapter 24 takes up.

Part 9 — Reasoning models, tools and agents

These terms get mixed up constantly. They are related but not the same, so here is the boundary:

```
Normal LLM          Prompt → LLM → Answer

Reasoning model     Prompt → LLM → Reasoning → Final answer
                    (still ONE model generating tokens)

Reasoning + tools   the LLM can decide to call a calculator, web search or code,
                    get the result back, and continue

Agent               Goal → Plan → Act → Observe → Re-plan → ... → Result
                    (a persistent execution LOOP over many steps)
```

System	Main capability
LLM	Generate
Reasoning model	Generate and reason
Tool-using LLM	Generate and use tools
Agent	Plan, execute, observe and adapt

As soon as you add search, verifiers, memory and tools around the model, you move from a reasoning *model* to a reasoning *system*, and the boundary with an agent gets thin. But keep one line clear:

> **Reasoning is a capability of the model and the inference process. Agentic behaviour is a capability of the surrounding execution system.**

An agent doesn't have to use a reasoning model: it can wrap a normal LLM in a loop. And a reasoning model doesn't need an agent: a question can go in, reasoning happen, and an answer come out. They complement each other, and Chapter 24 covers the system side.

Common misconceptions, cleared up early

Misconception	Reality
"Reasoning models have a different architecture."	Usually the same Transformer. They differ in training (RL with verifiers) and in how they are used (long reasoning at inference).
"There is a special reasoning engine inside."	Reasoning is ordinary next-token generation. The generated tokens become context that later tokens can use.
"The verifier runs every time you ask a question."	The verifier is a *training* tool, like the RLHF reward model. At inference there's usually just the model.
"Reasoning tokens are a new kind of token."	They are normal tokens, with a normal embedding, KV cache and cost.
"Low / Medium / High selects three different models."	It's the same weights under a different inference policy (budget, stopping, sampling and search). The exact mechanism varies by provider.
"A separate part of the model decides when to stop thinking."	There's no stop controller. Stopping is the model emitting the end-of-thinking token, chosen by the same softmax as any token, and learned through RL.
"The model counts its tokens to know when it has reached Medium."	There is no stored token number. Stopping is a probability at each checkpoint, shifted by the effort label, so Medium runs vary in length and overlap with Low and High.
"Medium means a fixed number of reasoning tokens."	The level scales the *amount* of checking and exploring, while the problem sets the baseline. A hard question at Low can take longer than an easy one at High.
"Extra reasoning comes from extra layers or a special reasoning module."	Nothing in the architecture changes. More reasoning means more tokens, each a normal forward pass through the same blocks.
"The effort setting only changes the length of the thinking."	It can also change whether the model thinks at all, how often it checks itself, and (in some designs) how many tool calls it makes and how much it explains.
"A thinking budget is something the model plans around."	Often it's enforced from outside: the serving layer cuts in, inserts a short "answer now" text and the end-of-thinking token, and the model continues from there.
"Greedy decoding is the safest setting for reasoning."	It can trap a long chain of thought in a loop that never reaches the end-of-thinking token. Reasoning models are usually run with moderate temperature.
"The thinking budget is a strict cap."	It's often a target. The model may stop early, and a separate maximum length is the hard limit.
"The visible reasoning is a full, faithful record of the model's thinking."	It's what the product chooses to show (raw, summarized or hidden), and it isn't guaranteed to explain why the model answered as it did.
"More thinking is always better."	It costs time, memory and money, and it can overthink easy problems. Effort should match difficulty.
"One verifier can judge everything."	Programmatic verifiers work for maths, code and puzzles. Elsewhere learned verifiers are used, and they can be exploited (reward hacking).
"Exploring several paths means waiting for each one in turn."	Tokens are sequential within a path, but separate paths can be batched in parallel, and shared prefixes can be cached.
"GRPO is a different model."	It's a training algorithm: PPO-style RL that uses a group of attempts as its baseline instead of a separate value model.

Misconception	Reality
"Reasoning can restore everything lost to quantization."	It can recover some quality, but cannot bring back knowledge the compressed model has lost.
"A reasoning model is an agent."	A reasoning model is one model that thinks. An agent is a loop of planning, acting and observing around it.

Quick reference

```
Why it works:  a Transformer does a fixed amount of work per token (N blocks);
               each written token becomes context → more sequential computation (a scratchpad)
Test-time compute:  spend more computation per problem at inference (depth and/or breadth)
               scaling axes: model size | training compute | test-time compute

Training:      RLVR = RL with VERIFIABLE rewards (correct = 1, wrong = 0)
               verifiers: programmatic (math, code) | learned (a judge model) | process (per step)
               outcome supervision (final answer) vs process supervision (each step)
               GRPO:  sample a group per question;  advantage = (reward − group mean) / group std
                      no value model → less memory than PPO
               DeepSeek-R1: R1-Zero (pure RL, emergent self-checking) → R1 (cold start, RL, SFT, RL)
                            → distill into small models
               verifier is discarded after training

Answer-time:   longer chain | self-consistency (majority vote) | best-of-N + verifier
               | search (Tree of Thoughts: branch → score → prune)
               sequential within a path, parallel across paths (batched; shared prefix cached)

Effort control: same model, different inference policy, in three layers:
                1 model (soft): effort label in the prompt, learned stopping ("Reasoning: high", /think, length target)
                2 serving (hard): token limits; forced "answer now" + end-of-thinking token; budget = target
                3 system: more samples, voting, verifiers, search
Stopping:       no controller: the model emits the end-of-thinking token (</think>), chosen by softmax
Mechanism:      label (text) in context → attention reads it at every step → hidden state nudged →
                P("Wait") vs P("</think>") shifts → compounds over the chain. NO architecture change;
                more reasoning = more forward passes (tokens).
Why it learned: reward = correct − λ × length; effort sets the PRICE λ of a token
                (low = expensive, high = cheap; medium = a check is worth it only if it buys enough accuracy)
Medium stops:   no stored token count. P(stop) at checkpoints (plan, answer, check 1, check 2 …) depends on
                the label and on the state of the reasoning → a spread of lengths that overlaps Low and High.
                Also: a rough length sense (positions; length-target training) and, where provided,
                an explicit countdown / control tokens in the context.
                RL teaches it: too early = wrong, never = no answer; effort label shifts P(stop) vs P("Wait")
                failures: never stops (loops, greedy), stops too early, overthinks
Settings:       low → short, few checks, fewer tool calls;  high → long, many checks, slow first word, costly
                latency ≈ thinking time;  avoid greedy decoding;  keep effort stable for cached conversations
Visible reasoning: product decision (raw / summary / hidden); not guaranteed faithful

Costs:         more tokens → more KV cache (128 KiB/token for an 8B-class model → 2,000 tokens ≈ 250 MiB)
               + latency, GPU time, energy, money;  overthinking is possible
Combos:        quantization + reasoning recovers SOME quality (not lost knowledge);
               distillation; routing: easy → small/low effort, hard → strong/high effort

Boundary:      reasoning = capability of the model/inference process;
               agent = capability of the execution system around it
```

What's next

That completes the model itself: how it is built (Chapters 2 to 8), trained (Chapters 9 to 11), run, adapted and compressed (Chapters 12 to 14), redesigned (Chapters 15 to 19) and taught to reason (this chapter). Chapter 21 puts it all together by opening up two real model families, Qwen and DeepSeek, so you can see how

these ideas (modern building blocks, attention variants, MoE and reasoning-focused training) combine in practice. After that, the handbook turns from the model to everything around it: serving it, scaling it, and building systems on top of it.

Chapter 21 — Inside Modern Models: How Qwen and DeepSeek Combine These Ideas

Introduction

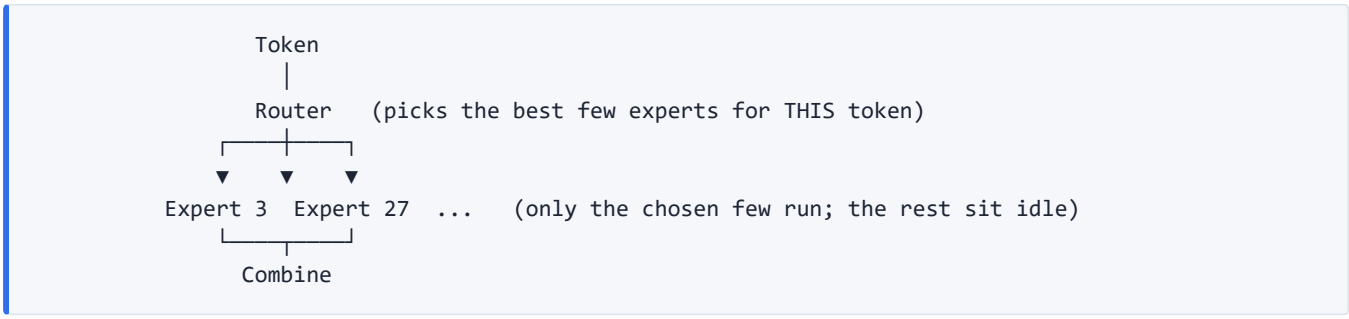
When people say models like Qwen and DeepSeek use a "better" architecture or training recipe, what do they actually mean? This chapter answers that by looking inside two real model families, using their published technical reports and model cards. You'll see that neither is a new kind of network. Both are **decoder-only Transformers** (Chapter 15) built from the pieces you now know: GQA or MLA (Chapter 17), RoPE, RMSNorm and SwiGLU (Chapter 16). What makes them stand out is a set of choices about how attention is stored, how the FFN is organized, and how the model is trained. Because this chapter comes late in the handbook, every ingredient below should already be familiar, so it works as a capstone.

A caveat up front: this field moves quickly, and the details below reflect the published reports and model cards available at the time of writing. Treat the specific numbers as a snapshot, and the *ideas* as the lasting part.

A quick reminder: Mixture of Experts

Both families rely on Mixture of Experts, covered in Chapter 18. Here is a short reminder of the idea.

In the FFN of Chapter 6, every token passes through the same big FFN. In a **Mixture-of-Experts (MoE)** layer, that single FFN is replaced by **many smaller FFNs, called experts**, plus a small **router** that picks a few of them for each token:



This gives a model with a huge number of **total** parameters but only a small number **active** for any one token. You'll see this written as, for example, "235B total, 22B active": the model stores 235 billion parameters of knowledge, but each token only pays the compute cost of about 22 billion. Attention stays dense; only the FFN becomes sparse. Left alone, a router tends to favor a handful of popular experts and neglect the rest, so MoE training adds an incentive to spread tokens across experts, as an extra term added to the loss from Chapter 9.

Part 1 — Qwen

Qwen3: the modern recipe, in dense and MoE forms

Qwen3 (released in 2025) comes in six **dense** sizes (0.6B up to 32B) and two **MoE** models. The dense models follow exactly the modern recipe from Chapter 16:

```
Attention:      GQA  +  RoPE  +  QK-Norm  (the old bias on Q, K, V was removed)
FFN:            SwiGLU
Normalization:  RMSNorm, pre-norm
Embeddings:     tied in the smaller models
```

The two **MoE** models are Qwen3-30B-A3B (30B total, 3B active) and Qwen3-235B-A22B (235B total, 22B active). Their design:

- **128 experts per MoE layer, 8 active per token.**
- **No shared experts.** The earlier Qwen2.5-MoE had them; Qwen3 dropped them.
- A **global-batch load-balancing loss.** This is the balancing incentive mentioned in the primer, which keeps the router from sending everything to a few popular experts. Computing that balance across the whole batch, rather than in small pieces, leaves each expert more room to specialize.

Qwen3's training recipe

Qwen3 was pre-trained on roughly **36 trillion tokens across 119 languages and dialects**, in three stages. Notice how the training data and the context length change as training proceeds:

```
Stage 1  General:        over 30 trillion tokens, 4,096-token sequences
Stage 2  Reasoning:      about 5 trillion more tokens, with extra STEM and coding data
Stage 3  Long context:   hundreds of billions of tokens, 32,768-token sequences
                         (using the context-extension techniques from Chapter 16)
```

Training on short sequences first is cheaper (attention cost grows with length, Chapter 17). The model is then extended to longer contexts with a smaller amount of long-text data.

After pre-training, a **four-stage post-training** process shapes the model's behavior:

```
1. Long chain-of-thought "cold start"   →  teach basic step-by-step reasoning from verified examples
2. Reasoning RL                         →  reinforcement learning (GRPO) on problems with checkable answers
3. Thinking-mode fusion                 →  merge "thinking" and "non-thinking" ability into ONE model
4. General RL                           →  broad improvement across many task types
```

The third stage is a distinctive idea: instead of shipping one model that always reasons at length and another that answers quickly, a single Qwen3 model can do either. A switch in the chat template (`/think` or `/no_think`) selects the mode. (Chapter 20 covers reasoning models and the RL methods behind them.)

For the **smaller** models, Qwen3 used **strong-to-weak distillation**: a small "student" model is trained to imitate a large "teacher", first on text the teacher generated, then by matching the teacher's output probabilities while generating its own text. The report says this took about a tenth of the GPU time that running reinforcement learning on each small model would.

Newer Qwen: hybrid attention

Qwen's more recent models (Qwen3-Next, and the Qwen3.5 and 3.6 families) changed the attention itself. Following Chapter 17, they use a **hybrid** of cheap linear-attention layers (Gated DeltaNet) and occasional full-attention layers, in a 3:1 pattern. Qwen3-Next, for instance, is an 80B-total, 3B-active MoE model with 48 layers and 512 experts per MoE layer (10 active per token), a native context of 262,144 tokens, and a multi-token-prediction training objective, an idea explained in the DeepSeek section below. The Qwen3.5 family spans models from under a billion parameters up to a roughly 400B-total MoE.

Part 2 — DeepSeek

DeepSeek-V3: three architectural ideas

DeepSeek-V3 (December 2024) is a **671B-total, 37B-active** MoE model with 61 layers and a hidden size of 7,168. Three architecture choices define it.

1. MLA for attention (Chapter 17). 128 heads with a per-head dimension of 128, but the cache stores only a compressed latent of 512 numbers (plus the small RoPE piece) per token per layer.

2. DeepSeekMoE for the FFN. In every layer except the first three (which stay as ordinary dense FFNs), the FFN is replaced by an MoE layer with **1 shared expert and 256 routed experts, of which 8 routed experts are active per token**. Two ideas make this design distinctive:

- **Many small experts.** Rather than a few large experts, there are many fine-grained ones, so the router can combine small specialties flexibly.
- **A shared expert that always runs.** Some knowledge is needed by *every* token (basic language patterns, for example). Giving that to an always-on shared expert frees the routed experts to specialize more sharply, instead of each having to relearn the common material.

3. Auxiliary-loss-free load balancing. As the primer noted, routers need some way to avoid overloading a few experts. The usual approach adds an extra *auxiliary loss* to the training objective (a second term added to the loss from Chapter 9), but that extra term can pull the model away from simply predicting well. DeepSeek-V3 removes the auxiliary loss and instead keeps a **bias for each expert**:

```
Routing:   each expert's score gets its bias added, and the top 8 are chosen
           (overloaded expert → its bias is pushed down;  underused expert → pushed up)

Weighting: how much each chosen expert contributes still uses the ORIGINAL score,
           without the bias
```

So the bias only nudges *which* experts get picked, to even out the load, and never distorts *how much* each one contributes to the answer.

DeepSeek-V3's training choices

Multi-token prediction (MTP). In Chapter 9, each position predicted the one next token. With MTP, each position **also predicts one more token beyond that** (V3 uses a depth of 1: the next token and the one after). That gives the model a denser learning signal from every position and encourages it to plan slightly ahead. The extra prediction can also later be used to speed up generation (Chapter 23).

FP8 mixed-precision training. Normally, the numbers inside the model are stored and multiplied in 16-bit or 32-bit formats. DeepSeek-V3 does most of its large matrix multiplications in **8-bit floating point**, which cuts memory use and speeds up the math. Precision-sensitive steps stay in higher precision, and the numbers are scaled in small tiles or blocks so that 8-bit's limited range isn't a problem. (Chapter 14 explains these number formats, and why using fewer bits per number is such a powerful idea.)

Efficient scale. DeepSeek-V3 was pre-trained on **14.8 trillion tokens using about 2.79 million H800 GPU-hours**, notably less than the compute budgets commonly reported for models of comparable quality at the time, which is a large part of why it drew so much attention. Its context length was extended in two stages: from 4K to 32K, then to 128K, the same short-first-then-extend pattern Qwen3 uses.

The later DeepSeek models

DeepSeek-V3.2 made exactly one architectural change to its predecessor: it added **DeepSeek Sparse Attention** (Chapter 17) through continued training, so each token attends to only the 2,048 most relevant earlier tokens, selected by a lightweight "lightning indexer."

DeepSeek-V4 (April 2026) comes in two sizes: V4-Flash (284B total, 13B active) and V4-Pro (1.6T total, 49B active), with a context length of **one million tokens**. According to its model card, V4 introduces three notable changes:

- **A hybrid of compressed and sparse attention** for long contexts, using about 27% of the per-token compute and 10% of the cache of V3.2 at that length.
- **Manifold-Constrained Hyper-Connections (mHC)**, a strengthened replacement for the plain residual connection of Chapter 7, intended to keep signals stable across layers.
- **The Muon optimizer** instead of AdamW (Chapter 11) for faster, more stable convergence. This is a reminder that even the optimizer, which seems like settled ground, is still being improved.

Training used more than 32 trillion tokens, with a mixed-precision scheme in which the MoE experts use an even smaller 4-bit format and most other parameters use FP8. Post-training had two steps: first, separate specialist models are trained for individual domains (with supervised fine-tuning and reinforcement learning using GRPO), then they are merged into one model through distillation.

Part 3 — The two side by side

	Classic GPT-2	Qwen3	DeepSeek-V3
Type	Decoder-only	Decoder-only	Decoder-only
Attention	Multi-head (MHA)	GQA + QK-Norm	MLA (compressed cache)
Positions	Learned table	RoPE	RoPE (with decoupled part for MLA)
Normalization	LayerNorm	RMSNorm (pre-norm)	RMSNorm (pre-norm)
FFN	GELU, dense	SwiGLU; dense, or MoE with 128 experts (8 active)	SwiGLU; MoE with 1 shared + 256 routed experts (8 active), first 3 layers dense
Total / active parameters	Same (dense)	Up to 235B / 22B (MoE)	671B / 37B
Pre-training data	Far smaller	~36T tokens, 3 stages	14.8T tokens
Training extras	—	Staged data; thinking-mode fusion; distillation for small models	MTP; FP8 training; auxiliary-loss-free balancing
Context handling	Fixed table	4K then 32K (then longer variants)	4K then 32K then 128K

Part 4 — So what does "better" actually mean?

Looking across both families, the improvements fall into a few themes:

```
1. Cheaper to RUN
   GQA, MLA, sliding window, sparse and hybrid attention → a smaller KV cache and less
   compute per token, especially at long context (Chapter 17)

2. More capacity for the same compute
   Mixture of Experts → many parameters stored, few used per token

3. Cheaper and more stable to TRAIN
   FP8 or FP4 arithmetic, QK-Norm, auxiliary-loss-free balancing, better optimizers,
   training on short sequences first and extending later

4. A richer learning signal
   Multi-token prediction; carefully staged and filtered training data

5. Better post-training
   Reasoning-focused reinforcement learning, thinking modes, distillation from larger models
```

It's worth being honest about two things. First, "better" is relative to a goal: these choices mostly buy *efficiency* (the same quality for less cost) and *long-context ability*. Second, we can only describe what has been published. The designs of many closed models aren't public, so comparisons here are among open

models. What you can rely on is the pattern: none of it replaces the core idea you learned in Chapters 2 to 11. Every one of these models still takes tokens, embeds them, passes them through stacked attention and FFN blocks with residuals and normalization, and predicts the next token. The progress is in the details of each box.

Common misconceptions, cleared up early

Misconception	Reality
"Qwen and DeepSeek use a fundamentally different kind of neural network."	They're decoder-only Transformers built from the same blocks. The differences are in attention storage, FFN organization (MoE), and training methods.
"A 671B model uses all 671 billion parameters for every token."	In an MoE model, only a small number of experts are active per token (37B in DeepSeek-V3). All parameters must be stored, but few are used at once.
"MoE models are smaller and cheaper to store."	They need memory for *all* parameters; the saving is in compute per token, not in stored size.
"DeepSeek's trick is a secret new algorithm."	Its pieces (MLA, fine-grained experts, FP8 training, multi-token prediction) are all described in public papers. The achievement is combining them carefully and efficiently.
"Multi-token prediction changes how the model generates text."	It's mainly a *training* objective that gives a denser signal. It can also help speed up generation, but the model is still a next-token predictor.
"Newer is always better on every measure."	These changes trade off cost, complexity and quality in different ways, and real models combine only the ones that suit their goals.

Quick reference

```
Qwen3 dense     GQA + RoPE + QK-Norm + SwiGLU + RMSNorm(pre-norm); 0.6B–32B
Qwen3 MoE       128 experts, 8 active, no shared expert; 30B-A3B and 235B-A22B
Qwen3 training  ~36T tokens: general (4K ctx) → reasoning (~5T) → long context (32K);
                post-training: CoT cold start → reasoning RL → thinking fusion → general RL
Qwen newer      hybrid Gated DeltaNet : full attention = 3 : 1, MoE, multi-token prediction

DeepSeek-V3     671B total / 37B active; MLA; 1 shared + 256 routed experts, 8 active;
                first 3 layers dense; aux-loss-free balancing (per-expert routing bias);
                multi-token prediction; FP8 training; 14.8T tokens; 4K→32K→128K context
DeepSeek-V3.2   adds DeepSeek Sparse Attention (top-2,048 tokens)
DeepSeek-V4     CSA+HCA attention, mHC residuals, Muon optimizer, FP4+FP8, 1M context
```

What's next

That completes our tour of the model itself: how it is built, trained, adapted, compressed and redesigned. The remaining chapters are about everything *around* the model, and they apply to any model. Chapter 22 covers how a model is packaged and served (GGUF, llama.cpp and vLLM), and Chapter 23 covers how

generation is made faster and spread across several GPUs. The last chapter, Chapter 24, steps back to the systems built around the model: routing between models, calling tools, memory, retrieval, agents, and keeping all of it observable and affordable.

Chapter 22 — Serving a Model: GGUF, llama.cpp and vLLM

Introduction

By now you can explain how a model is built (Chapters 2 to 8), trained (Chapters 9 to 11), run one token at a time (Chapter 12), adapted (Chapter 13) and compressed (Chapter 14), and you have seen how modern designs, from attention variants and Mixture of Experts to reasoning, build on that plain GPT. What we haven't asked is a very practical question:

> **You have a trained model. How do you actually get it running, for yourself on a laptop, and for thousands of users on a server?**

Three pieces of software answer it, and each one is a direct application of something you already know:

Piece	What it is	The problem it solves
GGUF	A file format	How do you *package* a (quantized) model so it loads fast and is self-contained?
llama.cpp	An inference engine	How do you *run* that file efficiently on ordinary hardware?
vLLM	A serving engine	How do you *serve* one model to many users at once without wasting the GPU?

Nothing in this chapter changes the model. There is no new attention, no new training. The Transformer from Chapters 2 to 8 is exactly what runs. This chapter is about **systems engineering around it**: files, memory, scheduling.

A note on where this chapter sits. Everything before it was about *the model*: its architecture and how it behaves. This chapter and the next are about **interfacing the model with its users**, and they are largely **model-independent**. The same ideas of packaging, memory and scheduling apply whether the model is a plain GPT, a Mixture of Experts or a multimodal model.

One framing to hold on to throughout:

> **llama.cpp is built to run a model efficiently. vLLM is built to serve that model efficiently to many users.**

Part 1 — What is a "model", as a file?

From the computer's point of view, a model is not "an AI". It is just:

```
Weights   +   Metadata
```

The **weights** are the huge collection of numbers from the earlier chapters: the embedding table, `W_Q`, `W_K`, `W_V`, `W_O`, the FFN matrices, the norm vectors, the LM head. The **metadata** is everything else the software needs to *use* those numbers: how many layers there are, how wide each one is, how long the context can be,

and how text is turned into tokens.

How models are stored for training

Models that come out of training, and that you download from Hugging Face, are usually stored in one of two formats:

- **PyTorch `.bin` files**: the older format, based on Python's `pickle`. Loading one can run arbitrary code, which is a security risk.
- `.safetensors`: the modern replacement. A safe, simple format: a small header describing each tensor, followed by the raw tensor bytes. It can also be memory-mapped (we'll see what that means in Part 3).

Both are excellent for training and for frameworks like PyTorch. But a safetensors model is spread across several files, and the information about the model lives next to the weights rather than inside them: the configuration is in a separate `config.json`, the tokenizer in other files, and so on.

What an inference engine like llama.cpp wants

An engine that runs models on laptops and phones has different priorities:

- **One self-contained file**, so you can copy and share a model easily.
- **Fast loading**, even for a 30 GB model.
- **Quantized weights stored natively**, in the block formats from Chapter 14.
- **Everything needed to run it inside the file**: the tokenizer and the chat format included.

So the llama.cpp project created its own format: **GGUF**. A useful analogy: `.docx` is a good format for *editing* a document, and `.pdf` is a good format for *reading* it. Same content, different optimization. Safetensors is for training and sharing; GGUF is for running.

(GGUF replaced the project's earlier formats, GGML, GGMF and GGJT. The key improvement was extensibility: hyperparameters are stored as flexible key–value pairs, so new model types and new fields can be added without breaking old files.)

Part 2 — Inside a GGUF file

A GGUF file has a simple, sequential layout:

```
┌──────────────────────────────────────────────────────┐
│ HEADER                                               │
│   magic          "GGUF"                              │
│   version        3                                   │
│   tensor_count   how many tensors the file holds     │
│   kv_count       how many metadata entries           │
├──────────────────────────────────────────────────────┤
│ METADATA   (key → value pairs)                       │
│   general.architecture        = "llama"              │
│   llama.context_length        = 8192                 │
│   llama.block_count           = 32                   │
│   llama.embedding_length      = 4096                 │
│   llama.attention.head_count  = 32                   │
│   tokenizer.ggml.tokens       = [ ...every token... ] │
│   tokenizer.chat_template     = "..."                │
├──────────────────────────────────────────────────────┤
│ TENSOR INFO   (one entry per tensor)                 │
│   name, shape, quantization type, offset in the file │
├──────────────────────────────────────────────────────┤
│ padding to an alignment boundary (default 32 bytes)  │
├──────────────────────────────────────────────────────┤
│ TENSOR DATA   (all the weights, as raw bytes)        │
└──────────────────────────────────────────────────────┘
```

So GGUF is **not just weights**. It is everything needed to run the model. Let's look at each part.

The header

Four small fields. The **magic** is the four bytes `G G U F`, which lets software recognize the file type immediately. Then a **version number** (currently 3), the **number of tensors**, and the **number of metadata entries**. That's all the program needs to know how to read the rest.

The metadata

Metadata is a list of named values. Keys are hierarchical, written like `llama.context_length`. This is where the model's architecture is described, in exactly the terms you've learned:

Metadata key (example)	What it says	Where we learned it
`general.architecture`	Which model family (`llama`, `qwen2`, ...)	Chapter 15
`llama.block_count`	Number of stacked Transformer blocks	Chapter 8
`llama.embedding_length`	Width of each token vector, d	Chapter 3
`llama.attention.head_count`	Number of query heads	Chapter 5
`llama.attention.head_count_kv`	Number of K/V heads (GQA)	Chapter 17
`llama.feed_forward_length`	Hidden width of the FFN	Chapters 6 and 16
`llama.context_length`	Maximum context	Chapter 12
`llama.rope.*`	RoPE settings	Chapter 16
`tokenizer.ggml.tokens` and merges	The whole vocabulary and merge rules	Chapter 2
`tokenizer.ggml.bos_token_id`, `eos_token_id`	Special tokens	Chapter 2
`tokenizer.chat_template`	How to format "system / user / assistant" turns	Chapter 13

Notice that the **tokenizer travels inside the file**. With safetensors you need to keep track of separate tokenizer files; with GGUF, one file is the model, the vocabulary and the chat format.

Tensor info and tensor data

For every tensor, the file stores its **name**, its **shape**, its **data type**, and its **offset** (where its bytes start inside the data section). The data type is where quantization enters: it can be `F32`, `F16`, or one of the quantized block formats (`Q8_0`, `Q4_K`, `Q6_K`, ...) from Chapter 14. Different tensors in one file can use different types, which is how a "mixed" format like `Q4_K_M` is stored.

Tensors follow a standard naming scheme, so a Transformer's parts are easy to recognize:

Tensor name	What it is	Chapter
`token_embd.weight`	The embedding table	3
`blk.N.attn_norm.weight`	Norm before attention in block N	7, 16
`blk.N.attn_q.weight`	`W_Q`	4
`blk.N.attn_k.weight`	`W_K`	4
`blk.N.attn_v.weight`	`W_V`	4
`blk.N.attn_output.weight`	`W_O`	5
`blk.N.ffn_norm.weight`	Norm before the FFN	7, 16
`blk.N.ffn_gate.weight`, `ffn_up.weight`, `ffn_down.weight`	The SwiGLU FFN	16
`output_norm.weight`	The final norm	8
`output.weight`	The LM head	8

Reading a GGUF file is therefore like reading the table of contents of the Transformer you've been building. (A Mixture-of-Experts model, Chapter 18, simply has many extra per-expert FFN tensors in each block, which is why its file is large even though only a few experts are used per token.) Here is what it looks like for an 8-billion-parameter Llama-style model (32 blocks, `d = 4096`, 32 query heads and 8 K/V heads of size 128, FFN width 14,336, vocabulary 128,256):

```
token_embd.weight          128,256 × 4,096     (the embedding table)

for each of the 32 blocks:
  attn_q.weight             4,096 × 4,096
  attn_k.weight             4,096 × 1,024      (8 K/V heads × 128: GQA makes K smaller)
  attn_v.weight             4,096 × 1,024
  attn_output.weight        4,096 × 4,096
  ffn_gate.weight           4,096 × 14,336
  ffn_up.weight             4,096 × 14,336
  ffn_down.weight          14,336 × 4,096
  attn_norm.weight, ffn_norm.weight    4,096 each

output_norm.weight           4,096
output.weight              128,256 × 4,096     (the LM head)
```

That is 9 tensors per block, so `32 × 9 + 3 = 291` tensors in total (a few models add a handful of extras). Adding up the matrices gives about 8.0 billion parameters, and the K and V matrices are visibly four times narrower than `W_Q`, which is GQA, from Chapter 17, written in a file. The small norm vectors are usually kept in `F32` even in a 4-bit file, since they are tiny and sensitive.

Alignment

Between the tensor info and the data, the file is padded so that every tensor starts at a multiple of 32 bytes (by default). This isn't wasteful. It lets the CPU and GPU read the data in the aligned chunks they are fastest at, and it is what makes the next trick possible.

Part 3 — Memory mapping: why a 30 GB model starts quickly

The slow way

A simple program loads a model like this:

```
Disk  →  read the whole file into RAM  →  copy it into the program's own memory  →  ready
```

For a 30 GB model, that means reading 30 GB and copying 30 GB before the first token. It is slow, and it needs enough RAM to hold *two* copies for a while.

Memory mapping

Operating systems offer a better tool, **memory mapping** (`mmap`). Instead of reading the file, the program asks the operating system to *map the file into its address space*, as if the whole file were already sitting in memory:

```
Disk
  ↓
The OS maps the file into the program's memory (instantly, nothing is read yet)
  ↓
When the program touches a part of a tensor, the OS loads just that part ("page fault")
  ↓
Pages stay cached in RAM, and are dropped by the OS when memory gets tight
```

The weights are loaded **on demand**, a small piece at a time, by the operating system. There is no second copy, start-up is fast, and if several processes load the same model file they can **share** the same cached pages. It works so well because a Transformer's weights are read-only: nothing ever writes to them, and the alignment from Part 2 means each tensor can be used straight from the mapped file.

The honest caveats

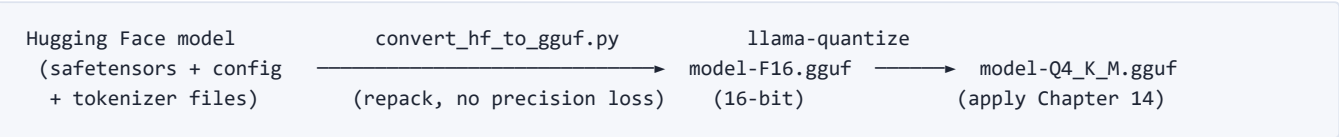
- **The first tokens can be slow.** If the model is on a slow disk and hasn't been read yet, the first pass through the model triggers many page loads. Once the pages are cached, it's fast.
- **If RAM is too small, the OS will drop and re-read pages**, and generation becomes painfully slow. Mapping doesn't make a model *fit*; it makes loading *lazy*.
- Flags let you choose: llama.cpp maps the file by default, `--no-mmap` loads it normally, and `--mlock` asks the OS to keep the model locked in RAM so it can't be paged out.

GPU offload: layers split between VRAM and RAM

If you have a GPU, the engine can place some or all of the **layers** in GPU memory (the `-ngl` option sets how many). A model too big for your GPU can still run with, say, 24 layers on the GPU and 8 on the CPU: slower than all-GPU, but much faster than all-CPU. Because Chapter 14's quantization shrinks every layer, more layers fit.

Part 4 — From Hugging Face to GGUF: what conversion does

You will often start with a model released as safetensors and need a GGUF. The process has two steps:



1. **Convert.** A script reads the safetensors tensors and the config and tokenizer files, renames the tensors to the standard GGUF names (`model.layers.7.self_attn.q_proj.weight` becomes `blk.7.attn_q.weight`), rearranges weights into the layout llama.cpp expects, writes the config into metadata keys, and embeds the tokenizer. The result is a 16-bit GGUF with no loss of precision.
2. **Quantize** (optional). A second tool compresses the weights into a quantized type, applying everything from Chapter 14: block-wise scales, mixed precision, and so on.

Three things are *not* touched. Conversion does not retrain the model, it does not change what the model computes, and (apart from the optional quantization step) it does not modify the weights. It repackages them. In practice you can also just download a ready-made GGUF, since many people publish models in several quantization levels, which is why the filenames from Chapter 14 (`Q4_K_M.gguf` and friends) are so common.

> **GGUF in one sentence:** an inference-optimized file format that packages the model's weights, tokenizer, metadata and quantization information so that engines like llama.cpp can load and run the model efficiently.

Part 5 — llama.cpp: running the Transformer you already know

llama.cpp is an inference engine written in C and C++, built on a small tensor library called **ggml**. It has no Python and no heavy dependencies, which is why it runs almost anywhere: laptops, phones, Raspberry Pis, servers. It has **backends** for CPUs (using vector instructions such as AVX and ARM NEON), for NVIDIA GPUs (CUDA), for Apple GPUs (Metal), and for others (Vulkan and more), and it can split a model across them.

What happens at start-up

When you launch it with a model:

```
llama-cli -m model.gguf
```

it does roughly this:

```
Open the GGUF file
        ↓
Read the metadata            (architecture, layer count, widths, context length)
        ↓
Load the tokenizer           (vocabulary, merge rules, chat template)
        ↓
Memory-map the weights       (Part 3), offloading some layers to the GPU if asked
        ↓
Allocate the KV cache        (sized from the context length you chose, Chapter 12)
        ↓
Wait for a prompt
```

Notice what *isn't* there: no training, no weight changes. Start-up only prepares the weights for fast inference. The KV cache is reserved up front for the context length you ask for (the `-c` option), using exactly the size formula from Chapter 12, so memory use is predictable.

What happens when you type a prompt

Take `What is AI?`. The whole run is the pipeline you've been studying, with each step labelled by its chapter:

```
"What is AI?"
     │
     ▼  Tokenizer (Chapter 2)              text → token IDs, e.g. [3923, 374, 15592, 30] (illustrative)
     ▼  Embedding lookup (Chapter 3)       each ID → a vector
     ▼  Transformer blocks × N             (Chapters 4 to 7, in the modern form of Chapter 16)
     │      attention uses Q, K, V;  the K and V of every token go into the KV cache (Chapter 12)
     ▼  LM head (Chapter 8)                → logits
     ▼  Sampling (Chapter 12)              temperature, top-k, top-p → one token
     │
     └──► the new token is fed back in; the KV cache is reused, so only the NEW token
          goes through the expensive part. Repeat until the end-of-text token.
```

The first pass over the whole prompt is the **prefill** (compute-heavy). Every pass after that is a **decode** step (memory-bound). Chapter 12 explained why, and llama.cpp's benchmarking tool reports them as two separate numbers: prompt-processing speed and token-generation speed.

Two things make it fast:

- **It runs quantized weights directly.** As Chapter 14 described, the matrix multiplications use kernels that unpack the 4-bit blocks and multiply in one go, rather than converting everything back to 16-bit first.
- **It uses all the hardware you give it:** vector instructions on the CPU, layers offloaded to the GPU, and a FlashAttention-style attention kernel (Chapter 17) where supported.

The important realization

> **llama.cpp does not invent a new algorithm. It executes the Transformer from Chapters 2 to 8, very efficiently.**

Everything you spent time learning is exactly what runs. It is *"the loop of Chapter 1"* implemented in highly optimized C.

Beyond the command line: llama-server

llama.cpp also includes an HTTP server, **llama-server**, that speaks an **OpenAI-compatible API** (chat completions, embeddings and so on), so an application can use a local model through the same kind of requests it would send to a hosted one. It supports several parallel "slots" so more than one conversation can be handled at once, continuous batching (which you'll meet below), and prompt caching, which reuses the cached prefix of a prompt it has seen before (the prefix caching idea from Chapter 12). The context size is shared among the slots.

What llama.cpp is best at

Running **one model for one person (or a few)** on whatever hardware you have, especially with quantized weights, CPUs, mixed CPU/GPU, or Apple silicon. It is the engine behind many desktop apps and personal assistants. But the design target changes when you have *hundreds* of users hitting a large GPU. That is vLLM's territory.

Part 6 — Many users, one GPU: why batching matters

Picture a server with two users. User A asks for a poem. User B asks for an explanation of AI. A simple engine handles them like this:

```
User A  ████████████
                      User B  ████████████
```

One after another. User B waits for all of A's tokens, and the GPU is mostly idle between and within steps. That is wasteful, and we can see exactly why from what you already know.

Why batching is nearly free

Recall from Chapters 12 and 14 that the **decode** phase is **memory-bound**: for each new token the GPU has to *read all the model's weights* from memory, but does very little arithmetic with them. Now, here's the key observation: **reading the weights once can serve many requests at the same time.** If ten users each need their next token, we can stack their vectors into one bigger matrix and multiply by the weights in one pass.

An idealized example. A 7B model in 16-bit is about 14 GB. On a GPU that reads roughly 1 TB per second, one decode step costs at least `14 GB / 1000 GB/s ≈ 14 ms` just to read the weights.

```
Batch of 1:    one step ≈ 14 ms  →   ~70 tokens/s total
Batch of 32:   one step ≈ 14 ms  →   32 tokens per step  →  ~2,300 tokens/s total
```

(The ceiling isn't reached in practice, because the KV caches also have to be read and the arithmetic grows, but the shape is the point.) **Batching multiplies throughput almost for free**, because GPUs love big matrix multiplications. A single request is a small, poorly used workload; thirty requests together are one large, efficient one.

So a serving engine's job is to keep the GPU's batch as full as possible, all the time. That raises two problems, which are exactly what vLLM is famous for solving.

Part 7 — Continuous batching: keep every slot busy

The problem with static batching

The first idea is **static batching**: collect, say, four requests, process them together, and start the next batch only when *all four* are done. But requests don't take the same time. One user wants a one-line answer; another wants an essay. Here is each request's slot over time (`▉` = working, `░` = idle, waiting for the others):

```
STATIC BATCHING          step:  1 2 3 4 5 6 7 8 9 10

slot 1   request A              ▉ ▉ ▉ ▉ ░ ░ ░ ░ ░ ░     A finished at step 4, slot idle
slot 2   request B              ▉ ▉ ▉ ▉ ▉ ▉ ▉ ▉ ▉ ▉
slot 3   request C              ▉ ▉ ▉ ▉ ▉ ▉ ░ ░ ░ ░
slot 4   request D              ▉ ▉ ▉ ░ ░ ░ ░ ░ ░ ░

new requests E, F, G wait until the whole batch ends        busy: 23 of 40 slot-steps (57%)
```

The whole batch moves at the speed of its slowest member, short requests leave idle slots behind them, and new users have to wait.

The fix: continuous (iteration-level) batching

Continuous batching makes the scheduling decision *at every single decoding step* rather than once per batch. As soon as a request finishes, its slot is handed to a waiting request, immediately:

```
CONTINUOUS BATCHING      step:  1 2 3 4 5 6 7 8 9 10

slot 1   A, then E              A A A A E E E E E E     E takes over the moment A finishes
slot 2   B                      B B B B B B B B B B
slot 3   C, then F              C C C C C C F F F F
slot 4   D, then G              D D D G G G G G G G

                                                        busy: 40 of 40 slot-steps (100%)
```

The GPU stays full, short requests finish quickly instead of waiting for long ones, and new requests start almost immediately. Think of a restaurant that seats a new party the moment any table frees up, instead of waiting for every table to finish before seating anyone.

A related detail worth knowing: processing a long new prompt (a big prefill) in one go could stall everyone else's generation. Modern engines handle this by splitting long prompts into chunks (**chunked prefill**) and mixing them into the same steps as other people's decoding.

Part 8 — The memory problem: every user has a KV cache

Continuous batching creates a new problem. Each request has its own **KV cache** (Chapter 12): request A's cache, request B's, and so on, each one growing token by token. How much memory is that? The vLLM paper gives a concrete example: for a 13-billion-parameter model (OPT-13B), the cache costs about **800 KB per token**. A request that grows to 2,048 tokens therefore needs about **1.6 GB** of KV cache, for one request. (A modern model with grouped-query attention is much smaller per token, for instance 128 KiB for Llama-3-8B, as Chapter 12 showed, but with hundreds of requests the total still dominates GPU memory.)

Why the simple approach wastes memory

The obvious way to store a request's cache is as **one contiguous block of memory**. But at the start, you don't know how long the answer will be. So the engine has to reserve space for the **maximum possible length**, say 2,048 tokens, even if the request ends after 50. This causes three kinds of waste:

```
Reserved but unused     space held for tokens that haven't been generated yet
Internal fragmentation  space reserved for the maximum, but the request ends early
External fragmentation  gaps left between differently-sized blocks of memory
```

The paper measured it: in existing systems, **only about 20% to 38% of the KV-cache memory actually held token data.** The rest was waste. And since memory limits how many requests fit in a batch, and batch size decides throughput, wasted memory directly means wasted speed.

Part 9 — PagedAttention: the cache as pages of memory

The vLLM authors noticed that this problem had been solved decades ago, by **operating systems**. A computer doesn't give each program one giant contiguous slab of RAM. It hands out small, fixed-size **pages**, which don't have to sit next to each other, and keeps a **page table** that maps the program's logical view to the physical pages.

PagedAttention applies exactly the same idea to the KV cache:

- The cache of each request is split into small **blocks**, each holding the Keys and Values of a fixed number of tokens (16 by default in the paper).
- Blocks are allocated **only as the request grows**: a new block is claimed when the previous one fills up.
- Blocks don't have to be contiguous in GPU memory. A **block table** records which physical block holds each part of the request's cache.

```
Request A's cache, as the request sees it (logical blocks):

   [ logical 0 ]  [ logical 1 ]  [ logical 2 ]
        │              │              │
        ▼              ▼              ▼         block table of request A
        7              1              3         (logical block → physical block)

GPU memory, as it really is (physical blocks, in any order):

   physical:   0    1    2    3    4    5    6    7    8    9
             ┌────┬────┬────┬────┬────┬────┬────┬────┬────┬────┐
             │ B0 │ A1 │free│ A2 │ B1 │free│free│ A0 │free│free│
             └────┴────┴────┴────┴────┴────┴────┴────┴────┴────┘
```

Request A's three logical blocks live in physical blocks 7, 1 and 3, scattered. Request B's two live in 0 and 4. Neither needed a contiguous slab. The attention computation looks up the block table to find the Keys and Values it needs.

Why it solves the problem

- **No big reservation.** A request only ever holds the blocks it has actually filled, so the only waste is the unused part of its *last* block (at most 15 tokens' worth).
- **No external fragmentation.** Any free block can be used by anyone.
- **Instant reuse.** When request A finishes, its blocks go straight back to the free pool for the next request.

So far, "like an operating system's memory manager". It also unlocks things a contiguous cache couldn't do.

Sharing blocks between requests

Suppose one prompt is used to generate several different answers at once (parallel sampling), or many users start with the *same long system prompt*. Those requests have **identical** Keys and Values for the shared part. With paging, they can point their block tables at the **same physical blocks**, storing the shared prefix **once**:

```
Request 1 block table:   [ P0 ] [ P1 ] [ X ]          X = request 1's own new block
Request 2 block table:   [ P0 ] [ P1 ] [ Y ]          Y = request 2's own new block
                           ▲      ▲
                           └──────┴── the same physical blocks, stored once
```

Each block keeps a **reference count**, the number of requests using it. If a request needs to *write* into a shared block (for example, the two answers diverge), the engine first makes a private copy of that block and writes into the copy. That is **copy-on-write**, borrowed straight from operating systems too.

The same idea extends across requests that arrive at different times. This is **automatic prefix caching**: each full block is identified by a hash of its tokens (and the blocks before it), so when a new request starts with text that has already been computed, such as a shared system prompt, the engine finds the existing blocks and skips recomputing them. Unused cached blocks are evicted when memory runs short, least recently used first. This is the engine-side machinery behind the "prefix caching" idea we met in Chapter 12.

When memory runs out

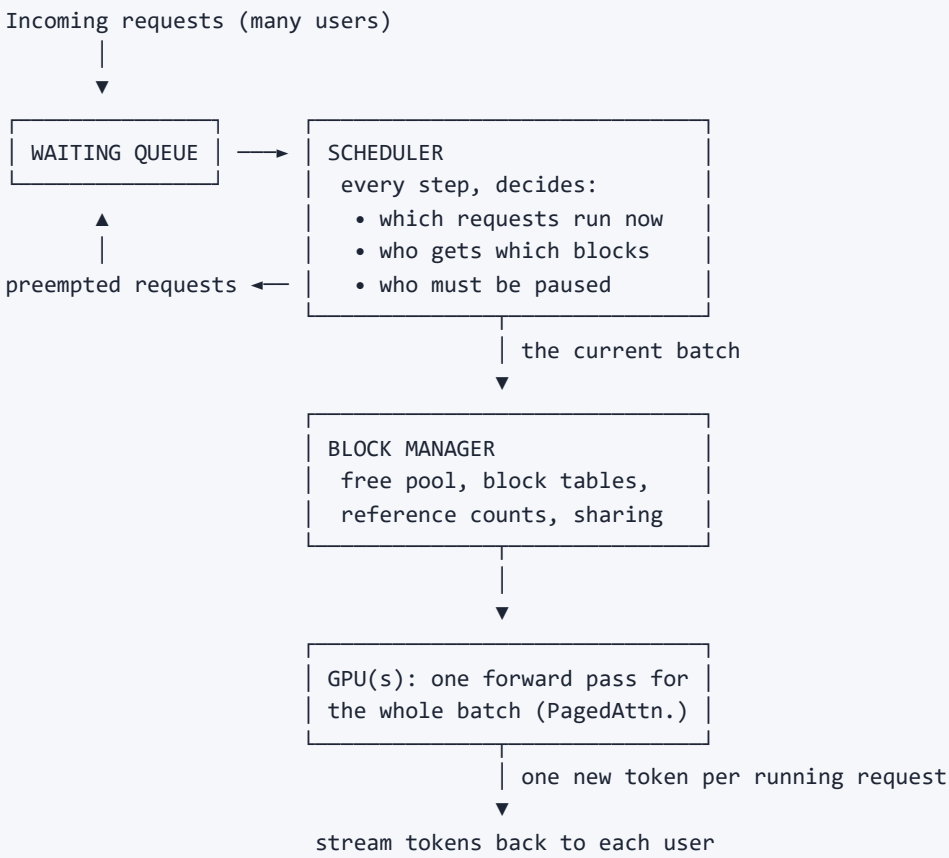
Even with paging, a busy server can run out of blocks. The engine then **preempts** some requests, pausing them to free their blocks, and resumes them later. There are two ways to get the cache back: **swap** the blocks out to CPU memory and copy them back in later, or simply **recompute** the cache when the request is rescheduled (a fresh prefill). Either way, nobody's answer changes.

The result

The paper reported **2× to 4× higher throughput** than the best existing systems at the same latency, with the biggest gains for longer sequences, larger models and more complex decoding. Not because the model got faster, but because wasted memory became useful batch capacity.

Part 10 — vLLM as a whole

vLLM is the serving engine built around these ideas. Here is the loop that runs continuously on the server:



Its pillars are the ones you've just seen:

- **Continuous batching**: the batch is rebuilt every step, so slots never sit idle.
- **PagedAttention**: efficient, shareable, non-contiguous KV-cache memory.
- **Prefix caching**: shared prompts are computed once.
- **High GPU utilization**: the consequence of the three above.

On top of that it provides an **OpenAI-compatible HTTP server**, supports quantized models, and can spread a model across several GPUs with tensor and pipeline parallelism (the subject of the next chapter).

Part 11 — llama.cpp or vLLM?

The difference is the design target, not what the model computes: in both cases it is the same Transformer. (llama.cpp's server also has parallel slots and continuous batching, so it can serve several users. It simply isn't built around squeezing the last bit of throughput from a large GPU.)

	llama.cpp	**vLLM**
Built for	Running a model efficiently, anywhere	Serving a model to many users, efficiently
Typical hardware	Laptops, desktops, phones, CPUs, Apple silicon, small GPUs	Data-centre GPUs
Model format	GGUF (quantized, self-contained)	Hugging Face-style weights (safetensors), including GPTQ, AWQ and FP8 quantizations (GGUF loading exists but is not its main path)
Typical users	One person, or a few	Many concurrent users
Strength	Portability, low memory, easy setup	Throughput, memory management at scale
Typical use	Desktop apps, local assistants, edge devices	API servers, production deployments

An application that sits on top of a model doesn't need to care which one is behind it: both expose OpenAI-style HTTP endpoints, so either can be one "model provider" among several.

> **One sentence to remember:** llama.cpp is optimized to run a model efficiently. vLLM is optimized to serve that model efficiently to many users.

Common misconceptions, cleared up early

Misconception	Reality
"A model file contains only the weights."	A GGUF file also holds the architecture description, the tokenizer, the chat template and the quantization metadata.
"Converting to GGUF changes or retrains the model."	Conversion only repackages (and optionally quantizes) the weights. The model computes the same thing.
"llama.cpp can't load safetensors because it's a bad format."	Safetensors is fine for training and sharing. GGUF is a different optimization: one self-contained file with native quantized blocks.
"Memory mapping makes a big model fit in a small RAM."	It makes loading lazy and cheap. If the model doesn't fit, pages get dropped and re-read, and generation becomes very slow.
"llama.cpp uses a different algorithm to run models."	It runs the standard Transformer, just implemented very efficiently, with quantized kernels.
"llama.cpp only works for one user at a time."	llama-server supports parallel slots and continuous batching. It just isn't tuned for very large-scale GPU serving.
"Batching users together slows each one down a lot."	Decode is memory-bound, so reading the weights once serves the whole batch. Throughput rises sharply for a small cost per user.
"Static and continuous batching are the same thing."	Static batching waits for the slowest request in a batch. Continuous batching refills freed slots at every step.
"PagedAttention changes the attention math."	It changes only where the cached Keys and Values are stored. The computed attention is the same.
"vLLM is faster because it uses a faster model."	The same model. The gain comes from not wasting KV-cache memory, which allows larger batches.
"The KV cache is stored contiguously per request."	In a naive engine, yes (and it wastes memory). In vLLM it is split into fixed-size blocks that can be anywhere.

Quick reference

```
Model file  =  weights + metadata
safetensors  training/sharing format (tensors only; config and tokenizer in other files)
GGUF         inference format: header | metadata | tensor info | (padding) | tensor data
             metadata includes architecture, tokenizer, chat template; tensors can be mixed-quantized
Tensor names token_embd | blk.N.attn_{q,k,v,output} | blk.N.ffn_{gate,up,down} | *_norm | output
mmap         map the file into memory; the OS loads pages on demand; no second copy
Convert      HF → convert script → F16 GGUF → llama-quantize → e.g. Q4_K_M

llama.cpp    C/C++ engine on ggml; CPU/CUDA/Metal/Vulkan; layer offload (-ngl);
             quantized kernels; KV cache preallocated for the context size; llama-server (OpenAI API)
             Run:  tokenize → embed → blocks (+KV cache) → LM head → sample → repeat

Batching     decode is memory-bound → one read of the weights serves the whole batch
Static       whole batch waits for the slowest request          (e.g. 57% busy)
Continuous   refill finished slots every step                   (e.g. 100% busy)

Naive KV cache   contiguous, reserved for max length → only ~20–38% holds real tokens
PagedAttention   fixed-size blocks (16 tokens), block table, allocate on demand
                 shared blocks + copy-on-write; prefix caching; preempt by swap or recompute
vLLM             scheduler + block manager + continuous batching + PagedAttention
                 (paper: 2–4× throughput)

llama.cpp = run a model efficiently      vLLM = serve a model to many users efficiently
```

What's next

We now know how a model is packaged, run on one machine, and served to many users. Two natural limits remain. Generation still produces **one token per forward pass**, however clever the batching. And a big enough model **doesn't fit on one GPU** at all. Chapter 23 tackles both: **speculative decoding**, where a small model drafts several tokens and the large model checks them in a single pass, and **parallelism**, the ways a model (and its data) can be split across many GPUs. It also steps back to look at the hardware those GPUs sit on, and why the industry is building chips designed specifically for LLMs.

Chapter 23 — Speed and Scale: Speculative Decoding, Parallelism and Hardware

Introduction

Chapter 22 showed how to package a model, run it, and serve it to many users. Two stubborn limits remain, and they are what this chapter is about:

1. **Generation produces one token per forward pass.** However cleverly requests are batched, each *individual* answer still grows one token at a time, and every one of those steps has to read all the model's weights (Chapter 12). How can a single request go faster?
2. **A big model doesn't fit on one GPU.** A 70-billion-parameter model in 16-bit numbers is about 140 GB, far more than the 80 GB of a high-end GPU. How do you run a model that is bigger than the machine?

The chapter has three parts:

Part	Question	Answer
Speculative decoding	How can one request generate faster?	A small model drafts several tokens, and the big model checks them all in one pass
Parallelism	How can a model (and its data) be spread over many GPUs?	Split the matrices, split the layers, or split the data
Hardware	Why are GPUs used, and why are new chips being built for LLMs?	Because LLM inference is limited by memory movement as much as by arithmetic

As in Chapters 14 and 22, **nothing here changes what the model computes.** Speculative decoding gives exactly the same output distribution as ordinary decoding. Parallelism splits the same arithmetic across machines. The Transformer is untouched.

Part 1 — Speculative decoding: a small model drafts, a big model verifies

The problem

A large model is slow at generating one token at a time. Each step is a full forward pass:

```
Large model  →  token 1
Large model  →  token 2
Large model  →  token 3
Large model  →  token 4
```

And remember why each step is slow. In the decode phase (Chapter 12) the GPU spends its time *reading the weights from memory*, and does very little arithmetic with them. The big model is mostly waiting for data.

The key observation

If a step is limited by reading the weights rather than by arithmetic, then **checking several tokens in one pass costs almost the same as producing one**. The weights are read once either way. This is the same fact that made batching nearly free in Chapter 22, used differently: instead of serving several *users* per weight-read, serve several *positions of one user*.

There is also a second fact, from Chapter 9. When we trained the model, a single forward pass over a sentence produced a next-token prediction at **every position at once**, thanks to the causal mask. The model can do the same at inference. If we hand it a prompt plus some *guessed* extra tokens, one forward pass tells us what the model would have predicted after each of them.

So here is the plan:

> Get *cheap guesses* for the next several tokens from somewhere, then ask the big model to check all of them in a single pass.

The idea

Use a **small, fast "draft" model** to guess several tokens ahead:

```
Draft model (small, fast):
   "I think the next tokens are:   the → capital → of → France → is → London"
```

Then ask the large model to **verify** them in one forward pass, giving it the prompt followed by all the drafted tokens. Because of the causal mask, that single pass produces the large model's own prediction after every position:

```
Large model input:   The capital of France is  London
Large model's view:  ✓ "capital"   ✓ "of"   ✓ "France"   ✓ "is"   ✗ "London" (it wanted "Paris")
```

The first four drafted tokens match what the large model would have said, so they are accepted. The fifth is wrong, so it is rejected, and the large model supplies the correct token instead. In one large-model pass we advanced **five tokens** (four accepted plus the corrected one) instead of one:

```
WITHOUT speculation:    Large → Large → Large → Large → Large          (5 passes)

WITH speculation:       Draft (cheap, several steps)
                              ↓
                        Large: verify all, in ONE pass                 (1 pass)
                              ↓
                        accept the good prefix, fix the first mistake
```

In one line:

> **Small model = guess. Large model = judge.**

The small model never *replaces* the large one. And it is an optimization in a very specific sense: **the model weights themselves don't change, and nothing is retrained.**

But is the output the same? Why this is exactly correct

An important worry: surely the answer depends on whose guesses we accept? It doesn't. The acceptance rule is designed so that the **final text follows exactly the large model's probability distribution**, as if the small model had never been involved. The idea (from the original papers) is a form of rejection sampling.

For each drafted token, with `q` the draft model's probability for it and `p` the large model's probability for it:

```
Accept the drafted token with probability   min(1, p / q)

If it is rejected, sample the replacement from the "leftover" distribution:
   (what the large model wanted more than the draft did), re-normalized.
```

A worked example. After the prompt `The capital of France is`, suppose the next-token probabilities are:

Token	Draft model `q`	Large model `p`
Paris	0.30	0.90
London	0.50	0.05
Rome	0.20	0.05

The draft model happens to pick **London**. The large model thinks London is unlikely (`p = 0.05` against `q = 0.50`), so the acceptance probability is `min(1, 0.05 / 0.50) = 0.10`. Most of the time it is rejected. Then the replacement is drawn from what the large model wants *more* than the draft did: `p − q` is positive only for Paris (`0.90 − 0.30 = 0.60`), so the leftover distribution is simply Paris.

Now add up every way the process can end:

```
End with Paris:   draft says Paris (0.30) → always accepted                      = 0.30
                  draft says London (0.50) → rejected (0.90) → replaced by Paris  = 0.45
                  draft says Rome   (0.20) → rejected (0.75) → replaced by Paris  = 0.15
                                                                      total  = 0.90  ✓
End with London:  draft says London (0.50) × accepted (0.10)                     = 0.05  ✓
End with Rome:    draft says Rome   (0.20) × accepted (0.25)                     = 0.05  ✓
```

The result is **Paris 0.90, London 0.05, Rome 0.05**: exactly the large model's own distribution, `p`. The draft model only affects *how fast* we get there, never *what* comes out. (With greedy decoding, temperature 0, the rule simplifies to "accept a token if it is the large model's top choice".)

How much faster?

Two numbers decide the speed-up: the **acceptance rate** `α` (how often a drafted token is accepted) and the **cost ratio** `c` (how expensive one draft step is compared with one large-model step). If the draft proposes `K` tokens per round, the expected number of tokens produced per large-model pass is:

```
expected tokens per pass  =  (1 − α^(K+1)) / (1 − α)
```

and the overall speed-up is that number divided by the cost of the round, `1 + K·c`.

Acceptance rate α	Tokens per large pass (K = 4)	Speed-up if the draft is 20× cheaper (c = 0.05)
0.5	1.9	≈ 1.6×
0.8	3.4	≈ 2.8×
0.9	4.1	≈ 3.4×

So the speed-up depends heavily on **how often the draft guesses right**. The original paper reported 2× to 3× speed-ups on a large model with no change to its outputs. Text that is predictable (code, boilerplate, quoted passages, formulaic answers) has a high acceptance rate, while open-ended creative text has a lower one.

Where the draft comes from: variants

A separate small model is only one way to get the guesses:

- **A smaller model from the same family**, such as a 1B model drafting for a 70B one. It must use the same tokenizer and vocabulary so the tokens line up.
- **Extra prediction heads on the model itself** (methods such as Medusa and EAGLE), so no second model is needed. A related idea is the **multi-token prediction** trained into models like DeepSeek-V3, whose extra prediction can later be reused to speed up generation (Chapter 21).
- **No model at all**: simply look for text in the prompt that the answer is likely to repeat (an "n-gram" or prompt-lookup draft). It works well when summarizing, editing, or answering about a document.

When it helps less

- **When the server is already busy.** Chapter 22's batching already uses the GPU's spare capacity. If a large batch makes the GPU compute-bound, there is less free capacity for the extra verification work.
- **When acceptance is low**, since wrong guesses are wasted work.
- It also needs memory for the draft model (or heads) and its cache.

It is best thought of as another example of the pattern that runs through this handbook: **don't make the expensive model do work that a cheaper source can propose first.**

Part 2 — Parallelism: running a model bigger than one GPU

The problem

Suppose a model needs 80 GB for its weights and your GPU has 24 GB. It obviously doesn't fit. Even with Chapter 14's quantization, the biggest models don't fit on one device, and the KV cache (Chapter 12) needs room as well. So we have to **split the work across several GPUs**. There are several ways to split, and each answers a different question:

```
Tensor parallelism    →  split the MATRICES inside each layer
Pipeline parallelism  →  split the LAYERS into groups
Data parallelism      →  keep a full COPY of the model, split the DATA
Expert parallelism    →  split the EXPERTS of a Mixture-of-Experts model
```

1. Tensor parallelism: split each layer across GPUs

Take the basic operation `Y = X·W`. Instead of one GPU holding all of `W`, each GPU holds a *piece*, computes its share of the answer, and the pieces are combined. There are two ways to cut a matrix, and a Transformer block uses both.

Cutting by columns. Each GPU holds some of the *columns* of `W` and produces some of the *output numbers*. Combining is just placing them side by side:

```
X = [1 2 3 4]        W (4×2) = [1 0]
                               [0 1]
                               [1 1]
                               [2 0]

GPU 1 holds column 1:   Y₁ = 1·1 + 2·0 + 3·1 + 4·2 = 12
GPU 2 holds column 2:   Y₂ = 1·0 + 2·1 + 3·1 + 4·0 = 5

Combine:   Y = [12  5]          (just concatenate, no arithmetic)
```

Cutting by rows. Each GPU holds some of the *rows* of `W` and receives the matching part of `X`. Each produces a *partial sum* of the same output, and the pieces have to be **added** together:

```
GPU 1 holds rows 1–2 of W and X[1, 2]:   partial = [1·1 + 2·0,  1·0 + 2·1] = [1  2]
GPU 2 holds rows 3–4 of W and X[3, 4]:   partial = [3·1 + 4·2,  3·1 + 4·0] = [11 3]

Combine:   Y = [1 2] + [11 3] = [12  5]      (an "all-reduce": add the partial results)
```

Same answer, `[12 5]`, as computing it on one GPU.

How a Transformer block uses these. The block is designed so that the two cuts chain neatly:

- **The FFN** (Chapter 6): cut the first matrix `W1` by **columns**. Each GPU computes its own slice of the hidden units. The activation function (GELU or SiLU) works on each number independently, so *no communication is needed* between the two matrices. Then cut the second matrix `W2` by **rows**, so each GPU produces a partial sum, and a single all-reduce adds them up.
- **Attention** (Chapter 5): the heads are independent of each other, so give each GPU a **group of heads**. Each GPU computes its heads (and keeps the KV cache for just those heads). Then `W_O` is cut by rows, and one all-reduce adds the partial results.

So each block needs just **two all-reduces**, one after attention and one after the FFN.

The big advantage is that tensor parallelism **cuts the latency of each token**. In decode, every GPU reads only *its share* of the weights, so a step is faster. For a 70B model in 16-bit (140 GB): split across 2 GPUs, each reads 70 GB per step; at roughly 3.35 TB/s of memory bandwidth that's about 21 ms per step instead of about 42 ms if the whole thing had fit on one GPU (communication time comes on top).

The cost is that those all-reduces happen **twice per block, for every block, for every token**, so the GPUs must talk to each other constantly and quickly. Compare the links that connect them:

Link	Speed (as NVIDIA quotes it for the H100, total both directions)
GPU memory (HBM)	about 3,350 GB/s
NVLink between GPUs in one server	about 900 GB/s
PCIe Gen 5	about 128 GB/s
Network between servers	far lower still

So tensor parallelism is normally used **inside one server**, across 2, 4 or 8 GPUs connected by a fast link like NVLink.

> **Mental model: split one layer across GPUs.**

2. Pipeline parallelism: split the layers into stages

Instead of cutting inside each layer, give each GPU a **consecutive group of whole Transformer blocks**:

```
GPU 1:  Blocks  1 – 8     ──►   activations   ──►
GPU 2:  Blocks  9 – 16    ──►   activations   ──►
GPU 3:  Blocks 17 – 24    ──►   activations   ──►
GPU 4:  Blocks 25 – 32  + LM head
```

The data flows through the GPUs like items along an assembly line. The communication is light: only the activation vectors at the boundaries travel between stages, and only once per stage rather than twice per block. That makes pipeline parallelism suitable for **linking separate servers**, where the network is slower.

The catch is that, naively, only one GPU is busy at a time while the others wait. This is called the **pipeline bubble**. The fix is to cut the batch into **micro-batches** so every stage always has something to work on:

```
NAIVE: one batch moves through four stages                time →
GPU 1   ▓▓ ·· ·· ··
GPU 2   ·· ▓▓ ·· ··
GPU 3   ·· ·· ▓▓ ··
GPU 4   ·· ·· ·· ▓▓          each GPU busy 1 slot in 4  (25%)

WITH 4 MICRO-BATCHES  (a, b, c, d)                        time →
GPU 1   a  b  c  d  ·  ·  ·
GPU 2   ·  a  b  c  d  ·  ·
GPU 3   ·  ·  a  b  c  d  ·
GPU 4   ·  ·  ·  a  b  c  d  busy 16 slots of 28  (57%)
```

With `m` micro-batches and `p` stages, the fraction of time the GPUs are busy is roughly `m / (m + p – 1)`: 25% for one batch, 57% for four, and about 84% for sixteen (with four stages). One more thing to remember: pipeline parallelism helps **throughput** and **memory**, but it does *not* shorten the time for one token, because that token still has to pass through every stage in turn.

> **Mental model: split the model vertically, by layers.**

3. Data parallelism: copy the model, split the data

Here every GPU holds a **full copy** of the model, and each handles a *different part of the workload*:

```
GPU 1 → full model copy → batch 1
GPU 2 → full model copy → batch 2
GPU 3 → full model copy → batch 3
GPU 4 → full model copy → batch 4
```

Its two uses are different:

- **In training**, each GPU computes gradients on its own slice of a large mini-batch, and then the gradients are **averaged across GPUs** (Chapters 10 and 11: a mini-batch gradient is already an average) before every copy applies the same update. This is how training scales to thousands of GPUs. Because the optimizer state (the momentum and variance from Chapter 11) is large, training systems often also **shard** it, and the weights, across the data-parallel GPUs instead of duplicating it.
- **In inference**, data parallelism is simply **more replicas**: two copies of the model serve twice as many users. It does nothing to make a too-big model fit.

> **Mental model: copy the model, split the data.**

4. Expert parallelism: split the experts

In a Mixture-of-Experts model (Chapter 18), the FFN of each block is replaced by many separate expert FFNs. **Expert parallelism** places different experts on different GPUs:

```
GPU 1 → Experts 1, 2
GPU 2 → Experts 3, 4
GPU 3 → Experts 5, 6
```

A router decides which experts each token uses, and the token's vector is sent to the GPU that holds them. Chapter 18 covers this in full.

5. One more: splitting the sequence

For extremely long contexts, even the attention itself can be spread out. **Context (or sequence) parallelism** splits the *tokens* of a long sequence across GPUs, so that no GPU has to hold the whole sequence's activations or attention at once.

Putting them together

Real deployments combine these. A typical large-model recipe:

```
Within one server (fast NVLink):        tensor parallelism across 8 GPUs
Across servers (slower network):        pipeline parallelism between groups of servers
To serve more users, or to train:       data parallelism, i.e. several copies of all of the above
For MoE models:                         expert parallelism as well
```

Method	What gets split	Communication	Typical use
Tensor parallel	The matrices inside each layer	Heavy (2 all-reduces per block per token) → needs fast links	Large-model inference and training, within a server
Pipeline parallel	Groups of layers	Light (activations between stages)	Very large models, across servers
Data parallel	The data / requests	Gradient averaging (training) or none (inference)	Training; more serving capacity
Expert parallel	MoE experts	Tokens routed to experts	MoE models
Context parallel	The sequence	Between attention chunks	Very long contexts

A last reminder from earlier chapters: parallelism is the **last** resort, not the first. A 70B model that needs 140 GB in 16-bit needs only about 35 GB in 4-bit (Chapter 14), which fits on a single 80 GB GPU. Quantization often turns a multi-GPU problem into a one-GPU problem, and engines like vLLM (Chapter 22) support both.

Part 3 — Hardware: why GPUs, and why new chips?

A natural question, and one that comes up as soon as you see how repetitive an LLM is: a Transformer is the same few operations over and over (matrix multiplications, attention, normalization, repeated across dozens of identical blocks). **Why run it on a general-purpose GPU? Why not build a dedicated "Transformer chip"?**

What a GPU is, and why it fits

A GPU is a highly **parallel, programmable** machine: thousands of small compute units that can perform the same operation on huge amounts of data at once. That matches matrix multiplication perfectly. It is also *programmable*, so the same chip runs a new attention variant, a new activation function or a new quantization format tomorrow, just by running new software.

The spectrum of chips

```
CPU                     general purpose, few powerful cores, flexible, slowest at this
 ↓
GPU                     programmable, massively parallel
 ↓
AI accelerator / TPU    built around matrix multiplication, less general
 ↓
LLM-specialized chip    designed around LLM workloads in particular
```

TPUs (Google's Tensor Processing Units) are purpose-built AI accelerators, designed around large matrix units rather than as a general graphics-derived design. And specialization is accelerating: the newest generation of chips is explicitly aimed at **LLM inference** (see below).

Why not hard-wire today's Transformer into silicon?

Because **Transformers keep changing**. A chip takes years to design and build. Today's model might use attention, an FFN and RMSNorm. A few years on, the same field has produced:

```
MoE layers  +  different attention (MLA, sparse, linear hybrids)  +  new position encodings
+  new quantization formats  +  reasoning-specific workloads
```

(Chapters 16, 17, 18 and 21 are a record of exactly that churn.) A chip hard-wired for last year's recipe could become an expensive paperweight. So the trade-off is fundamental:

```
GPU    →  programmability, at the cost of some efficiency
ASIC   →  efficiency, at the cost of flexibility
```

The bigger issue: memory, not arithmetic

Here is the most important point of this section. You've seen it in Chapter 12 (decode is memory-bound), Chapter 14 (fewer bytes per weight means faster generation), and Chapter 17 (FlashAttention reduces memory traffic). For LLM inference, the question is often not *"can I multiply these matrices quickly?"* but:

> **"Can I move all these weights and KV-cache values to the compute units fast enough?"**

That is sometimes called the **memory wall**. It is why accelerators are now designed around **compute, memory and interconnect together**, rather than raw arithmetic alone. For scale, an H100 has about 80 GB of main memory (HBM) at about 3.35 TB/s, and the tiny on-chip SRAM beside the compute units is much faster still. Whatever can be kept close to the compute is cheap to read, and that is the principle behind both FlashAttention (Chapter 17) and the new chips.

What the industry is doing (as announced in 2026)

These are recent announcements, so treat the details as a snapshot:

- **Google's eighth-generation TPUs**, announced in April 2026, were split into two chips: **TPU 8t** for training and **TPU 8i** for inference. Google describes the 8i as designed for low-latency inference, with **288 GB of high-bandwidth memory and 384 MB of on-chip SRAM** (three times the previous generation), so that large KV caches can sit close to the compute.
- **OpenAI and Broadcom** unveiled **Jalapeño** in June 2026, described as OpenAI's first chip designed from the ground up for LLM inference, built around the memory movement, kernels and serving patterns that matter for models like the ones it serves. Initial deployment was announced for late 2026.

Both are examples of the same idea: **an inference chip designed around the real bottlenecks of serving LLMs**, which are memory capacity and bandwidth, the KV cache, and the speed of communication between chips, rather than just peak arithmetic. The same logic explains why **reasoning models** (Chapter 20), which generate very long outputs and so keep large KV caches alive for longer, put even more pressure on memory.

A caution about the specifics: product details, names and availability change quickly, and the numbers above come from the companies' own announcements. The *principle* (programmability vs efficiency, and the memory wall) is the lasting part.

Common misconceptions, cleared up early

Misconception	Reality
"Speculative decoding changes the model's answers."	The accept/reject rule makes the final distribution exactly the large model's. The draft only affects speed.
"The small model's guesses are trusted."	Every drafted token is checked by the large model. Wrong ones are rejected and replaced.
"It needs retraining or a modified model."	The standard form needs neither: any suitably small model with the same tokenizer can draft.
"Verifying 5 tokens costs 5× as much as generating 1."	Decode is memory-bound: the weights are read once, so verifying several tokens in one pass costs about the same as one.
"Speculative decoding always gives the same speed-up."	It depends on the acceptance rate (predictable text gains more) and on how busy the GPU already is.
"More GPUs always means faster."	Splitting adds communication. Tensor parallelism needs very fast links, and pipeline parallelism has bubbles.
"Tensor parallelism and pipeline parallelism are the same thing."	Tensor splits the matrices *inside* each layer. Pipeline gives each GPU a group of whole layers.
"Pipeline parallelism makes one token faster."	It helps memory and throughput, but a token still passes through every stage in turn.
"Data parallelism lets a bigger model fit."	Every GPU holds a full copy, so the model must already fit. It adds throughput (inference) or training speed.
"You need multiple GPUs for any big model."	Often quantization (Chapter 14) removes the need: 70B in 4-bit is about 35 GB.
"A GPU is used because it's the best possible Transformer chip."	It's used because it is fast *and programmable* while models keep changing. Specialized chips trade flexibility for efficiency.
"LLM speed is all about arithmetic."	Decode is largely limited by memory bandwidth, which is why chips are being designed around memory and interconnect.

Quick reference

```
Speculative decoding   draft model guesses K tokens; large model verifies all in ONE pass
                       (causal mask → predictions at every position at once)
   accept drafted token with prob min(1, p/q); on rejection sample from normalized max(0, p−q)
   → output distribution is EXACTLY the large model's
   expected tokens per pass = (1 − α^(K+1)) / (1 − α);  speed-up ≈ that / (1 + K·c)
   variants: smaller model, extra heads (Medusa/EAGLE), multi-token prediction, n-gram lookup

Tensor parallel    split matrices: columns (concatenate) then rows (all-reduce add)
                   FFN: W1 by columns, W2 by rows;  attention: by heads, W_O by rows
                   2 all-reduces per block per token → needs NVLink, used within a server;
                   cuts per-token latency (each GPU reads a share of the weights)
Pipeline parallel  groups of layers per GPU; light communication; micro-batches fill the
                   bubble (busy ≈ m / (m + p − 1)); helps memory/throughput, not one token's latency
Data parallel      full copy per GPU; training: average gradients; inference: more replicas
Expert parallel    experts spread across GPUs; tokens routed to them (Chapter 18)
Context parallel   split a very long sequence across GPUs
Combine:  tensor inside a server + pipeline across servers + data replicas (+ experts)
Try quantization first: 70B → ~35 GB in 4-bit.

Hardware   GPU: parallel + programmable.  ASIC/TPU: efficient, less flexible.
           LLM inference is memory-bound → "memory wall" → chips designed around memory + interconnect
           (TPU 8i: 288 GB HBM + 384 MB SRAM; OpenAI/Broadcom Jalapeño; announced 2026)
```

What's next

We have now covered the model itself and how it is served and scaled: built, trained, adapted, compressed, redesigned, packaged, served, sped up and spread across GPUs. What remains is the layer on top. Chapter 24 steps back from the model to the **systems built around it**: routing between models, calling tools, memory, retrieval and agents, and keeping all of it observable and affordable. After that, Appendix A returns to the very small to make everything concrete, following one complete forward and backward pass through a tiny GPT with real numbers.

Chapter 24 — Production AI: Tool Calling, MCP, RAG, Agents and Reliability

Introduction

For twenty-three chapters we have studied the model itself: how it reads tokens, how it is trained, adapted, compressed, redesigned and served. But a model on its own is not a product. Nobody uses a bare next-token predictor. They use a chat assistant that remembers the conversation, looks things up, runs code, and calls other services. All of that is built *around* the model, and it is mostly ordinary software engineering.

The idea to keep in mind, which Chapter 1 introduced and every chapter since has reinforced, is this:

> **The model only ever reads tokens and writes tokens.** Everything else a "smart" system does is software that decides *what tokens to put in*, and *what to do with the tokens that come out*.

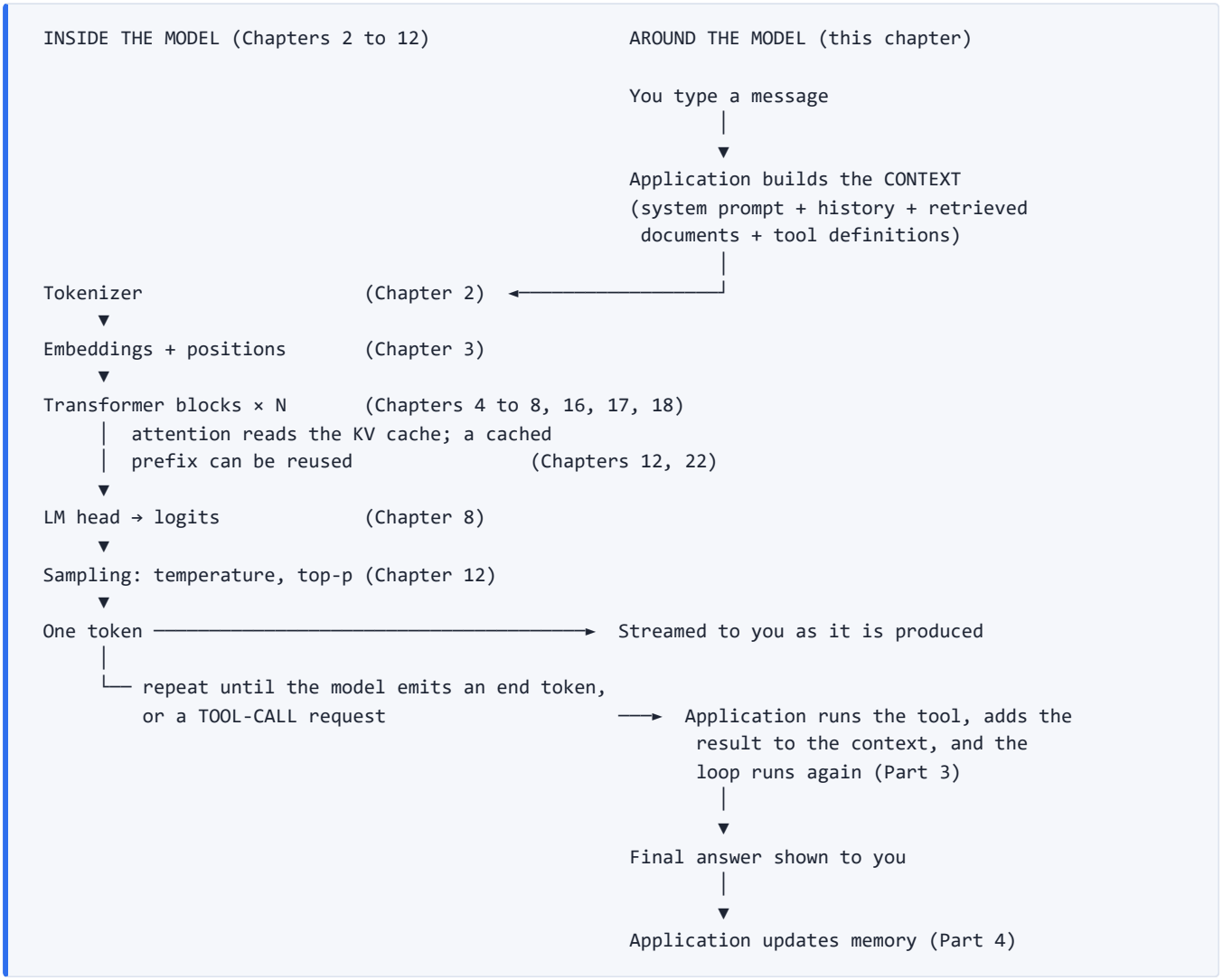
Memory, tools, retrieval and agents sound like new capabilities of the model. They are not. They are ways of **filling the context window** before the model runs, and **acting on its output** afterwards. The model's loop never changes.

This chapter is deliberately a tour rather than a manual. It covers, in order:

1. **One message, end to end:** the whole stack in one picture.
2. **The context is the interface:** what goes into the prompt, and why it is costly.
3. **Tool calling:** how a text predictor "uses" a calculator, a search engine or a database.
4. **Memory:** working memory versus persistent memory.
5. **Retrieval-augmented generation (RAG):** giving the model knowledge it was never trained on.
6. **Agents:** a loop of planning, acting and observing.
7. **Routing and cost:** spending the right amount on each request.
8. **Reliability:** logging, evaluation and guardrails.

Part 1 — One message, end to end

Let's trace a single user message through the entire system. Everything in the left column is something you have already learned. Everything in the right column is what this chapter adds.



Three things in this picture are worth underlining.

- **The application builds the context before the model ever runs.** What the model "knows" in a given moment is *exactly* what the application chose to include.
- **The model's output is either an answer or a request.** A model can end its turn with text for you, or with a structured request to use a tool. The application decides what happens next.
- **Everything is a loop.** Generation is a loop over tokens (Chapter 1). A tool-using system is a loop over *requests to the model*. An agent (Part 6) is a loop over *both*.

Part 2 — The context is the interface

The model has no memory of its own between requests (Chapter 12): every request starts from whatever text is sent to it. So the context window is the *only* way anything gets to the model. A typical request assembles:

Piece	What it is	Why it is there
System prompt	Standing instructions: role, tone, rules	Shapes behaviour for the whole conversation
Conversation history	Earlier user and assistant turns	The model can't recall them otherwise
Retrieved knowledge	Passages looked up for this question (Part 5)	Facts the model was never trained on
Tool definitions	Names and schemas of the tools it may call (Part 3)	So it knows what actions exist
Tool results	What the tools returned	Fresh information from the outside world
Reasoning tokens	The model's own thinking (Chapter 20)	Its scratchpad

Three consequences follow, each of which connects back to something you already know.

1. The context window is a budget. Everything above has to fit (Chapter 12), and every token costs: compute during prefill, memory in the KV cache, and money. Adding "just in case" content is not free.

2. Position matters. Models do not use all positions equally well. Research on long contexts found that models tend to use information at the **beginning and end** of the context more reliably than information buried in the **middle**. So put the most important material where it will be used well, and don't assume a bigger window means everything inside is equally "seen".

3. A stable prefix is cheap. Recall prefix caching (Chapters 12 and 22): if the *start* of a prompt is identical to one already processed, its KV cache can be reused. So order the context from **most stable to most changeable**:

```
[ system prompt ][ tool definitions ][ long reference document ][ history ][ new question ]
  ◄──────────── identical every request: cache hits ────────────►  ◄── changes ──►
```

If you put something that changes (like a timestamp) at the very start, you invalidate the cache for everything after it. Many providers also invalidate the cache when you change settings such as the reasoning level (Chapter 20).

Part 3 — Tool calling: how a text predictor uses a calculator

A language model can't browse the web, query a database or run code. It only produces text. So how do assistants do these things? With a simple trick:

> **The model writes a structured *request* for a tool. The application runs the tool and tells the model the result.**

The loop

```
1. The application sends the model:  the conversation  +  a list of available tools
2. The model replies, not with an answer, but with a TOOL REQUEST:
        { "tool": "get_weather", "arguments": { "city": "Mumbai" } }
3. The application ──► checks the request and RUNS the tool itself
4. The application adds the tool's result to the context:
        { "tool_result": "31°C, humid" }
5. The model reads the result and continues, possibly calling another tool,
   or finally writing the answer:  "It's 31°C and humid in Mumbai."
```

The model **never runs anything**. It produces a piece of text shaped like a function call, and the application, which is ordinary code, does the real work. That has an important consequence: the application is the one that can check, limit, log and refuse.

What the model is given

Each tool is described to the model, usually with a name, a plain-language description, and a schema for its arguments:

```
Tool:        get_weather
Description: Get the current weather for a city.
Arguments:   { "city": string }          (required)
```

These definitions are just text that goes into the context (Part 2). The model decides *whether* a tool is needed and *which* one, from the descriptions and the conversation. So **clear descriptions matter a lot**: they are the model's only manual.

How models learn to do this

Tool use is a skill acquired in fine-tuning (Chapter 13): models are trained on many examples of conversations that include well-formed tool requests and the results. At run time, systems often also constrain the output so that the request is **valid structured text** (valid JSON that matches the schema), by restricting which tokens the sampler may choose at each step (a use of Chapter 12's sampling).

Practical points

- **Parallel calls.** A model can request several independent tools at once. The application runs them together and returns all the results.
- **Errors are just results.** If a tool fails, the error message goes back as a result and the model can try something else.
- **Trust nothing blindly.** The arguments come from a model, and the *content* the model reads may come from untrusted places (a web page, a document, an email). Text inside a tool result can contain instructions aimed at the model. This is called **prompt injection**, and it is the main security risk of tool-using systems. So: give tools the least privilege they need, validate arguments, require human confirmation for actions with side effects (sending, paying, deleting), and treat all tool output as *data*, not as commands.

MCP: a standard plug

Every application used to wire up its tools its own way. The **Model Context Protocol (MCP)** is an open standard for connecting AI applications to external systems, and it is often described as a USB-C port for AI: one standard connector instead of a custom cable for every device. A tool provider writes an **MCP server** that exposes its tools (and also data "resources" and reusable prompts), and any MCP-aware application can connect to it. That turns an "N applications × M integrations" problem into "N + M".

```
            ┌── MCP server: files ──┐
Application ├── MCP server: database ├──  standard protocol
 (the host) └── MCP server: search ──┘
```

Nothing about MCP changes how the *model* works. It standardizes how the *application* finds and calls tools.

Part 4 — Memory: three different things that get confused

When people say an assistant "remembers", they may mean three very different things, and keeping them apart clears up a lot:

```
PERSISTENT MEMORY    the conversation history, notes, facts and preferences, stored by the
(a database, files)  APPLICATION. Permanent until deleted. The model never sees it directly.
        │
        ▼   for each request, the application CHOOSES what to include
CONTEXT WINDOW       the tokens the model sees right now. Limited in size. Assembled fresh per request.
        │
        ▼   the model processes that context
KV CACHE             the Keys and Values for those tokens (Chapter 12). Temporary working
(GPU memory)         memory for this run. Not your chat history.
```

The KV cache is working memory, the context window is what the model is looking at, and persistent memory is what the *application* knows. Only the middle layer reaches the model.

What to do when history outgrows the window

A long conversation will not fit forever. The usual options, each with a trade-off:

Approach	How it works	Trade-off
Truncate	Drop the oldest turns	Simple, but forgets early details
Summarize	Replace old turns with a short summary	Saves space; the summary can lose detail
Retrieve	Store everything; pull in only the relevant pieces for this question (Part 5)	Scales well; depends on retrieval quality
Structured memory	Extract durable facts ("prefers metric units", "works on project X") and store them as notes	Compact and precise; needs care about what to save
Start fresh	New context, carrying forward only essentials	Cleanest, loses continuity

Real systems combine them: recent turns verbatim, older turns summarized, durable facts stored, and relevant older material retrieved on demand.

Memory brings its own risks. Stored facts can be **stale** or wrong, they can contain **private** information that must be handled carefully, and they can be **poisoned**: if the system saves something from untrusted content, an attacker's text may come back in future conversations.

Part 5 — RAG: giving the model knowledge it was never trained on

A model's knowledge is frozen at training time, it knows nothing about your private documents, and it sometimes produces confident but false statements (hallucination). **Retrieval-augmented generation (RAG)** addresses this by *looking things up first* and putting what it finds in the context:

```
Question ⟶ find the most relevant passages in a knowledge base ⟶ put them in the prompt ⟶ model answers from them
```

RAG is the practical application of Part 2: instead of hoping the model remembers a fact, you put the fact in front of it. The original RAG work combined a generative model with a searchable index of documents, so that knowledge could be updated by changing the index, not retraining the model.

How the search works: embeddings and similarity

How do we find the "relevant" passages? With embeddings (Chapter 3). A separate **embedding model**, usually an *encoder-only* model of the kind in Chapter 15, turns every passage into a vector, so that passages with similar meaning land close together. The question is turned into a vector the same way, and we find the nearest neighbours. "Closeness" is usually measured by **cosine similarity**, the angle between vectors. A tiny example with 2-dimensional vectors:

```
Question: "How do I get my money back?"      q = [0.9, 0.1]

Passage A  "Refund policy"            [0.8, 0.2]    similarity with q = 0.99   ← closest
Passage C  "Returns and refunds"      [0.7, 0.3]    similarity with q = 0.96   ← close
Passage B  "Shipping times"           [0.2, 0.9]    similarity with q = 0.32
Passage D  "Office opening hours"     [0.1, 0.95]   similarity with q = 0.21

Take the top 2 (A and C) and put them in the prompt.
```

Notice that the question shares *no words* with "Refund policy", yet it still matches. That is the advantage of meaning-based search over keyword search.

The pipeline

```
INGEST (done ahead of time)
   Documents ⟶ split into chunks ⟶ embed each chunk ⟶ store vectors in an index

QUERY (done for each question)
   Question ⟶ embed ⟶ find nearest chunks ⟶ (re-rank) ⟶ insert into the prompt ⟶ model answers,
                                                                                  ideally citing sources
```

The choices that decide quality

- **Chunking.** Too big, and a chunk contains a lot of irrelevant text and wastes the context. Too small, and a chunk loses the context needed to make sense. Chunks of a few hundred tokens, often with some overlap, are common.

- **How many to retrieve (top-k).** Too few misses the answer. Too many fills the window with noise (and the "lost in the middle" effect of Part 2 makes extra text actively harmful).
- **Hybrid search.** Pure meaning-based search can miss exact terms such as product codes or names. Many systems combine it with keyword search.
- **Re-ranking.** Retrieve a generous set of candidates cheaply, then use a stronger model to re-order them and keep the best few.
- **Instructing the model.** Tell it to answer from the supplied passages and to say so when they don't contain the answer.

RAG, fine-tuning, or long context?

	Best at	Limits
RAG	Facts that change or are private; citing sources; large knowledge bases	Quality depends on retrieval; adds latency and moving parts
Fine-tuning (Chapter 13)	Changing style, format and behaviour	Poor for injecting many facts; needs retraining to update
Long context	Small documents you can just paste in	Costly (Part 2); a bigger window doesn't mean everything is used well

They are complements: fine-tune for behaviour, retrieve for facts.

How RAG fails

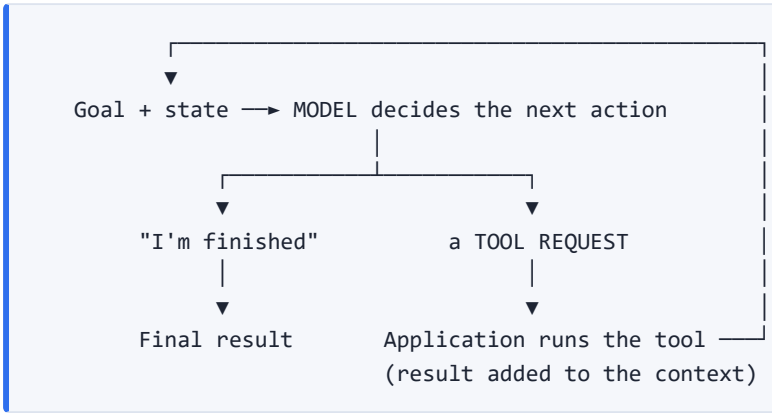
Mostly in the *retrieval*: the right passage wasn't found, irrelevant passages were found, or the answer is split across chunks. Then the generation step: the model ignores the passages, or blends them with its own memory. The documents themselves can also contain **prompt injection** (Part 3). When debugging, always check first *what was retrieved*. Often the model did exactly what it was asked, with the wrong evidence.

Part 6 — Agents: a loop of planning, acting and observing

Chapter 20 ended with a distinction, which we can now complete:

```
LLM               Prompt → LLM → Answer
Reasoning model   Prompt → LLM → Reasoning → Answer
Tool-using LLM    the model can request tools; the application runs them
Agent             Goal → Plan → Act → Observe → Re-plan → … → Result
```

An **agent** is a system that pursues a goal over **many steps**, deciding at each step what to do next, using tools, and looking at the results. The core is a loop:



A widely used pattern, **ReAct**, has the model interleave short reasoning with actions: *think about what is needed, act, observe the result, think again*. A tiny trace:

```
Goal: "Find the cheapest of these three laptops that has 16 GB of RAM."
 Thought: I need the specs and prices for each.     Action: search("laptop A specs")
 Observation: 8 GB RAM, $720.
 Thought: A has only 8 GB, so rule it out.          Action: search("laptop B specs")
 Observation: 16 GB RAM, $850.
 Thought: B qualifies. Check C.                     Action: search("laptop C specs")
 Observation: 16 GB RAM, $790.
 Thought: C is cheaper than B. Answer: C.           → Final answer
```

An agent has four parts: **a model** (decides), **tools** (act), **memory** (carries state, Part 4), and **a controller** (the loop that enforces limits and decides when to stop). As Chapter 20 stressed, an agent doesn't need a reasoning model, and a reasoning model doesn't need to be an agent. They complement each other.

Why agents are hard to make reliable

- **Errors compound.** If each step is right 95% of the time, ten steps in a row are right only about 60% of the time ($0.95^{10} \approx 0.60$). At 99% per step it is about 90%. At 90% per step, only about 35%.
- **Loops and drift.** Agents can repeat the same failing action, or wander away from the goal.
- **Cost grows fast.** Every step resends the growing context (Part 7).
- **Actions have side effects.** Unlike text, a deleted file or a sent email can't be un-sent.

The standard defences are the controller's job: a **maximum number of steps**, a **budget**, a check after each step that the goal is still on track, **human approval** before irreversible actions, narrow tools rather than powerful general ones, and detailed logs. Where a task can be done as a fixed sequence of steps, a plain workflow is often more reliable than an agent that decides everything itself. Use an agent when the path really isn't known in advance.

Larger systems sometimes use several agents, each with a narrow role, coordinated by another. The same principles apply, with an extra cost: more model calls and more places for errors to creep in.

Part 7 — Routing and cost: spend the right amount on each request

The earlier chapters gave you several dials that trade quality against cost, and production is about setting them per request:

Dial	Chapter	Effect
Which model (size, quantized or not)	14, 18	Cheaper and faster vs more capable
Reasoning effort	20	More thinking tokens vs latency and cost
Context size	12	Fewer input tokens, lower cost
Prompt caching (a stable prefix)	12, 22	Cheaper repeated input
Output limits (max tokens, step limits)	12, 20	A hard cap on runaway spend
Streaming	12	Doesn't cut cost, but the user sees text sooner

Routing means choosing among them per request:

```
Easy request    →  small (maybe quantized) model   →  low reasoning effort
Medium request  →  mid-size model                  →  medium effort
Hard request    →  strongest model                 →  high effort
```

A router can be rules (by task type, length or user tier), a small classifier, or even a small model that judges difficulty. (A small decision model of the kind described in Chapter 25 is a natural router: it reads the request and returns a calibrated probability for each route in milliseconds.) Add **fallbacks** too, so that if one provider fails or is slow, the request goes to another.

What a request costs

A rough model is:

```
cost  ≈  (input tokens × input price)  +  (output tokens × output price)
         with reasoning tokens counted as OUTPUT (Chapter 20), and cached input discounted
```

An illustrative example (made-up prices of $2 per million input tokens and $10 per million output tokens):

```
Input:   8,000 tokens × $2 / 1M           = $0.016
Output:  1,500 tokens × $10 / 1M          = $0.015      (e.g. 1,000 of them reasoning)
                                  Total  ≈ $0.031 per call

With prompt caching, if 5,000 of the 8,000 input tokens are a cached prefix billed at a tenth of the price:
Input:   3,000 × $2/1M  +  5,000 × $0.20/1M = $0.007     (about 56% cheaper than $0.016)
```

Why agent loops get expensive quickly

An agent calls the model many times, and each call **resends everything so far**. Suppose each step adds about 1,000 tokens to the context:

```
Call 1:  1,000 input tokens
Call 2:  2,000
Call 3:  3,000
 ...
Call 10: 10,000        Total input over 10 calls:  1k + 2k + … + 10k = 55,000 tokens
                       (not 10,000)
```

The total grows with the *square* of the number of steps. Prompt caching helps a great deal here, since each call's context starts with the previous call's, which is exactly a stable prefix. Context trimming and summarization (Part 4) help too. This is why step and budget limits (Part 6) are not optional.

Part 8 — Reliability: observing, testing and guarding

A system built around a probabilistic model needs more engineering around it than ordinary software, not less. Four practices cover most of it.

Observability. Record, for every request: the full assembled prompt, the tool calls and results, the retrieved passages, token counts (including reasoning tokens), latency (and time to the first token), and errors. When an answer is wrong, you can only find out *why* if you can see what the model actually saw. Tracing a whole multi-step run as one unit is especially valuable for agents.

Evaluation. Because outputs vary, you test on **sets of examples**, not single cases. Keep a collection of representative and tricky questions with expected behaviour, and re-run it whenever you change the model, the prompt, the retrieval, the tools, or the reasoning setting. Scoring can be exact (checking an answer, as with the verifiers of Chapter 20), or done by another model acting as a judge. Judge models are convenient, but they have biases and blind spots of their own, so spot-check them against humans. For RAG, evaluate retrieval (did the right passage come back?) separately from generation (did the answer follow the passages?).

Guardrails. Constraints that hold regardless of what the model says: input and output checks (for unsafe content or leaked sensitive data), least-privilege tools, confirmation before irreversible actions, limits on steps, spend and time, and treating everything the model reads as untrusted. These sit in the application, outside the model, so they can't be talked out of by a clever prompt.

Resilience. Timeouts, retries with back-off, fallbacks to another model or provider, and graceful degradation (for example, answering without retrieval if the search service is down). Version your prompts and model choices so that you can roll back, because a "small" prompt change can alter behaviour in ways you only notice later.

```
A healthy loop:   build → run → LOG → EVALUATE on your example set → change one thing → repeat
```

Common misconceptions, cleared up early

Misconception	Reality
"The model uses tools itself."	It only writes a structured request. The application validates it, runs the tool, and returns the result.
"The model remembers past conversations."	It has no memory between requests. The application stores history and re-sends what it chooses.
"KV cache, context window and memory are the same."	KV cache = temporary GPU working memory; context window = what the model sees now; persistent memory = the application's storage.
"A bigger context window means the model uses everything in it well."	Information in the middle of a long context tends to be used less reliably, and every token costs.
"RAG gives the model new knowledge permanently."	It adds passages to *this request's* context. Nothing in the model's weights changes.
"RAG is a replacement for fine-tuning."	RAG is for facts; fine-tuning is for behaviour and style. They complement each other.
"If the answer is wrong, the model hallucinated."	In RAG systems the cause is often retrieval: the wrong or missing passage. Check what was retrieved.
"MCP makes the model smarter."	It standardizes how applications connect to tools and data. The model is unchanged.
"An agent is a smarter model."	An agent is a loop of model calls, tools and state around a model.
"An agent loop costs N times one call."	Each call resends the growing context, so total input grows roughly with the square of the number of steps.
"Guardrails can live in the prompt."	A prompt can be overridden or ignored. Hard limits belong in the application.
"Text returned by a tool is safe to follow."	It may contain instructions planted by an attacker (prompt injection). Treat it as data.

Quick reference

```
Core idea:   the model only reads and writes tokens. Production = software that
             (1) decides what goes into the context, and (2) acts on what comes out.

Context:     system prompt + history + retrieved passages + tool definitions + tool results
             budget: tokens cost compute, KV cache and money; ends and beginnings are used best
             order stable → changeable for prefix-cache hits

Tool calling: model emits a structured request → APPLICATION validates and runs it →
              result added to context → model continues.  MCP = open standard plug for tools/data.
              Risks: prompt injection → least privilege, validate, confirm side effects.

Memory:      persistent (app database) → chosen into context window → KV cache (temporary).
             Overflow: truncate | summarize | retrieve | structured notes | fresh start.

RAG:         ingest: chunk → embed → index.   query: embed → nearest chunks (cosine) → (re-rank) → prompt.
             levers: chunk size, top-k, hybrid search, re-ranking.   Fails mostly in retrieval.
             RAG = facts;  fine-tuning = behaviour;  long context = small documents.

Agents:      goal → model decides → act (tool) → observe → repeat, with a CONTROLLER (max steps, budget).
             errors compound (0.95^10 ≈ 0.60);  cost grows ~ steps²;  prefer a fixed workflow if the path is known.

Routing:     choose model, reasoning effort, context size, caching, limits per request; add fallbacks.
             cost ≈ input×price_in + output×price_out (reasoning counts as output) − cache discount.

Reliability: log everything; evaluate on example sets (judge models have biases);
             guardrails in the application, not the prompt; timeouts, retries, fallbacks, versioning.
```

What's next

You now have the whole path from a sentence to a system, from tokenization through the Transformer, training, serving and reasoning, up to the application built around the model. **Chapters 25 to 27** look at the other side of that application: a new family of fast models that do not write text at all, but make a typed, calibrated *decision* from a fixed list of answers, in one pass. Chapter 25 introduces them, and Chapters 26 and 27 take two real designs apart and show how such a model is trained, calibrated and tested. After that, the **Appendix** makes everything concrete one last time. It follows a complete forward pass and a complete backward pass through a tiny GPT using small real numbers, so you can see every matrix multiplication, softmax and gradient that the chapters described in words.

Chapter 25 — System 1 Decision Models: Fast, Typed, Calibrated Decisions Without Generating Text

Introduction

Every model in this handbook so far *writes*. Even when we asked a question with a one-word answer ("is this email spam?"), the model produced tokens one at a time, in the loop of Chapter 12 (inference, the KV cache and sampling), and we read the answer out of the text it wrote.

A lot of real software does not need writing. It needs a **pick**: which team gets this ticket, which tool should run next, is this command safe, does this customer want a refund. A pick has a fixed list of possible answers. It often has to arrive in milliseconds. And the program that receives it needs to know **how sure** the model is, as a number, not as a sentence.

This chapter is about a new family of models built for that job: **decision models**, also called **System 1 models**.

The name is borrowed from psychology. Daniel Kahneman describes two modes of human thinking: fast, automatic judgement (System 1) and slow, deliberate reasoning (System 2). Chapter 20's reasoning models are the System 2 side: they spend thousands of tokens thinking before they answer. A decision model is the System 1 side: it reads, decides in a single pass, and stops.

> **A decision model is usually not a new kind of Transformer.** It is a Transformer (often a BERT-style encoder, sometimes a decoder LLM) with a different *input layout*, a different *readout* and a different *training objective*. The novelty is in the job, and in the discipline of making the output typed and honest.

This chapter is the concept and the landscape: what a decision is, the three kinds of question, the vocabulary, where decision models fit in a system, and the models that exist today (Jev, Laya, Vega and others). It stays at the level of ideas. Chapter 26 builds one, starting from BERT, and Chapter 27 covers its data, training, calibration and evaluation.

The parts of this chapter:

1. **The problem:** why a text generator is an awkward tool for choosing.
2. **What a decision is:** state, question, allowed answers, result.
3. **The three kinds of question:** choice, score and noul (yes/no), and why there are three.
4. **Vocabulary:** the terms you will meet.
5. **Decision versus generation:** where decision models fit, and why not a plain classifier or embeddings.
6. **The landscape (October 2026):** Jev, Laya, Vega and the others.
7. **What is genuinely new here:** and what is an old idea in a new package.

> **How current this is.** The first widely noticed decision-model product, TypeSafe AI's Jev, was announced in September 2026, and most of the facts in this chapter are weeks old. Where a statement comes from a vendor, or from a model's own authors, the text says so. Treat names, sizes, prices and

> benchmark numbers as a snapshot. The ideas are the lasting part.

Part 1 — The problem: why use a text generator to make a choice?

Take a support inbox. A message arrives:

> *"Hi, we were billed twice for March. Please refund the duplicate today or we will cancel our plan."*

Software needs three things from it: which team should handle it (billing, technical, other), how urgent it is, and whether the customer is threatening to leave.

The language-model way

```
prompt: "Classify this message... Answer with one word: billing, technical or other."
   ↓
the model writes:   "bill"   "ing"   "."          ← one forward pass per token (Chapter 12)
   ↓
the program parses the text and hopes it is one of the allowed words
```

This works, and for many applications it is fine. But look at what it costs when you do it thousands of times a minute inside a larger system:

1. **A loop for a one-word answer.** Generating text means one forward pass per token. The answer "billing" may be two or three tokens, plus whatever the model chooses to add around it.
2. **The answer is free text.** The program must parse it. The model may say "Billing team", "billing." or a sentence of explanation. You can force a format (structured output modes exist), but you are still asking a writer to behave like a switch.
3. **The confidence is words, not a number.** "I'm fairly sure it's billing" can't be put in an `if` statement. Token probabilities exist inside the model, but they are not calibrated (calibration, defined in Part 4 and explained in Chapter 27), and they are probabilities over the next *token*, not over *your list of allowed answers*.
4. **The expensive tokens are the output tokens** (Chapter 24, routing and cost). For a decision, the output is the one part you did not need to pay for.

What we actually want

```
state (the message) + a question + a list of allowed answers
                          │
                          ▼
                ONE forward pass, no writing
                          │
                          ▼
        a probability for every allowed answer
```

One pass. A fixed answer list. A number for every option. That is a decision model.

"Couldn't a small language model do this?"

Partly, and the idea is worth seeing because it shows what the new models are doing. A decoder LLM can be used this way directly: put the options in the prompt, run **one** forward pass, and read the probabilities of the option labels at the next-token position instead of letting the model write. One independent test of decision models used exactly such a baseline (a 1.5B-parameter Qwen model with "parallel constrained decoding": score each allowed answer in parallel, never generate freely). The dedicated decision models go further, with a purpose-built readout and training, and they report probabilities that are meant to be *honest* (see Calibration in Part 4, and Chapter 27, Part 4).

Two names for the same idea

Kahneman's *System 1 / System 2* split was adopted by TypeSafe AI, whose model Jev is called a "System One model". Some vendors avoid the term and simply say **decision model**. This chapter uses both.

Part 2 — What a decision is

A **decision** has four parts:

Part	Meaning	Example
State	the text (or data) to judge	"Hi, we were billed twice for March. Please refund the duplicate today or we will cancel our plan."
Question	what we are asking about it	"Which department should handle this?"
Allowed answers	a closed list, written by *you*, at request time	billing, technical, other
Result	a probability for each allowed answer	billing 0.91, technical 0.05, other 0.04 (illustrative)

That is all a decision model does. The state goes in, the question and the answer list go in, and probabilities come out. No text is generated.

Writing a decision down: three fields

The open decision models in this chapter (Laya and Vega) share a request shape. A decision is written as a **question** with three fields:

Field	What it is	Needed for
`type`	which kind of question: `choice`, `score` or `noul` (Part 3)	every question. It tells the system how to read and judge the answer.
`instructions`	the question itself, in plain English	every question
`criteria`	the **allowed answers**, each with a short description	`choice` (a label plus a description for each), `score` (the ordered levels). For `noul` it is optional: the defaults are "no, the statement does not hold" and "yes, the statement holds".

Each question also has a name of your choosing (such as `department`). The field names above are the ones Laya and Vega use. TypeSafe's Jev offers the same three kinds of question, but its exact field names are not covered here.

Here is the ticket written down, with each line explained:

```
state = "Hi, we were billed twice for March. Please refund the duplicate today or we will cancel our plan."

questions = {
  "department": {                                # the name of this question
      "type": "choice",                          # the kind: pick one
      "instructions": "Which department should handle this?",       # the question
      "criteria": {"billing":   "invoices, payments, refunds",      # answer: description
                   "technical": "bugs, outages, system errors",
                   "other":     "everything else"}},
  "urgency": {
      "type": "score",                           # the kind: an ordered scale
      "instructions": "How urgent is this?",
      "criteria": ["not urgent", "soon", "blocking"]},              # levels 0, 1, 2 in order
  "churn_risk": {
      "type": "noul",                            # the kind: yes/no
      "instructions": "Does the user threaten to cancel or leave?"},  # no criteria: yes/no defaults
}
```

One request carries **several** questions about the **same** state.

> **The criteria are text the model reads.** That is the key design fact. The allowed answers are not baked into the model's weights the way the classes of an ordinary classifier are. They are part of the *input*, so the same model can answer a different list tomorrow without retraining. It also means that each answer must explain itself: "billing: invoices, payments, refunds" gives the model something to read, while a bare "A" gives it nothing.

A question learners ask: "Is the state like a prompt?"

Yes, with one refinement. In a language model, instructions and evidence share one prompt. Here the **state is the evidence**, kept apart from the question and the answer list, and then everything is laid into one input sequence. The "prefill" idea from Chapter 12 is a good mental model: a decision model is essentially **prefill only**. It reads the whole input in one pass (the prefill) and then stops. There is no decoding loop.

Part 3 — The three kinds of question

Decision models offer three kinds of question. TypeSafe calls them *primitives*. An independent paper on Jev describes the three as choosing among candidate options (Choice), making binary judgments (Noul) and rating an input against predefined levels (Score). Laya uses the names `choice`, `score` and `noul`. ("Noul" is just the name these systems use for a yes/no check.)

Kind	Plain meaning	Allowed answers	What comes back (illustrative)
choice	pick one from a list	billing / technical / other	0.91 / 0.05 / 0.04
score	place it on an ordered scale	not urgent (0) / soon (1) / blocking (2)	0.10 / 0.25 / 0.65, and an expected level of 0.10×0 + 0.25×1 + 0.65×2 = **1.55**
noul	is this statement true?	yes / no	P(yes) = 0.88

Why there are three kinds: errors are not all alike

Suppose the true urgency is level 2 ("blocking"). Compare five predictions. Two ways of measuring the error are shown: **cross-entropy** (Chapter 9), and the **ranked probability score** (RPS, lower is better), which compares *cumulative* probabilities and so cares about *how far* the probability is from the right level.

```
                                  cross-entropy    ranked probability score
A  [0.10, 0.25, 0.65]  right           0.43               0.13
B  [0.10, 0.65, 0.25]  off by one      1.39               0.57
C  [0.65, 0.25, 0.10]  off by two      2.30               1.23
D  [0.30, 0.60, 0.10]  near miss       2.30               0.90
E  [0.60, 0.30, 0.10]  far miss        2.30               1.17
```

Look at D and E. Both give the true level 0.10, so cross-entropy scores them identically at 2.30. But E puts its weight two levels away and D puts it one level away, and RPS tells them apart. Cross-entropy treats every wrong class as equally wrong, which is right for `choice`: "billing" is not closer to "technical" than to "other". It is wrong for `score`, where order matters. This is why `score` is a separate kind with its own measure of error.

A question learners ask: "Is a score one number?"

It depends on what you mean, and the answer is worth getting right.

A score question does **not** return a single probability somewhere between 0 and 2. Keep two scales apart: **probabilities** always run from 0 to 1, and **levels** (0, 1, 2) are the *answers*. A score returns:

- a **probability for each level**, which together sum to 1 (for example 0.10, 0.25, 0.65), and
- one derived **expected level** (1.55 in the example), which does sit in the 0-to-2 range.

The single number is calculated *from* the distribution. Laya's code computes it literally as the sum of level × probability.

A different design is possible: the model outputs one number and cut-offs decide the level (this is called ordinal regression). It is simpler, but it throws away the model's **uncertainty**. Consider two situations with the *same* expected level:

```
sure it's level 1:        [0.0, 1.0, 0.0]   expected level = 1.0
torn between 0 and 2:     [0.5, 0.0, 0.5]   expected level = 1.0
```

A single number cannot tell these apart, but they should lead to different actions: act on the first, escalate the second. The expected level can even land on a level the model considers **impossible**: in the second case it is 1.0, yet level 1 has probability zero. So when you need a final level:

Method	How	Watch out for
Most likely level	take the level with the highest probability	the safest default
Round the expected level	1.55 → 2	can pick a level the model gave zero probability
Cut-offs on the expected level	the "one number" design	the same trap, and the thresholds are yours to choose

For anything that triggers an action, look at the confidence or the distribution first, and treat the single number as a summary.

> **Choice and score have outputs of the same shape** (one probability per option). What differs is the *meaning* of the options: unordered for `choice`, ordered for `score`. The `noul` kind is a two-option question (false, true) reported as the probability of "true".

What a whole answer looks like

For the ticket above, a request returns something like this (illustrative values):

```
department   billing 0.91   technical 0.05   other 0.04          → pick: billing
urgency      level 0: 0.10  level 1: 0.25   level 2: 0.65        → expected level 1.55
churn_risk   P(yes) = 0.88
```

Three answers, one state, one pass.

Part 4 — Vocabulary

A short glossary, so that the rest of the chapter reads easily. Most of these words are explained properly where they first matter.

Term	Meaning
State	the text or data being judged (the evidence)
Question / instructions	what is being asked, in plain words
Criteria / options / allowed answers	the closed list of answers, each with a short description
Primitive (kind of question)	`choice`, `score` or `noul`
Typed output	an output whose form is fixed in advance (a label from a list, a level, a yes/no probability), so a program can use it without parsing text
Non-autoregressive	not generating token by token; the answer comes from one forward pass
System 1 / System 2	fast automatic judgement versus slow deliberate reasoning; decision models versus reasoning models
Logit ("dial")	the raw score the model produces for an option, before it becomes a percentage
Softmax	the function that turns the dials into percentages that sum to 1 (Chapter 8)
Temperature	a number that divides the dials before the softmax; a higher value gives softer, less sure percentages
Confidence	how sure the model says it is. Laya uses the name for two different quantities.
Calibration	the property that "80% sure" is right about 80% of the time
ECE	*expected calibration error*: the average gap between stated confidence and actual accuracy
Teacher	a stronger model (or people) whose answers are used as the training answer key
Gold / soft label	the training target. A *soft* label is a probability for every option instead of a single right answer.
Slot / endpoint	the position in the input whose output vector is read to score one option
Option budget	the limited number of tokens available to describe all the options
Threshold / coverage / abstention	answering automatically only above a confidence level; coverage is the fraction of questions answered; abstaining means handing the rest to someone else
Gating / escalation	using the probability to decide whether to act, ask a bigger model, or ask a person
Zero-shot vs fine-tuned	used as shipped, versus trained further on your own decisions
Held-out	data kept out of training so it can test the model fairly
Backbone / head	the main body of the model (for example a BERT-style encoder) versus the small layers added on top to produce the decision

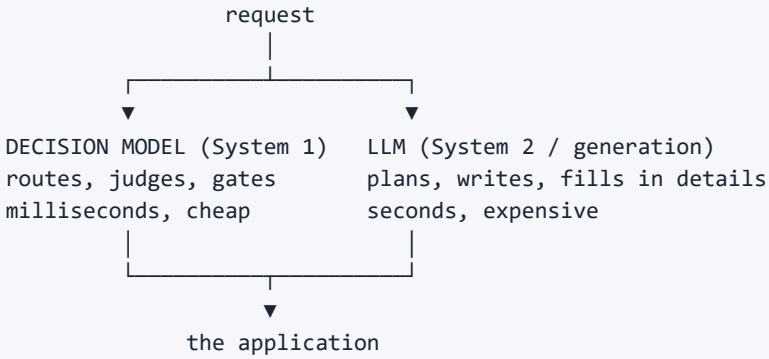
Part 5 — Decision versus generation: where decision models fit

The dividing line

If the right answer **can be listed in advance**, it is a decision. If it cannot, it is generation.

Decision (closed answers)	Generation (open answers)
Which tool should handle this?	Write the reply to the customer
Is this command safe to run?	Fill in the arguments for the tool
How severe is this finding?	Plan the next five steps
Does the user want a refund?	Summarise the thread

The pattern: a generator writes, a decision model routes and gates



A 2026 paper on penetration-testing agents ("Calibrated Decision Models for Autonomous Penetration-Testing Harnesses") formalises four decision points where a System 1 model replaces free-form LLM judgement in an agent: confirming a finding, grading severity, choosing which agent to run, and confirmation loops. Its authors are candid that their evidence is an exploratory case study that motivates the design and is not a controlled experiment.

What the probability is for

A probability is only useful if the system *does something* with it. Three common policies:

- **Act / ask / escalate.** Above 0.9, act automatically. Between 0.6 and 0.9, ask a bigger model. Below 0.6, ask a person.
- **Cost-based.** Suppose acting correctly is worth +1, acting wrongly costs 3, and escalating costs 0.5. Acting beats escalating when `p − 3(1 − p) > −0.5`, that is `4p − 3 > −0.5`, so **p > 0.625**. (These numbers are the ones Laya's author describes for its act/escalate head, used here as an illustration of the reasoning.)
- **Coverage target.** Answer only the most confident half of the questions, and send the rest elsewhere.

All three assume the probabilities are **honest**. That is the subject of calibration (Part 4, and Chapter 27, Part 4), and the policies above depend on it.

"Why not just a classifier?"

An ordinary classifier (for example a BERT encoder with a classification head, Chapter 15, Part 4 and Chapter 26, Part 1.6) has its labels fixed inside it: one output per class. Add a class and you retrain. A decision model reads the allowed answers as *part of the input*, so the list can change per request. That is the difference, and the whole reason for the extra machinery.

"Why not embeddings and a similarity comparison?"

This is the natural alternative, and it deserves a careful answer. Embedding models (Chapter 15, Part 5; Chapter 24, RAG) turn a text into a vector, and you pick the label whose vector is closest. It is cheap, quick to set up, and a good baseline. There are three reasons it is not the whole answer.

1. The text and the options never meet. An embedding comparison encodes each text separately and compares the results afterwards (a *bi-encoder*). Similarity measures closeness of *topic*, not "which option is true for this text". Take the state "Please do **not** cancel my subscription" with the options "cancel subscription" and "keep subscription". Both options are on-topic. Telling them apart needs the state and the option read **together**, which is what a decision model does (a *cross-encoder*: everything is in one sequence, so attention connects them). This is a typical weakness of similarity matching, not a measured claim about any particular model.

2. Not every question is "which label is closest". "How urgent, 0 to 2?" and "is this statement true?" are not similarity-shaped. You can train a small classifier on top of the embeddings, but then you are training a head anyway, and the options still cannot influence how the state is read.

3. Cosine similarity is not a probability. A similarity of 0.71 does not mean "71% likely". You would still need to calibrate it.

In practice the two combine. Search systems retrieve candidates with embeddings and then re-rank them with a model that reads the query and each candidate together. Laya does the same: for questions with more than about 20 options, it shortlists the labels by embedding similarity (using vectors from its own encoder) and then decides with a single joint pass.

"The tool list changes between systems. How does it adapt?"

Because the options are text, the same model can be given a different list. It adapts the way an LLM adapts to tool descriptions: by *reading them*. The limits are real:

- It sees only what is in the state and the option text. If it needs context, that text has to supply it.
- The shipped base models are weaker zero-shot than their authors' fine-tuned versions. Writing good descriptions is the free first step, and fine-tuning on your own decisions is the step that changes behaviour.

The framing checklist

Before touching a model:

1. **List the decisions the software makes.** Each needs a question and a closed list of answers.
2. **Decision or generation?** If the right answer cannot be listed in advance, it is generation.
3. **Pick the kind:** choice, score or noul.
4. **Put the evidence in the state.** The model cannot look anything up.

5. **Write each answer so it explains itself.**
6. **Decide what the probability will do:** act, ask or escalate.
7. **Count the options.** Laya's answer text shares a budget of 192 to 256 tokens (roughly 20 options), and Jev's documented limit is 255 options. Beyond that, shortlist or route in two steps.

Limits to keep in mind from the start

- **A closed list cannot be surprised.** If the right answer is not on it, the model will still pick one.
- **No invented text, but still sometimes wrong.** A typed output cannot hallucinate a sentence, but it can be confidently wrong. That is why calibration and evaluation are part of the design.
- **Everything it needs must be in the state.**

Part 6 — The landscape (October 2026)

This part describes the models that exist, using each maker's own materials, and flags which claims are the maker's. The field is only weeks old, so expect it to move.

6.1 Jev (TypeSafe AI): the closed one that started it

Who and when. TypeSafe AI announced Jev on **15 September 2026**. The company (founded in 2024) says it worked on the model in stealth for about two years. Its chief executive, Diogo Almeida, previously worked at OpenAI on RLHF and ChatGPT, according to Wikipedia and TechCrunch. The name is a nod to the economist William Stanley Jevons and the "Jevons paradox": when something gets cheaper, people use more of it.

What it does. State plus typed questions in, probabilities out: the three kinds of question from Part 3. TypeSafe describes Jev as turning unstructured state into typed, probabilistic decisions, and as giving up string generation entirely.

What is public and what is not.

Published	Not published
the API: input and output formats, how confidence numbers are computed	the weights
pricing and limits	the architecture and model size
evaluation write-ups by TypeSafe	a technical paper
	the training data (the launch post has a FAQ heading asking where the training data comes from, with no answer under it)

TypeSafe describes the model as transformer-based. Outside observers have speculated that it might be built on an open-weight language model or a BERT-style encoder, but that is speculation.

Claims (all TypeSafe's own).

- End-to-end response between 70 ms and 500 ms.
- On its own workflow evaluations, 193.6× faster and 444.6× cheaper than frontier models. The launch post says those workflows were made by people on TypeSafe's own team, that the reference answers are the

average of two frontier models (not human labels), and that, it says, the workflows are outside its training distribution.

- Price: **$0.042 per million input tokens, with output tokens free**. A decision has no generated output, so only the input is billed.
- Up to **255 options** for a choice question; text only (it says images are not supported yet). One summary of the API documentation reports an input limit of 64k tokens, which the launch post itself does not state.
- Training method named **"Reinforcement Learning for Calibrated Decisions" (RLCD)**. Wikipedia and TechCrunch both report that Jev was trained exclusively on synthetic data, attributing that to Almeida. TypeSafe's own page does not confirm it.
- Early users reported in the press: a safety classifier at Vercel running 5 to 18 times faster than an OpenAI model with greater accuracy, and an email classifier at Bryo AI 10 to 20 times cheaper than Gemini with better confidence scores but slightly lower accuracy. These are company statements.

An ecosystem appeared quickly. An independent paper counted **2,170 public GitHub projects** using Jev as of 22 September 2026, a week after launch. That measures adoption, not accuracy.

6.2 Laya (Convai Innovations): the open one we will take apart

Who and what. Laya is an open-weight decision model from Convai Innovations, built by Nandakishor M, released under the Apache 2.0 licence with its code, benchmark scripts and a fine-tuning notebook. It was presented as a free, local alternative to Jev.

The three checkpoints.

Checkpoint	Backbone	Parameters	Context	Purpose
`laya`	ModernBERT-large	421M	512 tokens	English text, the base model
`laya-multilingual`	mmBERT-base	322M	1,024 (can run up to 8,192)	100+ languages
`laya-typed-decisions`	ModernBERT-large	421M	1,024	fine-tuned on a four-workflow decision benchmark

Claims (the authors'). About 33 ms for one question on a T4 GPU, calibration error cut from 0.466 to 0.081 after temperature fitting, and 0.766 accuracy for the fine-tuned checkpoint on a typed-decisions test. All three are the authors' own figures (treat them as claims, not measurements; Chapter 27, Part 6 examines them).

Lineage. The author had earlier published two papers on related ideas: a March 2025 model that predicts the probability a sales conversation will convert, and a September 2025 paper on estimating, *before* a language model answers, whether it is likely to hallucinate, and routing the query accordingly. In his own write-up he says the underlying concepts predate Jev. The papers do describe related ideas (probability outputs, reinforcement learning, routing before generation). They do not describe the typed, request-time interface, and how Jev itself was developed is not public.

Status in one sentence. Laya's base checkpoints are weak zero-shot on some tests, and its authors say the value comes from fine-tuning on your own decisions. Its design is fully open, which is why it is the one decision model whose internals anyone can inspect. Chapter 26 takes it apart.

6.3 Decision 2.0 and Vega (vLLM Semantic Router team): the decoder way

Who and what. Decision 2.0 is a family of six open decision models from the team behind the vLLM Semantic Router, released under Apache 2.0. They are built on **Qwen** language models. **Vega** is the largest. It is *not* a release from Qwen's own team. It is built on a Qwen base model.

How it differs. Where Laya is a 421M-parameter bidirectional encoder, the Decision 2.0 models are **decoder LLMs** adapted with LoRA (Chapter 13) and a small readout head, and they accept the same request shape as Laya (`state`, and questions with `type`, `instructions`, `criteria`). Chapter 26, Part 3 takes Vega's design apart.

Model	Parameters	Context	Median latency per single-question request (card)
Kai	0.60B	8,192	4.9 ms
Eos	0.75B	16,384	6.0 ms
Sol	1.88B	16,384	7.2 ms
Nox	4.21B	16,384	12.9 ms
Lux	7.94B	16,384	18.4 ms
Vega	**29.37B**	**32,768**	**71.4 ms**

The latencies are the cards' own ("on a single GPU", hardware not named). The pattern is the one theory predicts: small models sit on a fixed overhead floor of a few milliseconds, and big ones grow roughly in proportion to size. Vega supports up to 255 options.

6.4 Others reported so far

This list comes from news coverage and an independent comparison in the first weeks of October 2026. Treat every entry as "reported" and check it before relying on it.

- **Microsoft-Decision-1**, reported to be built on Alibaba's Qwen3.5-9B, text-only, priced at the same $0.042 per million input tokens as Jev. Microsoft's performance claims are its own.
- **OpenAI** is reported to have opened a "Decisions" API in public beta.
- **Cloudflare**'s hosted Clef 27B and Clef-flash 9B.
- **Perplexity Decider 1.1 (27B)**, an open-weight model, reported to top one independent comparison.
- Further open or hosted entries: Liquid AI's `d1`, meraGPT's Decider 1, Featherless's Simple Jev, OpenDecider (several sizes, including a 400M encoder), and the Open-Jev datasets and models.
- A news headline also reports that AWS, Upstage and Ollama have agreed on a common decision-model API, a sign that the field is converging on one request shape.

6.5 Side by side

	Jev	Laya	Vega (Decision 2.0)
Open weights	no (hosted API)	yes, Apache-2.0	yes, Apache-2.0
Backbone	not disclosed	BERT-style encoder (ModernBERT-large; mmBERT-base)	Qwen decoder with LoRA
Size	not disclosed	421M (322M multilingual)	29.4B listed
Context	not stated by TypeSafe (64k reported for the API)	512 to 1,024 by default	32,768
Options per question	up to 255	about 20 (token budget)	up to 255
Speed claim	70 to 500 ms end to end	about 33 to 40 ms on a T4	71.4 ms, hardware not named
Who measured	TypeSafe	the authors, plus third parties	the team, plus its stated reproduction

6.6 How to read the claims

Whenever a number appears, ask four questions (Chapter 27, Part 6 turns these into a longer checklist):

1. **Who measured it?** The vendor, the model's authors, or an outsider?
2. **On what data?** Was the test data part of the training mix?
3. **Which version?** A base checkpoint used as shipped, or one fine-tuned on that very test's training split?
4. **Compared with what?** A baseline that ignores the input can score surprisingly well.

As one data point: the independent tests so far found larger decoder-based models, and Jev, ahead of Laya's *base* checkpoints when used zero-shot, which is consistent with Laya's authors saying that fine-tuning does most of the work.

Part 7 — What is genuinely new here

It helps to separate what is old from what is new, because the topic is surrounded by marketing.

Old ideas, repackaged:

- **Encoders with a classification head** (BERT, 2018). See Chapters 15 and 26.
- **Zero-shot classification with labels given as text.** Models that take the candidate labels at request time have existed for years (for example NLI-based zero-shot classifiers, and "label-agnostic" pretraining). Vega is even tagged as a zero-shot-classification model on Hugging Face.
- **Cross-encoders** for re-ranking, where the query and candidate are read together.
- **Cross-entropy and temperature scaling**, the standard tools for training and calibrating a classifier.
- **Routing** with a small classifier (Chapter 24, Part 7), which decision models generalise.

What looks new in 2026:

1. **A shared interface.** Typed question kinds, answer lists given per request, and several questions about one state answered in one pass.
2. **Calibrated probabilities as the product.** The number is meant to be trusted and acted on (act, ask, escalate), not just ranked.
3. **Training on many decision datasets at once**, plus teacher-labelled or synthetic data, so that one model reads whatever answer list it is given.
4. **Packaging and price:** milliseconds, input-only billing, and an emerging common API.

What is not known: how Jev works inside. Everything in this chapter about internals comes from the *open* models.

> **Where this sits among the chapters.** System 2 (Chapter 20) buys accuracy by spending tokens. System 1 buys speed and a trustworthy number by spending none. In an agent (Chapter 24) the two work together: the fast model routes and gates, and the slow model plans and writes.

Common misconceptions, cleared up early

Misconception	Reality
"A decision model is a small language model."	Not necessarily. Laya is a BERT-style encoder. Vega is a large decoder with a readout head. None of them generate text.
"It cannot be wrong, because it cannot hallucinate."	It cannot invent a sentence, but it can pick the wrong option, with confidence.
"The list of answers is learned inside the model."	The list is request-time *text* that the model reads. That is why it can change per request, and why each answer must explain itself.
"A score question returns one number."	It returns a probability for each level, and the single number (the expected level) is calculated from them.
"Calibrated means accurate."	Calibrated means *honest about its uncertainty*. A calibrated model can still be wrong often. (See Calibration in Part 4 and Chapter 27, Part 4.)
"Embeddings and similarity do the same job."	Similarity compares topics and reads the text and the options separately. A decision model reads them together and gives a probability.
"Jev's design is known."	TypeSafe has not published the architecture, weights, training data or a technical paper.
"Vega comes from the Qwen team."	It comes from the vLLM Semantic Router team, built on a Qwen base model.
"A decision model replaces the LLM in an agent."	It pairs with it: the fast model routes and gates, the LLM plans and writes.
"System 1 models are a new architecture."	Mostly a new job, readout and training objective on familiar Transformers.

Quick reference

```
Decision = STATE (evidence) + QUESTION + ALLOWED ANSWERS (request-time text) → PROBABILITY per answer
One forward pass (prefill only). No generation. Several questions per state, answered together.

Request fields (Laya / Vega):  type · instructions · criteria      (criteria = the answers, each described)

Three kinds:   choice  pick one            unordered options        → probability per option
               score   ordered levels      ordered options          → probability per level + expected level
               noul    yes/no statement    two options (false/true) → P(true)
Why three:     errors differ: cross-entropy treats all wrong answers alike; ordered levels need a
               distance-aware score (ranked probability score)

Fits where:    decision (closed list) → decision model;  open answer → LLM.   Pair them: route/gate vs plan/write
Probability →  act / ask / escalate (thresholds, cost rule p > 0.625 in the example), needs calibration

Landscape (Oct 2026):  Jev (TypeSafe, closed, undisclosed internals) · Laya (open, ModernBERT, 421M)
               · Decision 2.0 / Vega (open, Qwen decoder + LoRA, 0.6B to 29B) · others reported
Read claims:   who measured · on what data · which version · compared with what
```

What's next

You now have the concept, the vocabulary and the map. **Chapter 26** takes two real designs apart: Laya, built on a BERT-style encoder, and Vega, built on a Qwen decoder, starting with what BERT and its `[CLS]` token really are. **Chapter 27** covers what turns such an architecture into a model people can rely on: where the labelled data comes from, how it is trained and what that costs, how to keep its confidence honest, and how to test it fairly. The sources behind all three chapters are listed in References, section 14. After that, the **Appendix** makes everything concrete one last time, following a complete forward pass and a complete backward pass through a tiny GPT with small real numbers.

Chapter 26 — Inside a Decision Model: BERT, Laya and Vega

Introduction

Chapter 25 explained *what* a decision model is: a state, a question and a list of allowed answers go in, and one probability per answer comes out, in a single pass and without generating text. This chapter and the next explain *how one is built*. This chapter is about the **architecture**. The next is about **data, training, calibration and evaluation**.

We take two real designs apart, because they make opposite choices from the same starting point:

- **Laya**, an open model built on a small **bidirectional encoder** (a BERT descendant). Everything about it is public: weights, code, training loop, benchmark scripts.
- **Vega**, an open model built on a large **causal decoder** (a Qwen language model) with a LoRA adapter and a small readout head.

Laya is the main worked example, so we start where it starts: with BERT. The handbook has only sketched BERT so far (Chapter 15, Part 4), and a decision model leans on details that sketch left out: what `[CLS]` and `[SEP]` are, how BERT is pretrained, how a head is added on top.

The parts of this chapter:

1. **Encoders first:** BERT properly. Why it exists, its input, the `[CLS]` token, pretraining, fine-tuning, and ModernBERT.
2. **Laya, layer by layer:** the input layout, the `[MASK]` slots, the head, and the size of every piece.
3. **Vega, the decoder way:** a different answer to the same problem, and a speed comparison.

> **Sources and numbers.** Facts about Laya come from its repository, model cards and documentation; facts about Vega from its model card and code; facts about BERT and ModernBERT from their papers. Where a claim is the authors' own, the text says so. Small worked examples use illustrative numbers. Some of them were produced by short programs, included next to this chapter as `Chapters-26-27-experiments.js`, so that you can rerun them (`node Chapters-26-27-experiments.js all`).

Part 1 — Encoders first: BERT, properly

1.1 Why an encoder-only model exists

The problem. A GPT reads left to right. When it processes "bank" in "the **bank** of the river", it has not yet seen "river". That is exactly what the causal mask of Chapter 4 enforces, and it is what makes generation possible. But suppose we are *not* generating. Suppose we only want to understand a complete piece of text, for instance to say "this email is about billing".

Why the previous step is not enough. The causal mask throws information away: early tokens never see later ones.

The fix: remove the mask. Let every token look at every other token, in both directions.

```
GPT (causal mask)                BERT (no mask)
 each row = a token,              every token sees every token
 each ✓ = "can look at"

       t1 t2 t3 t4                      t1 t2 t3 t4
  t1   ✓                           t1   ✓  ✓  ✓  ✓
  t2   ✓  ✓                        t2   ✓  ✓  ✓  ✓
  t3   ✓  ✓  ✓                     t3   ✓  ✓  ✓  ✓
  t4   ✓  ✓  ✓  ✓                  t4   ✓  ✓  ✓  ✓
```

What it costs. With no mask, a token cannot be "the next word to predict", because the answer is already visible. A BERT cannot write text. It produces **one understanding vector per input token**, and you build on those vectors.

The architecture is Chapters 4 to 7 with the mask removed: attention, feed-forward network, residuals and LayerNorm, stacked. The original BERT-base has 12 layers, a hidden size of 768, 12 attention heads and about 110 million parameters. BERT-large has 24 layers, a hidden size of 1,024, 16 heads and about 340 million. Its vocabulary is 30,000 WordPiece tokens, and it was pretrained on BooksCorpus (800 million words) and English Wikipedia (2,500 million words).

1.2 What BERT is used for

BERT is a **reader**, not a writer. Its usual output is a vector, and what you do with the vectors depends on the task.

Translation: no. Translation needs output of a different length in another language, written one token at a time. BERT has no way to write. That job belongs to encoder–decoder models and decoder LLMs (Chapter 15).

Fill in the blank: yes. That is what BERT is trained on (Part 1.5). It is rarely a product by itself. By learning to fill blanks across billions of words, BERT ends up with good vectors for everything else.

The real jobs, from the BERT paper and its benchmarks:

Task	Input shape	Where the answer is read from
Sentiment, spam, intent	**1 text**	the `[CLS]` vector, into a small classifier
Named-entity tagging ("which words are people or places?")	**1 text**	each token's vector, one label per token
Fill in the blank	**1 text** with a `[MASK]`	the vector at the `[MASK]` position
Embedding one text for search	**1 text**	one pooled vector
Question answering from a passage	**question + passage**	per-token scores for where the answer starts and ends
Does sentence A imply sentence B?	**2 texts**	`[CLS]`, a three-way classifier
Are these two questions duplicates?	**2 texts**	`[CLS]`, yes/no
Re-ranking search results	**query + document**	`[CLS]`, a relevance score

BERT always takes **one input sequence**, which holds one text or two. Most uses need only one. Two-text inputs are used when the *task itself* is about the relationship between two texts. The paper describes the single-text case as "a degenerate text-∅ pair": a pair with nothing in the second half.

> **A common question: "Isn't BERT just an LLM without the decoder?"** Close, with one hidden trap. The blocks inside GPT and the blocks inside BERT are almost the same. The difference is the **causal mask** (Chapter 15, Part 2):
>
> ```
> GPT  = Transformer blocks + causal mask (cannot look ahead)  + a head that predicts the next word
> BERT = Transformer blocks, no mask       (sees everything)   + heads that read vectors out
> ```
>
> The "decoder" of the original Transformer did two extra things: it used a causal mask *and* it attended to an encoder's output (cross-attention). GPT-style models kept the mask and dropped the cross-attention. So BERT is not "an LLM minus a part". It is the same blocks with the mask removed, which gives up writing and gains seeing everything.

1.3 What goes in: tokens, `[CLS]`, `[SEP]`, segments, positions

The problem. The model takes a list of tokens. BERT is used on a single text or on a **pair**. How does it tell the pieces apart, and where does the *summary* of the whole input come from?

The fix: three small ideas.

```
[CLS]  I  was  charged  twice  [SEP]  please  refund  it  [SEP]
  |    └──────── sentence A ────────┘      └─── sentence B ───┘
  |
  special first token: the "summary slot"
```

1. `[CLS]` is always the first token. It is a placeholder, not a word. The BERT paper says its final hidden state serves as the summary of the whole sequence for classification tasks.
2. `[SEP]` separates sentence A from sentence B.

3. **A segment embedding.** A learned vector added to every token saying "I belong to sentence A" or "sentence B".

What the model actually receives per token is the sum of three vectors of the same size (768 for BERT-base):

```
input vector =  token embedding    (which word piece: a row in the embedding table, Chapter 3)
             + segment embedding   (A or B: one of two learned vectors)
             + position embedding (which slot: 1st, 2nd, 3rd ...)
```

An illustrative sum, with 4 numbers instead of 768:

```
token "charged"           [ 0.50   0.10  -0.30   0.20]
+ position (3rd slot)     [ 0.05  -0.10   0.20   0.00]
+ segment A (row 0)       [ 0.10  -0.20   0.00   0.30]
= input to layer 1        [ 0.65  -0.20  -0.10   0.50]
```

Fixed, learned or computed?

Thing	Kind
The `[CLS]` token's *input* vector (plus position and segment A)	**learned**, then the same for every input
The segment A and B vectors	**learned**
The `[CLS]` position's *output* vector after the last layer	**computed** freshly for each input

A common question: "Where do segment embeddings come from?" Three separate things happen:

Step	What happens	Learned?
Deciding the label	The data-preparation code numbers each token: everything up to and including the first `[SEP]` gets 0 (A), everything after gets 1 (B).	**no**, plain code
The two vectors	A tiny table with **2 rows**, each as wide as the token embedding. It starts random and is trained along with everything else.	**yes**
Using it	Look up the row for the token's label and **add** it to the token and position vectors.	computed

It is the same idea as the position table of Chapter 3, with a different table: about 30,000 rows for tokens, up to 512 for positions, and 2 for segments. **If you feed BERT a single sentence, the segment embedding is still used**: every token is labelled A, so the same vector is added to every token, and it tells the model nothing. Training teaches the model to use the two vectors because pair tasks ("do these sentences go together?", "which part of the passage answers the question?") cannot be solved without knowing which token belongs to which part.

Later encoders found they can do without it. ModernBERT drops BERT's pair-matching task, and Laya's code passes only token ids and an attention mask to the encoder, with no segment ids.

1.4 What `[CLS]` really is

The problem. BERT gives one vector per token. But many tasks need *one* vector for the whole text. How do we get a single summary from many vectors?

Why the obvious options are weak. Using one real word's vector biases the summary toward that word. Averaging all the vectors works, and is a common choice, but treats every token equally and has nothing in it that is *trained for the task*.

The fix: a slot with no word of its own. `[CLS]` is an ordinary token whose only job is to give a summary somewhere to live. Because it carries no word meaning, nothing biases it, and because BERT has no mask, it can look at every token in every layer.

Why it is placed *first*. The position does not matter to attention, so first is just a convention. In a GPT the situation is different: with the causal mask only the **last** token has seen everything, so a GPT-style classifier reads the last token. In BERT every token has seen everything, so any token could hold the summary, and a dedicated neutral one is used.

How it becomes a summary: one attention step. Take one head in one layer, with made-up 2-number vectors so the arithmetic can be followed:

```
tokens:                 [CLS]     charged    twice     refund
CLS asks (its query):   [1.0, 0.5]

key of each token:      [0.2,0.1] [0.9,0.3]  [0.5,0.4] [0.8,0.8]
score = q·k / √2:        0.177     0.742      0.495     0.849
softmax → weights:       0.164     0.289      0.226     0.321      (sum = 1)

value of each token:    [0.1,0.0] [1.0,0.2]  [0.4,0.6] [0.2,1.0]

new [CLS] = Σ weight × value = [0.46, 0.51]
```

This is Chapter 4 applied to one position. The new `[CLS]` vector is a blend of the other tokens, weighted by relevance (it also looks at itself, with weight 0.164). The same thing happens at every position in parallel. After 12 layers with 12 heads in each, the final vector `C` has absorbed information from the whole sentence in many layered ways.

Why it becomes a *useful* summary: the loss. Nothing in the architecture forces `[CLS]` to summarise well. A loss is attached to it. In pretraining, `C` is used for next-sentence prediction; in fine-tuning, a classification layer reads `C` with the loss $\log(\text{softmax}(C \cdot W^T))$ (Part 1.6). Backpropagation (Chapter 10) then changes the attention weights so that `[CLS]` pulls in whatever the task needs. A sentiment task teaches it to gather sentiment cues; a topic task teaches different cues. The slot is the same, and what it learns to hold is whatever the loss rewards.

A warning: an untrained `[CLS]` is a poor summary. The Sentence-BERT paper reports that taking BERT's raw `[CLS]` output, or averaging its outputs, gives poor sentence embeddings, often worse than simply averaging older GloVe word vectors. The summary becomes good only after training with a loss that rewards good sentence vectors.

> **A common question: "Is a dedicated summary token needed in a GPT classifier too?"** A first answer ("needed in BERT, not in GPT") overstates it. A dedicated token helps in both kinds of model. What differs is **where it can sit**:

```
BERT, token FIRST:              GPT, token FIRST (a [CLS] copy):
 [CLS] the cat sat               [CLS] the cat sat
  ✓     ✓   ✓   ✓  sees all       ✓     ✗   ✗   ✗   sees only itself: useless

GPT, token LAST (an end/summary token):
 the cat sat [SUM]
  ✓   ✓   ✓    ✓   the only position that can see everything
```

Decoders do use such tokens. GPT-1 (2018) added randomly initialised start and end tokens to every task input, and fed the last position's final activation to a linear output layer. E5-Mistral turns a decoder LLM into an embedding model by appending an `[EOS]` token and taking the last layer's vector at that token. A "pause token" paper appended learnable tokens to give a decoder extra computation before answering, with gains reported only when the model was both pretrained and fine-tuned with them.

Why a dedicated token helps, even in a GPT (the usual explanations): it is **neutral** (the last real word's vector is busy predicting the next word, so a summary competes with that job), **consistent** (always the same token), and **extra working space**. The trade-off is that a brand-new token in a model that has never seen it starts untrained. Vega, in Part 3, uses exactly this idea: the last token of its prompt is the "summary" the head reads.

1.5 Pretraining BERT: what `[MASK]` means in BERT

The problem. BERT sees the whole sentence, so it cannot be trained to predict the next word: the answer would be sitting in its input. We need a training task that needs **no human labels** and **forces the model to use both left and right context**.

The fix: fill in the blanks. This is the old "cloze test" from language classes. Hide some words and make the model guess them from the words on both sides. Any text becomes training data, because the hidden word is its own label.

How it works, on "my dog is hairy":

1. **Choose positions.** Pick 15% of the tokens at random. Say it picks "hairy".
2. **Corrupt them, with a three-way split** (the paper's recipe):

Chance	What replaces the chosen token	Result
80%	the `[MASK]` token	my dog is **[MASK]**
10%	a random word	my dog is **apple**
10%	nothing (kept as is)	my dog is **hairy**

3. **Run the whole sequence through BERT.** No mask in the attention, so the vector at the chosen position has seen words on both sides.
4. **Read the vector at the chosen position** and send it through an output layer that produces a probability for each of the 30,000 vocabulary words ("as in a standard LM", the paper says).
5. **Score with cross-entropy** (Chapter 9) against the original word. With illustrative numbers: if the model gives "hairy" 0.62, the loss is $-\ln 0.62 = 0.48$ (a good guess); if it gives 0.05, the loss is 3.00.

Only the chosen positions count towards the loss. The other 85% of tokens are read but not scored.

Why the odd 80/10/10 split? The paper gives two reasons.

- **A mismatch problem.** `[MASK]` never appears in real text, so it will not appear when BERT is later used for classification. If the model saw `[MASK]` only at the positions it had to predict, it could learn "I only need to think hard at `[MASK]`". Keeping or randomising some chosen words breaks that habit.
- **No hiding place.** As the paper puts it, the model cannot tell which words it will be asked to predict or which were swapped for random ones, so it has to keep a useful contextual representation of every input token. Random replacement touches only 1.5% of all tokens (10% of 15%), which the paper says does not seem to hurt understanding.

The second task: next-sentence prediction. BERT also trained on pairs of sentences. In 50% of examples sentence B really follows A (label "IsNext"); in the other 50% it is a random sentence ("NotNext"). The `[CLS]` output `C` predicts which. That is the loss attached to `[CLS]` in pretraining. The training loss is the sum of the two: the mean masked-word loss plus the mean next-sentence loss.

The scale. BERT-base trained for 1,000,000 steps, with 256 sequences of 512 tokens per step (128,000 tokens per step), about 40 passes over a 3.3-billion-word corpus, with Adam at a learning rate of 1e-4, a 10,000-step warm-up and linear decay. BERT-base used 4 Cloud TPUs in a Pod configuration (16 chips); BERT-large used 16 (64 chips). That is Chapter 11, applied to a real model.

> **A common question: "Is Laya's `[MASK]` the same as BERT's?"** No, and this is the most common confusion in this topic. In BERT, `[MASK]` is a **blank**: something was hidden there, and the model must guess it. In Laya (Part 2), `[MASK]` is a **slot** placed in front of each option: nothing was hidden, and the output vector at that position is read to score the option. Laya never asks BERT to guess a word under the mask. One plausible reason for reusing the token is that the encoder spent pretraining learning to produce a useful, context-aware vector at exactly that kind of position. The authors do not state this reason, so treat it as an inference.

1.6 Fine-tuning: putting a head on a pretrained BERT

The problem. After pretraining, BERT gives a good vector for every token, but it cannot *answer* anything. How do we turn a general reader into a specific task solver, with only a few thousand labelled examples?

The fix, in three moves.

1. **Start from the pretrained weights**, not from random ones.
2. **Add a tiny new output layer**, the "head".
3. **Train everything**, BERT's layers and the head, on your labelled examples, with a small learning rate.

The paper says fine-tuning needs only one extra output layer, and that every parameter is updated. Each task gets its own fine-tuned copy, all starting from the same pretrained weights.

The head changes shape with the task.

Task	What the head reads	What it outputs
Classify a whole text (sentiment, intent, entailment)	the `[CLS]` vector `C`	probabilities over K classes
Tag each word (names, places)	every token's vector T_i	one label per token
Pull an answer out of a passage	every passage token's vector	a start position and an end position

For question answering the paper adds two learned vectors, `S` (start) and `E` (end). The score for an answer spanning tokens i to j is $S \cdot T_i + E \cdot T_j$, with j ≥ i. For "there is no answer", the paper scores the span that starts and ends at `[CLS]`.

One worked example: the classification head. With K = 3 classes (billing, technical, other), the head is a table `W` with one row per class (`W` is **learned**; `C` is **computed** for each text). Illustrative numbers with a 4-number vector instead of 768:

```
C (the [CLS] output)     = [ 0.6, -0.2,  0.9,  0.3]

W row "billing"          = [ 0.5,  0.1,  0.4, -0.2]   →  logit  0.58
W row "technical"        = [-0.3,  0.6,  0.0,  0.5]   →  logit -0.15
W row "other"            = [ 0.2, -0.4, -0.5,  0.1]   →  logit -0.22

softmax → probabilities    billing 0.518, technical 0.250, other 0.233
```

If the true label is "billing", the loss is −ln 0.518 = 0.66 (the paper's $\log(\text{softmax}(C \cdot W^T))$, Chapter 9). If it were "technical" the loss would be 1.39. Backpropagation starts from the simple signal "probability minus answer":

```
gradient on the logits:     billing −0.48 (push UP),  technical +0.25, other +0.23 (push DOWN)
gradient on W's billing row = −0.48 × C = [−0.29, 0.10, −0.43, −0.15]
```

The signal does not stop at `W`. It keeps flowing back through `C` into every attention and feed-forward weight in BERT. That is what "train everything" means: the encoder is gently reshaped so `[CLS]` collects the evidence the task needs.

Why it works so well and so cheaply.

- **Almost nothing is new.** `W` has 3 × 768 = 2,304 numbers, against about 110 million in BERT-base: roughly 0.002%.
- **It is fast.** The paper says all of its results can be replicated in at most 1 hour on a single Cloud TPU, or a few hours on a GPU, starting from the same pretrained model.
- **Small learning rates.** The recipe uses learning rates of 5e-5, 3e-5 or 2e-5, batch sizes of 16 or 32, and 2 to 4 epochs. Pretraining used 1e-4. Small steps nudge the pretrained weights without overwriting what they learned.

The limitation that matters for this chapter. A head with K rows has its classes **baked in**. Add a class and you need a new row and retraining. A decision model needs a head whose number of outputs changes with every request. That is what Part 2 builds.

1.7 Modern encoders: what ModernBERT changes

BERT dates from 2018. Since then, decoder LLMs developed better building blocks that encoders mostly did not receive. The ModernBERT paper (December 2024) describes encoders as the "workhorse of numerous production pipelines" that saw "limited Pareto improvements" after BERT, and applies the modern parts to an encoder. It is still BERT in spirit: no causal mask, `[CLS]`, `[SEP]`, `[MASK]`, fill-in-the-blank training. What changed is the machinery, much of which this handbook already covers.

Part	BERT (2018)	ModernBERT	Why	Handbook
Positions	learned absolute table, 512 slots	**RoPE**	proven in short- and long-context models, "ease of context extension"	Chapter 16
Norm placement	post-norm	**pre-norm**, plus one extra LayerNorm after the embeddings	stabilises training	Chapters 7, 16
FFN activation	GELU	**GeGLU**, a gated FFN	consistent gains in recent work	Chapter 16
Bias terms	present	**removed** in linear layers (except the final output layer) and in LayerNorm	spend the parameters on weights that matter	Chapter 16
Attention	every layer global	**every third layer global, the rest a 128-token sliding window**	long inputs get cheap	Chapter 17
Speed tricks	padded batches	**unpadding**, FlashAttention 3 (global layers) and 2 (local), `torch.compile`	no wasted compute on padding	Chapter 17
Shape	12 layers, wide	**22 layers (base), 28 (large)**, "deep and narrow"	better quality at the same size, sized to common GPUs	
Tokenizer	WordPiece, 30,000	modified OLMo **BPE, 50,368 tokens**, same special tokens	better token efficiency and code handling	Chapter 2
Context	512	**8,192 natively**		

Two details are worth keeping. The tokenizer keeps BERT's special tokens (`[CLS]` , `[SEP]` , `[MASK]`) and templating for backwards compatibility, so everything in Parts 1.3 to 1.6 still applies. And the sliding-window layers matter for Laya's code: when it builds its own attention masks, it passes the encoder's sliding-window setting along so its mask respects the local layers.

Training	BERT	ModernBERT
Data	~3.3 billion words	**2 trillion tokens** (web, code, science)
Masking	15%	**30%**
Next-sentence task	yes	**dropped**
Optimiser	Adam	**StableAdamW**
Learning-rate schedule	warm-up, then linear decay	**warm-up, a long constant stretch, a short decay**
Base-model rate	1e-4	8e-4

(Words and tokens are different units, so treat the data comparison as rough, but it is several hundred times more text.) The large model was initialised from the trained base model rather than from random weights.

Sizes, with a check you can reproduce. Base: 22 layers, hidden size 768, 149M parameters. Large: 28 layers, hidden size 1,024, 395M. The paper's GLU expansion figures, 2,304 for base and 5,248 for large, give the width of the gated input projection (two halves). With that reading, the arithmetic for the large model is:

```
per layer:  attention 4 × 1024²                      = 4.19M
            gated FFN: 1024 × 5248 + 2624 × 1024      = 8.06M
            total per layer                           = 12.26M      × 28 layers = 343M
            + embeddings 50,368 × 1024                = 51.6M
            ≈ 395M      matches the paper

base:       5.01M × 22 = 110M + embeddings 38.7M ≈ 149M   matches
```

The other reading of "expansion" gives 207M and 620M, which do not match, so this interpretation is the one that reproduces the paper.

What the authors report, and admit. On classification and retrieval, ModernBERT reaches state-of-the-art results among encoders of similar size, processing short inputs about twice as fast as DeBERTaV3 and long inputs about twice as fast as the next fastest model (the authors' own benchmarks). Stated limits: English only, trained largely on web data and so inheriting its biases, trained with fill-in-the-blank only (they suggest adding a second objective), and not a text generator.

1.8 The mask is a dial, not a switch

Because the only structural difference between BERT and GPT is the mask, it is tempting to treat encoders and decoders as two separate species. Research suggests they are better seen as two ends of a dial.

- **One network with several mask patterns.** UniLM trains one shared Transformer with bidirectional, left-to-right and sequence-to-sequence masks; GLM and the UL2 family read the prompt bidirectionally and write the output left to right.
- **Decoder to encoder works with adaptation.** LLM2Vec replaces a decoder's causal mask with an all-ones matrix, adds a short masked-prediction stage and contrastive learning, and reports strong text encoders (a new unsupervised state of the art on the MTEB embedding benchmark).
- **Converting one kind into the other is only partly successful.** The Ettin study trained encoders as decoders and decoders as encoders for 50 billion extra tokens each way. The converted models did not surpass models trained on the target objective from the start.

The dial reappears in this chapter: Laya uses a custom mask so that its options cannot attend to each other (Part 2), and Vega's choice of a causal backbone decides where it must put the state and the readout (Part 3).

> **A note on embedding models.** Embedding models (Sentence-BERT, E5, BGE, and large ones such as Gemini Embedding) are also encoders, with the same kind of layers, trained with a different objective: texts that mean the same thing should land close together, rather than texts being classified. They are what Chapter 25, Part 5 compares decision models against.

Part 2 — Laya, layer by layer

2.1 Where it came from

Laya did not appear from nowhere. Its author, Nandakishor M, had published two earlier papers on related ideas, and Laya generalises both.

Date	Work	Core idea	Notes
30 March 2025	*SalesRLAgent* (arXiv:2503.23303)	Treat "will this sale convert?" as a sequential decision, with a policy network producing a conversion probability, a value network supporting reinforcement learning, and a module estimating its own confidence.	The input is **frozen** 3,072-dimension embeddings from an Azure OpenAI model, so no reinforcement learning ever reached a transformer. Trained on **synthetic GPT-4o conversations**, in about 6 hours on a CPU. The paper reports 96.7% accuracy and 85 ms against 3,450 ms for GPT-4 (self-reported).
23 September 2025	*Confidence-Aware Routing* (arXiv:2510.01237)	Estimate, **before** a language model answers, whether it is likely to hallucinate, and route the query to local generation, retrieval, a larger model or a human.	Combines three signals (agreement with a reference embedding, how consistently the layers converge, and a learned confidence estimator). Tested on a 360M-parameter model. The paper lists its own limits: dependence on the reference model, fixed thresholds, domain specificity, and small scale.
September 2026	**Laya**	Make probabilities a typed, request-time product, trained **end to end** on an encoder.	The act/escalate head descends from the routing paper; the probability outputs from the sales model.

2.2 The input: one sequence per question

Chapter 25 showed a decision as three fields. Laya turns **each question** into one text sequence. For the department question about our ticket, the layout is:

```
[CLS] choice question: Which department should handle this? [SEP]
[MASK] billing: invoices, payments, refunds
[MASK] technical: bugs, outages, system errors
[MASK] other: everything else [SEP]
Hi, we were billed twice for March. Please refund the duplicate today or we will cancel our plan. [SEP]
└──────────── the question and options ────────────┘ └──────────── the state ────────────┘
```

The rules, taken from the code:

- `type` and `instructions` become the first line: "choice question: ...".
- Each `criteria` entry becomes one line behind its own `[MASK]` **marker**, written "label: description". For a `score` question the lines read "level 0: not urgent", "level 1: soon", "level 2: blocking". For a `noul` question there are always two lines, "false: no, the statement does not hold" and "true: yes, the statement holds" (unless you override the wording).
- The question and options share a **budget**: 192 tokens in the English checkpoint, 256 in the others. Each option is cut off at 48 tokens, and when there are too many options each gets a smaller share. This is where the "roughly 20 options" limit comes from.
- Whatever remains of the context window (512 or 1,024 tokens) goes to the **state**, cut if necessary.
- If a request asks several questions about one state, the state is **tokenised once** and reused. Each question is still its own row of a batch. "All the questions are answered in one forward pass" means one *batched* pass, not one giant sequence.

2.3 The `[MASK]` slots

Each `[MASK]` is a **slot**: a position with no word of its own, placed before each option. This is the same trick as `[CLS]` (Part 1.4): *a token with no word of its own, whose output vector is read out for a decision.* Where `[CLS]` is one slot for the whole text, Laya gives **each option its own slot**.

Because the encoder is bidirectional, the slot in front of "billing" can see the question, its own option text, the other options and the state, all at once. After the last layer, its vector summarises "this option, in this question, against this state". A small scorer then turns each slot vector into one number.

> **A common question: "Does BERT's masked training still happen in Laya?"** The base encoder was pretrained with fill-in-the-blank, long before Laya. Laya's own training options are a reinforcement-style objective or plain cross-entropy (Chapter 27, Part 2). Its code contains no masked-word objective.

2.4 The pipeline, with shapes

```
tokens (say 60)                                  shape [60]
  │
  ▼ ModernBERT encoder (28 layers)               shape [60 × 1024]
  │   one final hidden vector per token. These are the encoder's last hidden
  │   states, NOT its word-prediction layer.
  ▼ + a learned "question type" vector           (a table of 3 rows: choice / score / noul),
  │   added to every position
  ▼ 2 extra Transformer layers (the "head")      shape [60 × 1024]
  │   attention over all positions, trained from scratch, pre-norm, no positional encoding
  ▼ pick out the vector at each [MASK] position  shape [3 × 1024]     (3 options)
  │
  ▼ scorer: LayerNorm → Linear → GELU → Linear   shape [3]            one logit per option
  │
  ▼ softmax (after a calibrated temperature)     shape [3]            probabilities
```

A small worked step, with illustrative numbers. Three options give three slot vectors and three logits:

```
logits [2.1, -0.4, -1.3]
  temperature 1.0 → [0.896, 0.074, 0.030]     sharp
  temperature 1.5 → [0.774, 0.146, 0.080]     softer: less over-confident
```

Dividing by a temperature above 1 flattens the distribution. Chapter 27, Part 4 returns to this.

Fixed, learned or computed?

Piece	Kind
Where the `[MASK]` markers go	computed by plain code from your options
Encoder weights	learned in pretraining, then fine-tuned
Question-type vectors, the 2 head layers, the scorer	learned, from scratch, in fine-tuning
The calibration temperatures (one per question type)	fitted afterwards (Chapter 27, Part 4)
How many options there are	chosen by you, per request

2.5 Why each piece exists

The slots and the shared scorer. The scorer is a *single* small network applied to every slot with the same weights. It cannot know which option it is looking at except through the slot's vector, so that vector has to carry "this option, in this question, against this state". Bidirectional attention provides exactly that. Because the scorer is shared, adding an option just adds a slot, and nothing in the model has a fixed number of outputs.

The extra layers. The public material does not give an ablation, so this is an inference. The encoder was pretrained to predict words. The two extra layers are new, cheap (about 25 million parameters) and trained for this task alone. They let the slot vectors attend to each other and to the state once more with decision-specific weights. The documentation does say that training only the head with a frozen encoder "gains much less" than a full fine-tune.

What goes in and out of the head. The head receives the encoder's **final hidden vectors**, one per token, plus the question-type vector. It hands back vectors of the same shape, and only the slot positions are read. Nothing comes from BERT's word-prediction layer.

Order-blindness. In a plain sequential layout, an option's place in the list changes its position vector, and the code reports that five identical options produced logits spread by 3.59. Laya's code has a **parallel layout** in which every option starts at the same position and masks stop options from attending to each other. The head has no positional encoding, so reordering the options just reorders the logits. This is the mask dial of Part 1.8 in action: a custom pattern, neither causal nor fully open. The README still lists "position sensitivity" among the limitations and suggests averaging over several orderings, so the design intent is clear, but a given checkpoint should not be assumed fully order-blind.

2.6 The size of every piece (a check against the model card)

Two head layers at width 1,024 cost about 25.2M parameters (12.6M each). The scorer adds about 1.05M, the act head about 0.26M, and the type table 3,072 numbers, about 26.5M in all. Add ModernBERT-large's 395M and the total is **about 421M**, matching the model card.

2.7 The act/escalate head, and its status

There is one more small head. It reads the `[CLS]` vector plus four summary numbers (the top probability, the gap between the top two, the entropy, and the option count) and is meant to output "act" or "escalate". The model card is candid that "`action.act_probability` carries no usable signal yet" (AUROC 0.30, which is worse than chance). So the head exists, but its output should not be used. Use the calibrated probabilities and your own threshold (Chapter 27, Part 4).

2.8 What you actually download

For a GPT-style model, the download is the trained weights. For Laya, the checkpoint folder holds:

- `model.safetensors`: the encoder and the head (843 MB for the English model, in half precision, which matches 421M parameters at two bytes each),
- the tokenizer files,
- `rl_agent_config.json`: the settings that must match training (the token budgets, the number of head layers, the act-head costs, **and the calibration temperatures**).

Calibration lives in that config, not in the weights, which matters in Chapter 27, Part 5.

2.9 Speed in practice

On a T4 GPU the model card reports about 39.5 ms for one question (English checkpoint) and 32.8 ms (multilingual), 158.6 ms and 72.3 ms for ten questions in one call, and 103 to 332 questions per second batched. On a CPU the README reports roughly 193 to 464 ms for the multilingual router. These are the authors' own figures; Part 3 compares them with Vega's.

2.10 Limits

- **Few options.** The shared option budget means that about 20 options is the practical ceiling. On Banking77 (77 labels) the model card reports 0.425 accuracy against Jev's 0.870, and both Laya checkpoints score exactly 0.425, which looks like a budget ceiling and not a capability gap.
- **Order sensitivity** (above).
- **Short context.** 512 to 1,024 tokens by default.
- **Text only.**
- **The `score` kind is its weakest**, per the model card (0.372 on a five-level sentiment benchmark), and, the card warns, `noul` can follow the wording of its option labels instead of the state.
- **The base checkpoints are weak zero-shot** on its own typed-decisions test (0.362, below a baseline that always guesses the common answer, 0.461). Fine-tuning does most of the work (Chapter 27, Parts 2 and 6).

Part 3 — Vega, the decoder way

Laya answers the question "how do you get one probability per option?" with a bidirectional encoder and slots. Vega (the largest of the Decision 2.0 models, Chapter 25, Part 6.3) answers it with a **causal decoder**. Comparing the two shows how much the attention mask dictates the design.

3.1 The backbone

Vega is built on `Qwen/Qwen3.8-27B`. Its configuration (the model's own config file) shows:

Property	Value
Layers	64: **48 linear-attention** and **16 full-attention**, in a repeating pattern of three linear then one full
Hidden size	5,120
Attention heads	24 query heads sharing 4 key/value heads (grouped-query attention)
Vocabulary	248,320 tokens
Native context	262,144 positions (Vega supports 32,768)

The mixed linear and full attention is the hybrid design of Chapter 17, and the 24-versus-4 head split is grouped-query attention (GQA). Chapter 21 describes the Qwen family.

3.2 The input: state first

Vega builds a text prompt, with the **state first**:

```
Context:
<state>

Task type: choice
Question:
<instructions>
Options:
<option>{"key": "billing", "description": "invoices, payments, refunds"}</option>
<option>{"key": "technical", "description": "bugs, outages, system errors"}</option>
...
Select the single option best supported by the context and instructions.
Decision:
```

The fields match Chapter 25: `type`, `instructions` and `criteria` (here as `key` plus `description`).

3.3 The readout: endpoints and a query

```
tokens → Qwen backbone (causal, no vocabulary head, no KV cache) → one hidden vector per token
           │
           ├─ take the vector at the LAST token of each <option> block    ("endpoint" vectors)
           └─ take the vector at the LAST token of the whole prompt       (the "query" vector)
                      ▼
        score_i = key(LN(endpoint_i)) · query(LN(query)) / √256
                  + w · GELU( W_c·LN(endpoint_i) + W_q·LN(query) )
                      ▼
              softmax over the options → probabilities
```

A check of this reading: four projections of 5,120 → 256, plus a scalar and layer norms, give 5.26 million parameters. Stored in 32-bit floats that is 21.1 MB, which is exactly the size of the head file in the repository.

3.4 What Vega keeps from Qwen

A natural question: is only *part* of the Qwen system retained? The loader takes the language-model part of the full multimodal model and uses its final hidden vectors.

Part of Qwen3.8-27B	In Vega?
Input word embeddings	**kept**
All **64 transformer layers** and the final norm	**kept**, none removed
The output layer that predicts the next word (the "LM head", 248,320 outputs, about 1.27B parameters, untied from the embeddings in this model)	**dropped**
The vision tower (this base model is natively multimodal)	**dropped** (a code comment says "vision tower unused")
The generation loop and KV cache	**not used** (`use_cache=False`)

Added on top: a LoRA adapter (below) and the 5.3M-parameter head.

An accounting check: the base repository holds about 27.78B parameters. Vega's config counts 25.62B for the text model, and the untied LM head is 1.27B, which leaves about 0.89B. That is consistent with the vision tower, though it has not been checked tensor by tensor.

So Vega removes no blocks. It keeps all 64 layers, discards the word-prediction output, and retrains behaviour with LoRA. The speed difference from generating text comes from skipping the decode loop, not from a smaller network.

> **A common question: "What is the vision tower?"** It is a **separate stack of Transformer layers dedicated to images**, what Chapter 19 calls the vision encoder. In this Qwen model: 27 layers, width 1,152, 16 heads, images cut into 16 × 16 patches, each 2 × 2 group of patches merged, then projected up to the language model's width of 5,120 so that image vectors look like text-token vectors. The config also defines special token ids for "image", "video", and "vision start" and "vision end", marking where picture tokens sit in the sequence.
>
> There are two levels to keep apart. **The modules** (vision tower, projector, text model) are chosen by the architect and are separable: they are separate tensors, and Vega shows it by loading only the language model. **What each head or layer does inside a module** is learned, not assigned (Chapter 5: real heads range from clearly specialised to generic to nearly redundant). It is the fixed / learned / computed lens again. Joint training tunes the pieces to each other, but they stay separable. (Joint training may have shifted the text layers slightly, so a tower-less model is not guaranteed identical to one trained on text alone.)

3.5 How it is adapted: a LoRA "soup"

The adapter file holds **3.74 billion** trainable parameters, in 32-bit floats (14.9 GB), which is 13% of the base. It is rank 512, but the config shows it was built as a **soup**: **four separate LoRA runs of rank 128 each**, stitched together along the rank dimension (B matrices scaled by 1/4, the head averaged). So the team trained at least four adapter runs and merged them. The base model is not in the repository: it is a separate 55.6 GB download.

The training loss combines cross-entropy over the valid options, an optional Brier term and a replay KL term, configured as `training_mode: lora`, starting from Qwen's post-trained (chat-tuned) model with a randomly initialised head. That is **supervised cross-entropy with no reinforcement learning** (Chapter 27, Part 2). The KL term on replayed rows keeps the model from forgetting general decisions it already handled.

3.6 Laya and Vega side by side

	Laya	Vega
Backbone	395M ModernBERT, bidirectional	27B Qwen, causal, hybrid attention
State position	last	first
Slot for each option	**before** the option text (`[MASK]`)	**after** the option text (its last token)
Summary token	`[CLS]` at the front	the final token of the prompt
Head	2 Transformer layers + MLP scorer (~26M)	bilinear + MLP over two vectors (~5M)
Adaptation	full fine-tune	LoRA, rank 512 (a soup of four)
Context and options	512 to 1,024 tokens; about 20 options	32,768 tokens; up to 255 options

3.7 The mask decides the design

Everything in the table's "position" rows follows from one fact. **Vega is causal**, so each of its choices is forced:

- The state comes **first**, so that later tokens can see it.
- The readout sits at the **end** of each option, because a token can only see what came before it.
- The summary token is the **last token of the prompt**, exactly the "`[CLS]` at the end" placement of Part 1.4.

Laya is bidirectional, so it can put its slots *before* the option text and the state *last*, and still have every slot read everything.

Do the two recompute the state for every question? By default, **both do**. Vega's default "exact path" encodes each question as its own row `[state + question + options]`, so, as its own docstring says, the shared part of the input is encoded again for every question. Vega also has an **opt-in** `share_context` mode, off by default, that computes the shared prefix once and continues each question from it: the full-attention layers reuse the prefix's stored keys and values (Chapter 12's KV cache, Chapter 22's prefix caching), and the linear-attention layers carry a recurrent state. It is not bit-for-bit identical (near ties can flip, and a `tau` setting sends close calls back to the exact path), and it only applies with at least two questions. Laya only tokenises the state once; it has no way to share encoder work, because its question tokens come *before* the state and attention runs both ways, so every state vector depends on the question.

How much sharing saves, for a 300-token state and 80 tokens per question-plus-options:

```
5 questions:   default 5 × 380 = 1,900 token-passes    shared 300 + 5×80 = 700      → 2.7× fewer
10 questions:  default 3,800                            shared 1,100                 → 3.5× fewer
```

3.8 Speed: the theory and the claims

Compute per token. A forward pass costs about 2 × (non-embedding parameters) FLOPs per token:

	per token	300-token request
Vega (about 24.3B non-embedding)	48.7 GFLOP	**14.6 TFLOP**
Laya (about 370M: encoder 343M + head 26M)	0.74 GFLOP	**0.22 TFLOP**
Ratio	**about 66×**	

On the same hardware, then, Vega should be about **66 times slower** in pure compute.

What the model cards report. All six Decision 2.0 cards report a median latency for a single-question request on a single GPU (Chapter 25's table). From Lux (7.94B) to Vega (29.4B), parameters grow 3.7× and latency 3.9×: roughly proportional, as theory predicts for large models. At the small end latency barely moves (4.9 ms for 0.6B, 6.0 ms for 0.75B): a fixed overhead floor.

Why the measured gap to Laya looks small (about 2×). Laya's published latency (32.8 to 39.5 ms) is on a **T4 GPU**, about 65 FP16 TFLOPS at peak. Three reasons it is not 66×:

1. **Different hardware.** Vega's cards say only "single GPU". Its optimised path in the code is written for AMD ROCm GPUs of the MI300 family (`gfx942`), and the same team's write-up for a smaller model reports training on an AMD MI325X. If the figure was measured on such a GPU (about 1,307 BF16 TFLOPS

peak, roughly 20 times a T4), the card's number is plausible: a 300 to 500 token prompt would imply 240 to 400 effective TFLOPS, 19 to 31% of that peak. That fits, but it is an inference and not a stated fact. A 27B model in bf16 needs about 50 GB for weights and could not run on a 16 GB T4 at all.

2. **The overhead floor.** Laya's compute alone on a T4 would be about 4 to 19 ms for 100 to 512 tokens (at 30% of peak), yet it measures 33 to 40 ms.
3. **Vega's engineering:** HIP graphs, fused kernels and a leaner LoRA path, all aimed at removing overhead.

On the **same GPU**, Laya should be many times faster than Vega, tens of times in compute. Nobody has published that comparison, so it is an estimate and not a measurement.

Memory and size. Vega's base alone is 55.6 GB (plus the 14.9 GB adapter), so it wants an 80 GB-class GPU or a quantised build. Laya is **843 MB** and runs on a CPU.

3.9 What the extra compute buys

- **Length:** 32,768 tokens versus 512 to 1,024. A user report (a Laya issue) describes near-chance accuracy for Laya on long, noisy multilingual posts, where a decoder-based model handled the same task.
- **Options:** up to 255 versus about 20.
- **Reading strength:** a 27B backbone reads text much more deeply.

Laya wins on speed, size and cost. A reasonable split is Laya or a smaller family member for short, high-volume routing, and a larger decoder-based model where inputs are long or option lists are large.

Common misconceptions, cleared up early

Misconception	Reality
"BERT is a classifier."	BERT is an **encoder** that outputs vectors. A classifier is a head added on top. Embedding models are encoders too, trained with a different objective.
"BERT can translate or write."	It cannot write. It reads. Translation needs an encoder–decoder or a decoder.
"`[CLS]` carries the sentence's meaning from the start."	Its *input* is a constant. The meaning is gathered layer by layer, and only because a loss is attached to it. A raw `[CLS]` is a poor summary.
"`[CLS]` is only for BERT."	A dedicated summary token helps in decoders too, but it must sit **last**, not first.
"Laya's `[MASK]` is BERT's `[MASK]`."	BERT's is a blank to guess. Laya's is a **slot** whose output vector is read to score an option.
"The extra layers are a different kind of Transformer."	They are ordinary Transformer layers. What makes them a decision head is what they are trained to do.
"Vega removed blocks from Qwen."	It kept all 64 layers and dropped only the word-prediction output and the vision tower.

Quick reference

```
ENCODER (BERT family): no causal mask → every token sees every token → one vector per token
  input   = token + segment + position embeddings; [CLS] first, [SEP] between sentences
  [CLS]   = a neutral summary slot; useful only because a loss reads it
  pretrain= fill in blanks: 15% chosen → 80% [MASK] / 10% random / 10% unchanged
  finetune= pretrained weights + tiny head, train everything with small learning rates
  ModernBERT = RoPE, GeGLU, pre-norm, alternating local/global attention, 8,192 tokens, 2T tokens

LAYA (encoder design)
  per question: [CLS] <type> question: <instructions> [SEP] [MASK] opt0 [MASK] opt1 ... [SEP] state [SEP]
  encoder → + type vector → 2 head layers → read each [MASK] slot → scorer → logit → softmax(÷T) → probabilities
  421M parameters = encoder 395M + head ~26M;  limits: ~20 options, 512-1,024 tokens, text only

VEGA (decoder design)
  Qwen 27B (64 layers, hybrid attention) + LoRA soup (3.74B params) + 5M-param head
  prompt: state FIRST ... options ... "Decision:"; readout at the last token of each option and of the prompt
  causal mask forces state first and readout last; supports up to 255 options and 32,768 tokens
```

What's next

This chapter took the architecture apart: an encoder with a slot for each option (Laya) and a decoder with a readout at the end of each option (Vega). The next chapter covers what turns such a network into a useful decision model: where the labelled data comes from, how the model is trained to give good probabilities, what training costs, how to keep its confidence honest, and how to test it fairly.

Chapter 27 — Training and Trusting a Decision Model: Data, Calibration and Evaluation

Introduction

Chapter 26 took the architecture apart: an encoder with a `[MASK]` slot for each option (Laya) and a decoder with a readout at the end of each option (Vega). An architecture alone gives random numbers. This chapter is about what turns it into a decision model people can rely on: the **data** it learns from, the **training** that teaches it to give good probabilities, the **cost** of that training, the **calibration** that keeps its confidence honest, and the **evaluation** that tells you whether any of it worked.

Laya is again the main worked example, because its code, documentation and benchmark files are public. Vega and Jev appear where their makers say something about their own data or training.

The parts of this chapter:

1. **Data:** where labelled decisions come from.
2. **Training:** how the model learns to give good probabilities, including the "reinforcement learning" in the name.
3. **What training costs:** and why fine-tuning is so much cheaper than pretraining.
4. **Calibration:** making the probabilities honest.
5. **Keeping calibration alive:** updating it as the world drifts.
6. **Evaluation:** testing honestly.
7. **Limits and open questions.**

> **Sources and numbers.** As in the previous chapter, facts about Laya come from its repository, model cards and documentation, facts about Vega from its model card and code, and the authors' own claims are labelled as such. Small worked examples use illustrative numbers, and some were produced by the short programs in `Chapters-26-27-experiments.js` so that you can rerun them (`node Chapters-26-27-experiments.js all`).

Part 1 — Data: where labelled decisions come from

The problem. To train the model of Chapter 26, Part 2 you need many examples of **a state, a question with its allowed answers, and the correct answer**. Where do thousands of those come from, and what *form* should "the correct answer" take?

1.1 The sources, from most human to least

Source	How it works	Strength	Weakness
Human labels	people answer each question	trustworthy	slow, costly, people disagree
Existing labelled datasets, recast as decisions	an old classification dataset becomes `choice`, `score` or `noul` questions	cheap, real text	the labels were designed for something else
A teacher model	one or more strong LLMs answer each decision, and their answers (averaged) are the label	scalable, gives probabilities	the student cannot beat the teacher's blind spots
Synthetic states	an LLM also *writes* the example tickets and emails	unlimited volume	may not look like real traffic
Real logs with outcomes	production decisions plus what happened next	realistic	needs the outcome to be observed

1.2 What the systems say they do

Laya (documentation and repository):

- Its training mix includes **public labelled datasets recast as decisions**. The benchmark files name AG News, BoolQ, email spam, phishing, RAG passage relevance and a support-triage set as "in training". Each dataset's label set simply becomes the `criteria` text. Training on many datasets with many different label sets is what teaches a model to *read* the options and not memorise a label list. (Related earlier work: "label-agnostic pre-training" for zero-shot text classification.) The author's own account describes the base mix as "100% human-labeled" public datasets with no synthetic labels, augmented by shuffling option order, paraphrasing, alternating raw text and JSON, and adding distractor questions. No dataset names are given, so this is a claim.
- Its **fine-tuning loop learns from a teacher's probabilities**, which its documentation calls `gold`: the probability the teacher gave each option.
- Its fine-tuned checkpoint was trained on the public `typed-decisions` dataset (Part 1.4): 1,200 training cases (6,000 decisions), tested on 400 cases (2,000 decisions) across four workflows: agent-trace observability, customer service, invoice processing and security incidents.

Jev (TypeSafe's launch post and press): the launch post's FAQ heading "Where does our training data come from?" has no answer under it. Wikipedia and TechCrunch report training on synthetic data only. For its *evaluations*, the reference answers are the average of two large models.

Decision 2.0 / Vega: the model card says only that the training data was checked row by row against all the Index test items, a contamination check. A write-up from the same team about a *different* family (Vela, built on Decision 2.0) describes label resampling (options are anonymised, dropped and paraphrased so the model learns to read labels), synthetic rows generated by a Qwen3-30B model and kept only when an independent re-label agreed (71% did), and deduplication against evaluation sets. That is Vela's recipe, not Vega's.

1.3 The form of the label: a distribution, not one answer

Laya's training items come in two shapes:

- `expected`: one hard answer (a choice label, `true` / `false` for a noul, a level for a score; a fractional level like 1.5 splits its weight between levels 1 and 2). A label-smoothing option moves a tenth of each answer's weight to the other options.
- `gold`: the teacher's **probability for every option**, such as `{"billing": 0.9, "technical": 0.1}`.

Why a distribution matters. Suppose two teacher models disagree on a ticket: one says billing, the other says other. The soft target is `[billing 0.5, technical 0, other 0.5]`. The loss is cross-entropy (Chapter 9), computed for three candidate model outputs:

```
model output                          loss vs the soft target    loss vs a hard label "billing"
overconfident [0.98, 0.01, 0.01]              2.31                       0.02   (looks great!)
matches teachers [0.495, 0.01, 0.495]         0.70                       0.70
wrong side [0.02, 0.01, 0.97]                 1.97                       3.91
```

Trained on the hard label, the overconfident model looks nearly perfect. Trained on the soft target, it is punished heavily, because the teachers were *not sure*. A soft label teaches the model **how sure to be**. That is where calibration starts (Part 4).

1.4 The `typed-decisions` dataset

`LocalLLaMA/typed-decisions` on Hugging Face is the dataset Laya's fine-tuning notebook loads. Its card says:

- Apache-2.0 licence, tagged synthetic, and **independent of TypeSafe** (the card says it is not affiliated with the company and does not reproduce Jev). The schema follows the System 1 kinds of question.
- Each case has five typed questions about one piece of state; the test split is 400 cases and 2,000 decisions.
- **Gold is the mean of three samples from a teacher of roughly 4B-class capability** (not named). So a score "measures agreement with that teacher, not correctness". The card gives a missed duplicate invoice as an example of the teacher being wrong.
- Reference points on its test: always answering the common label scores 0.470, a model that perfectly recovered the hidden factors that generated each case scores 0.704, and the **teacher agreeing with itself** (a fresh sample against gold built from the others) scores **0.735**. The card warns that scores well above 0.735 suggest a model is learning the teacher's quirks. Per-question ceilings range from 0.560 (agent-trace urgency) to 0.937 (customer-service category).

1.5 Open datasets on Hugging Face

Laya's authors have not published a dataset repository of their own. Others are open:

Dataset	What it is	Licence	Notes
`LocalLLaMA/typed-decisions`	the benchmark Laya fine-tunes on	Apache-2.0	synthetic; gold from a ~4B teacher
`typesafe/evalsafe-*` (5 sets)	TypeSafe's own evaluation sets (customer service is 204 cases, 3,287 question instances)	not shown in the metadata	"labels are model-generated references"
`ZefanCai/Open-Jev`	typed-decision training data, up to about 160k rows	CC0	mostly synthetic; says it is not official TypeSafe data
`tasksource/tasksource-jev-typed-decisions`	2.5 million typed decisions from 670 sources	"other" (read it)	recast public datasets
`tasksource/procedural-typed-decisions`	procedurally generated problems	Apache-2.0	
`pngwn/system-one-decisions`	synthetic IT tickets	**CC BY-NC 4.0**	non-commercial

Check licences before releasing a fine-tune: the data's licence matters, and one of these is non-commercial.

1.6 Seen versus unseen data

Laya's benchmark file marks each dataset as "in training" or "held out" (not in the training mix):

Task	Accuracy	Status
Email spam	0.993	in training
Phishing	0.993 (multilingual)	in training
Jailbreak guardrails	0.708 to 0.762	**held out**
Moderation (toxicity)	0.530	**held out**

Only the held-out rows tell you how the model handles new data. The authors say so plainly: moderation is barely above chance on a balanced split, and examples picked by hand work while real traffic does not. The same pattern shows in the author's own summary: 83.8% accuracy in-task (on data in the training mix) against 65.1% zero-shot.

1.7 How much data, and what it buys

On the typed-decisions test, Laya's base checkpoint scores **0.362** and the fine-tuned one **0.766** (the authors' figures; the README notes that the fine-tuned row has no committed result file behind it yet). That used 6,000 decisions and took about 4 to 6 minutes on two T4 GPUs (Part 3). Small, well-aimed data makes a very large difference.

A typical data plan. Write the questions and criteria (Chapter 25). Collect real states from the system the model will serve. Get labels from people, a teacher, or several teachers averaged into probabilities. **Hold out two slices before training**: a small slice to fit calibration (Laya uses up to 400 items or 10%), and a test set to judge the result. Fitting the temperature on training items "measures the fit rather than the calibration" (Laya's docs).

Part 2 — Training: how the model learns to give good probabilities

This part is told as one idea in six steps, with a single small example. (Laya's documentation calls its training "reinforcement learning", and the last steps explain what that really means.)

2.1 The idea: a student and an answer key

Imagine a student studying with an **answer key**, but a special key: instead of "the answer is B", it says "70% of experts say B, 20% say A, 10% say C". The student reads a question, gives percentages, compares them with the key, and adjusts to get closer. After thousands of questions, the student's percentages match what the experts would say. That is the whole objective: **the student** is the network, **the answer key** is the teacher's probabilities from Part 1, the comparison is a score called the loss, and "adjusting" is the nudge.

2.2 The dials

For each option the network ends up with one number, a "dial" setting (the technical name is a *logit*). A softmax turns the dials into percentages. A higher dial means a higher percentage.

```
question + options → [the whole network] → one dial per option → softmax → percentages
                                                                          ↓
                                                          compare with the teacher's key
```

2.3 One worked example

Three options. The teacher's key says **70% / 20% / 10%**. The network's dials are `[0.50, 0.40, -0.20]`, which gives **42% / 38% / 21%**. Far from the key. The loss, cross-entropy (Chapter 9), asks "how surprised would the key be by the student's answer?" and here it is **0.966**.

The **nudge** is *key minus student*, option by option: `70−42, 20−38, 10−21` → `+0.28, −0.18, −0.11`. It says: turn the first dial up, the other two down. Following it repeatedly:

```
step 0: percentages 42/38/21   loss 0.966   nudge [+0.28, −0.18, −0.11]
step 1:             63/23/14   loss 0.815   nudge [+0.07, −0.03, −0.04]
step 2:             68/20/12   loss 0.804
step 4:             70/19/11   loss 0.802   nudge ≈ 0
step 8:             70/20/10   loss 0.802   done
```

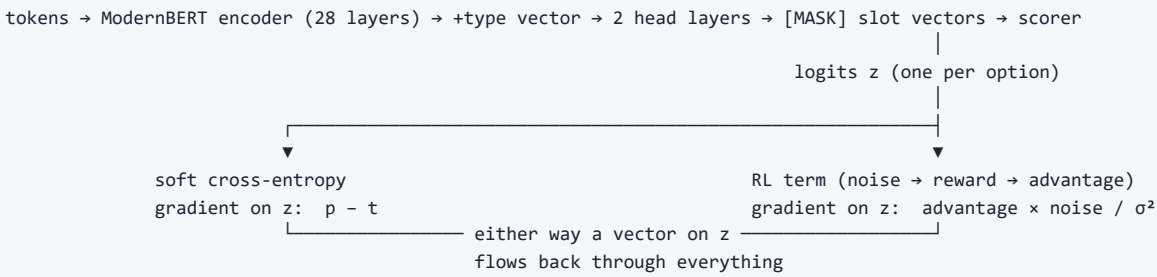
Three things to notice:

1. **The nudge shrinks by itself** as the student gets closer, which is why training settles down.
2. **The loss never reaches zero.** It stops at **0.802**, the teacher's own uncertainty. A key that says "70/20/10" cannot be matched with zero surprise.
3. With a **single hard label** `[1, 0, 0]` from the same start, the student ends at **96% / 2% / 2%**, far too sure. The soft key teaches the student *how unsure to be*.

2.4 How the nudge reaches the transformer

In real training the dials are not turned directly. They come out of the network (the scorer, the two head layers, the 28 encoder layers). So the nudge `+0.28, −0.18, −0.11` is handed **backwards** through the network (backpropagation, Chapter 10; Chapter 26, Part 1.6). Every layer gets a small push so that next time

the dials land a bit closer to the key. Each question gives one small push, and thousands of them make a trained model. The transformer's shape never changes, and only its numbers move.



2.5 Ordered levels (`score` questions)

For ordered levels, cross-entropy only looks at how much probability sits on the *correct* level. Chapter 25, Part 3 showed two answers that both give the right level 0.10, one putting the rest of its weight one level away and the other two levels away: cross-entropy scores both 2.30, while the **ranked probability score** gives 0.90 and 1.17. Laya adds this term for `score` questions only. It is a direct, differentiable score, so no reinforcement learning is involved.

2.6 What the "reinforcement learning" part adds

In plain terms it is a coaching method: **try small random variations of your answer, see which scored better, and move toward those**. Laya's loss also contains a cross-entropy term at full weight. The RL step, in Laya's code, for each training question:

1. **Explore.** Add Gaussian noise to the dials (noise size σ = 0.4 early, 0.1 late), four times. Each noisy version becomes a "reported" distribution.
2. **Reward each report** with a proper scoring rule: log score + 0.75 × spherical score, minus the ranked probability score on `score` questions. The log score is exactly negative cross-entropy.
3. **Advantage** = reward minus the average reward of the group of four, then scaled. This is the group-relative idea of GRPO from Chapter 20, where the "group" is the four noisy attempts at the same input.
4. **Gradient** on the dials = advantage × noise ÷ σ^2. Noise that helped is reinforced; noise that hurt is reversed.
5. **Add soft cross-entropy** at full weight.

(A detail from the community: the code builds the noisy dials from a *detached* copy of the logits but computes the log-probability from the live ones. A user who ported it without the detach found the RL term silently vanished.)

With the same example (key 70/20/10, student 42/38/21):

Try	Resulting answer	Score	Better or worse than the average try?
1	71 / 16 / 13	−0.26	better
2	30 / 47 / 23	−0.73	**worse**
3	42 / 25 / 32	−0.51	slightly worse
4	59 / 31 / 10	−0.30	better

Moving toward the tries that scored better and away from those that scored worse nudges the dials the right way, **about the same way as the plain nudge**. In a simulation of this exact example, the update direction of the plain cross-entropy nudge and the exact reward gradient were nearly identical (cosine 0.999). The four-try RL estimate agreed with the exact gradient on average (cosine 0.83, spread 0.20), reaching 0.99 with 64 tries, and its size grows about fourfold when σ drops from 0.4 to 0.1 because of the $1/\sigma^2$ factor.

Because the score and the key are both known, the best direction can be **calculated directly**: that is cross-entropy's nudge. The random tries are a noisy route to the same place. A third-party A/B test of Laya's own fine-tuning notebook, on its own data and settings (single run, one seed, issue #741), compared the options on about 1,200 test decisions:

```
plain soft cross-entropy only             80.50%
Laya's RL + cross-entropy (notebook)      79.08%
the reward computed directly (no noise)   79.92%
```

On roughly 1,200 questions those gaps are inside the noise (Part 6). The evidence says the RL step is not helping, and Laya's documentation itself offers a plain cross-entropy mode.

When would RL matter? When the score cannot be computed directly: when the cost of acting wrongly versus escalating is a trade-off, or when the true outcome only shows up later (a sale closing weeks afterwards). Laya's helper code contains pieces for both (action costs for the act/escalate head, and a TD(λ) function that credits earlier turns of a conversation with the final outcome), but the public training loop does not use them. Its forward helper throws the act head's output away.

2.7 What a real training run looks like

1. Shuffle the training examples and take a small batch of 8.
2. Run the model on them, get the dials, compute the loss against the teacher keys.
3. Nudge everything backwards. After 64 examples have been seen (the "effective batch of 64"), update the weights.
4. Make four passes over the data (four epochs).
5. Use **two learning rates**: gentle (2.5e-5) for the already-trained encoder and bigger (1e-4) for the brand-new head, like asking an experienced reader to adjust slightly while a new assistant learns quickly. The optimiser is AdamW (Chapter 11) with a cosine schedule and gradient clipping at 1.0.
6. Fit the temperature on held-out examples (Part 4), then test on separate data (Part 6).

2.8 How it evolved

When	What
2025	Reinforcement learning on top of frozen embeddings (the sales model, Chapter 26, Part 2.1)
At release, by the author's account	Pure RL training an end-to-end encoder: REINFORCE with a group-mean baseline over 8 noisy samples (noise 1.0 down to 0.3), reward = log score + 0.5 × spherical − ranked probability score, **no cross-entropy**, plus TD(λ) for multi-turn data and action costs (+1 correct, −3 wrong, −0.5 escalate)
19 to 20 September 2026	Users ask for the original training scripts and how to train
29 September	The independent A/B test above: no measurable gain from the RL term
2 to 7 October	A `laya.train` module merges three drifting copies of the training loop, with CSV loading, a plain `soft-ce` mode, option-order shuffling and abstention-threshold fitting
Now	Notebook default: 4 samples, noise 0.4 to 0.1, spherical weight 0.75, **plus full-weight cross-entropy**

The trend is clear: pure RL on frozen features, then RL plus supervised learning end to end, then community evidence that supervised learning alone is enough. Even the reward function's own defaults (spherical weight 0.5) still carry the original value. (The "original recipe" comes from the author's account, and the repository history available covers only the period from September 2026.)

Vega uses the same family of ideas without RL: a cross-entropy loss with an optional Brier term and a KL term that keeps the model from forgetting (Chapter 26, Part 3.5).

2.9 Which loss, when

The default is **soft cross-entropy on teacher distributions**. For `score` questions, add the ranked-probability term as an ordinary differentiable loss. Reach for RL only when the reward is genuinely non-differentiable or delayed, such as action costs or outcomes observed later.

Part 3 — What training costs

Is fine-tuning still like training a GPT? No. It is the cheapest tier of training there is, closer to teaching a trained employee a new form than to raising and educating one from scratch.

3.1 Three scales side by side

Compute is estimated with the standard rule of about 6 × parameters × tokens. The Laya row rests on an assumed ~250 tokens per training item, so treat it as an estimate.

What	Data processed	Approx. compute	Hardware and time
BERT-base pretraining	1,000,000 steps × 128,000 tokens ≈ 128 billion tokens	~6.5×10^{19} FLOP	16 TPU chips (from the paper)
ModernBERT-large pretraining	2 trillion tokens	~4×10^{21} FLOP	a large GPU cluster
Laya fine-tune (6,000 decisions, 4 epochs)	~6 million tokens (assumed)	~1.3×10^{16} FLOP	**about 4 to 6 minutes on two T4 GPUs** (from Laya's docs)

A Laya fine-tune is roughly **5,000 times** less compute than pretraining BERT-base and about **300,000 times** less than pretraining ModernBERT-large.

Why it is so cheap. It starts from a pretrained model (Chapter 26, Part 1.6); it uses little data; it makes few passes (4 epochs); inputs are short; and the model is small. Training needs about 16 bytes per parameter in mixed precision (a rule of thumb), so 421M parameters is roughly **7 GB** before activations, which fits a 16 GB card. That is what Laya's notebook targets (it runs on Kaggle's free two-T4 tier).

Where cost can still hide.

- **Labels.** Getting good answers for a few thousand examples is often a bigger job than the GPU time.
- **Experiments multiply it.** You will run it more than once (tuning, comparing, recalibrating).
- **Longer inputs cost more.** Cost scales with *tokens*, not examples. Laya's docs give about 4 to 6 minutes for 6,000 decisions but roughly 4 to 5 hours for about 30,000 questions: 5× the data, about 50× the time, which suggests the larger run has much longer inputs (an inference, since the docs do not say).
- **A bigger backbone costs far more** (below).

It is simple to run. Laya's documentation shows the whole thing as one command over a CSV with a text column and a label column:

```
laya-train --data tickets.csv --text-column body --label-column department --base english --out ./tickets-checkpoint
```

There is a `--dry-run` to check your data first, and it runs on CUDA, Apple Silicon (MPS) or CPU. A `--freeze-encoder` option trains only the head and fits a small GPU or CPU, but the docs say it "gains much less" than a full fine-tune.

3.2 What LoRA saves, and what it does not (Vega)

Vega is a LoRA adapter on a full Qwen base, so it is tempting to assume its training is nearly free. It is cheaper than pretraining Qwen, which Vega's team inherits. But:

- LoRA freezes the base and trains small add-on matrices. That reduces **how many numbers get updated**, not **how many get computed**. Every step still runs forward through all 64 layers, and the backward pass still carries gradients through the frozen layers to reach the adapters. It skips only the weight-gradient work for frozen weights, so the cost per token drops from about 6 × parameters to about **4 × parameters** (and back toward 6 × with activation checkpointing). The real saving is **memory**: no gradients or Adam statistics for 27B weights.
- Vega's adapter is **not small** (3.74 billion parameters, 13% of the base), and it is a soup of at least four runs (Chapter 26, Part 3.5).

	Laya full fine-tune	LoRA on the 27B (one rank-128 member)
Compute per token	~2.2 GFLOP	**~103 GFLOP** (about 152 with checkpointing), **46 to 68×** more
Training memory	~7 GB	~51 GB frozen base + ~15 GB for the adapter's training state ≈ **66 GB**, before activations
Hardware	one or two 16 GB GPUs	an 80 GB+ class GPU
Runs	one, maybe a few seeds	at least four (the soup)

For an illustration of time at an assumed 400 effective TFLOPS per GPU (Vega's token budget is not published): 100 million training tokens would take about **7 GPU-hours per member, roughly 29 for a four-member soup**, and a billion tokens about 290. Laya's 6-million-token demo comes to well under a GPU-minute on the same assumption. Hours to days on serious hardware: not free, but nothing like pretraining.

LoRA's economics scale with the base model's size. A 4B base needs about 8 GB for frozen weights and a 9B base about 16 GB, plus the adapter. A smaller base keeps training and serving cheap. A Laya-style encoder is the cheapest of all, at the price of its input and option limits.

Part 4 — Calibration: can you trust the "90%"?

4.1 The weather forecaster

A forecaster says "70% chance of rain" on 100 days. A good forecaster gets rain on about 70 of them. A bad one says 70% and gets rain on 40. Both give a number, but only the first number *means* something.

A decision model is the same. When it says "billing, 90%", is it right about 90 times in 100? That property is **calibration**.

Why you care. The policies of Chapter 25, Part 5 (act above 0.9, ask a bigger model between 0.6 and 0.9, ask a person below 0.6; or act only above 62.5% when a wrong action costs three times a right one) all assume the percentages are honest. If a model says 95% and is right 60% of the time, an automatic rule lets a lot of mistakes through.

4.2 An experiment

A toy model was built to be over-confident, as many are: it exaggerates its certainty. It was given 4,000 test questions with 3 options each, and its answers were grouped by how sure it said it was:

```
BEFORE calibration
  says 33-50% sure   → actually right 37% of the time
  says 50-70% sure   → right 47%
  says 70-90% sure   → right 54%
  says 90-100% sure  → right 71%      ← "I'm certain" was wrong 29 times in 100
```

On average the model claimed 74% confidence but was right only 54% of the time. That gap is the problem.

4.3 The fix: one dial

The raw scores go through a softmax, and a **temperature** divides them first. A bigger temperature flattens the answer:

```
raw scores [2.1, -0.4, -1.3]
  temperature 1.0 → 90%, 7%, 3%      sharp
  temperature 1.5 → 77%, 15%, 8%     softer
```

Two things to notice:

- **The top choice never changes.** In the experiment all 4,000 answers were identical before and after. Only the *confidence* changes, so calibration makes the model **honest, not more accurate**: accuracy stayed at 54.4%.
- **The dial is found by trying values** on examples the model has not trained on, and picking the one that makes it least surprised by the true answers. That is cross-entropy again (Chapter 9). The search found 2.45, and the true value used to build the data was 2.5.

```
AFTER calibration
  says 33-50% sure   → right 44%
  says 50-70% sure   → right 57%
  says 70-90% sure   → right 75%
  says 90-100% sure  → right 100%   (only 15 questions fell in this group)
```

The measure is **ECE** (expected calibration error): group the answers by stated confidence, find the gap between "what it said" and "how often it was right" in each group, and average the gaps, weighted by group size. It dropped from **0.197 to 0.016**.

4.4 What honest numbers buy you

Once the numbers mean what they say, you can choose how much to trust:

```
answer only when confidence ≥ 0.5   → answers 61% of questions, 61% of them right
answer only when confidence ≥ 0.7   → answers 14% of questions, 76% of them right
```

That is a trade-off between **how many** questions you answer and **how often you are right**. (Without calibration, the same "90% or more" rule would have answered 24% of questions at only 71% right.) The toy data is deliberately noisy, so ignore the low overall accuracy: the point is the shape.

4.5 In Laya

All from its own documents and benchmark files:

- **Both published checkpoints ship over-confident.** The English one's ECE is **0.466** raw and **0.081** after temperature fitting (Jev's is reported as 0.246). The multilingual checkpoint ships with no fitted temperatures at all.
- **Its recipe:** hold out a calibration slice before training (up to 400 items or 10%), train, then fit one temperature per question type on that slice (a standard optimiser minimising the negative log-likelihood). The docs recommend going further, one per (question type, option count). The runtime clamps temperatures to 0.5 to 5.0.

- **Why the slice must be unseen:** on items it has already trained on, the model is near-certain and near-correct, so the fit has nothing to correct and returns a meaningless value.
- **Beyond temperature:** Laya's calibration module also fits **per-bucket abstention thresholds** (so that a gate keeps a target error rate in every bucket) and a **histogram-binning map** (a non-parametric correction for buckets whose reliability curve is not a simple softening). A **calibration file** (a small JSON holding the temperatures and any binning map, with no weights) can be installed on a loaded model.
- **Traps:** a stale `temperature_by_options` entry in a copied config silently overrides a new fit; for questions with 11 or more options the shipped bucket acted as a sharpener (a documented issue). There are also **two different "confidence" numbers**: `answer_confidence` is the top probability after temperature, while the plain `confidence` is entropy-based and sits on a different scale. Do not compare both to the same threshold.

4.6 Calibration's blind spot

Laya's own benchmark shows its English checkpoint scoring **0.0% accuracy at 95% confidence** on Khmer text. The model had stepped outside its world and was still sure of itself. Calibration fitted on in-distribution data does not catch this. That is why Laya routes by language **before** the model runs, and does not rely on confidence to notice.

Part 5 — Keeping calibration alive

Models drift, because the world changes. Recalibration is the right first response to drift, because it is cheap: it changes a handful of numbers, not the network, and Laya stores calibration separately from the weights (Chapter 26, Part 2.8).

5.1 How many labelled examples before a refit?

On the toy model (true temperature 2.5), a temperature was fitted repeatedly on calibration sets of different sizes:

Examples	Fitted temperature (typical error)
20	±63%
50	±35%
100	±21%
200	**±16%**
400	±9%
1,000	±6%

So **200 is workable for one temperature and 400 is comfortable**. A real model with more options needs more. Laya's own floors are: 10 examples for a per-type temperature (below that it stays at 1.0), 100 per bucket for abstention thresholds, 200 per bucket for a binning map, and 2,000 per bucket for per-option-count temperatures.

5.2 Two gates

Calibration fixes **honesty**; fine-tuning fixes **accuracy**. So a loop needs two gates:

```
log (input, answer, scores, later outcome)
      │
      ▼
GATE 1, frequent and cheap:     at least 200 new labelled items AND the last 200 look dishonest
      │                          (ECE above a threshold)
      ▼                          refit the temperature → smooth → clamp → deploy the calibration file
GATE 2, rare and expensive:     accuracy over the last ~400 falls below your floor
      │                          EVEN AFTER recalibration (or errors cluster in one slice)
      ▼
queue a fine-tune: batch the corrected examples, train offline, validate, ship a new checkpoint
```

5.3 A simulation with drift

8,000 questions were streamed through a toy model, with the world made harder at question 3,000. Three policies: fit once and never again, refit every 200 labelled items, and refit only when the last 200 looked dishonest (ECE above 0.08).

Scenario A: the world gets noisier.

Policy	Refits	ECE before drift	ECE, first 1,000 after	ECE, late	Accuracy before → late
Fit once	1	0.023	0.118	0.086	55.9% → 47.3%
Refit every 200	40	0.024	0.048	0.011	55.9% → 47.3%
Gated	**17**	0.017	0.051	0.011	55.9% → 47.3%

Scenario B (the model's scores stop tracking reality) has the same shape: the static ECE rose after the drift, both live policies brought it back to about 0.013 to 0.016 (the gated one used 21 refits against 40), and accuracy fell from 55.9% to 52.8% under every policy.

What this shows:

1. Without recalibration, the confidence slowly became dishonest.
2. The gate matched refit-every-200 with roughly half the refits.
3. **Accuracy fell and no policy brought it back.** That drop is what should trigger the second gate and a fine-tune. Calibration restored honesty, not accuracy.

A gate costs little here, because refitting one number is cheap. It matters more when each refit needs real work, such as re-running the model over a buffer (Laya's helper re-runs the model on labelled examples to collect scores, so log the *inputs and outcomes*, not only the answers).

5.4 Smoothing and cautions

Smoothing (a suggestion, not simulated): take the median of the last three fits or a moving average of the logarithm of the temperature, clamp it, and only deploy a new value when it differs from the old by a margin (say 10%), so it does not flicker.

Cautions.

1. **Where do the labels come from?** If only low-confidence items get human review, you only learn the truth about those. Add a **random audit sample** of automatically handled items, or calibration will be biased.
2. **Labels arrive late** (a refund dispute, an outcome weeks later): calibrate on a rolling window of items whose outcome is known.
3. **Calibrate per slice** (language, workflow). One global temperature can hide a slice that is going wrong.
4. **Calibration will not catch out-of-world inputs** (Khmer): route those before the model.
5. **Keep a frozen test set and a rollback**, or recalibration can quietly mask a model that is getting worse. Laya's calibration file records which checkpoint it was fitted for and warns on a mismatch.

What Laya has and lacks. It has the pieces (the function that collects scores from labelled examples, the temperature, threshold and binning fitters, the calibration file, and an audit hook for logging requests). It has no automatic drift monitor or online updater that could be found, so the glue is yours.

Part 6 — Evaluation: testing honestly

6.1 The driving-test idea

A student who practises on last year's exam paper and then sits *this* year's tells you about memory, not skill. A number only means something if you know what the model has seen, what it is compared with, who set the answers, and how much luck is in the result. Five questions, asked of any score:

1. Did it train on this? Laya's benchmark file marks every test "in training" or "held out" (Part 1.6). Spam scores 0.993 (in training); moderation, which the model has never seen, scores 0.530.

2. Compared with what? A number needs a reference point. On the typed-decisions test:

```
random guessing                      0.318
always pick the common answer        0.461
Laya, base checkpoint                0.362     ← below "always pick the common answer"
Laya, fine-tuned (author's figure)   0.766
Jev (published figure)               0.727
```

3. Who decided the right answers? Gold can itself be shaky. Typed-decisions gold is the average of three samples from a ~4B teacher (Part 1.4). One independent test's gold was one human reviewer correcting an AI draft. Another independent comparison used AI-assisted cases with no independent human annotation, and its 520 responses came from just 130 scenarios because variants of the same scenario are correlated.

4. How much of it is luck? Every accuracy number carries a margin of error. If each question were independent, the 95% margin on a 77% score is roughly:

Questions	Margin
100	±8.2 points
400	±4.1
1,200	±2.4
2,000	±1.8

Two examples: **0.766 versus 0.727** on 2,000 decisions is a 3.9-point gap against a margin of about ±2.7 even if every question were independent (they are not: five questions share each case, so the true margin is wider), and the A/B test's **80.5% versus 79.1%** on 1,200 decisions is a 1.4-point gap against a margin of about ±3, within noise. A careful test resamples whole cases together (a clustered bootstrap), corrects for testing many suites, and **declines to claim** a gap that does not survive: one independent test refused to claim a 5.7-point Jev lead on triage (adjusted p = 0.063).

5. Can any metric be gamed? Yes. The typed-decisions card notes that a model that ignores the input and answers the commonest label has the **best ECE** (0.088) while knowing nothing, so read KL and Brier alongside ECE. Look at several measures together: accuracy, Brier or KL, ECE, **accuracy at each coverage level** (Part 4.4), and speed and cost.

6.2 Break it on purpose

Stress test	What the sources show
Reverse the option order	On typed-decisions the base checkpoint changes **35.7%** of its choice answers when the options are reversed. Fine-tuning cuts that to about 6 to 8%. At 20 options: 18.7% base, falling to 1.9 to 2.6% with fine-tuning (and shuffling options during training).
Many options	Banking77 (77 labels): **0.425 against Jev's 0.870**, because the option text runs out of its token budget.
A wrong "reviewer" comment	One independent comparison added a deliberately misleading note to each case. **Laya's accuracy dropped to 19%.**
Unfamiliar language	The English checkpoint scored **0.0% at 95% confidence** on Khmer.
Negation and labels	The model card warns that `noul` can follow the wording of its option labels instead of the state.
Long, noisy text	A user report found near-chance accuracy on long multilingual posts.

6.3 What the independent tests found

Issue #555 (1,240 decisions across 9 suites; the default English Laya checkpoint, version 0.3.11, against Jev 1.13.0 and a small Qwen baseline):

	Laya	Jev	Qwen2.5-1.5B baseline
All suites	**0.686**	**0.907**	0.605
Triage	0.875	0.932	0.781
Moderation	0.757	0.944	0.729
5-level score	0.333	0.650	0.525

Jev led on all nine suites, and eight of the nine gaps held up after correction. Calibration was close for all three on choice questions. The test used the *English* checkpoint on every suite (including the multilingual one), and did not evaluate the fine-tuned checkpoint or the language router.

A comparison of about 16 decision models (Fabio Akita, 10 October 2026): on its compact set, Perplexity's 27B model scored 99.4%, Jev and Microsoft's Decision-1 97.6% each, a stock Qwen 4B with no decision training 74.4%, and a 144M model 31.5% (coin-flip level). Its verdict is that the factory Laya checkpoint is not suited to broad tasks, though it is among the few options that fit on weak devices; fine-tuned Laya was not tested.

A fair reading. These tests used the *base* English checkpoint zero-shot. That is consistent with Laya's own documentation, which says the base checkpoints are weak and that fine-tuning on your domain does most of the work. The honest summary: large decoder-based models and Jev were stronger out of the box, while Laya's strengths are size, speed, open weights, and what fine-tuning can do on your own data. Both tests have limits of their own (AI-assisted labels, correlated cases).

6.4 A checklist for testing a decision model

1. Collect **real examples from the target system**, 200 to 500 of them, with answers checked by a person.
2. Keep them **out of training**.
3. Report per question type and per slice (language, workflow), not only the average.
4. Compare against baselines: always the common answer, and a simple alternative such as embeddings plus similarity.
5. Report calibration **and** accuracy at several coverage levels.
6. Run the stress tests: reorder options, rename them, add distractors, add a misleading comment, try unfamiliar inputs.
7. Measure speed and cost.
8. Repeat the run, or use a clustered bootstrap, before believing a gap of a couple of points.

Laya ships a harness for much of this (`laya-evals`): scoring a labelled set, slice gates, abstention at a threshold, and comparing a run with a saved baseline as a build gate.

Part 7 — Limits and open questions

- **Jev's internals are not public:** architecture, size, weights, training data and recipe. Everything in this chapter about *how* decision models work comes from the open ones.
- **Laya's original training scripts and data were not released** at launch (users asked for them in the first days); a training module and a public fine-tuning dataset have appeared since.
- **Vega's training data is undocumented** on its model card.
- **The act/escalate head does not work yet** in Laya's published checkpoints.
- **Independent evidence is thin.** Two independent tests exist, with the limits listed in Part 6, and none of them tested a Laya checkpoint fine-tuned on the test domain.
- **Order effects, many-option collapse, negation errors, unfamiliar languages and long noisy documents** are all documented weak spots (Part 6.2).
- **Calibration does not travel.** A temperature is right for the data it was fitted on, and needs refitting per setting (Part 5).
- **Priority and provenance claims** (who had the idea first) are claims, and the documents do not settle them.

Common misconceptions, cleared up early

Misconception	Reality
"LoRA makes training nearly free."	It saves **memory**, not compute, and Vega's adapter is 3.74 billion parameters.
"Fine-tuning is like training a GPT."	It is thousands to hundreds of thousands of times less compute than pretraining.
"RL is what makes Laya work."	A third-party test found no measurable gain over plain cross-entropy. Vega uses cross-entropy only.
"Calibration makes the model better at the task."	It makes the confidence honest. It never changes which answer is picked.
"Calibrate once and forget."	The right temperature drifts with the data. Use two gates: cheap recalibration, rare fine-tuning.
"A high benchmark score means it will work for me."	Only if the data was held out and resembles yours, and the gap exceeds the noise.

Quick reference

```
DATA      teacher probabilities (soft labels) teach honesty; hold out a calibration slice and a test set
TRAINING  cross-entropy on soft targets (+ ranked probability score for ordered levels); RL adds nothing measurable
COST      fine-tune ~10^16 FLOP vs pretraining 10^19-10^21; LoRA saves memory, not compute
CALIBRATE temperature fitted on held-out data: confidence honest, answers unchanged; refit via two gates
EVALUATE  held-out data · baselines · who made the gold · margin of error · stress tests · coverage curves
```

What's next

That completes the tour of the model and the systems built around it. Appendix A shows one complete training step of a tiny GPT with every number visible, and the References list the papers, model cards and repositories behind this chapter and the two before it.

Appendix A — TinyGPT by Hand: One Complete Training Step with Real Numbers

Introduction

Throughout this handbook we have described a GPT in words, diagrams and small fragments of arithmetic. This appendix does something different: it runs **one complete training step on a miniature GPT, with every number visible**. We follow a single training sentence from raw text, through the forward pass, to a loss, back through every operation to get gradients, and finally through an AdamW update. Then we run the forward pass a second time and watch the loss go down.

The model is tiny, but it is a *real* GPT in structure. It has token and position embeddings, a causal multi-head attention layer, residual connections, LayerNorm, a feed-forward network, an LM head, a cross-entropy loss, backpropagation and AdamW. Every operation is exactly the one from Chapters 2 to 11, just with 4 numbers per token instead of 4,096.

A few notes on how to read it:

- **Every number was computed by a program, not invented.** Each value comes from the previous one, in the same order a framework like PyTorch would compute them. The tables show numbers rounded to three decimals (the program keeps full precision), so if you redo a step by hand with the rounded numbers, the last digit may differ slightly. The program itself, `Appendix-A-tinygpt.js`, sits next to this file. Run it with `node Appendix-A-tinygpt.js` to print every number again.
- **The gradients were checked.** The backward pass was done with the manual formulas of Chapter 10, and then verified against an independent method: nudging each of the 204 parameters by a tiny amount and measuring how the loss changes. The two agree to about 1 part in 10 million (largest relative difference 4.5e-7).
- **One simplification:** the FFN has a hidden width of 8 (twice the model width) instead of the usual 4× (which would be 16), only so that the matrices fit on a page. Biases are left out, as in many modern models.
- **The starting values are arbitrary.** A real model starts from random numbers, and so does this one. The numbers were chosen by hand or filled in with small one-decimal values. What matters is not the particular values but that every later number follows from them.

Part 1 — The model and its 204 numbers

The configuration

```
Vocabulary size        = 6 words
Embedding dimension    = 4        (call it E)
Attention heads        = 2
Head dimension         = 2        (heads × head dimension = 2 × 2 = 4 = E)
Transformer blocks     = 1        (original "post-norm" order of Chapter 7)
FFN hidden width       = 8
Sequence length        = 3 tokens
```

The vocabulary and the training sentence

ID	Word
0	The
1	cat
2	sat
3	dog
4	ran
5	END

We train on a single sentence, **"The cat sat"**, whose token IDs are `[0, 1, 2]`. Recall from Chapter 9 that one forward pass over this sentence produces *three* training examples at once, one per position, thanks to the causal mask:

Position	The model has seen	It must predict
1	The	cat
2	The cat	sat
3	The cat sat	END

Every learned parameter

Parameter	Shape	Count
Token embedding table	6 × 4	24
Position embedding table	3 × 4	12
W_Q, W_K, W_V for 2 heads (6 matrices)	6 × (4 × 2)	48
W_O	4 × 4	16
LayerNorm 1 (γ and β)	4 + 4	8
FFN W1 (expand)	4 × 8	32
FFN W2 (compress)	8 × 4	32
LayerNorm 2 (γ and β)	4 + 4	8
LM head	4 × 6	24
Total		**204**

Here are all of them. These are the only numbers the model "knows". Everything else in this appendix is computed from them and the input.

Token embedding table (Chapter 3), one row per word:

```
The     0.20   0.60   0.40   0.10
cat     0.70   0.10   0.20   0.80
sat     0.50   0.80   0.90   0.20
dog     0.90   0.20   0.40   0.60
ran     0.30   0.40   0.70   0.10
END     0.10   0.90   0.30   0.50
```

Position embedding table (Chapter 3), one row per position:

```
pos 0   0.10   0.00   0.20   0.10
pos 1   0.00   0.30   0.10   0.00
pos 2   0.20   0.10   0.00   0.40
```

Attention matrices (Chapters 4 and 5), each 4 × 2. Head 1 and head 2 have different matrices, as Chapter 5 explained:

```
W_Q (head 1)          W_K (head 1)          W_V (head 1)
  0.4   0.1             0.3  -0.1             0.5   0.1
  0.3  -0.2             0.2   0.4            -0.2   0.3
 -0.2   0.6            -0.3   0.2             0.4  -0.1
  0.5   0.2             0.1   0.5             0.2   0.6

W_Q (head 2)          W_K (head 2)          W_V (head 2)
 -0.3   0.2             0.4   0.3             0.2  -0.3
  0.5   0.1            -0.1   0.2             0.4   0.5
  0.1  -0.4             0.3  -0.2            -0.1   0.2
  0.2   0.3             0.2   0.1             0.3   0.1
```

Output projection W_O (4 × 4), **LayerNorm** parameters (γ = 1 and β = 0 at the start, for both LayerNorms), and the **FFN** and **LM head** matrices:

```
W_O (4 × 4)
  0.2   0.1  -0.1   0.3
  0.5  -0.2   0.4   0.1
 -0.3   0.6   0.2  -0.2
  0.1   0.2   0.3   0.4

W1 (4 × 8)
  0.1  -0.2   0.1   0.4   0.1   0.1  -0.3   0.3
  0.2   0.3   0.2  -0.1   0.2  -0.2   0.1  -0.2
 -0.2   0.2   0.3   0.3  -0.3   0.0   0.2  -0.1
 -0.2  -0.1   0.3   0.3  -0.1  -0.1   0.3  -0.3

W2 (8 × 4)
 -0.3  -0.1  -0.1   0.0
  0.4   0.0   0.2  -0.2
 -0.1   0.3   0.3  -0.2
  0.0   0.3   0.2  -0.3
  0.1  -0.3   0.2  -0.1
  0.2  -0.2  -0.1  -0.1
 -0.1   0.4  -0.1  -0.4
 -0.2  -0.3   0.1  -0.4

LM head (4 × 6)
 -0.2  -0.2  -0.1  -0.2   0.2   0.0
  0.0   0.3   0.1   0.2  -0.4   0.0
  0.0   0.0   0.1  -0.2   0.0   0.4
 -0.2   0.0   0.3   0.1  -0.4   0.1
```

Part 2 — The forward pass

We follow the order of the architecture diagram from Chapter 8, using the post-norm block of Chapter 7.

Step 1 — Tokenize and look up embeddings (Chapters 2 and 3)

`"The cat sat"` becomes `[0, 1, 2]`. Each ID selects a row of the token table, and each position selects a row of the position table. They are **added**, and the sum is the first real input of the Transformer. From now on the token IDs are gone, and the model only sees vectors:

```
Token    Word embedding              + Position embedding         =  X (input to the block)
The      0.20  0.60  0.40  0.10       0.10  0.00  0.20  0.10       0.30  0.60  0.60  0.20
cat      0.70  0.10  0.20  0.80       0.00  0.30  0.10  0.00       0.70  0.40  0.30  0.80
sat      0.50  0.80  0.90  0.20       0.20  0.10  0.00  0.40       0.70  0.90  0.90  0.60
```

`X` is a 3 × 4 matrix: 3 tokens, 4 numbers each.

Step 2 — Head 1: Q, K and V (Chapter 4)

Each head projects every token through its own three matrices: `Q = X·W_Q`, `K = X·W_K`, `V = X·W_V`. As one worked multiplication, **Q for "The"** in head 1 is the row `[0.30, 0.60, 0.60, 0.20]` times the 4 × 2 matrix `W_Q`:

```
first number:   0.30×0.4 + 0.60×0.3 + 0.60×(−0.2) + 0.20×0.5 = 0.12 + 0.18 − 0.12 + 0.10 = 0.28
second number:  0.30×0.1 + 0.60×(−0.2) + 0.60×0.6 + 0.20×0.2 = 0.03 − 0.12 + 0.36 + 0.04 = 0.31
So Q(The) = [0.28, 0.31]
```

Doing this for every token and all three matrices gives (head 1):

```
Token        Q                 K                 V
The       0.280   0.310     0.050   0.430     0.310   0.270
cat       0.740   0.330     0.280   0.550     0.550   0.640
sat       0.670   0.550     0.180   0.770     0.650   0.610
```

Step 3 — Head 1: scores, causal mask, scaling, softmax

The raw score for query token *i* and key token *j* is the dot product `Q(i)·K(j)`. For example, "cat" asking about "cat" is `0.74×0.28 + 0.33×0.55 = 0.389`. The whole matrix `Q·Kᵀ`:

```
             The      cat      sat
The      0.147    0.249    0.289
cat      0.179    0.389    0.387
sat      0.270    0.490    0.544
```

Now three things happen, in the order a typical implementation does them. **Scale** every score by `1/√2 ≈ 0.707` (the head dimension is 2). **Mask** the future (replace the upper triangle by −∞). Then **softmax** each row:

```
After scaling and masking:
             The      cat      sat
The      0.104     -inf     -inf
cat      0.127    0.275     -inf
sat      0.191    0.347    0.385

After softmax (attention weights, each row adds up to 1):
             The      cat      sat
The      1.000    0.000    0.000
cat      0.463    0.537    0.000
sat      0.296    0.345    0.359
```

Read the "cat" row: when the model builds a new representation for *cat*, it uses **46% of "The"'s information and 54% of "cat"'s own**, and **0% of "sat"**, which is the future. The attention weights are close to uniform because the model has not been trained yet. That will matter in the backward pass.

Step 4 — Head 1: the weighted sum of V

Each token's output is its attention row times the V vectors. For "cat": `0.463×V(The) + 0.537×V(cat)` = `[0.439, 0.469]`.

```
Head 1 output (3 × 2):
The      0.310    0.270
cat      0.439    0.469
sat      0.515    0.520
```

Step 5 — Head 2, concatenation and W_O (Chapter 5)

Head 2 repeats Steps 2 to 4 with its own matrices. Its queries, keys and values are:

```
Token        Q                  K                  V
The       0.310  -0.060      0.280   0.110      0.300   0.350
cat       0.180   0.300      0.490   0.310      0.510   0.130
sat       0.450   0.050      0.580   0.270      0.590   0.480
```

After the same scale, mask and softmax steps, its attention weights and outputs are:

```
Head 2 attention weights:                    Head 2 output (3 × 2):
              The      cat      sat
The      1.000    0.000    0.000            0.300    0.350
cat      0.483    0.517    0.000            0.409    0.236
sat      0.314    0.338    0.348            0.472    0.321
```

The two heads' outputs are placed side by side (2 + 2 = 4 numbers per token), and `W_O` lets the heads mix:

```
Concatenated C (3 × 4):                      After W_O (Attn):
The     0.310   0.270   0.300   0.350          0.142   0.227   0.242   0.200
cat     0.439   0.469   0.409   0.236          0.223   0.243   0.296   0.191
sat     0.515   0.520   0.472   0.321          0.253   0.295   0.347   0.240
```

Step 6 — Residual connection and LayerNorm 1 (Chapter 7)

The attention output is **added back** to the block's input (the residual connection), and then each row is **normalized**:

```
R1 = X + Attn                                H1 = LayerNorm(R1)
The     0.442   0.827   0.842   0.400        -0.896   0.961   1.033  -1.098
cat     0.923   0.643   0.596   0.991         0.787  -0.850  -1.121   1.184
sat     0.953   1.195   1.247   0.840        -0.628   0.809   1.120  -1.301
```

A worked LayerNorm for the row of "The": the mean of `[0.442, 0.827, 0.842, 0.400]` is 0.6278, the variance is 0.0430, so the standard deviation is 0.2074 (a tiny $\varepsilon = 10^{-5}$ is added inside the square root). Each number becomes `(x - 0.628) / 0.207`, giving `[-0.896, 0.961, 1.033, -1.098]`, which has mean 0 and standard deviation 1. With $\gamma = 1$ and $\beta = 0$, that is the output.

Step 7 — The feed-forward network (Chapter 6)

Each token is processed **independently** (no mixing between tokens here). Expand 4 → 8 with `W1`, apply GELU, and compress 8 → 4 with `W2`:

```
Z = H1·W1  (3 × 8)
  0.116   0.784   0.083  -0.474  -0.098  -0.172   0.242  -0.235
 -0.104  -0.755  -0.072   0.419   0.126   0.130  -0.190   0.163
  0.135   0.723   0.045  -0.387  -0.107  -0.095   0.103  -0.072

G = GELU(Z)   (for example GELU(0.784) = 0.614)
  0.063   0.614   0.044  -0.151  -0.045  -0.074   0.144  -0.096
 -0.048  -0.170  -0.034   0.277   0.070   0.072  -0.081   0.092
  0.075   0.553   0.023  -0.135  -0.049  -0.044   0.056  -0.034

F = G·W2  (3 × 4)
  0.208   0.076   0.074  -0.094
 -0.039  -0.017   0.040  -0.061
  0.184   0.015   0.069  -0.074
```

Step 8 — Residual connection and LayerNorm 2

```
R2 = H1 + F                                  H2 = LayerNorm(R2)
The    -0.688   1.037   1.107  -1.192          -0.738   0.950   1.019  -1.231
cat     0.747  -0.867  -1.081   1.123           0.793  -0.877  -1.098   1.182
sat    -0.444   0.824   1.189  -1.376          -0.482   0.759   1.117  -1.394
```

`H2` is the block's output: one **contextual embedding** (Chapter 8) per token. It is a 3 × 4 matrix, the same shape as the block's input, which is what lets blocks be stacked.

Step 9 — LM head, softmax and the loss (Chapters 8 and 9)

The LM head turns each 4-number vector into 6 scores (**logits**), one per word, and softmax turns each row into probabilities:

```
Logits (3 × 6):
          The     cat     sat     dog     ran     END
The     0.394   0.433  -0.099   0.011  -0.035   0.284
cat    -0.395  -0.422   0.078   0.004   0.037  -0.321
sat     0.375   0.324  -0.182  -0.114   0.157   0.307

Probabilities (softmax):
          The     cat     sat     dog     ran     END
The     0.205   0.213   0.125   0.140   0.133   0.184
cat     0.130   0.127   0.209   0.194   0.200   0.140
sat     0.205   0.195   0.117   0.126   0.165   0.192
```

The untrained model is nearly clueless: every word gets roughly 1/6 ≈ 0.167 of the probability. Now compare each row with its target, using cross-entropy `-log(probability of the correct word)`:

Position	Target	Probability given to it	Loss = −log(p)
1 (after "The")	cat	0.213	1.546
2 (after "The cat")	sat	0.209	1.567
3 (after "The cat sat")	END	0.192	1.652
Average			**1.588**

A sanity check: a model that guessed uniformly among 6 words would have loss `ln 6 = 1.792`. Ours is 1.588, about the same. This single number, **1.588**, is what the backward pass starts from.

The same pass, as tensor shapes

Real implementations carry an extra *batch* dimension `B` (here `B = 1`). Here is the pass above written the way framework code writes it, with `T = 3` tokens, `E = 4`, `H = 2` heads and head dimension `D = 2`:

Step	Shape	Our numbers
Input embeddings `X`	`(B, T, E)`	(1, 3, 4)
After `X·W_Q` (all heads at once)	`(B, T, H·D)`	(1, 3, 4)
Split into heads and move heads forward	`(B, H, T, D)`	(1, 2, 3, 2)
Attention scores `Q·Kᵀ`	`(B, H, T, T)`	(1, 2, 3, 3)
After softmax	`(B, H, T, T)`	(1, 2, 3, 3)
Weighted sum with `V`	`(B, H, T, D)`	(1, 2, 3, 2)
Concatenate heads	`(B, T, E)`	(1, 3, 4)
After `W_O`, residual, LayerNorm	`(B, T, E)`	(1, 3, 4)
FFN expand / compress	`(B, T, 8)` / `(B, T, E)`	(1, 3, 8) / (1, 3, 4)
Logits	`(B, T, vocab)`	(1, 3, 6)

The rule that makes the block stackable: **it takes `(B, T, E)` and returns `(B, T, E)`**. A line of real code such as `q = self.wq(x).view(B, T, n_heads, head_dim).transpose(1, 2)` now reads as a sentence: *"project the tokens into queries, then split the embedding dimension into heads and move the head dimension forward."*

Part 3 — The backward pass

Now everything runs in reverse (Chapter 10). Remember what flows backward: not the loss itself, but the **gradient of the loss** with respect to each intermediate value. Every operation does two jobs: it computes the gradient for its own **parameters** (if it has any), and the gradient for its **inputs**, which it hands to the operation before it.

Step B1 — Where it starts: the gradient of the logits

For softmax followed by cross-entropy, the gradient of the loss with respect to the logits is wonderfully simple: **the predicted probabilities minus the one-hot target**, divided by the number of positions (3), because the loss is an average. For position 1, whose target is *cat*:

```
probabilities:   [0.205, 0.213, 0.125, 0.140, 0.133, 0.184]
minus one-hot:   [0, 1, 0, 0, 0, 0]
divided by 3  →  [0.0683, -0.2623, 0.0417, 0.0466, 0.0445, 0.0612]
```

The correct word gets a **negative** gradient ("raise this logit") and every other word a small positive one ("lower these"). For all three positions:

```
d(loss)/d(logits):
          The      cat      sat      dog      ran      END
The     0.0683  -0.2623   0.0417   0.0466   0.0445   0.0612
cat     0.0434   0.0422  -0.2637   0.0646   0.0668   0.0467
sat     0.0684   0.0650   0.0392   0.0419   0.0550  -0.2694
```

Each row adds up to zero, as it must.

Step B2 — The LM head

The LM head computed `logits = H2 · W_head`. For a matrix multiplication, the gradient of the weights is `H2ᵀ · d(logits)` and the gradient passed back to its input is `d(logits) · W_headᵀ`. As one worked entry, the gradient of `W_head[0][1]` (the weight from the first embedding number to the *cat* logit) is the sum over the three positions of *(that embedding number) × (that position's logit gradient for cat)*:

```
dW_head[0][1] = -0.738 × -0.2623  +  0.793 × 0.0422  +  -0.482 × 0.0650  =  0.1957
```

The full gradient matrix for the LM head:

```
dW_head (4 × 6):
          The      cat      sat      dog      ran      END
e1     -0.0490   0.1957  -0.2589  -0.0033  -0.0064   0.1218
e2      0.0788  -0.2369   0.3007   0.0194   0.0254  -0.1874
e3      0.0983  -0.2410   0.3758   0.0233   0.0334  -0.2898
e4     -0.1281   0.2822  -0.4176  -0.0394  -0.0525   0.3554
```

Steps B3 to B7 — back through the block

The gradient now travels back through the block in the exact reverse order of the forward pass. Each step uses a rule you already know:

Backward step	What it does
LayerNorm 2	Computes gradients for γ_2 and β_2, and passes a gradient to `R2`. (The formula is longer than for a plain multiplication, because every number in a row affects that row's mean and variance, and so every other number.)
Residual connection	An addition **copies** the incoming gradient to both of its inputs: one copy goes down into the FFN, and one goes straight to `H1`. This "shortcut" is why gradients reach the early layers so easily.
FFN, `W2`	`dW2 = Gᵀ · dF`; the gradient passed back is `dF · W2ᵀ`.
GELU	Multiplies the gradient by GELU's slope at each `Z` value (a fixed function with no parameters, Chapter 6).
FFN, `W1`	`dW1 = H1ᵀ · dZ`; the gradient to `H1` is `dZ · W1ᵀ`, **added** to the shortcut copy from the residual.
LayerNorm 1, then residual	Same as before: gradients for γ_1 and β_1, then the gradient is copied to both the attention path and the shortcut to `X`.

Step B8 — W_O and the attention heads

`dW_O = Cᵀ · dAttn`, and the gradient passed back, `dC`, is split into one piece per head. Inside each head, the weighted sum `O = A·V` sends one gradient to `V` (and to `W_V`) and one to the attention weights `A`. The gradient on `A` then passes backward through **softmax** (each row: `dS = A ⊙ (dA − Σ A·dA)`), through the **scaling** by 1/√2, and into `Q·Kᵀ`, which splits into gradients for `Q` and `K` and, from there, for `W_Q` and `W_K`. The masked (future) positions had attention weight exactly 0, so they receive exactly zero gradient.

Step B9 — The embeddings

The input `X` was the **sum** of a token embedding and a position embedding, and the gradient of a sum goes to both parts. Each token's row is collected into the embedding table. Notice what that means: **only the rows for words that were actually used get a gradient.** "The", "cat" and "sat" receive gradients. **"dog" and "ran" receive exactly zero**, because they were never looked up in this sentence. Position rows 0, 1 and 2 each receive a gradient.

What the gradients look like

Every one of the 204 parameters now has a gradient. A summary of how big they are (the overall size, or norm, of each gradient matrix):

Parameter	Size of its gradient
LM head	1.00e+0
Token embeddings	7.54e−1
Position embeddings	7.54e−1
W_O	5.68e−1
LayerNorm 2 β / γ	2.20e−1 / 1.07e−1
W_V, head 2	1.63e−1
W_V, head 1	1.20e−1
LayerNorm 1 β / γ	1.31e−1 / 2.98e−2
FFN W1	5.32e−2
FFN W2	2.54e−2
W_Q, head 2	3.12e−3
W_K, head 1	2.92e−3
W_Q, head 1	1.92e−3
W_K, head 2	1.85e−3

(The token and position embeddings have identical norms because each word appears exactly once in this sentence, so each position's gradient flows into exactly one token row.)

Two things stand out. The gradients are largest **near the loss** (the LM head) and smaller further back, which is the usual picture. And the gradients for `W_Q` and `W_K` are **tiny**, hundreds of times smaller than the LM head's. That is not a bug: because the untrained attention weights are nearly uniform (Step 3), changing the

query–key scores barely changes the outputs yet, so the loss is not sensitive to them. They will matter once the model is a little less clueless. This is a nice example of why **AdamW**, which adapts the step to each parameter's own gradient size, is useful, as the next part shows.

The gradient check

To make sure none of this is wrong, the program also measured each parameter's effect on the loss by brute force: nudge the parameter up by 0.00001, then down by 0.00001, recompute the loss both times, and take the slope. The slope from backpropagation and the slope from this brute-force method **agree for all 204 parameters**, with a largest relative difference of 4.5e-7. That is the whole content of "backpropagation computes exact gradients, only much faster".

Part 4 — The AdamW update

We now have a gradient for every parameter, and we apply one **AdamW** step (Chapter 11) to all of them. The settings:

```
learning rate  η  = 0.05       (large, so the effect is visible in one step)
β₁ = 0.9      β₂ = 0.999    ε = 10⁻⁸
weight decay   λ  = 0.01
```

For each parameter, with gradient `g`, and `m` and `v` both starting at 0:

```
m  = β₁·m + (1 − β₁)·g              (momentum: smoothed gradient)
v  = β₂·v + (1 − β₂)·g²             (smoothed squared gradient)
m̂  = m / (1 − β₁ᵗ)     v̂ = v / (1 − β₂ᵗ)   (bias correction, t = step number)
w ← w − η · m̂ / (√v̂ + ε)  −  η·λ·w          (the step, plus weight decay)
```

The first step has a special shape

On the very first step, `m = 0.1·g` and `v = 0.001·g²`. After bias correction, `m̂ = g` and `v̂ = g²`, so `m̂ / √v̂ = g / |g|`, which is just **+1 or −1**. So on step 1, **every parameter moves by about η = 0.05, in the direction opposite to its gradient's sign, no matter how large or small the gradient is.** Three examples:

Parameter	Old value	Gradient	m̂	v̂	New value	Change
`W_head[0][1]`	-0.2000	1.96e-1	1.96e-1	3.83e-2	-0.2499	-0.0499
`W_Q head 1 [0][0]`	0.4000	5.07e-4	5.07e-4	2.57e-7	0.3498	-0.0502
`embedding of 'dog' [0]`	0.9000	0.00e+0	0.00e+0	0.00e+0	0.8996	-0.0004

- `W_head[0][1]` has a large gradient (0.196) and moves by about 0.050.
- `W_Q` **of head 1** has a gradient several hundred times smaller (5.1e-4), yet it also moves by about 0.050. That is Adam's per-parameter scaling (dividing by √v̂) at work. Plain gradient descent would barely have touched it.
- **The embedding of "dog"** has a gradient of exactly 0, so `m̂ = 0` and the Adam part contributes nothing. It only shrinks slightly through weight decay (0.9000 to 0.8996). A word that doesn't appear in the data

isn't learned.

(Real training doesn't take such a bold first step. Chapter 11 mentioned that the learning rate usually starts small and "warms up".)

Part 5 — Did it learn? The second forward pass

We now run the forward pass again with the updated parameters, on the same sentence:

Position	Target	P(target) before	P(target) after	Loss before	Loss after
1 (after "The")	cat	0.213	0.302	1.546	1.197
2 (after "The cat")	sat	0.209	0.276	1.567	1.289
3 (after "The cat sat")	END	0.192	0.289	1.652	1.243
Average loss				**1.588**	**1.243**

One update, using nothing but the arithmetic above, moved the loss from **1.588** to **1.243**, and each of the three target words became more probable. Repeat this for millions of sentences, and the random matrices become a language model.

Don't over-read the size of the drop: we trained on a single sentence with a deliberately large learning rate, so the model is memorizing it. The point is the *mechanism*. Every step in the loop is exactly what happens at full scale.

Part 6 — What TinyGPT teaches

What is random, and what is mathematics?

The first question many people have is whether a GPT is "magically" learning from random numbers. Now we can answer it precisely. The only random things are the **starting parameters**: the token and position embeddings, `W_Q`, `W_K`, `W_V`, `W_O`, `W1`, `W2`, the LM head, and (in a real model) the LayerNorm parameters, plus things like the order of the training data. Everything else is **deterministic mathematics**: embedding lookup, matrix multiplication, dot products, scaling, masking, softmax, weighted sums, residual adds, LayerNorm, GELU, cross-entropy, backpropagation and AdamW. There is no hidden step.

Where each step is explained

In this appendix	Chapter
Tokenizing	2
Embeddings and position embeddings	3
Q, K, V, scores, the causal mask, softmax	4
Two heads, concatenation, `W_O`	5
FFN and GELU	6
Residuals and LayerNorm; the block	7
Stacking, the LM head, logits	8
Next-token targets and cross-entropy	9
The backward pass	10
AdamW	11

TinyGPT versus a real model

	TinyGPT	A modern LLM
Parameters	204	billions
Embedding width	4	4,096 or more
Blocks	1	30 to 100+
Heads	2, each of size 2	32 or more, each of size 128
FFN hidden width	8	about 4× the width (or the gated SwiGLU form, Chapter 16)
Vocabulary	6 words	30,000 to 250,000 tokens
Context	3 tokens	thousands to a million
Batch	1 sentence	thousands of sequences
Position information	learned table	usually rotary (RoPE, Chapter 16)
Norm placement	after (post-norm)	before (pre-norm), usually RMSNorm
Attention	full multi-head	GQA, MLA, sliding window and more (Chapter 17)

Everything in the right-hand column is a change to *one* part of the left-hand model. The overall loop is the same: **embed, attend, think, repeat, predict, measure the error, send it back, nudge every weight.**

The sources behind the whole handbook (papers, documentation and the study material it was built on) are listed in **References**, the next section.

References and Sources

Created by Anand (LinkedIn · GitHub). Drafted with AI assistance (ChatGPT, Claude) and reviewed by the author. Version 1.0, October 2026. Errors and suggestions are welcome as GitHub issues.

Licence: the text is released under CC BY 4.0 and the code (`Appendix-A-tinygpt.js`) under the MIT licence.

This section lists where the handbook's material comes from: the learning material it was built on, the research papers it draws on, and the documentation, model cards and websites that were consulted. It is organized by topic, and each entry says which chapters use it.

How to read this list

- **Papers** are cited by first author, year, title and arXiv identifier (links go to the arXiv abstract page). Every identifier was confirmed to resolve to the stated paper, either through arXiv's own catalogue or by opening its abstract page.
- **Documentation, model cards and websites** were opened and read in the course of writing, and are listed with their addresses. Documentation changes frequently, so details such as model names, defaults and prices may have moved by the time you read this. The date of the last check was **October 2026**.
- Entries marked **(search result only)** were seen as a headline or summary in a web search but not opened in full, so they are weaker evidence than the rest.
- A few explanations in the handbook come from general background knowledge and are not tied to one source. They are noted at the end, in "Notes on verification".

---

1. The learning backbone and the source conversation

The handbook grew out of a guided study session built around one lecture series. Two items sit underneath everything:

- **The YouTube lecture series *Building LLMs from scratch*, by the channel Vizuara**, which provided the order of topics for Chapters 2 to 11 (tokenization, embeddings, attention, the Transformer block, training). Many thanks to the Vizuara team for making it freely available. If you want to follow along with the original lectures, start here.
  - **First lecture:** *Lecture 1: Building LLMs from scratch: Series introduction*: https://youtu.be/Xpr8D6LeAtw
  - **Playlist:** *Building LLMs from scratch*: https://youtube.com/playlist?list=PLPTV0NXA_ZSgsLAr8YCgCwhPIJNNtexWu
  - The handbook does not reproduce the lectures. It uses their sequence as a skeleton, and expands or reorders where more explanation was useful.
- **The source conversation:** a ChatGPT tutoring conversation titled *"LLM building overview"*, which walked through the lecture topics and went on to cover fine-tuning, quantization, serving, MoE, reasoning and multimodal models. The explanations, worked examples and the questions asked along the way come from it, reorganized and corrected where needed. Chapters 1 to 15 and 18 to 20 draw on it heavily,

Chapters 22 and 23 only lightly (the conversation covered those topics briefly), and Chapters 16, 17, 21, 24, 25, 26 and 27 were written mainly from the outside sources listed below. Chapters 25 to 27 also grew out of a follow-up study session and were then checked against the primary sources in Section 14. Address: https://chatgpt.com/share/6a83f0ae-b5d8-83e8-aba9-4d63e88465db
    - The web citations that appeared inside that conversation were not preserved in the exported copy, so they cannot be listed here. Where a claim needed support, it was re-checked against the sources below.
- **No other YouTube playlists or videos** were searched or used.

---

2. Foundations: the Transformer, tokenization, normalization, optimization (Chapters 2 to 11)

- Vaswani et al. (2017). *Attention Is All You Need.* arXiv:1706.03762. Chapters 4, 5, 7, 15.
- Sennrich et al. (2015). *Neural Machine Translation of Rare Words with Subword Units.* arXiv:1508.07909. Byte-pair encoding. Chapter 2.
- Ba et al. (2016). *Layer Normalization.* arXiv:1607.06450. Chapter 7.
- He et al. (2015). *Deep Residual Learning for Image Recognition.* arXiv:1512.03385. Residual connections. Chapter 7.
- Hendrycks et al. (2016). *Gaussian Error Linear Units (GELUs).* arXiv:1606.08415. Chapter 6.
- Kingma et al. (2014). *Adam: A Method for Stochastic Optimization.* arXiv:1412.6980. Chapter 11.
- Loshchilov et al. (2017). *Decoupled Weight Decay Regularization.* arXiv:1711.05101. AdamW. Chapter 11 and Appendix A.
- Brown et al. (2020). *Language Models are Few-Shot Learners.* arXiv:2005.14165. GPT-3. Chapters 5, 7.

3. The Transformer family (Chapter 15)

- Devlin et al. (2018). *BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding.* arXiv:1810.04805. Chapters 15 and 26.
- Raffel et al. (2019). *Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer.* arXiv:1910.10683. T5.
- Lewis et al. (2019). *BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension.* arXiv:1910.13461.
- Liu et al. (2019). *RoBERTa: A Robustly Optimized BERT Pretraining Approach.* arXiv:1907.11692.
- He et al. (2020). *DeBERTa: Decoding-enhanced BERT with Disentangled Attention.* arXiv:2006.03654.
- Reimers et al. (2019). *Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks.* arXiv:1908.10084. Embedding models; also Chapters 24 (RAG) and 26.
- Radford et al. (2022). *Robust Speech Recognition via Large-Scale Weak Supervision.* arXiv:2212.04356. Whisper, an encoder–decoder model. Chapters 15, 19.

4. Inference: attention cost, the KV cache and long context (Chapters 12 and 17)

- Dao et al. (2022). *FlashAttention: Fast and Memory-Efficient Exact Attention with IO-Awareness.* arXiv:2205.14135. Chapter 17.
- Dao (2023). *FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning.* arXiv:2307.08691. Chapter 17.
- Shah et al. (2024). *FlashAttention-3: Fast and Accurate Attention with Asynchrony and Low-precision.* arXiv:2407.08608. Chapter 17.
- Liu et al. (2023). *Lost in the Middle: How Language Models Use Long Contexts.* arXiv:2307.03172. Chapter 24 (context placement).

5. Fine-tuning and alignment (Chapter 13)

- Ouyang et al. (2022). *Training language models to follow instructions with human feedback.* arXiv:2203.02155. InstructGPT; the RLHF pipeline.
- Schulman et al. (2017). *Proximal Policy Optimization Algorithms.* arXiv:1707.06347. PPO.
- Rafailov et al. (2023). *Direct Preference Optimization: Your Language Model is Secretly a Reward Model.* arXiv:2305.18290. DPO.
- Hu et al. (2021). *LoRA: Low-Rank Adaptation of Large Language Models.* arXiv:2106.09685.
- Dettmers et al. (2023). *QLoRA: Efficient Finetuning of Quantized LLMs.* arXiv:2305.14314. Also Chapter 14.

6. Quantization (Chapter 14)

- Dettmers et al. (2022). *LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale.* arXiv:2208.07339. Outliers in activations.
- Frantar et al. (2022). *GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers.* arXiv:2210.17323.
- Lin et al. (2023). *AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration.* arXiv:2306.00978.
- The GGUF file format and the llama.cpp quantization formats are covered in the documentation listed in Section 12.

7. Modern building blocks and attention variants (Chapters 16 and 17)

- Su et al. (2021). *RoFormer: Enhanced Transformer with Rotary Position Embedding.* arXiv:2104.09864. RoPE.
- Peng et al. (2023). *YaRN: Efficient Context Window Extension of Large Language Models.* arXiv:2309.00071.
- Zhang et al. (2019). *Root Mean Square Layer Normalization.* arXiv:1910.07467. RMSNorm.
- Shazeer (2020). *GLU Variants Improve Transformer.* arXiv:2002.05202. SwiGLU.
- Henry et al. (2020). *Query-Key Normalization for Transformers.* arXiv:2010.04245. QK-Norm.
- Shazeer (2019). *Fast Transformer Decoding: One Write-Head is All You Need.* arXiv:1911.02150. Multi-query attention.

- Ainslie et al. (2023). *GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints.* arXiv:2305.13245.
- Chowdhery et al. (2022). *PaLM: Scaling Language Modeling with Pathways.* arXiv:2204.02311. A model using multi-query attention.
- DeepSeek-AI (2024). *DeepSeek-V2: A Strong, Economical, and Efficient Mixture-of-Experts Language Model.* arXiv:2405.04434. Multi-head latent attention (MLA).
- Jiang et al. (2023). *Mistral 7B.* arXiv:2310.06825. Sliding-window attention.
- Beltagy et al. (2020). *Longformer: The Long-Document Transformer.* arXiv:2004.05150.
- Gu et al. (2023). *Mamba: Linear-Time Sequence Modeling with Selective State Spaces.* arXiv:2312.00752.
- Yang et al. (2024). *Gated Delta Networks: Improving Mamba2 with Delta Rule.* arXiv:2412.06464. Gated DeltaNet, used in the hybrid Qwen models.
- DeepSeek-AI (2025). *DeepSeek-V3.2: Pushing the Frontier of Open Large Language Models.* arXiv:2512.02556. DeepSeek Sparse Attention. Also Chapter 21.

8. Mixture of Experts (Chapter 18)

- Lepikhin et al. (2020). *GShard: Scaling Giant Models with Conditional Computation and Automatic Sharding.* arXiv:2006.16668.
- Fedus et al. (2021). *Switch Transformers: Scaling to Trillion Parameter Models with Simple and Efficient Sparsity.* arXiv:2101.03961. Top-1 routing and the load-balancing loss.
- Jiang et al. (2024). *Mixtral of Experts.* arXiv:2401.04088.
- Dai et al. (2024). *DeepSeekMoE: Towards Ultimate Expert Specialization in Mixture-of-Experts Language Models.* arXiv:2401.06066. Fine-grained and shared experts.
- DeepSeek-AI (2024). *DeepSeek-V3 Technical Report.* arXiv:2412.19437. Also Chapters 17 and 21.

9. Multimodal models (Chapter 19)

- Dosovitskiy et al. (2020). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale.* arXiv:2010.11929. Vision Transformer.
- Radford et al. (2021). *Learning Transferable Visual Models From Natural Language Supervision.* arXiv:2103.00020. CLIP.
- Alayrac et al. (2022). *Flamingo: a Visual Language Model for Few-Shot Learning.* arXiv:2204.14198.
- Liu et al. (2023). *Visual Instruction Tuning.* arXiv:2304.08485. LLaVA.
- Radford et al. (2022). Whisper, listed in Section 3, for audio encoders.

10. Reasoning models (Chapter 20)

- Wei et al. (2022). *Chain-of-Thought Prompting Elicits Reasoning in Large Language Models.* arXiv:2201.11903.
- Kojima et al. (2022). *Large Language Models are Zero-Shot Reasoners.* arXiv:2205.11916.
- Wang et al. (2022). *Self-Consistency Improves Chain of Thought Reasoning in Language Models.* arXiv:2203.11171.

- Yao et al. (2023). *Tree of Thoughts: Deliberate Problem Solving with Large Language Models.* arXiv:2305.10601.
- Lightman et al. (2023). *Let's Verify Step by Step.* arXiv:2305.20050. Outcome vs process supervision.
- Snell et al. (2024). *Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters.* arXiv:2408.03314.
- Shao et al. (2024). *DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models.* arXiv:2402.03300. GRPO.
- DeepSeek-AI (2025). *DeepSeek-R1: Incentivizing Reasoning Capability in LLMs via Reinforcement Learning.* arXiv:2501.12948.
- Muennighoff et al. (2025). *s1: Simple test-time scaling.* arXiv:2501.19393. Budget forcing.
- Aggarwal et al. (2025). *L1: Controlling How Long A Reasoning Model Thinks With Reinforcement Learning.* arXiv:2503.04697. Length-controlled policy optimization.
- Sun et al. (2025). *BudgetThinker: Empowering Budget-aware LLM Reasoning with Control Tokens.* arXiv:2508.17196. Seen in search results; abstract-level.
- Yao et al. (2022). *ReAct: Synergizing Reasoning and Acting in Language Models.* arXiv:2210.03629. Also Chapter 24.

11. Serving, speed and scale (Chapters 22 and 23)

- Kwon et al. (2023). *Efficient Memory Management for Large Language Model Serving with PagedAttention.* arXiv:2309.06180. vLLM. The 20.4% to 38.2%, 800 KB-per-token and 2 to 4× figures come from this paper.
- Leviathan et al. (2022). *Fast Inference from Transformers via Speculative Decoding.* arXiv:2211.17192.
- Chen et al. (2023). *Accelerating Large Language Model Decoding with Speculative Sampling.* arXiv:2302.01318.
- Cai et al. (2024). *Medusa: Simple LLM Inference Acceleration Framework with Multiple Decoding Heads.* arXiv:2401.10774.
- Li et al. (2024). *EAGLE: Speculative Sampling Requires Rethinking Feature Uncertainty.* arXiv:2401.15077.
- Shoeybi et al. (2019). *Megatron-LM: Training Multi-Billion Parameter Language Models Using Model Parallelism.* arXiv:1909.08053. Tensor parallelism.
- Huang et al. (2018). *GPipe: Efficient Training of Giant Neural Networks using Pipeline Parallelism.* arXiv:1811.06965. Pipeline parallelism.

12. Documentation, model cards and websites

Serving software and file formats (Chapter 22)

- GGUF file-format specification (ggml project): https://github.com/ggml-org/ggml/blob/master/docs/gguf.md
- llama.cpp server documentation (parallel slots, continuous batching, mmap, offload): https://github.com/ggml-org/llama.cpp/blob/master/tools/server/README.md
- vLLM design documents: PagedAttention https://docs.vllm.ai/en/latest/design/paged_attention.html and automatic prefix caching https://docs.vllm.ai/en/latest/design/prefix_caching.html. The first is marked by its authors as a historical description of the original paper.

Hardware (Chapter 23)

- NVIDIA H100 product page (memory, bandwidth, NVLink, PCIe figures): https://www.nvidia.com/en-us/data-center/h100/
- Google, *Our eighth generation TPUs: two chips for the agentic era* (TPU 8t and TPU 8i): https://blog.google/innovation-and-ai/infrastructure-and-cloud/google-cloud/eighth-generation-tpu-agentic-era/
- StorageReview, *Google Announces TPU 8t Sunfish and TPU 8i Zebrafish* (memory and bandwidth details for the 8i): https://www.storagereview.com/news/google-announces-tpu-8t-sunfish-and-tpu-8i-zebrafish
- The Decoder, *OpenAI and Broadcom unveil "Jalapeño," a custom chip built for LLM inference* (June 2026): https://the-decoder.com/openai-and-broadcom-unveil-jalapeno-a-custom-chip-built-for-llm-inference/. Further coverage by TechPowerUp and others appeared in search results only.
- Both the TPU 8i and Jalapeño material are company announcements, and the Jalapeño performance claims were not independently verified.

Reasoning effort, thinking budgets and tool use (Chapters 20 and 24)

- Anthropic documentation: *Extended thinking* https://platform.claude.com/docs/en/build-with-claude/extended-thinking; *Effort* https://platform.claude.com/docs/en/build-with-claude/effort; *Steering thinking* https://platform.claude.com/docs/en/build-with-claude/thinking-steering-and-cost; *Thinking* https://platform.claude.com/docs/en/build-with-claude/thinking; *Task budgets* https://platform.claude.com/docs/en/build-with-claude/task-budgets.
- OpenAI documentation: *Reasoning models* https://developers.openai.com/api/docs/guides/reasoning.
- Model Context Protocol, *What is MCP?*: https://modelcontextprotocol.io/docs/getting-started/intro

Model cards and technical reports (Chapters 20 and 21)

- Qwen Team (2025). *Qwen3 Technical Report*. arXiv:2505.09388. Thinking mode, thinking budget, and the training stages.
- Qwen3-8B model card (thinking switch, sampling settings): https://huggingface.co/Qwen/Qwen3-8B
- Qwen3-Next-80B-A3B model card (hybrid Gated DeltaNet and attention layout): https://huggingface.co/Qwen/Qwen3-Next-80B-A3B-Instruct
- Qwen3.5 and 3.6 families: described in Chapter 21 from their model cards on Hugging Face and secondary coverage in search results. These were **not individually re-verified**, so treat the details there as a snapshot.
- DeepSeek-V3.2 model card: https://huggingface.co/deepseek-ai/DeepSeek-V3.2
- DeepSeek-V4-Pro model card (hybrid compressed attention, mHC, Muon): https://huggingface.co/deepseek-ai/DeepSeek-V4-Pro
- OpenAI gpt-oss-120b model card (reasoning level set in the system prompt): https://huggingface.co/openai/gpt-oss-120b
- Grattafiori et al. (2024). *The Llama 3 Herd of Models*. arXiv:2407.21783. Model sizes used in several examples.
- Gemma Team (2025). *Gemma 3 Technical Report*. arXiv:2503.19786.
- Kimi Team (2025). *Kimi K2: Open Agentic Intelligence*. arXiv:2507.20534.

13. Production systems (Chapter 24)

- Lewis et al. (2020). *Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks*. arXiv:2005.11401.
- Liu et al. (2023), *Lost in the Middle*, and Yao et al. (2022), *ReAct*: listed in Sections 4 and 10.
- Model Context Protocol: listed in Section 12.

14. System 1 decision models (Chapters 25 to 27)

The field is weeks old, so most sources here are model cards, source code, announcements and a few very recent papers. Entries marked **(search result only)** were seen as a headline or summary but not opened in full; entries marked **(not opened)** are known only from another source's reference list. Everything else was opened and read for the parts the chapters use.

Jev (Chapters 25 to 27)

- TypeSafe AI. *Introducing System One Models and Jev* (15 September 2026). https://typesafe.ai/blog/introducing-system-one-models-and-jev. Source of the pricing, latency and speed claims, the 255-option limit, the evaluation method, the blank training-data FAQ heading, and the list of what is not disclosed.
- Wikipedia, *Jev (AI model)*. https://en.wikipedia.org/wiki/Jev_(AI_model). Background on the company and the reported synthetic-data training.
- TechCrunch (18 September 2026). *A new kind of AI model from a ChatGPT inventor is thrilling developers*. Early-user statements and background on the company.
- Ling et al. (2026). *Jev in the Wild: A Data-Driven Analysis of the Jev Model's Functionality, Applications and Ecosystem*. arXiv:2609.30216. Abstract, introduction and conclusion read. Source of the project count and the description of the three interfaces.
- *Calibrated Decision Models for Autonomous Penetration-Testing Harnesses: JEV and Laya as System One Decision Layers for LLM-Driven Pentest Agents*. arXiv:2609.28940. Abstract read. The four decision points, and the authors' statement that their evidence is exploratory.
- Akita, F. (10 October 2026). *Which Jev-like AI decision/classifier model is best?* https://akitaonrails.com/en/2026/10/10/which-jev-like-ai-decision-classifier-model-is-best/. An independent comparison of about 16 models (Chapter 27, Part 6). Its author states that its test cases were AI-assisted and reviewed, with no independent human annotation.
- News coverage of Microsoft-Decision-1, an OpenAI "Decisions" API, Cloudflare's Clef models, Perplexity's Decider and a common decision-model API (October 2026). **(search result only)** Those entries in Chapter 25, Section 6.4, are labelled "reported" for that reason.

Laya (Chapters 25 to 27)

- Convai Innovations / Nandakishor M. *Laya*. Repository https://github.com/NandhaKishorM/laya, documentation https://nandhakishorm.github.io/laya/, and model cards https://huggingface.co/convaiinnovations/laya. The README, benchmark file, fine-tuning and evaluation documents and the source files that build the input, the model, the training loop and the calibration were read. All statements about Laya's architecture (Chapter 26) and training and calibration (Chapter 27) come from these. The performance figures are the authors' own.
- Laya's GitHub issues: #4 and #26 (requests for training scripts), #555 (an independent evaluation against Jev), #741 (a controlled comparison of training losses, single run) and #887 (the `laya.train` module).

Used in Chapter 27, Parts 2 and 6.

- Nandakishor M. (2025). *SalesRLAgent: A Reinforcement Learning Approach for Real-Time Sales Conversion Prediction and Optimization.* arXiv:2503.23303. The earlier sales-conversion work (Chapters 25 and 26).
- Nandakishor M. (2025). *Confidence-Aware Routing for Large Language Model Reliability Enhancement.* arXiv:2510.01237. The earlier routing work (Chapters 25 and 26).
- Nandakishor M., *I Built Non-Autoregressive Decision Models a Year Ago* (DEV Community). The author's own account of the earlier work and of the original training recipe (Chapter 27, Part 2). It links a Reddit post, which blocks automated access, so that post was not read.

Vega and Decision 2.0 (Chapters 25 to 27)

- vLLM Semantic Router team. *Decision-2.0-Vega-27B* and the smaller Decision 2.0 model cards. https://huggingface.co/vllm-sr/Decision-2.0-Vega-27B. Cards, configuration files, the adapter file's header and source code were read. The latency figures are the cards' own.
- Qwen. *Qwen3.8-27B* configuration. https://huggingface.co/Qwen/Qwen3.8-27B. Configuration only. Source of the layer, head, vocabulary and vision-tower figures in Chapter 26, Part 3.
- vLLM Semantic Router team. *Vela 2.0* write-up. https://vllm-sr.ai/blog/vela-2-0-open-foundation-routing-models/. Describes a different model family built on Decision 2.0; used only for the team's data practices (Chapter 27, Part 1).

Encoders and BERT-family background (Chapter 26, Part 1)

- Devlin et al. (2018), *BERT*, and Reimers et al. (2019), *Sentence-BERT*: listed in Section 3. For Chapter 26 the BERT paper is the source of the `[CLS]`, `[SEP]` and segment inputs, the 15% and 80/10/10 masking recipe, next-sentence prediction, the fine-tuning heads and hyperparameters, and the sizes and training scale quoted; Sentence-BERT is the source of the statement that raw `[CLS]` or averaged BERT outputs make poor sentence embeddings.
- Warner et al. (2024). *Smarter, Better, Faster, Longer: A Modern Bidirectional Encoder for Fast, Memory Efficient, and Long Context Finetuning and Inference* (ModernBERT). arXiv:2412.13663. The encoder Laya is built on; source of the architecture, training and size figures in Chapter 26, Part 1.7.
- Radford et al. (2018). *Improving Language Understanding by Generative Pre-Training* (GPT-1), OpenAI. Start and end tokens and the use of the last position's activation (Chapter 26, Part 1.4).
- Wang et al. (2024). *Improving Text Embeddings with Large Language Models* (E5-Mistral). arXiv:2401.00368. Method passage read. Appending an `[EOS]` token and taking its last-layer vector (Chapter 26, Part 1.4).
- Goyal et al. (2023). *Think before you speak: Training Language Models With Pause Tokens.* arXiv:2310.02226. Abstract read (Chapter 26, Part 1.4).
- Dong et al. (2019). *Unified Language Model Pre-training for Natural Language Understanding and Generation* (UniLM). arXiv:1905.03197. **(search result only)** (Chapter 26, Part 1.8).
- Du et al. (2021). *GLM: General Language Model Pretraining with Autoregressive Blank Infilling.* arXiv:2103.10360, and the UL2 20B announcement on the Google Research blog. **(search result only)** (Chapter 26, Part 1.8).
- BehnamGhader et al. (2024). *LLM2Vec: Large Language Models Are Secretly Powerful Text Encoders.* arXiv:2404.05961. **(search result only)** (Chapter 26, Part 1.8).
- Weller et al. (2025). *Seq vs Seq: An Open Suite of Paired Encoders and Decoders* (Ettin). arXiv:2507.11412. **(search result only)** (Chapter 26, Part 1.8).

- Clarke et al. (2023). *Label agnostic pre-training for zero-shot text classification.* Findings of ACL 2023, pp. 1009 to 1021. **(not opened)**; it appears in the reference list of the "Jev in the Wild" paper (Chapters 25 and 27).

Data and datasets (Chapter 27, Part 1)

- `LocalLLaMA/typed-decisions` on Hugging Face (Apache-2.0). The full dataset card was read. Source of the teacher description (mean of three samples from a roughly 4B-class model), the 0.735 teacher self-agreement figure and the reference points quoted.
- `typesafe/evalsafe-*`, `ZefanCai/Open-Jev` (CC0), `tasksource/tasksource-jev-typed-decisions`, `tasksource/procedural-typed-decisions` (Apache-2.0) and `pngwn/system-one-decisions` (CC BY-NC 4.0). Hugging Face metadata and descriptions only (the Open-Jev card was partly read).

General

- Kahneman, D. (2011). *Thinking, Fast and Slow.* The origin of the System 1 and System 2 terms (Chapter 25). Used only for that general idea.

Notes on verification

- **Papers.** Every arXiv identifier above was confirmed to match the stated paper. The abstracts were read for the papers from which the handbook quotes specific results: Mixtral (47B total, 13B active), Switch Transformer (speed-up), DeepSeekMoE, self-consistency, Tree of Thoughts, process supervision, test-time compute, chain of thought, s1, L1, DeepSeek-R1, DeepSeekMath, vLLM and speculative decoding (the 2 to 3× result), CLIP, ViT, Flamingo and LLaVA, and the Qwen3 report. For the remaining papers (for example RoPE, RMSNorm, SwiGLU, GQA, FlashAttention, LoRA, DPO, PPO, GPTQ, AWQ), the handbook relies on well-established descriptions of the methods and does not quote numbers from them.
- **Documentation and model cards.** Statements about reasoning-effort controls, thinking budgets, countdown markers, token billing and token limits come from the provider documentation listed in Section 12, as read in October 2026. Provider behaviour changes quickly.
- **Illustrative numbers.** Many numbers in the handbook are deliberately made up to show a principle: the stop-probability tables in Chapter 20, the cost and price examples in Chapters 22 and 24, the 2-D embedding vectors in Chapter 24, and the "quality" scores in Chapters 14 and 20. They are always labelled as illustrative in the text.
- **Appendix A** is different: every number in it was computed by a program (`Appendix-A-tinygpt.js`, included next to it), and its gradients were checked against numerical differentiation.
- **Hardware figures (Chapter 23)** such as H100 bandwidth, NVLink and PCIe speeds are as quoted by NVIDIA, and the NVLink and PCIe figures there are total both directions. The TPU 8i and Jalapeño details are company announcements.
- **Chapters 16, 17 and 21** (the modern-models chapters) were written earlier in the project and checked against the official papers and model cards for the models discussed. The papers listed above are the primary sources. Page-level addresses for individual claims in those chapters were not recorded, so check a specific figure against the cited paper or model card before relying on it.
- **Not used.** No book, course or website beyond those listed was consulted, and no other YouTube playlist or video was searched or sourced.

- **Decision models (Chapters 25 to 27).** The field is weeks old. Statements about Laya's architecture were checked against its source code and documentation, and statements about Vega's against its model cards, configuration and code. Performance and speed figures are the makers' own unless an independent test is named, and Jev's internals are not public. The worked numbers in Chapters 25 to 27 (probabilities, expected level, ranked probability score, the calibration, drift, margin-of-error and cost examples) are illustrative: the ones in Chapters 26 and 27 were produced by the seeded toy programs in `Chapters-26-27-experiments.js`, which you can rerun. They show mechanisms and are not measurements of Laya or Vega. The compute and memory estimates use the standard rules of thumb (about 6 × parameters × tokens for training, 2 × parameters per token for a forward pass, about 16 bytes per parameter for mixed-precision training).
- **Not used (Chapters 25 to 27).** The Reddit post that the Laya author links could not be opened, the articles on Microsoft-Decision-1, OpenAI's decisions API and Cloudflare's Clef models were seen only as search results, and the arXiv preprints and papers marked "(search result only)" in Section 14 were not read in full.